



Palo Alto Networks Certified Next-Generation Firewall Engineer

NGFW-ENGINEER • STUDY REFERENCE AND NOTES

A working reference built for the Next-Generation Firewall Engineer certification and its learning path. Every page carries the detail, the CLI, the common mistakes, and a set of recall questions to test yourself against.

The three domains

1 • PAN-OS Networking Configuration

interfaces • zones • HA • routing • GlobalProtect • tunnels

2 • PAN-OS Device Setting Configuration

authentication • VSYS • logging • updates • certificates • User-ID • web proxy

3 • Integration and Automation

deployment options • APIs • orchestration • Panorama • ACC

Plus the context you need in practice

NAT • security policy • App-ID and Content-ID • objects and EDLs • troubleshooting

Exam at a glance

Exam code

NGFW-Engineer

Duration

90 minutes

Format

Multiple choice

Level

Specialist

Delivered by

Pearson VUE

Recommended

2 to 3 years IT security, 2 with PAN NGFW

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Contents



Topics by Part

Click a topic to jump to it and open it, or type below to filter, the search looks inside the topic, the detail, the CLI and the check yourself answers.

1 Part 1 • Platform and Deployment

pages 1 to 4

- 1 Single Pass Parallel Processing and the planes
- 2 Management access and initial configuration
- 3 Licences and subscriptions
- 4 PA-Series hardware
- 5 VM-Series
- 6 CN-Series and Cloud NGFW
- 7 Choosing a deployment option
- 8 Why quantum safety is on a firewall release
- 9 Quantum Readiness Dashboard and cipher translation
- 10 Understanding the quantum threat and the migration clock
- 11 Post quantum cryptography: algorithms, standards, detection
- 12 Governance: Y2Q, the legal mandate, becoming quantum ready
- 13 Passwordless: the threat and the standards
- 14 Passwordless architecture and use cases
- 15 Configuring and troubleshooting passwordless
- 16 Orion scale and performance changes
- 17 The two Orion fabrics
- 18 Orion lifecycle and operational changes
- 19 Modernizing PAN-OS: commit process and telemetry
- 20 Putting Orion in context

2 Part 2 • Prisma AIRS

pages 5 to 6

- 21 What Prisma AIRS is and what it protects
- 22 Network Intercept versus API Intercept
- 23 The AI threat categories
- 24 AI Model Security
- 25 AI agent security
- 26 AI Red Teaming
- 27 Posture management and discovery

3 Part 3 • Interfaces and Zones

pages 7 to 9

- 28 Interface modes: tap, virtual wire, L2, L3, tunnel
- 29 Layer 2, VLAN objects and VLAN interfaces
- 30 Layer 3 addressing and link settings
- 31 Aggregate Ethernet and LACP
- 32 Subinterfaces and loopbacks
- 33 Management interface and interface management profiles
- 34 Virtual wire in depth
- 35 Security zones and zone types
- 36 Zone Protection, DoS Protection, packet buffers

4 Part 4 • Routing

pages 10 to 11

37 Virtual routers vs the Advanced Routing Engine

38 Static routes, AD and path monitoring

39 OSPF on PAN-OS

40 BGP on PAN-OS

41 Redistribution and routing policies

42 BFD, ECMP and route monitoring

5 Part 5 • NAT and Traffic Steering

pages 12 to 13

43 Source NAT types

44 Destination NAT and port forwarding

45 U turn (hairpin) NAT

46 NAT and security policy interaction

47 Policy Based Forwarding

48 DHCP and IPv6

49 Session management and QoS

6 Part 6 • Policy and Objects

pages 14 to 16

50 How security rules are evaluated

51 Rule types: universal, intrazone, interzone

52 The policy types and their order

53 application-default versus any

54 Application dependencies and override

55 Shadowing, unused rules, Policy Optimizer

56 Address, service and application objects

57 External Dynamic Lists

58 Tags and Dynamic Address Groups

59 Custom applications, threats, data patterns

7 Part 7 • App-ID and Content-ID

pages 17 to 19

60 The App-ID identification sequence

61 App-ID values in the Traffic log

62 Custom App-IDs and content updates

63 Security profiles versus policy rules

64 Antivirus and Anti-Spyware

65 Vulnerability Protection

66 URL Filtering and credential detection

67 File Blocking, WildFire and Data Filtering

68 Identifying the culprit and tuning safely

8 Part 8 • Certificates and Decryption

pages 20 to 23

69 The purpose and value of decryption

70 Certificates and PKI integration

71 Certificate profiles and SSL/TLS service profiles

72 SSL Forward Proxy, trust and untrust

73 Subordinate CA from enterprise PKI

74 SSL Inbound Inspection

75 Decryption policy and SSH Proxy

76 Decryption profiles

77 Exclusions, pinning and mutual TLS

78 Decryption broker and mirroring

79 Deploying decryption safely

80 Troubleshooting decryption

81 Decryption and network design

82 Detecting PQC via threat signatures

83 Decrypting PQC traffic and cipher translation

9 Part 9 • Authentication and Identity

pages 24 to 28

- 84 The authentication stack: profiles and sequences
- 85 The authentication protocols compared
- 86 Administrator roles and access control
- 87 Authentication Policy and the portal
- 88 Multi factor authentication
- 89 Passwordless authentication
- 90 Certificate based authentication
- 91 User-ID concepts and the two halves

- 92 The mapping methods
- 93 Agent versus agentless User-ID
- 94 Group mapping and directory sync
- 95 User context and Device-ID
- 96 Identity data redistribution and segments
- 97 What the Cloud Identity Engine is
- 98 SCIM and the two management modes
- 99 Health alerts and stolen credentials

10 Part 10 • VPN and Tunnels

pages 29 to 30

- 100 IKE Phase 1 and Phase 2
- 101 Proxy IDs and tunnel interfaces
- 102 Tunnel monitoring and dead peer detection
- 103 GRE tunnels on PAN-OS
- 104 Quantum resistant cryptography on IPSec
- 105 Configuring RFC 8784 and hybrid KEM step by step
- 106 Quantum Key Distribution

11 Part 11 • GlobalProtect

pages 31 to 32

- 107 Portal, gateway and agent architecture
- 108 The sequential implementation, step by step
- 109 Gateway selection and connect methods
- 110 Split tunnelling by route, domain and application
- 111 Authentication and SAML on GlobalProtect
- 112 HIP objects and HIP profiles
- 113 Clientless VPN

12 Part 12 • High Availability

pages 33 to 34

- 114 Active/Passive HA and the HA links
- 115 HA states and election
- 116 Link monitoring and path monitoring
- 117 Active/Active: session owner, setup, floating IPs
- 118 Config sync and split brain
- 119 HA upgrade sequence

13 Part 13 • Virtual Systems

pages 35 to 36

- 120 Virtual systems and multi tenancy
- 121 Interfaces, zones and routers inside a VSYS
- 122 Shared versus VSYS specific objects
- 123 Inter VSYS routing: external zones and shared gateway
- 124 VSYS administrators and resource limits
- 125 When to choose VSYS over separate firewalls

14 Part 14 • Web Proxy and Logging

pages 37 to 39

- 126 Explicit versus transparent web proxy
- 127 PAC files and proxy authentication
- 128 Log types and what writes to each
- 129 Log forwarding profiles and match lists

130 Built in actions and tagging

131 Designing a logging and forwarding strategy

132 Strata Logging Service

133 Log collectors and log collector groups

134 Forwarding preference and redundancy

15 Part 15 • Panorama and ACC

pages 40 to 43

135 Panorama modes and adding firewalls

136 Device groups and hierarchy

137 Device group objects and policies

138 Pre rules, post rules and evaluation order

139 Templates and template stacks

140 Designing a device group and template hierarchy

141 The commit model: candidate, running, commit vs push

142 Panorama HA

143 Log collection and forwarding through Panorama

144 Administrative accounts, RBAC and access domains

145 ACC dashboards and custom reports

146 Panorama troubleshooting

147 Quantum Readiness view in SCM

16 Part 16 • Software Updates

page 44

148 Dynamic content updates and the new App-ID risk

149 PAN-OS upgrade paths and version compatibility

150 Configuration backup and restore

17 Part 17 • Cloud and Orchestration

pages 45 to 48

151 Self managed VM-Series versus Cloud NGFW

152 Bootstrapping: init-cfg.txt and bootstrap.xml

153 Autoscaling and lifecycle automation

154 VMware NSX-T service insertion

155 CN-Series and Kubernetes

156 Multicloud Security Fabric (MSF): the four pillars

157 Hyperscale Security Fabric (HSF): P-Nodes and S-Nodes

158 XML API, REST API and the Terraform provider

159 CLARA: Cloud Network & AI Risk Assessment

18 Part 18 • Troubleshooting

pages 49 to 51

160 Packet flow: first packet versus fast path

161 The session table and session states

162 Global counters

163 Packet capture: the four stages

164 A structured troubleshooting approach

165 Common symptom to cause patterns

166 GUI log queries: the filter syntax

167 CLI log commands and live tailing

168 Choosing GUI versus CLI, and combining both

Part 1 • Platform and Deployment 1

Architecture, Management and Licensing

Everything later in this guide is configured on top of a platform that is already reachable, licensed and time synchronised. Start here.

1 Single Pass Parallel Processing and the planes

Single pass: the packet is inspected **once**. App-ID, User-ID, Content-ID and threat prevention all operate against that one pass rather than each engine rescanning the traffic.

Parallel processing: networking and security functions run on **dedicated hardware at the same time** rather than in a chain.

The two planes

A firewall splits into a **management plane** and a **data plane**.

Management plane: web interface, CLI, logging, configuration, routing protocol daemons, User-ID agent processes.

Data plane: forwarding and inspection. On larger platforms it subdivides into **signature match**, **security processing** and **network processing**.

Why the split matters operationally

A management plane problem (slow GUI, log ingestion lag, commit taking forever) is **not** the same as a data plane problem (throughput, session capacity, packet loss), and they are diagnosed separately:

`show system resources` : management plane CPU and memory

`show running resource-monitor` : data plane CPU per core

`show system state filter sw.mprelay.*` : plane to plane communication

In practice

A hung or sluggish web interface with traffic still flowing perfectly is the classic management plane symptom.

Don't reboot the firewall for it, check management plane CPU first, because a rebooting firewall interrupts traffic that was fine.

The two planes, and which symptom belongs to which

the most useful first question in any performance complaint

Management plane

web UI • CLI • configuration • commits

logging • reporting • routing protocol daemons

User-ID processes • Panorama connection

`show system resources`

Management plane symptoms

slow GUI • slow commit • log lag • traffic unaffected

Data plane

forwarding • NAT • session table

signature match • security processing

network processing • decryption

`show running resource-monitor`

Data plane symptoms

throughput drop • packet loss • session table full

② Management access and initial configuration

Defaults: management address **192.168.1.1/24**, credentials **admin/admin** with a forced password change on first login, console **9600 8N1**.

The **MGT interface is out of band** and has **its own default gateway**, configured at

Device > Setup > Management . It **does not use the data plane routing table**.

Initial CLI configuration

configure

set deviceconfig system ip-address 10.0.0.5 netmask 255.255.255.0 default-gateway 10.0.0.1

set deviceconfig system dns-setting servers primary 10.0.0.53

set deviceconfig system hostname fw-edge-01 timezone Europe/London

set deviceconfig system ntp-servers primary-ntp-server ntp-server-address 10.0.0.123

commit

Service routes at Device > Setup > Services send specific services (updates, DNS, syslog, LDAP, WildFire, Palo Alto Networks Services) out a **data interface instead of MGT**.

Get NTP and DNS right at this stage. **Certificate validation and log correlation both depend on accurate time**, and a firewall with bad time produces decryption failures that look like configuration errors.

In practice

"The firewall routes production traffic fine but can't download content updates" is MGT's separate gateway. Fix it with a service route, not a static route.

③ Licences and subscriptions

The base platform gives App-ID, User-ID, the policy engine and networking. Everything else is a **subscription**:

Threat Prevention → antivirus, antispysware, vulnerability protection

Advanced Threat Prevention → adds inline ML based detection

URL Filtering (PAN-DB) → web categories

WildFire / Advanced WildFire → sandbox analysis and rapid signatures

DNS Security → cloud DNS lookups and sinkhole intelligence

GlobalProtect → needed for **HIP checks and clientless VPN**, not for a plain VPN

SD-WAN, IoT Security, Enterprise DLP, AIOps as further options

Support is separate from subscriptions. Without a valid support licence you **cannot download PAN-OS software**, even while content updates keep arriving.

request license info

· request license fetch

· show system info

In practice

The diagnostic tell: content updates still arriving but a PAN-OS upgrade refusing to download means **support has lapsed**, not the subscriptions. Two different renewals, two different failure modes.

✓ Check yourself

architecture and management

1. The GUI is crawling but traffic is unaffected. Which plane, and which command?

Management plane. `show system resources` . Don't reboot, you'd interrupt traffic that was fine.

2. Which command shows data plane CPU per core?

`show running resource-monitor` .

3. Why can a firewall route perfectly and still fail to reach the update server?

The MGT interface has its own default gateway and does not use the data plane routing table. Fix with a service route.

4. Which two settings must be right before deploying certificates, and why?

NTP and DNS. Certificate validation and log correlation depend on accurate time; bad time looks like a config error.

5. Content updates arrive but PAN-OS won't download. What has expired?

Support, not the subscriptions. No support licence, no software download.

6. Does a basic site to site style remote access VPN need the GlobalProtect subscription?

No: the subscription is needed for HIP checks and clientless VPN, not plain connectivity.

✓ Anchor facts

- ✓ Defaults: **192.168.1.1/24**, admin/admin, console **9600 8N1**.
- ✓ **MGT has its own gateway**, separate from the data plane.
- ✓ Management plane = GUI, CLI, logging, routing daemons, User-ID.
- ✓ Data plane = forwarding, signature match, security and network processing.
- ✓ **No support licence, no software download.**
- ✓ GlobalProtect subscription is for **HIP and clientless**, not basic VPN.



Easy to get wrong

- Rebooting for a slow GUI and interrupting healthy traffic.
- Assuming MGT uses the data plane routing table.
- Deploying certificates before NTP is correct.
- Confusing support expiry with subscription expiry.
- Buying GlobalProtect licences for a deployment that doesn't need them.

Symptom	Plane	Command	Do not
Slow web UI, slow commit	Management	<code>show system resources</code>	Reboot reflexively
Log ingestion lag	Management	<code>show log-collector-es-cluster health</code>	Blame throughput
Throughput below line rate	Data	<code>show running resource-monitor</code>	Check the GUI first
Session table full	Data	<code>show session info</code>	Raise timeouts blindly

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 1 • Platform and Deployment 2

The Deployment Options

The foundation for everything else in this part, every orchestration decision that follows operates on a form factor you have chosen.

4 PA-Series hardware

The physical appliance, with **dedicated hardware for each plane** and the highest throughput per unit. Ranges run from small branch models through mid range campus units to chassis based data centre platforms.

What differs by model: **throughput** (App-ID, Threat Prevention and IPsec figures are quoted separately, and the Threat Prevention number is the realistic one), **maximum sessions**, **new sessions per second**, **maximum security rules**, **maximum VSYS**, and whether the platform supports **multi VSYS** at all.

Smaller models have a **single combined plane**, so a management plane spike affects traffic in a way it would not on a larger platform. That is worth knowing before sizing a branch.

Hardware features on larger units: **redundant power supplies**, **hot swappable fans**, **dedicated HA ports**, and a **log card** slot on some chassis.

`show system info` for model and serial, `show system environmentals` for power, fans and temperature.

In practice

Size on the **Threat Prevention throughput** figure, not the App-ID one. App-ID numbers assume no threat inspection, which is not how anyone runs a firewall in production.

5 VM-Series

The **virtual machine** form factor, running the same PAN-OS on a hypervisor or in a public cloud.

Private cloud: VMware ESXi and NSX-T, KVM, Hyper-V, Nutanix, OpenStack.

Public cloud: AWS, Azure, GCP, Oracle Cloud, Alibaba Cloud.

Licensing is either by **fixed model** (VM-50 through VM-700, sized by vCPU and memory) or by **flexible / credit based licensing**, where you allocate credits to instances and choose vCPU count and which subscriptions to enable. Flexible licensing is the current model and suits autoscaling.

Bootstrapping lets an instance configure itself at first boot from `init-cfg.txt` and optionally `bootstrap.xml`, so a new firewall comes up already licensed and connected to Panorama with no manual step. Covered in full in the cloud and orchestration part.

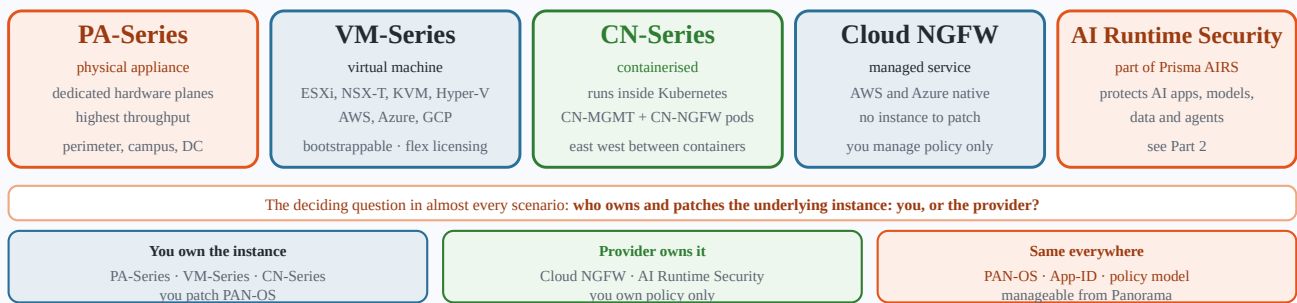
Performance depends on the hypervisor: **SR-IOV** or **DPDK** give far better throughput than a standard virtual NIC, and **CPU pinning** matters.

In practice

VM-Series in a cloud is **not** the same as Cloud NGFW. With VM-Series you own the instance, patch PAN-OS, and size the vCPUs. That distinction answers most "which form factor" questions.

The five deployment options

all run the same PAN-OS and policy model: what differs is who owns the instance



6 CN-Series and Cloud NGFW

CN-Series · the containerised firewall

Runs **inside Kubernetes** as two pod types:

CN-MGMT: the management plane, deployed as a StatefulSet, holding configuration and logs.

CN-NGFW: the data plane, deployed as a DaemonSet so there is one per node, inspecting pod traffic.

Its purpose is **east west traffic between containers**, which a perimeter firewall never sees. Policy can use **Kubernetes metadata** (namespaces, labels, service names) rather than pod IPs, which matter because pod IPs are ephemeral. It is managed from **Panorama with the Kubernetes plugin**.

Cloud NGFW · the managed service

cloud provider managed NGFW in **AWS and Azure**. You subscribe through the provider's marketplace, attach it to a VPC or VNet, and **manage only policy**: there is no instance to size, patch or scale.

Policy is managed either through the cloud provider's console or through **Panorama** using **Cloud Device Groups**. Availability and scaling are the provider's responsibility.

In practice

CN-Series is the answer when the requirement names **pod to pod** or **container** traffic. Cloud NGFW is the answer when the requirement stresses **no operational overhead** or **no instances to manage**.

7 Choosing a deployment option

Choosing the right deployment option comes down to matching a requirement to a form factor.

Physical site, high throughput, hardware control → PA-Series.

Virtualised data centre, or IaaS where you want full control → VM-Series.

Traffic between containers in a cluster → CN-Series.

Cloud native with minimum operational overhead → Cloud NGFW.

AI applications, models, data and agents → AI Runtime Security.

Mixed estates are normal, and all of them are **manageable from Panorama**, which is why the automation domain pairs deployment with centralised management.

Sizing considerations that cut across all of them: **throughput with threat inspection enabled**, **session count**, whether you need **multi VSYS**, and whether the deployment must **autoscale**, which pushes you toward VM-Series with flexible licensing or Cloud NGFW.

In practice

Read the requirement for the operational ownership signal. "We have no platform team" points at Cloud NGFW. "We need SR-IOV performance and custom sizing" points at VM-Series. The throughput number alone rarely decides it.

✓ Check yourself

deployment options

1. Which throughput figure should you size on, and why?

Threat Prevention throughput. The App-ID figure assumes no threat inspection, which is not how anyone runs production.

2. Name the two CN-Series pod types and their Kubernetes workload types.

CN-MGMT as a StatefulSet (management plane), CN-NGFW as a DaemonSet, one per node (data plane).

3. Why does CN-Series policy use Kubernetes metadata rather than IPs?

Pod IPs are ephemeral. Namespaces, labels and service names are stable identifiers.

4. VM-Series in AWS versus Cloud NGFW in AWS: what actually differs?

Who owns the instance. VM-Series: you size, patch and scale it. Cloud NGFW: you manage policy only.

5. The requirement stresses no platform team and no instances to patch. Which option?

Cloud NGFW. The operational ownership signal decides it, not throughput.

6. Which two form factors suit autoscaling, and what licensing supports it?

VM-Series with flexible / credit based licensing, and Cloud NGFW, which scales by itself.

✓ Anchor facts

- ✓ Five options: **PA-Series**, **VM-Series**, **CN-Series**, **Cloud NGFW**, **AI Runtime Security**.
- ✓ CN-Series = **CN-MGMT StatefulSet** + **CN-NGFW DaemonSet**.
- ✓ CN-Series policy uses **Kubernetes metadata**, not pod IPs.
- ✓ Cloud NGFW is **provider managed**, policy via console or **Cloud Device Groups** in Panorama.
- ✓ VM-Series licensing: **fixed model or flexible credits**.
- ✓ **SR-IOV / DPDK** and CPU pinning drive VM-Series performance.



Easy to get wrong

- Sizing on **App-ID throughput** instead of Threat Prevention.
- Treating VM-Series in cloud as equivalent to **Cloud NGFW**.
- Expecting a perimeter firewall to see **pod to pod** traffic.
- Forgetting small PA models may share a **single combined plane**.
- Writing container policy against pod IPs.

Requirement signal	Option	You manage	Managed from Panorama?
Physical site, max throughput	PA-Series	Everything	Yes
Hypervisor or IaaS, full control	VM-Series	PAN-OS and the instance	Yes
Pod to pod, east west in a cluster	CN-Series	Pods and PAN-OS	Yes, with the K8s plugin
No operational overhead	Cloud NGFW	Policy only	Yes, Cloud Device Groups
AI apps, models, data, agents	AI Runtime Security	Policy	Via the AIRS platform

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 1 • Platform and Deployment 3

PAN-OS 12.1 Orion • Quantum Readiness

Orion's headline theme is quantum safety, which is a strong signal of how recent this material is. Three topics below slow down for the threat, the standards, and the legal mandate behind them, before the tooling.

⑧ Why quantum safety is on a firewall release at all

Palo Alto Networks' name for this whole capability area is **Quantum Resistant Encryption (QRE)**: it's the umbrella term covering the threat itself, the algorithms and standards answering it, and Orion's specific tooling to deploy it. This topic and the next cover why QRE exists and what Orion does about it; the two after go deeper into the threat and the standards behind it.

The driver is **harvest now, decrypt later**. Traffic captured today can be stored and decrypted once the cryptography protecting it becomes breakable. So a VPN tunnel carrying data with a **long confidentiality life** is exposed by a **future** capability, not a present one.

That is why standards bodies moved before the capability existed. NIST standardised the replacement algorithms, and the ones to know by name are **ML-KEM** for key encapsulation, plus **ML-DSA** and **SLH-DSA** for signatures.

Two migration problems Orion addresses

You do not know what you are using. Cryptographic usage across an estate is largely invisible, so you cannot plan a migration.

Some things cannot be upgraded. Legacy applications may never support quantum safe ciphers natively.

Orion's answers are a **Quantum Readiness assessment and dashboard** for the first problem and **cipher translation** for the second.

This is **recognition and configuration awareness**, not algorithm internals. Know why you would enable it, and where the setting sits relative to conventional IPsec parameters.

In practice

The question to be able to answer is "which tunnels should be quantum safe first", and the answer is those carrying data that must stay confidential for years, not the ones carrying the most traffic.

The Orion quantum toolkit

assess what you have, upgrade what you can, translate what you cannot

1 • Assess

Quantum Readiness Dashboard

- inventory of cryptographic assets
- compliance validation against PQC
- inline remediation guidance

2 • Upgrade

Quantum ready VPN

- post quantum options on IPsec
- PQ preshared keys • RFC 8784
- multiple key exchanges • RFC 9370

3 • Translate

Cipher translation

- for apps that cannot be upgraded
- firewall upgrades the session
- legacy app becomes quantum safe

Harvest now, decrypt later: traffic captured today is decrypted when the capability arrives, so long lived confidentiality is the risk, not current traffic volume.

NIST standards worth naming: **ML-KEM** (key encapsulation) • **ML-DSA** and **SLH-DSA** (signatures)

9 Quantum Readiness Dashboard and cipher translation

Quantum Readiness Dashboard gives a **real time inventory of cryptographic assets**, validates them for compliance against post quantum standards, and offers **inline remediation**, so the assessment and the fix are in the same place. This, plus cipher translation below, is Orion's actual delivery of **QRE**.

Its value is planning. A PQC migration is a multi year exercise, and you cannot sequence it without knowing which tunnels, certificates and cipher suites are in use and which are already compliant.

Cipher translation handles the applications that **cannot be upgraded**. The firewall terminates and reestablishes with quantum safe cryptography, so a legacy application becomes quantum safe **without any change to the application itself**.

Alongside the software, Orion introduced **quantum optimised hardware**: 14 new fifth generation NGFW models engineered for high performance processing of post quantum cryptography, including a **PA-5500 Series** for data centres and a **ruggedised PA-455R-5G** with native 5G for industrial environments.

In practice

The pairing is the point. The dashboard tells you what to migrate and in what order; cipher translation covers the things that will never migrate themselves. Neither alone solves the problem.

10 Understanding the quantum threat: what a quantum computer actually is, and the migration clock

Topic 8 said this is **recognition and configuration awareness, not algorithm internals**. This topic and the next one are the internals, taken slowly enough to actually stick, so the dashboard and cipher translation stop being a black box.

What a quantum computer actually is

Quantum computers (QCs) represent a new supercomputing platform based on the laws of quantum mechanics, not a faster version of a normal computer:

- **Quantum algorithms:** solve problems that classical computers structurally cannot handle.
- **Qubits:** unlike a classical bit, always either 0 or 1, a qubit can exist in **simultaneous states**.
- **Entanglement:** qubits can be linked to other qubits, correlating their states.
- The result: a QC can **sift through enormous amounts of data and spot subtle patterns** that classical algorithms simply miss.

Why migration is genuinely hard, not just theoretically necessary

Four separate obstacles stack up together:

- **Time consuming:** the transitions themselves take real time.
- **Expensive:** the work carries a real cost.
- **Workforce gap,** not enough people who actually understand post quantum cryptography.
- **Early adoption reduces risk:** waiting doesn't make any of the other three easier.

The clock, stated as four numbers

NIST frames the migration timeline directly:

- **Prep and planning:** about a year.
- **Migration itself:** a completion goal of two to five years.
- **Time data needs to stay secure** once migrated: roughly eight years.
- **CRQC:** a **Cryptographically Relevant Quantum Computer**, one actually capable of breaking today's encryption, currently estimated **five to fifteen years away**.

Run those four numbers forward from today, and an organisation that waits for more certainty on the fifteen year end of that range has already missed the one year prep window and most of the migration window too.

Harvest now, decrypt later

is what actually collapses "years away" into "start now." Harvesting attacks are already happening, traffic is being captured and stored today, encrypted, waiting for the capability to decrypt it to arrive. Anything with a long confidentiality lifetime that's been captured since roughly 2023 is already sitting in that exposure window, whether or not the CRQC that eventually breaks it exists yet.

Two issues, and only one is covered here

Palo Alto Networks frames the immediate remediation problem as two separate issues:

- **Data harvesting attacks:** the harvest now decrypt later problem above.
- **Unsanctioned post quantum ciphers:** a bad actor already using an available, unapproved PQC to hide traffic on your network today. This one, and the standards that answer both issues, is the next topic.

In practice

If asked why this is urgent rather than a five to fifteen year problem, the answer is harvest now, decrypt later: the attack already happened for anything captured today, the decryption is just deferred. The prep and planning year should already be running.

11 Post quantum cryptography: the two algorithms breaking things, the standards fixing them, and catching unsanctioned use

What PQC actually is

Post quantum cryptography (PQC) is the essential advancement in encryption built specifically to **resist quantum attacks**. Together, these are the guidelines for implementing quantum resistant algorithms so data stays secure through the migration described in the previous topic:

- **FIPS 203, FIPS 204, FIPS 205:** three NIST approved encryption standards.
- **RFC 8784, RFC 9242, RFC 9370:** address the same threat inside existing protocols like IKEv2.

Shor's algorithm, breaks asymmetric encryption outright

Shor's algorithm is a method of **factoring complex numbers**, including finding the large prime numbers that **asymmetric encryption**, and by extension **PKI**, is built on. Shor's doesn't need to brute force a private key: it **derives the key directly**, based on the large primes used to decrypt messages. That's why asymmetric encryption needs entirely new maths, not a longer key, a longer key built on the same factoring problem is still solved by the same algorithm.

Grover's algorithm, weakens symmetric encryption, doesn't break it

Grover's algorithm targets **symmetric encryption** instead, and does something structurally different from Shor's: it gives a **quadratic speed up** on brute force key search, which **halves the effective strength** of a symmetric key rather than defeating it outright. A 256 bit key halved by Grover's still leaves 128 bits of real security; a 128 bit key halved does not. That single fact is why **AES-256 is the going forward recommendation over AES-128**: doubling the key length is a sufficient defence against Grover's in a way it can never be against Shor's.

FIPS 203, general encryption

FIPS 203 is the primary standard for general encryption:

- **Small encryption keys:** comparatively small, so two parties can exchange them easily.
- **Speed of operation.**
- Based on **CRYSTALS-Kyber**, renamed **ML-KEM**: Module-Lattice-Based Key Encapsulation Mechanism.

FIPS 204, the primary signature standard

FIPS 204 is the primary standard for protecting digital signatures:

- Based on **CRYSTALS-Dilithium**, renamed **ML-DSA**: Module-Lattice-Based Digital Signature Algorithm.

FIPS 205, the deliberate backup, on different maths

FIPS 205 is also for digital signatures:

- Based on **Sphincs+**, renamed **SLH-DSA**: Stateless Hash Based Digital Signature Algorithm.
- Built on a **different mathematical approach entirely** from ML-DSA.
- Exists specifically as a **backup in case ML-DSA is later found to be vulnerable**: a deliberate hedge, not duplication.

The RFCs, carrying all of this over IKEv2

- **RFC 8784:** combines a post quantum pre shared key (PPK) with IKEv2, protecting specifically **against Shor's algorithm**.
- **RFC 9242:** defines the **IKE_INTERMEDIATE** exchange.

- **RFC 9370:** multiple key exchanges (hybrid keys); its extra post quantum key material actually needs 9242's exchange to have somewhere to go.

Together, all three RFCs mark significant progress in strengthening the security of digital communication and data protection.

Catching the other issue from the previous topic: unsanctioned PQC

Unsanctioned PQCs means there are open source and other PQC implementations freely available, and a bad actor using one can potentially hide on a network, obscuring a secure shell (SSH) session inside traffic that looks compliant, for example. Palo Alto Networks NGFWs can recognise the behaviour of a PQC even when it's an unapproved or open source implementation, which is exactly what makes it possible to find and stop. It's imperative to be able to identify whether someone is using an unauthorised PQC inside your network at all.

Enhanced Decryption Policies and Threat Signatures

are the mechanism: they prevent unknown threats hidden by post quantum cryptography in SSL. The three things this buys, named directly:

- **Traceability:** you can see it happening.
- **Control:** you can act on it.
- **Protection:** the session doesn't get a free pass just because it's encrypted with something new.

In practice

Keep the two algorithms straight in one line: **Shor's derives the asymmetric key outright; Grover's only halves symmetric strength.** That's why PKI needs three entirely new FIPS standards while AES just needs a longer key, and why FIPS 205 exists on purpose, as insurance against FIPS 204 rather than a second copy of it.

12 The governance side: Y2Q, the legal mandate, and how an organisation actually becomes quantum ready

Everything so far has been the technical answer. This topic is the **regulatory and organisational pressure** sitting behind it, why a migration plan isn't optional for a lot of organisations, and what actually doing one looks like.

Y2Q, the governance clock started on a specific date

Y2Q (Years to Quantum) is framed as having **started ticking on 13 August 2024**. It deliberately echoes Y2K: a fixed, dated point after which the remaining runway to migrate is shrinking on a known schedule, not an open ended someday.

The National Quantum Initiative

is the **federal source and gateway to quantum R&D** across the US government, the umbrella programme that the legal requirements below sit under.

The actual legal mandate

The **Quantum Computing Cybersecurity Preparedness Act**, passed **July 2022**, is where "someday" became a dated requirement:

- Within a year of NIST announcing its new PQC standards, the **Director of OMB must issue guidance**.
- That guidance requires **each executive agency to develop a migration plan**, including **interim benchmarks**, to move the agency's IT to post quantum cryptography.

This is what turns Part 1's QRE material from "a good idea" into "**a plan with benchmarks a federal agency is legally required to produce**."

Mapping to DoD Zero Trust

Palo Alto positions QRE directly against the **DoD's own Zero Trust principles**, three pillars at a time:

- **Consolidate**: use existing NGFW platform capability, centrally managed. No new box to deploy for this.
- **Integrate**: open standards, an integrated platform, **modular PQC deployment** rather than a rip and replace.
- **Accelerate**: leverage the investment already made; simple to deploy, change and evolve; a **phased approach** rather than one cutover.

How an organisation actually becomes quantum ready, five steps

This is a genuine operating cycle, not a one time project, sourced from *A guide to a quantum safe organization* (QED-C, December 2021):

- **1 • Assign resources and build awareness**: designate a team to manage the transition, build an internal budget, and engage vendors. Reading this guide is literally step 1 in motion.
- **2 • Define responsibilities**: break responsibilities out by use case. This is named directly as **a huge undertaking**: an organisation cannot become quantum safe in one swoop.
- **3 • Develop an inventory and priority list**: review every device, system, platform and vendor for three things: **how long it needs to stay secure**, the **lifetime of the asset** (will it even still be around in five years?), and its **susceptibility to attack**.
- **4 • Evaluate solutions, experiment, and test**: evaluate how PQC, QKD and other quantum technologies will secure the organisation's assets. This can include **layering PQC on existing RSA architecture**, the same additive pattern as RFC 8784 and RFC 9370 on the VPN page, while preparing for future implementations.
- **5 • Continue, monitor progress**: review, monitor and refine policies, and develop relationships with vendors and experts to track advancements in quantum safe solutions

and quantum computing itself. The cycle feeds back into step 1 rather than ending.

In practice

The five step cycle and the Consolidate/Integrate/Accelerate framing describe the same migration from two angles: one is the organisational process, the other is how Palo Alto's own platform maps onto it. Step 3's inventory question, how long does this asset need to stay secure, is exactly the question the Quantum Readiness Dashboard from the earlier topics answers.

✓ Check yourself

the quantum threat, PQC and governance

1. What does QRE stand for, and what does it cover?

Quantum Resistant Encryption: Palo Alto Networks' umbrella term for the threat, the standards answering it, and Orion's tooling to deploy it.

2. What does QC stand for, and what makes a qubit different from a classical bit?

Quantum Computer. A qubit can exist in simultaneous states and link to other qubits through entanglement: a classical bit is always just 0 or 1.

3. Explain harvest now, decrypt later in one sentence.

Traffic captured today is stored and decrypted once the cryptography becomes breakable, so long lived confidentiality is the exposure, not current traffic volume.

4. Name the four numbers in NIST's migration timeline.

~1 year prep and planning, 2 to 5 years migration, ~8 years data needs to stay secure, inside an overall 5 to 15 year threat window.

5. What does CRQC stand for?

Cryptographically Relevant Quantum Computer: one actually capable of breaking today's encryption, currently estimated 5 to 15 years away.

6. What does PQC stand for, and what is it for?

Post Quantum Cryptography: encryption built specifically to resist quantum attacks.

7. What's the difference between what Shor's algorithm does and what Grover's algorithm does?

Shor's factors large primes and derives an asymmetric private key directly. Grover's only halves the effective strength of a symmetric key. It weakens, it doesn't break outright.

8. Name the algorithm and full spelled out name behind FIPS 203, 204 and 205.

203: CRYSTALS-Kyber → ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism). 204: CRYSTALS-Dilithium → ML-DSA (Module-Lattice-Based Digital Signature Algorithm). 205: SpHincS+ → SLH-DSA (Stateless Hash Based Digital Signature Algorithm).

9. Which FIPS standard exists purely as a backup, and why?

FIPS 205: built on different maths from FIPS 204, specifically in case ML-DSA is later found vulnerable.

10. What does RFC 8784 specifically defend against, and what does RFC 9242 actually enable?

RFC 8784's post quantum PSK defends specifically against Shor's algorithm. RFC 9242 defines the IKE_INTERMEDIATE exchange that RFC 9370's multiple key exchanges need to carry larger PQ key material.

11. How does a firewall catch someone using an unsanctioned PQC to hide a session?

It recognises the behaviour of PQC in the traffic itself, via Enhanced Decryption Policies and Threat Signatures, even from an unapproved implementation.

12. What does the Quantum Readiness Dashboard give you, and why does it matter?

A real time inventory of cryptographic assets, **compliance validation and inline remediation**. You cannot sequence a multi year migration without it.

13. What date is Y2Q framed as having started, and what does the Quantum Computing Cybersecurity Preparedness Act actually require?

13 August 2024. **Within a year of NIST's PQC standards**, OMB must issue guidance requiring every executive agency to produce a migration plan with interim benchmarks.

14. Name the three DoD Zero Trust pillars QRE is mapped against.

Consolidate, Integrate, Accelerate.

15. What are the five steps to becoming quantum ready, in order?

Assign resources and build awareness → define responsibilities → develop an inventory and priority list → evaluate, experiment and test → continue and monitor progress.

✓ Anchor facts

- ✓ **QRE** (Quantum Resistant Encryption) is Palo Alto's umbrella term for the threat, the standards, and Orion's tooling together.
- ✓ **Quantum computers (QCs)** use **qubits** (simultaneous states) and **entanglement**, not just faster classical bits.
- ✓ Migration clock: **~1yr prep, 2 to 5yr migrate, ~8yr data stays secure**, inside a 5 to 15yr threat window.
- ✓ **CRQC** = Cryptographically Relevant Quantum Computer, one actually capable of breaking today's encryption.
- ✓ The driver is **harvest now, decrypt later**: already happening, not a future risk.
- ✓ **PQC** = Post Quantum Cryptography, resisting quantum attacks specifically.
- ✓ **Shor's** derives the asymmetric key outright; **Grover's** only halves symmetric key strength.
- ✓ **FIPS 203** = ML-KEM/Kyber (encryption). **FIPS 204** = ML-DSA/Dilithium (signatures). **FIPS 205** = SLH-DSA/Sphincs+ (backup signatures, different maths).
- ✓ **RFC 8784** defends against Shor's. **RFC 9242** carries what **RFC 9370** needs to send.
- ✓ **Quantum Readiness Dashboard** = inventory, compliance, inline remediation. **Cipher translation** upgrades apps that cannot upgrade themselves.
- ✓ **Y2Q** started **13 August 2024**. The Preparedness Act requires OMB guidance and agency migration plans within a year of NIST's PQC standards.
- ✓ DoD Zero Trust mapping: **Consolidate, Integrate, Accelerate**.
- ✓ Five step readiness cycle: **assign → define → inventory → evaluate → monitor**, then repeat.



Easy to get wrong

- Studying the algorithm internals as the main goal. **Recognition and configuration awareness** is the level Orion's tooling operates at, the maths is background, not the test.
- Prioritising quantum safe migration by traffic volume rather than **confidentiality lifetime**.
- Treating "PQC" and "quantum safe" as vague synonyms rather than PQC being the specific, named standard set (FIPS 203/204/205 plus the RFCs).
- Assuming FIPS 205 is redundant with FIPS 204. It's a deliberate hedge on different maths, not duplication.
- Forgetting RFC 9242 exists at all. It's easy to only remember 8784 and 9370, but 9370 depends on 9242's exchange to work.
- Thinking cipher translation changes the application. It does not.
- Treating passwordless as a replacement for auth profiles and MFA rather than an addition.

Standard / RFC	Job	Underlying algorithm
FIPS 203 (ML-KEM)	General encryption	CRYSTALS-Kyber, small keys, fast
FIPS 204 (ML-DSA)	Primary digital signatures	CRYSTALS-Dilithium
FIPS 205 (SLH-DSA)	Backup digital signatures	Sphincs+, different maths from 204
RFC 8784	Defends against Shor's specifically	Post quantum PSK + IKEv2
RFC 9242 + 9370	Hybrid key exchange over IKEv2	IKE_INTERMEDIATE exchange + multiple key exchanges
Enhanced Decryption Policies	Catches unsanctioned PQC hiding traffic	Behavioural recognition, even unapproved implementations

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 1 • Platform and Deployment ⁴

PAN-OS 12.1 Orion • Passwordless Authentication

A password is a single, reusable, stealable piece of information. Passwordless removes it from the equation entirely rather than trying to protect it better, the threat, the architecture, and how to actually build and troubleshoot it.

13 Passwordless: the threat it closes, and the standards it's built on

Why this exists at all

Credential compromise is the **number one attack vector** in public breaches, and passwords account for **49 percent** of publicised cyber attacks in recent years. Strong authentication and MFA are still recommended, but the course is direct about their limit: **these measures are no longer sufficient on their own**.

The reason a password is uniquely weak: it's a **single, reusable, often weak piece of information**, not something tied to a physical device. Once stolen, it hands an attacker the literal keys to an account, and it's **easy to steal and exploit at scale** in a way a device bound credential isn't.

Four ways attackers bypass strong authentication anyway

- **Man in the Middle (MitM)**: intercepting the authentication exchange itself.
- **SIM swapping**: hijacking the phone number an SMS or call based factor relies on.
- **MFA fatigue**: bombarding a user with approval prompts until one is accepted by exhaustion or accident.
- **Stolen session cookies**: taking the already authenticated session token, sidestepping the login step entirely.

Three tangible outcomes, stated directly

every improvement in authentication reduces the available attack surface, because authentication is the **primary gatekeeper** for every protected resource behind it:

- **Improve security posture**: prevents commonly exploited identity based attacks: credential compromise, MFA bypass, MFA fatigue, SIM swapping.
- **Simplify the end user's experience**: one biometric scan (fingerprint, face, or retinal), with no MFA prompt needed on top.
- **Reduce the password management burden**: removes constant password resets and refreshes, which cost IT organisations **millions of dollars a year in OPEX**.

The standards it's actually built on

a passwordless future has to rely on rigorous, open industry standards, so devices, browsers and platforms all authenticate securely and consistently:

- **FIDO2** (Fast Identity Online 2), the modern open authentication standard, described directly as **secure, easy to use, and phishing resistant**. This is the standard the whole architecture, covered on the next topic, is primarily built on.
- **Passkey, CTAP, and WebAuthn**: the other named open standard components working alongside FIDO2.

Licensing

is acquired and enabled the same way as other PAN-OS NGFW options. Worth knowing as a matter of principle: the **standards based components of passwordless authentication remain open and accessible to the public**: the license covers Palo Alto's own implementation, not the underlying open standards themselves.

In practice

If asked why passwordless is worth the migration effort, the cleanest answer is the one sentence version: a password is information that can be stolen at scale, and every one of the four bypass methods above exploits that fact in a different way, passwordless removes the shared secret entirely rather than trying to protect it better.

14 Passwordless architecture: three layers, and three ways to deploy it

The architecture, primarily based on FIDO2, is built to replace a **vulnerable shared password** with **secure public key cryptography**. The core move is decentralising authentication: the secret, the **private key**, moves from a server's database to the **user's own device**, which is what makes the whole model **immune to server side data breaches and phishing**: there's no central password database to steal, and nothing to phish because there's nothing the user types.

Three layers, one workflow

- **User device layer**: where the private key actually lives, and never leaves.
- **Authentication server layer**: the firewall itself, holding **Kerberos Constrained Delegation (KCD)**.
- **Service application layer**: the app the user is actually trying to reach.

The five step flow, worth having in order

1 the user is authenticated to the firewall. 2 the user requests a service from the target app. 3 the firewall **impersonates the user** to retrieve a ticket for that app, against the Kerberos server, using KCD. 4 the firewall **inserts a SPNEGO token** into the HTTP header and sends the request on to the app. 5 the user gets the service back from the app, with no password ever entered for that application.

Three supported use cases

PAN-OS now provides comprehensive passwordless support across genuinely different scenarios:

- **IdP managed applications**: a central IdP (Okta, Microsoft Entra ID, Ping Identity) becomes the single trusted authority, verifying identity with a passkey or other passwordless method once, then granting access to every federated app behind it without reauthentication.
- **Kerberos applications**: the mechanism the Orion release actually introduces: users reach Kerberos protected network resources with no password entered at all, which is exactly what makes it highly resistant to phishing and credential theft. There's no password to steal in the first place.
- **Legacy and custom applications**: the use case worth remembering specifically for apps that **cannot be modified**: implementing a modern identity solution as a secure gateway in front of the app gives it a robust security shield **without touching existing code** at all. The legacy app itself never needs its own password or SSO capability, PAN-OS proxies access on its behalf.

In practice

The legacy app use case is the one worth having ready for a design conversation: it's the answer whenever a scenario describes an application that's too old or too fragile to modify but still needs modern, phishing resistant authentication in front of it.

15 Configuring and troubleshooting Kerberos passwordless: the profile, the CLI, and the logs

Configuring passwordless means two things together: enabling modern authentication standards inside the organisation's IdP, and guiding users through a **one time registration** of their secure authenticator. The goal throughout is shifting identity verification from something the user knows (a password) to something they have or are (biometrics, a physical key).

Step one: the Kerberos Server Profile

this gives the NGFW the **IP address of the Kerberos server**, and the firewall's policy uses it when requesting **constrained delegation on the user's behalf**: the exact mechanism from the previous topic's five step flow. The profile holds a name, an **Administrator Use Only** option, and a servers table (name, Kerberos server address, port, port **88** is the Kerberos standard).

CLI troubleshooting starts with authd

a core OS service that acts as the **central broker for every user authentication request** and exposes runtime counters from the moment it starts. Troubleshooting begins by retrieving its statistics.

MP commands

the management plane is responsible for **learning user identities** after they authenticate passwordlessly, then **distributing that information to the data plane**. The commands worth knowing by name: **show session ID**, **check the connection to identity sources**, and **show authentication delegation**.

DP commands

run on the data plane to verify the firewall actually **learned the user's identity and correctly mapped it to their IP address**:

- **User to IP address mappings:** `show user ip-user-mapping all`, displaying the table mapping users to their IP addresses.
- **View live traffic sessions.**
- **Check user group mappings.**
- **Debugging on the data plane.**

Why logs matter more here than elsewhere

passwordless logs are described as **indispensable**: they give a definitive, step by step record of authentication events, clearly identify the point of failure, and precisely state the reason, turning an **ambiguous problem into a specific, solvable one**, rather than leaving the administrator guessing which of the five workflow steps actually failed.

Three log sources, each answering a different question

- **System logs:** the definitive chronological record of every event on the box; crucial for troubleshooting failures, investigating security incidents, and auditing activity generally.
- **Authd.log, even when tickets are cached:** authd logs **every local authorisation and session event**, not only network authentication, so it still shows output when a Kerberos ticket is simply being served from cache rather than freshly requested.
- **Authd.log generally:** its main value is a **low level, OS focused perspective**, revealing hardware interactions and local policy failures that are **invisible to browser or server side logs** entirely.

In practice

When a passwordless failure doesn't show up anywhere a browser or the target application logs, that's exactly when authd.log is worth checking. It sees local, OS level events the rest of the stack simply has no visibility into.

✓ Check yourself

passwordless authentication

1. What percentage of publicised cyber attacks do passwords account for?

49 percent, with credential compromise the number one attack vector.

2. Name all four ways attackers bypass strong authentication, covered here.

Man in the Middle, SIM swapping, MFA fatigue, and stolen session cookies.

3. Which open standard is the architecture primarily built on?

FIDO2: secure, easy to use, and phishing resistant.

4. Where does the private key live, and why does that make the model phishing resistant?

On the user's own device, and it never leaves: there's nothing typed, so nothing to phish.

5. Walk through the five step flow for a Kerberos protected app.

User authenticates to the firewall → requests the app → firewall impersonates the user via KCD to get a ticket → firewall inserts a SPNEGO token into the HTTP header → user gets the response.

6. Which use case is the answer for an application that cannot be modified?

Legacy and custom applications: PAN-OS proxies access as a secure gateway in front of it, no code changes needed.

7. What does the Kerberos Server Profile actually give the firewall?

The IP address of the Kerberos server, used when requesting constrained delegation on the user's behalf.

8. Which command shows the user to IP address mapping table?

`show user ip-user-mapping all`, run on the data plane.

9. Why does authd.log still show output even when a Kerberos ticket is served from cache?

Authd logs every local authorisation and session event, not only network authentication.

✓ Anchor facts

- ✓ Passwords: 49% of publicised attacks; credential compromise is attack vector #1.
- ✓ Four bypass methods: **MiTM, SIM swapping, MFA fatigue, stolen session cookies**.
- ✓ Architecture is built on **FIDO2**; the private key never leaves the **user's device**.
- ✓ Five step flow ends with the firewall inserting a **SPNEGO token** via **KCD**.
- ✓ Three use cases: **IdP managed, Kerberos apps, legacy/custom apps** (no code changes needed).
- ✓ Kerberos Server Profile carries the server IP; standard port is **88**.
- ✓ `show user ip-user-mapping all` is the DP command to know.



Easy to get wrong

- Assuming MFA alone is sufficient, the course is explicit that it no longer is on its own.
- Thinking a legacy app needs code changes to gain passwordless protection. It doesn't.
- Confusing s4u2self (ticket for the KCD agent itself) with s4u2proxy (ticket for the target app).
- Overlooking authd.log in favour of only checking browser or server side logs.
- Forgetting that authd.log logs local events too, not just network authentication.

Term	Definition
Data plane (DP)	High speed engine that processes all network traffic through the firewall
Management plane (MP)	The interface administrators use to configure, monitor and manage the device
Kerberos Constrained Delegation (KCD)	Lets a service impersonate a user to access specific authorised backend services on their behalf, limiting which services an app can reach while acting as the user
KCDReq	A request to authd to create a SPNEGO token to access a Kerberos protected application
s4u2self	Kerberos protocol to retrieve the ticket for the KCD agent itself, on behalf of the user
s4u2proxy	Kerberos protocol to retrieve the ticket for the target Kerberos application, on behalf of the user
pan_com	The PAN-OS communication infrastructure the MP uses to send data to the DP
SPNEGO token	A message packet carrying authentication data, letting client and server securely negotiate the auth protocol with no manual credential entry

• NETWORKING
• DEVICE SETTINGS
• INTEGRATION
• CONTEXT

KNOW
CONFIGURE
VERIFY

Part 1 • Platform and Deployment ⁵

PAN-OS 12.1 Orion • Scale, Fabrics and Lifecycle

The rest of Orion: what got bigger, the two fabrics, and the lifecycle changes that matter most if you run firewalls at scale.

16 Scale and performance changes

Orion is positioned as **optimised for cloud**, scaling policy and identity enforcement for large, dynamic deployments.

Up to 10 million user contexts, described as a **20x increase** over previous generations. That is the headline identity scale figure, and it exists so User-ID and user based policy remain usable in the largest environments.

Enhanced session handling: session scale improvements for better performance and resilience under heavy load.

High speed payload analysis pipelines between the data planes and cloud analysis engines, using a real time payload exchange. Beyond raw speed, this enables **cross session correlation**: detecting activity that hides across multiple sessions, which is how advanced persistent threats and staged data exfiltration evade per session inspection.

Also in the release: **decryption enhancements** for performance and coverage of encrypted traffic inspection, and **expanded Device-ID capabilities**.

In practice

Cross session correlation is the conceptually interesting one. Everything earlier in this guide inspects a session; this looks across sessions, which is a different detection model and the answer to threats deliberately split up to stay under per session thresholds.

17 The two Orion fabrics

Hyperscale Security Fabric (HSF) brings **clustering for VM-Series**, so multiple software firewalls act as one logical high capacity system rather than as individually sized instances. This is the answer to throughput requirements that exceed a single VM.

Multicloud Security Fabric addresses the operational problem of security spread across several clouds. **Software Firewall, Cloud Firewall and Prisma AIRS instances are deployed automatically**, cloud infrastructure is configured to **steer traffic**, and a **secure multi cloud networking mesh** is established. Scaling and load balancing are **native**, removing the need for separate load balancers.

Alongside it, **Cloud and AI Risk Assessment** continuously evaluates every cloud and AI asset for risks from weak or missing protection, highlighting where controls are needed.

Both fabrics reappear in the cloud and orchestration part, because that is where you deploy them. Here the point is **what problem each solves**: HSF is about **capacity**, Multicloud Security Fabric is about **consistency and reach across clouds**.

In practice

Keep the two apart in your head. If the requirement is "more throughput than one VM-Series can deliver", that is **HSF**. If it is "consistent policy and traffic steering across AWS, Azure and GCP without hand built plumbing", that is the **Multicloud Security Fabric**.

Orion at a glance five themes, and which part of this guide each connects to

Quantum safe

readiness dashboard
cipher translation
PQ ready VPN
→ Part 10 Tunnels

Identity scale

10M user contexts
20x increase
passwordless / KCD
→ Part 9 Identity

Inspection

decryption enhancements
payload pipelines
cross session correlation
→ Part 8 Decryption

Fabrics

HSF, VM-Series cluster
Multicloud Security Fabric
native scaling and LB
→ Part 17 Cloud

Lifecycle

hot / warm / cold patches
HA upgrade orchestration
health checks
→ Parts 15, 16

Hyperscale Security Fabric = capacity

more throughput than one VM-Series can deliver

Multicloud Security Fabric = consistency

one policy and traffic steering across several clouds

Patch deployment changes the upgrade conversation

a **hot patch** addresses a critical vulnerability without the disruption of a full image upgrade

18 Lifecycle and operational changes

Orion's operational theme is removing the manual work of running a large estate, and it lands mostly through Panorama.

PAN-OS patch deployment

Instead of a full image upgrade for every fix, you can deploy lightweight **hot, warm or cold patches**. These target critical vulnerabilities with **minimal disruption**, which changes the calculus of responding to a CVE, you no longer need a full upgrade window for a single fix.

HA upgrade orchestration

Historically upgrading an HA pair meant a manual sequence: upgrade the passive unit, fail over, watch for stability, then upgrade the former active. Panorama now **automates that orchestration** from the UI.

Also: **intelligent health checks** before and after deployment, **plugin dependency identification** for upgrades, and **optimised UI performance** for large scale deployments.

The three pillars stated for the release are **frictionless upgrades**, **operational simplification** and **optimised user experience**.

In practice

If you run a large estate, patch deployment and HA orchestration are the features that change your week. The HA upgrade sequence is still worth knowing manually, Part 16 covers it, because you need to understand what the orchestration is doing before you trust it.

19 Modernizing PAN-OS: a decoupled commit process, and telemetry down to the process level

This is the architecture underneath everything else Orion adds: engineered for **greater agility and scale**, accommodating both traditional on box management and modern **API driven automation**, so operational consistency holds whether you're managing one appliance or a global deployment.

Why it matters for Strata Cloud Manager specifically

the modernised architecture is **foundational to deep SCM integration**: it streams richer, **near real time telemetry** for AIOps, and provides the hooks for **faster, more granular policy pushes** from the cloud. Together, that's what enables genuine **zero touch provisioning** and consistent, AI driven policy management across the entire enterprise security fabric.

The commit process, rearchitected

the change is a genuine structural one, worth holding as a before/after:

- **Legacy commit**: the entire configuration was parsed, validated, and applied in **one single sequential process**. The administrator waited for all of it to finish, and a failure at any stage could mean a **full rollback**.
- **Modernised commit**: **validation is separated from application**. The configuration is validated first, for syntactical and logical correctness; only then are the changes applied to the management plane and dataplane processes, **often in parallel**, which is what actually cuts the total time down.
- **The operational benefit, stated directly**: commit duration drops from **minutes to seconds** for many common changes. That lets administrators push security policy with real agility and confidence, **even during production hours**, shrinking how much change work has to be crammed into a dedicated maintenance window.

New visibility down to the individual process

effective troubleshooting needs deep visibility into what's actually running, and the modernised architecture adds a genuine new suite of diagnostic tooling for exactly that: **enhanced CLI commands and API endpoints** exposing far more granular data points than before, giving both administrators and support engineers real insight into system health and traffic processing logic rather than a black box.

Per process telemetry, specifically

every major software process inside the modernised PAN-OS now exposes its own detailed telemetry, **CPU and memory consumption, thread counts**, and other key performance indicators, reachable through the same new CLI commands and API calls. The practical payoff: a process experiencing genuine **resource contention** can be identified **precisely**, rather than inferred from the whole system's aggregate load.

In practice

The decoupled commit process is the detail worth having ready if asked what changed architecturally in Orion: validation happening before, and separately from, application to the management and data planes is exactly what turns a multi minute commit into a multi second one.

20 Putting Orion in context

Orion is positioned as a platform shift rather than a point release, with the stated theme of securing the **quantum, multicloud and AI era**, which maps neatly onto the three things this guide covers that older material does not: **post quantum crypto, the cloud fabrics, and AI security**.

For study purposes, the useful framing is which Orion feature connects to which part of the syllabus:

Quantum → tunnels and IPsec configuration.

Passwordless and 10M user contexts → authentication and identity.

Decryption enhancements → certificates and decryption.

HSF and Multicloud Security Fabric → cloud and orchestration.

Patches and HA orchestration → Panorama and software updates.

Worth remembering that **Orion appears twice in the Software Firewalls course**, once as the AI Hyperscale Security Fabric and once as the Multi Cloud Security Fabric, so the fabrics get their configuration detail there rather than here.

In practice

Treat Orion as a lens rather than a chapter. Almost every feature is an enhancement to something covered elsewhere in this guide, so learning it as "what changed in the thing I already know" sticks better than learning it as a feature list.

✓ Check yourself

Orion scale, fabrics and lifecycle

1. What is the headline identity scale figure, and why does it exist?

10 million user contexts, a 20x increase, so User-ID and user based policy stay usable in the largest environments.

2. What does cross session correlation detect that per session inspection cannot?

Activity deliberately split across sessions: APTs and staged data exfiltration hiding under per session thresholds.

3. Distinguish the two Orion fabrics in one line each.

HSF = capacity, clustering VM-Series into one logical system. Multicloud Security Fabric = consistency and traffic steering across clouds.

4. What changes about responding to a critical CVE?

Hot, warm or cold patches instead of a full image upgrade, minimal disruption, no full upgrade window.

5. What did HA upgrade orchestration replace?

The manual sequence: upgrade passive, fail over, monitor, upgrade the former active. Panorama now automates it.

6. Which three eras does Orion position itself against?

Quantum, multicloud and AI, which maps to tunnels, the cloud fabrics, and Prisma AIRS.

✓ Anchor facts

- ✓ **10 million user contexts**, a 20x increase.
- ✓ **Cross session correlation** catches threats split across sessions.
- ✓ **HSF** = VM-Series clustering for capacity.
- ✓ **Multicloud Security Fabric** = auto deploy, traffic steering, native scaling and LB.
- ✓ Patches come as **hot, warm or cold**.
- ✓ Panorama automates **HA pair upgrade orchestration**.



Easy to get wrong

- Confusing the two fabrics, one is capacity, one is reach.
- Assuming every fix now needs a full image upgrade.
- Trusting HA orchestration without knowing the manual sequence.
- Treating Orion as a separate subject rather than enhancements to existing features.

Orion feature	What it changes	Covered in detail in
Quantum readiness, cipher translation	Crypto inventory and migration	Part 10 · VPN and Tunnels
Passwordless / KCD, 10M user contexts	Identity scale and phishing resistance	Part 9 · Authentication and Identity
Decryption enhancements	Encrypted inspection performance	Part 8 · Certificates and Decryption
HSF, Multicloud Security Fabric	Capacity and multi cloud consistency	Part 17 · Cloud and Orchestration
Patches, HA orchestration	Upgrade and lifecycle work	Parts 15 and 16

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

The Platform and the Two Intercepts

AI Runtime Security is a fifth deployment option alongside the four covered so far, and Prisma AIRS is the platform it belongs to. The shape to learn: what it protects, and the two ways it intercepts.

21 What Prisma AIRS is, and what it protects

Prisma AIRS, **AI Runtime Security**: is a **purpose built, centralised AI security platform** covering the **entire AI ecosystem**. The four things it protects are worth memorising as a set, because the whole platform is organised around them:

AI applications · AI models · AI data · AI agents

It is described as securing the **entire AI attack lifecycle**, across **three layers: in the code, at runtime, and in the cloud**. That framing explains why the platform has both a development time component and an inline one.

The five capabilities

AI Runtime Firewall: inline network protection for AI traffic (Network Intercept).

AI Runtime API: API level protection embedded in application code (API Intercept).

AI Model Security: protects model artifacts and training pipelines.

AI Red Teaming: automated adversarial testing.

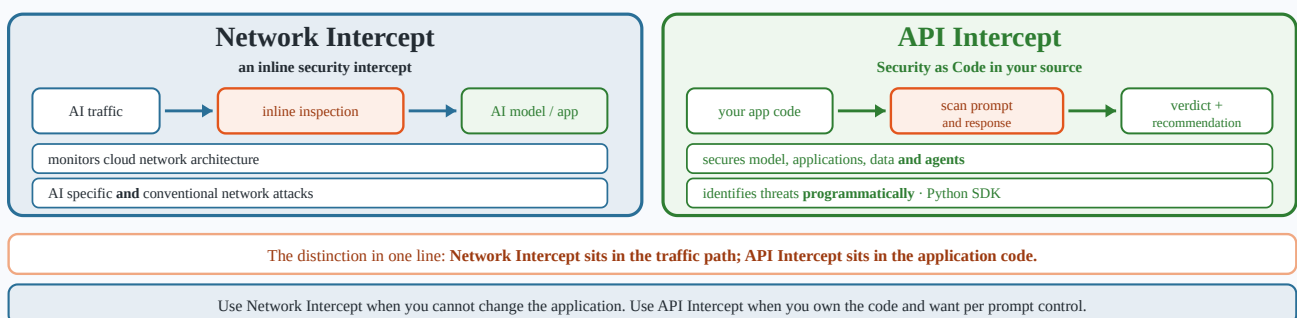
Posture Management: continuous compliance and risk monitoring.

It also secures the surrounding workloads: **AI and traditional applications on VMs, containers or cloud native platforms, Kubernetes clusters**, and **all network traffic** in the cloud environment, inbound, outbound and east west.

In practice

Hold onto the four protected things and the five capabilities. Almost every question about AIRS resolves to "which of those four is at risk, and which of the five capabilities addresses it".

The two intercepts the single most important distinction on this page



22 Network Intercept versus API Intercept

Prisma AIRS AI Runtime: Network Intercept is an **inline security intercept** providing real time, AI powered network protection. It monitors your **cloud network architecture** for **AI specific and conventional network attacks**, so it covers both the new threat class and the ordinary one.

Prisma AIRS AI Runtime: API Intercept secures your AI model, applications, data and agents by embedding **Security as Code directly into your source code**. The APIs **scan prompts and model responses** to identify threats **programmatically** and return **actionable recommendations**. There is an official **Python SDK** for integration.

Choosing between them

Network Intercept when you cannot modify the application, a third party AI tool, or traffic you only see on the wire.

API Intercept when you own the code and need **per prompt, per response decisions** inside the application flow.

They are complementary, not alternatives.

The same architecture point as the firewall itself applies: Network Intercept is **inline and transparent to the app**; API Intercept requires a code change but gets far richer context about what the application is actually doing.

In practice

This is the AIRS equivalent of the tap versus inline decision. Network Intercept needs no application change and sees traffic; API Intercept needs a code change and sees intent.

23 The AI threat categories

The named threat classes are the most examinable detail, because they define what the platform actually stops:

Prompt injection: malicious prompt manipulation that subverts model intent or leaks information. The classic example is a jailbreak instructing the model to adopt an unrestricted persona.

Malicious code: code generated or carried through the model.

Malicious URL: links in prompts or responses.

Sensitive data loss, with a **mask sensitive data** option as an alternative to blocking.

Database security attack: the model being used to reach a backing store.

Toxic content: filtered across several categories.

Resource overload: the AI equivalent of a denial of service.

Hallucination attacks and **contextual grounding**: checking a response is grounded in its source material.

AI agent threats: covered on the next page.

Custom topic guardrails: your own subject boundaries.

Secure MCP: protection for Model Context Protocol exposure.

In practice

Group them to remember them: **input threats** (prompt injection, malicious URL), **output threats** (sensitive data loss, toxic content, hallucination), **availability** (resource overload), and **agent and tooling threats** (agent threats, MCP).

1. What does AIRS stand for?

AI Runtime Security.

2. Name the four things Prisma AIRS protects.

AI applications, AI models, AI data, AI agents. **The whole platform is organised around this set.**

3. Name the five capabilities.

AI Runtime Firewall, AI Runtime API, AI Model Security, AI Red Teaming, Posture Management.

4. Distinguish the two intercepts in one line.

Network Intercept sits in the traffic path; API Intercept sits in the application code.

5. You cannot modify a third party AI tool. Which intercept?

Network Intercept: **inline, no application change needed.**

6. What does contextual grounding check?

That a response is grounded in its source material: the defence against hallucination attacks.

7. Which three layers does AIRS secure across?

In the code, at runtime, and in the cloud.

✓ Anchor facts

- ✓ **AIRS = AI Runtime Security.**
- ✓ Protects **AI applications, models, data and agents.**
- ✓ Five capabilities: **Runtime Firewall, Runtime API, Model Security, Red Teaming, Posture Management.**
- ✓ **Network Intercept** = inline. **API Intercept** = Security as Code in your source.
- ✓ API Intercept **scans prompts and responses** and returns recommendations. Python SDK available.
- ✓ Covers **AI specific and conventional** network attacks.
- ✓ Three layers: **code, runtime, cloud.**



Easy to get wrong

- Treating the two intercepts as alternatives. They are **complementary**.
- Assuming AIRS only handles AI specific threats. It covers conventional ones too.
- Forgetting **agents** are a protected category in their own right.
- Thinking API Intercept is a proxy. It is **embedded in your code**.

	Network Intercept	API Intercept
Sits	In the traffic path	In the application code
Requires a code change?	No	Yes
Sees	Network traffic	Prompts and responses
Covers	AI specific and conventional attacks	Model, apps, data, agents
Use when	You cannot modify the app	You own the code

Model Security, Agents, Red Teaming and Posture

The remaining capabilities, plus the Discover, Protect, Defend arc the course is built around. Agent security is the newest material and the part most worth attention.

24 AI Model Security

Protects models **throughout their lifecycle, from development to deployment**, so this is the "in the code" layer rather than the runtime one.

What it does: **comprehensive model scanning, detecting vulnerabilities, preventing model poisoning, and ensuring integrity from training through deployment**. It also covers **policy enforcement** and **compliance** for models.

Ongoing controls include **continuous monitoring of model behaviour, automated vulnerability scanning, and policy based access controls** over model assets.

The on premises scanning option

Models can be scanned **entirely on premises**, so **sensitive weights and training data never leave your control**. That matters for intellectual property and for regulated environments where model artifacts cannot be sent to a cloud service.

The threat this addresses is distinct from runtime: a **poisoned or tampered model** is compromised before it ever serves a prompt, so no amount of prompt inspection would catch it.

In practice

The distinction to hold: **Model Security protects the artifact; Runtime Security protects the conversation**. A poisoned model passes every runtime check because the malicious behaviour is the model, not the traffic.

25 AI agent security

Agents differ from chat applications because they **take actions and use tools**, which gives them a genuinely new attack surface. AIRS secures agents **including those built on no code and low code platforms**, where the builder may not be a developer at all.

The three named agentic threats

Identity impersonation: an agent acting as someone it is not.

Memory manipulation: tampering with the agent's retained context so it behaves differently later.

Tool misuse: the agent's legitimate tools being turned to unintended ends.

Learn those three as a set. They map to the three things an agent has that a chatbot does not: **an identity, persistent memory, and tools**.

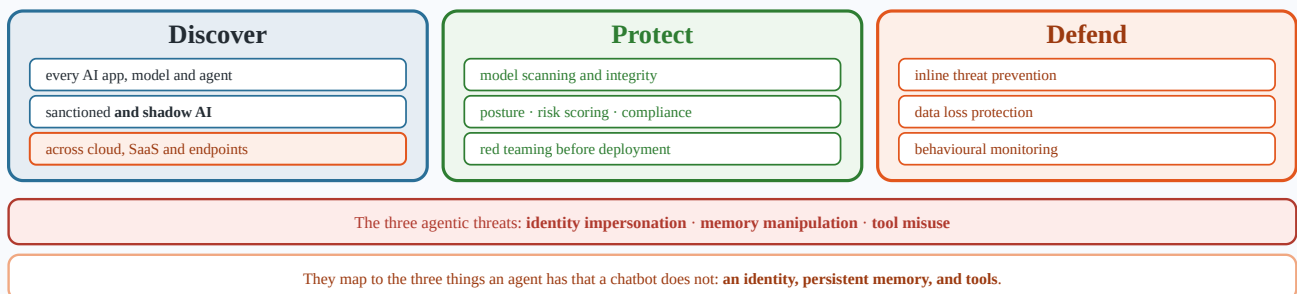
The governance controls follow from that: **agent discovery** across cloud, SaaS and endpoints, **agent identities with role based access control and audit trails**, and **secure MCP** for the Model Context Protocol through which agents reach tools.

The scale argument for central governance: hundreds of agents deployed without it becomes ungovernable, because nobody knows what exists or what it can reach.

In practice

Memory manipulation is the one that is easiest to underestimate. It is a **persistence** attack, poison the memory once and the agent misbehaves on future interactions, with nothing malicious in the later traffic to detect.

Discover, Protect, Defend the arc the course follows, and where each capability lands



26 AI Red Teaming

Automated adversarial testing of your AI applications and models, the platform runs **red team simulations** against them rather than waiting for a real attacker.

Positioned as a **comprehensive security assessment**, it belongs in the **Protect** phase: you test before and during deployment, so weaknesses are found by you rather than in production.

The value is coverage and repeatability. Manual prompt injection testing finds a handful of techniques; automated red teaming exercises a large library of them consistently, and can be rerun after every model or prompt change.

This is the AI equivalent of vulnerability scanning, and it pairs with **Posture Management** the same way scanning pairs with configuration compliance, one tests behaviour, the other assesses configuration and risk.

In practice

The operational argument is regression. Every change to a system prompt, a model version or a tool definition can reintroduce a weakness, so testing has to be automated and repeatable rather than a one off exercise before launch.

27 Posture management and discovery

AI Posture Management continuously evaluates security posture, vulnerabilities and compliance across all AI assets, with **automated risk scoring**. It is the ongoing assessment layer rather than an inline control.

Discovery is its necessary partner: **complete visibility into every AI application, model and agent** across the organisation, **whether sanctioned or shadow AI**. You cannot assess or protect assets you do not know exist, and shadow AI is the realistic case, teams adopt AI tools without telling security.

Discovery spans **cloud, SaaS and endpoints**, which is what makes it capable of finding unsanctioned usage rather than only the deployments you built deliberately.

How the pieces fit

Discovery finds the assets → **Posture Management** scores their risk and compliance → **Model Security** and **Red Teaming** harden them → **Runtime Security** defends them in production. Each stage depends on the one before it.

Deployment is primarily **SaaS via the Palo Alto Cloud Platform**, and it integrates with the wider Prisma platform including **CNAPP**.

In practice

Shadow AI is the phrase that makes discovery meaningful. The assets most likely to be unprotected are precisely the ones security does not know about, which is why discovery comes first in the arc rather than being an afterthought.

✓ Check yourself

model security, agents, posture

1. Why would runtime inspection not catch a poisoned model?

The **malicious behaviour** is the model, **not the traffic**. Model Security protects the artifact; Runtime protects the conversation.

2. Why can models be scanned on premises?

So sensitive weights and training data never leave your control: IP protection and regulated environments.

3. Name the three agentic threats and what they map to.

Identity impersonation, memory manipulation, tool misuse: mapping to an agent's identity, memory and tools.

4. Why is memory manipulation easy to underestimate?

It is a persistence attack, poison once, and later interactions misbehave with nothing malicious in that traffic.

5. What makes red teaming an ongoing need rather than a launch task?

Regression: every change to a prompt, model version or tool can reintroduce a weakness.

6. Why does discovery come first in the arc?

You cannot assess or protect what you don't know exists, and shadow AI is exactly what security hasn't been told about.

✓ Anchor facts

- ✓ **Model Security** = lifecycle, scanning, poisoning prevention, integrity. On prem scanning available.
- ✓ Agentic threats: **identity impersonation, memory manipulation, tool misuse**.
- ✓ Agent governance: **discovery, agent identities with RBAC and audit, secure MCP**.
- ✓ **Red Teaming** = automated adversarial simulation, in the Protect phase.
- ✓ **Posture Management** = continuous risk, vulnerability and compliance scoring.
- ✓ **Discovery** covers **sanctioned and shadow AI** across cloud, SaaS and endpoints.



Easy to get wrong

- Expecting runtime controls to catch a **poisoned model**.
- Treating agents as chat applications. They have **identity, memory and tools**.
- Running red teaming once at launch instead of after every change.
- Assuming discovery only finds deployments you built deliberately.
- Confusing Posture Management (assessment) with Runtime Security (inline enforcement).

Capability	Phase	Protects	Inline or assessment?
Discovery	Discover	Visibility, incl. shadow AI	Assessment
AI Model Security	Protect	The model artifact and pipeline	Assessment / scanning
AI Red Teaming	Protect	Behaviour under attack	Testing
Posture Management	Protect	Configuration and compliance	Assessment
Runtime (Network / API)	Defend	Live prompts, responses, traffic	Inline

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 3 · Interfaces and Zones 8

Interface Modes

Interfaces and zones are the foundation, and every device setting is configured on top of an interface that already works. Learn what each mode can and cannot do before anything else.

28 The interface modes and what each can do

Tap: receives a **copy** of traffic from a SPAN or mirror port. Full App-ID, User-ID and threat visibility, but **cannot block anything** because it is not in the path. Used for evaluation and for baselining before going inline.

Virtual Wire: two interfaces **bound together** so traffic passes between them transparently. **No IP, no MAC, no routing, no ARP participation.** Invisible to the surrounding network, so you can insert the firewall without readdressing anything. It **can** enforce policy, NAT and inspection.

Layer 2: interfaces are assigned to a **VLAN object** and the firewall switches between them while inspecting. Add a **VLAN interface** to provide routing into that broadcast domain.

Layer 3: interfaces have **IP addresses** and the firewall routes. The most common production mode. Requires assignment to a **virtual router or logical router**.

Tunnel: a logical interface with no physical port, used as the endpoint for **IPSec and GRE**. Assigned to a zone and a router like any other interface.

HA: dedicated interfaces for HA1 control and HA2 state sync.

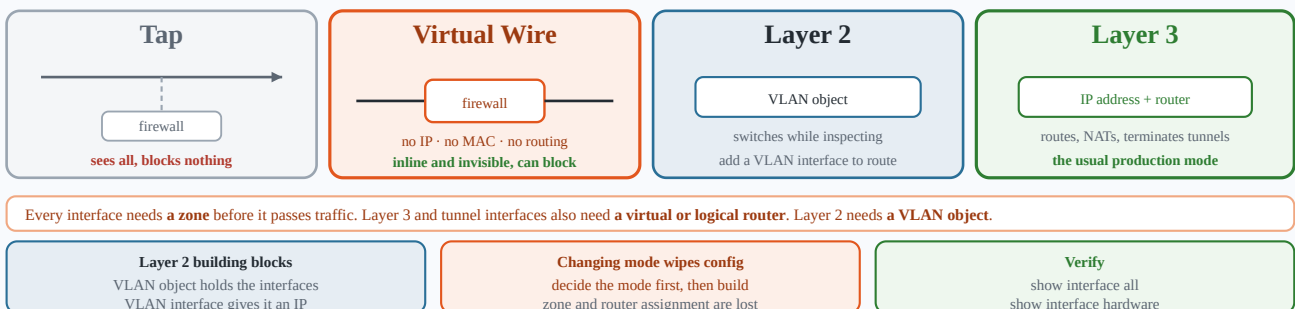
Log Card and **Decrypt Mirror:** specialised roles on supporting platforms.

Interfaces are configured at `Network > Interfaces > Ethernet`. Verify with `show interface all`, `show interface ethernet1/1`, and `show interface hardware` for physical state.

In practice

Changing an interface mode **clears its configuration**, so decide the mode before building on it. If a requirement says "insert a firewall without changing the IP design", the answer is **virtual wire**. If it says "inspect without any risk of blocking", it's **tap**.

Choosing a mode how much of the network design may change, and do you need to block?



29 Layer 2 mode, VLAN objects and VLAN interfaces

In Layer 2 mode the firewall behaves like a switch that inspects. The **VLAN object** at `Network > VLANs` is the container: it lists the Layer 2 interfaces in that broadcast domain and optionally references a **VLAN interface**.

The **VLAN interface** is a logical Layer 3 interface (named `vlan.1` and so on) that gives the VLAN an IP address so the firewall can route into and out of it. It is assigned to a **zone** and a **router** like any other Layer 3 interface.

The firewall maintains a **MAC address table** and an **ARP table** in this mode. Verify with `show mac all`, `show arp all`, and `show vlan all`.

Traffic **within** the VLAN is switched, and because those interfaces are usually in the same zone it hits the **intrazone-default allow** rule. Traffic **leaving** the VLAN goes via the VLAN interface and is evaluated as normal interzone traffic.

In practice

Layer 2 mode without a VLAN interface gives you inspection but no gateway, hosts still point at their existing router. Adding the VLAN interface is what lets the firewall become the gateway.

30 Layer 3 addressing and link settings

A Layer 3 interface takes an IP address **statically**, by **DHCP client**, or by **PPPoE**, and supports **multiple addresses** on one interface. IPv6 is configured alongside IPv4 on the same interface.

Required assignments: a **security zone**, a **virtual or logical router**, and optionally a **Virtual System** if multi-vsys is enabled.

Under the **Advanced** tab: **link speed**, **duplex** and **link state** (auto or forced), **MTU**, **adjust TCP MSS** with an IPv4 and IPv6 adjustment value, **untagged subinterface**, an **Interface Management Profile**, and **ARP entries** or **ND entries**.

Adjust TCP MSS matters whenever traffic crosses a tunnel. It clamps the MSS so packets do not need fragmenting later. **LLDP** is configured per interface via an LLDP profile.

Verify with `show interface ethernet1/1`, `show arp all`, and `show lldp neighbors all`.

In practice

Forcing speed and duplex on one side while the other autonegotiates causes duplex mismatch, which shows as throughput far below the link rate with rising errors, not as a down interface.

✓ Check yourself

interface modes

1. Which interface mode gives full visibility but cannot block traffic?

Tap. It receives a mirrored copy and is not in the forwarding path.

2. Name the four things virtual wire does *not* have.

No IP address, MAC address, routing participation or ARP. It is transparent to the surrounding network.

3. What two objects does Layer 2 mode need before it can route out of the VLAN?

A VLAN object to hold the interfaces, and a VLAN interface (`vlan.1`) to give it an IP, zone and router.

4. What happens to an interface's configuration when you change its mode?

It is cleared: zone and router assignment are lost. Decide the mode before building on the interface.

5. Which setting clamps TCP segment size so packets do not need fragmenting across a tunnel?

Adjust TCP MSS, on the interface's Advanced tab, with separate IPv4 and IPv6 adjustment values.

6. Which two commands would you use to confirm physical versus logical interface state?

show interface hardware for physical state, show interface all or show interface ethernet1/1 for logical.

✓ Anchor facts

- ✓ **Tap sees but cannot block.** Virtual wire is inline and invisible.
- ✓ Virtual wire has **no IP, no MAC, no routing, no ARP.**
- ✓ Layer 2 needs a **VLAN object**; add a **VLAN interface** to route.
- ✓ Layer 3 and tunnel interfaces need a **virtual or logical router.**
- ✓ Every interface needs a **zone** before it passes traffic.
- ✓ Layer 3 addressing: **static, DHCP client or PPPoE**, multiple addresses allowed.



Easy to get wrong

- Expecting a tap interface to block. It is out of path.
- Building on an interface then changing its mode, **config is wiped.**
- Layer 2 without a VLAN interface: inspection but **no gateway.**
- Forcing speed/duplex on one side only → mismatch, not a down link.
- Forgetting **MSS adjustment** on interfaces feeding tunnels.

Mode	IP address?	Routes?	Can block?	Needs
Tap	No	No	No	Zone; a mirror source
Virtual Wire	No	No	Yes	Zone; a virtual wire object
Layer 2	Via VLAN interface	Via VLAN interface	Yes	Zone; VLAN object
Layer 3	Yes	Yes	Yes	Zone; virtual/logical router
Tunnel	Optional	Yes	Yes	Zone; router; IPSec or GRE config

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 3 • Interfaces and Zones 9

Aggregate Ethernet, Subinterfaces, Loopbacks and Management

Aggregate Ethernet is frequently paired with virtual wire in scenarios about inserting a firewall without redesigning the network.

31) Aggregate Ethernet and LACP

An **Aggregate Ethernet (AE)** group bundles physical interfaces into one logical interface named `ae1`, giving both more bandwidth and link redundancy.

Order of configuration

You must **create the AE interface first**, set its **interface type** (Layer 3, Layer 2, virtual wire) and its **zone and router**, and only then assign physical interfaces as members. PAN-OS will not let you point a member at an AE group that does not exist.

The **zone and IP address go on the AE interface**, never on the physical members. Members contribute only capacity.

Members must match

Interface **speed and duplex**, and they should be the same media type. Mismatched members will not join the bundle.

LACP

Optional, without it the bundle is static. When enabled you set:

Mode: active (initiates) or passive (responds only).

Transmission rate: fast (1 second) or slow (30 seconds).

Max ports and **system priority** (lower wins the election).

Fast failover for subsecond recovery.

LACP in an HA pair

Pre negotiation lets the **passive** firewall negotiate LACP in advance so the bundle is already up at the moment of failover, cutting recovery time significantly. Available in active/passive HA.

Verify with `show lacp aggregate-ethernet all`, `show interface ae1`, and `show lacp counters`.

In practice

Two rules save most AE faults: **build the AE interface before the members**, and **put the zone on the bundle**. If one side runs LACP passive and the other is static "on", the bundle never forms, at least one side must actively negotiate.

Aggregate Ethernet: what goes where

and why pre negotiation matters in an HA pair

Configure in this order

- 1 · create **ae1** and set its interface type
 - 2 · assign **zone, IP and router** to ae1
 - 3 · add physical members, matching speed and duplex
- members carry **no zone and no IP** of their own

LACP settings

Mode · active initiates, passive only responds

Rate · fast = 1s, slow = 30s

System priority · lower wins the election

Max ports and fast failover

passive + static "on" never forms, one side must negotiate

LACP pre negotiation in active/passive HA

the **passive** firewall negotiates LACP in advance, so the bundle is already up when failover happens
without it, the new active firewall must negotiate from scratch and failover takes seconds longer

32 Subinterfaces and loopbacks

Subinterfaces carry **802.1Q tags** and are named `ethernet1/1.100` or `ae1.100`. The suffix is an arbitrary identifier and the **VLAN tag is configured separately**: they do not have to match, though keeping them identical is good discipline.

Available on **Layer 3, Layer 2 and virtual wire** interfaces, and on AE groups. Each subinterface takes its own **zone, router and IP**, which is how one physical link serves several segments with different policy.

An **untagged subinterface** option lets the parent handle untagged frames while subinterfaces handle tagged ones.

Loopback interfaces are logical and always up. Used for the **GlobalProtect gateway address**, router IDs, DNS proxy, and as a stable management target.

Tunnel interfaces are the same idea for VPN endpoints, no physical port, assigned to a zone and router.

In practice

Subinterfaces on a **virtual wire** are how you apply different policy to different VLANs crossing the same cable, which is a common transparent insertion requirement.

33 The management interface and Interface Management Profiles

The **MGT interface** is out of band. It has its **own IP, netmask and default gateway** configured at

`Device > Setup > Management`, and it **does not use the data plane routing table**. Permitted IP addresses and services are set there too.

Service routes at `Device > Setup > Services > Service Route Configuration` let you send specific services (updates, DNS, syslog, LDAP, WildFire) out a **data interface instead of MGT**. This is how a firewall with no MGT internet path still reaches the update server.

By default a **data plane interface answers nothing**, no ping, no SSH, no HTTPS. An **Interface Management Profile** at `Network > Network Profiles > Interface Mgmt` selectively enables **Ping, Telnet, SSH, HTTP, HTTPS, SNMP, Response Pages, User-ID, User-ID Syslog Listener**, and is attached on the interface's **Advanced** tab.

The profile also takes a **permitted IP list**, which is the recommended control rather than relying on a security rule.

In practice

Two classic faults. "I can't ping the inside interface". There is **no management profile attached**, not a routing problem. And "the firewall routes fine but can't download updates", that is **MGT's separate gateway**, fixed with a service route. Never enable HTTP or Telnet, and never attach a management profile to an untrusted facing interface.

✓ Check yourself

aggregate, subinterfaces, management

1. In what order must you configure an AE group, and where does the zone go?

Create ae1 and set its type first, assign zone, IP and router to ae1, then add members. Members carry no zone or IP.

2. Two firewalls: one LACP passive, one static "on". Does the bundle form?

No. At least one side must actively negotiate. Passive responds but never initiates.

3. What does LACP pre negotiation do, and in which HA mode?

In active/passive, the passive firewall negotiates LACP in advance so the bundle is already up at failover, cutting recovery time.

4. Is the subinterface number the same as its VLAN tag?

No: they are configured separately. Keeping them identical is discipline, not a requirement.

5. The firewall routes traffic fine but cannot reach the update server. Most likely cause?

MGT has its own default gateway and no path out. Fix with a service route sending updates via a data interface.

6. Why does a ping to the firewall's inside interface fail even with a permit rule?

No Interface Management Profile is attached. Data interfaces answer nothing by default.

✓ Anchor facts

- ✓ **AE interface first, members second.** Zone and IP on the bundle.
- ✓ AE members must match **speed and duplex**.
- ✓ LACP: **active initiates, passive responds**; rate fast 1s or slow 30s.
- ✓ **Pre negotiation** speeds HA failover on LACP bundles.
- ✓ Subinterface suffix ≠ VLAN tag; each takes its own zone, router and IP.
- ✓ **MGT has its own gateway.** Service routes move services onto data interfaces.
- ✓ Data interfaces answer nothing without an **Interface Management Profile**.



Easy to get wrong

- Assigning a zone to AE **members** instead of the AE interface.
- Trying to add members before the AE interface exists.
- Passive + passive, or passive + static, and expecting a bundle.
- Assuming MGT uses the data plane routing table.
- Enabling HTTP or Telnet in a management profile.

Logical interface	Named	Takes a zone?	Typical use
Aggregate Ethernet	ae1	Yes, on the bundle	Bandwidth and link redundancy
Subinterface	ethernet1/1.100	Yes, its own	One link, several tagged segments
Loopback	loopback.1	Yes	GP gateway, router ID, DNS proxy
Tunnel	tunnel.1	Yes	IPSec and GRE endpoints
VLAN	vlan.1	Yes	Routing into a Layer 2 domain

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | **CONFIGURE** | VERIFY

Part 3 · Interfaces and Zones¹⁰

Virtual Wire, Zones and Zone Protection

Virtual wire earns its own page because its behaviour is least like a conventional interface. Zones then close the foundation, since policy is written between zones, never between interfaces.

34 Virtual wire in depth

A **virtual wire object** at `Network > Virtual Wires` binds exactly **two interfaces**. Traffic arriving on one is inspected and passed out the other. The firewall does not participate in ARP, does not appear as a hop, and needs no IP addressing.

Tag Allowed

Controls which **802.1Q VLAN tags** may pass. **0 means untagged**. Left at the default it passes untagged traffic only, which is the most common reason tagged traffic is silently dropped on a new virtual wire.

Multicast Firewalling

Off by default, meaning multicast passes without security policy evaluation. Enable it to apply policy to multicast.

Link State Pass Through

When enabled, losing link on one side **brings down the partner interface too**. This matters because the devices either side rely on link state to detect failure, without it they keep sending into a black hole and upstream failover never triggers.

Virtual wire subinterfaces

Subdivide a wire by **VLAN tag**, or by **VLAN tag plus IP classifier**, so one physical pair carries several logical wires in different zones with different policy.

Supported: security policy, NAT, App-ID, User-ID, Content-ID, decryption, QoS.

Not supported: routing, VPN termination, DHCP, GlobalProtect, and it cannot host management services.

Verify with `show interface all` and `show session all filter ...` to confirm traffic is traversing the wire.

In practice

Two settings account for most virtual wire faults: **Tag Allowed** not listing the VLANs in use, and **link state pass through** left disabled so neighbour failover never fires. Check both before looking at policy.

Zones, and the two default rules you never see in the rulebase

policy is between zones, and an interface belongs to exactly one

Zone = a group of interfaces

ethernet1/2

ethernet1/3

zone: TRUST

one interface → exactly one zone

Zone type must match the interface

Layer 3 · Layer 2 · Virtual Wire

Tap · Tunnel · External (vsys)

a Layer 3 interface cannot join a virtual wire zone

Attached at zone level

Zone Protection profile, floods, recon, packet attacks

Packet Buffer Protection, per zone toggle

User-ID enable list and exclude list

Log Setting for zone level logs

The two implicit rules at the bottom of every rulebase

intrazone-default: traffic within the same zone is **ALLOWED**

interzone-default: traffic between different zones is **DENIED** · neither logs until you override it

35 Security zones and zone types

A zone is a **logical grouping of interfaces of the same type**, and **security policy is written between zones**, never between raw interfaces.

Rules: an interface **must be in a zone** to process traffic; an interface can be in **only one zone**; a zone may hold many interfaces; the **zone type must match the interface type**: Layer 3, Layer 2, Virtual Wire, Tap, Tunnel, or **External** for inter VSYS traffic.

Default behaviour comes from two implicit rules: **intrazone-default allows** within a zone, **interzone-default denies** between zones. **Neither logs** until you override them, which is why "the deny isn't in the log" is so common.

Attached at zone level: a **Zone Protection profile**, **Packet Buffer Protection**, **User-ID enable** and **exclude lists**, and log settings.

Configured at `Network > Zones`. Verify with `show zone all`.

In practice

Override both implicit rules on day one to enable logging. Without it you have no record of what the firewall drops, which turns every investigation into guesswork.

36 Zone Protection, DoS Protection and packet buffers

All three defend against volumetric and malformed traffic, and **none is a security profile**: they are separate objects evaluated **before a session is created**, which is why they can stop floods that would otherwise exhaust the session table.

Zone Protection attaches to a **zone** and acts on the **aggregate** of traffic entering it:

Flood protection: SYN with **random early drop or SYN cookies**, ICMP, ICMPv6, UDP, Other IP, each with alert, activate and maximum rates.

Reconnaissance protection: TCP and UDP port scan, host sweep, with an action and block duration.

Packet based attack protection: spoofed IP, fragmented traffic, malformed options, specific TCP flag combinations.

Protocol protection: non IP protocol allow or block lists.

DoS Protection is applied through a **DoS policy rule** and is far more granular, targeting specific source and destination pairs. Thresholds are **aggregate** (shared by everything matching) or **classified** (per source IP, destination IP, or both).

Packet Buffer Protection protects the firewall itself from sessions consuming excessive buffer, enabled globally at `Device > Setup > Session` and toggled per zone.

In practice

The ordering point matters: **zone protection acts earliest, before a session exists; security profiles act last, on the content of an allowed session**. If a flood is exhausting the session table, no security profile will help you.

✓ Check yourself

virtual wire and zones

1. Tagged traffic is being dropped on a new virtual wire. First thing to check?

Tag Allowed **on the virtual wire object**. Default passes untagged only, 0 means untagged.

2. What does link state pass through prevent?

Neighbours sending into a black hole. Losing link one side brings down the partner, so upstream failover triggers.

3. Name three things virtual wire cannot do.

Routing, VPN termination, DHCP, also GlobalProtect and hosting management services.

4. Same zone traffic: allowed or denied by default, and is it logged?

Allowed by intrazone-default, and not logged until you override the rule.

5. How many zones can one interface belong to?

Exactly one. A zone can hold many interfaces, but not the reverse.

6. A SYN flood is exhausting the session table. Which control helps, and why not a security profile?

Zone Protection flood protection: it runs before session creation. Security profiles inspect content only after a session is allowed.

✓ Anchor facts

- ✓ A virtual wire binds **exactly two** interfaces.
- ✓ **Tag Allowed 0 = untagged.** Tagged traffic needs its tags listed.
- ✓ **Link state pass through** is what lets neighbours detect failure.
- ✓ Virtual wire subinterfaces split by **VLAN tag** or **tag plus IP classifier**.
- ✓ **intrazone-default allows, interzone-default denies**, and neither logs by default.
- ✓ Zone Protection = per zone, aggregate. DoS Protection = per rule, classified or aggregate.



Easy to get wrong

- Tagged traffic dropped because **Tag Allowed** was never set.
- Expecting a virtual wire to terminate a VPN or serve DHCP.
- Putting a Layer 3 interface in a virtual wire zone, types must match.
- Wondering why denies aren't logged: **override the implicit rules**.
- Calling Zone Protection a security profile. It is not, and it runs far earlier.

	Zone Protection	DoS Protection	Packet Buffer Protection
Attached to	A zone	A DoS policy rule	Global, toggled per zone
Scope	Aggregate entering the zone	Specific source/destination pairs	The firewall's own buffers
Thresholds	Aggregate only	Aggregate or classified	Latency and buffer utilisation
Covers	Floods, recon, packet attacks, protocols	SYN, UDP, ICMP, Other IP, session limits	Abusive sessions
Evaluated	Before session creation: earlier than any security profile		

Routing Foundations

PAN-OS has moved its routing implementation forward, so know the current engine rather than legacy virtual-router behaviour. This is where study material written a few years ago will catch you out.

37 Legacy virtual routers versus the Advanced Routing Engine

PAN-OS now offers **two routing modes**, and which one is active changes the entire configuration layout.

Legacy mode · virtual routers

The traditional model. A **virtual router (VR)** holds interfaces, static routes and routing protocol configuration, each with its own RIB and FIB. Protocol configuration lives **inside** the virtual router object, and redistribution uses PAN-OS specific redistribution profiles.

Advanced Routing Engine · logical routers

The current implementation, built on a standards based routing stack. The container is now a **logical router**, and protocol configuration is separated into **routing profiles** that are created once and attached where needed, BGP profiles, OSPF profiles, filters, redistribution profiles, BFD profiles.

The benefits are a **more standard configuration model**, richer route filtering with **route maps**, **prefix lists**, **access lists** and **AS path access lists**, and closer parity with how routing is configured on dedicated routers.

Enabling it is a **device wide change** under `Device > Setup > Management > Advanced Routing`, and it **requires a reboot**. Existing virtual routers are not automatically converted. You rebuild the configuration as logical routers.

In practice

Check which mode a firewall is in before you start looking for configuration, because the menu tree differs. If a scenario mentions **route maps** or **prefix lists**, that is the Advanced Routing Engine; if it mentions **redistribution profiles inside a virtual router**, that is legacy mode.

How PAN-OS picks a route

and where Policy Based Forwarding sits relative to the routing table

Route selection order

- 1 · Policy Based Forwarding, before the route lookup
- 2 · Longest prefix match wins
- 3 · Lowest administrative distance
- 4 · Lowest metric within the same protocol

PBF overrides the routing table

matched on zone, source, destination, application, user

Default administrative distances

Static	10		
OSPF internal	30	RIP	120
OSPF external	110		
BGP internal (iBGP)	200	lower wins	
BGP external (eBGP)	20		

Path monitoring keeps it honest

on failure, PBF falls back to the normal route lookup

38 Static routes, administrative distance and path monitoring

A static route needs a **destination prefix**, a **next hop** (IP address, the next VR or logical router, an FQDN, or **Discard** to blackhole), and an **interface**. It also takes an **administrative distance** (default **10**) and a **metric** (default 10).

Route selection order: longest prefix match first, then lowest administrative distance, then lowest metric.

Path monitoring on a static route sends ICMP probes to one or more destinations. If they fail, the route is **withdrawn from the RIB** so a backup route with a higher distance takes over. Configurable are the **ping interval**, **ping count**, **failure condition (any or all)**, and a **preemptive hold time** before the route is reinstated.

Verify with `show routing route`, `show routing fib`, and for the Advanced Routing Engine `show advanced-routing route`.

In practice

A floating static backup route only works if the primary is **actually removed** from the table. Without path monitoring, a static route stays up as long as the *interface* is up, so a failure further along the path never triggers failover.

✓ Check yourself

routing foundations

1. What is the container for routing in legacy mode, and what replaces it under the Advanced Routing Engine?

Virtual router **in legacy mode**; logical router **under the ARE**, with **protocol config moved into reusable** routing profiles.

2. Does enabling the Advanced Routing Engine convert your existing configuration?

No. **It is a device wide toggle that** requires a reboot, **and you rebuild the configuration as logical routers**.

3. State the route selection order.

Longest prefix match → lowest administrative distance → lowest metric. **PBF is evaluated before any of it.**

4. Default administrative distance for a static route, OSPF internal, and eBGP?

Static 10, OSPF internal 30, eBGP 20. OSPF external is 110 and iBGP is 200.

5. Why does a floating static backup route sometimes never take over?

No path monitoring. Without it the route stays up as long as the interface is up, so a failure further along the path never removes it.

6. Which commands show the routing table and the forwarding table?

`show routing route` and `show routing fib`; **under the ARE**, `show advanced-routing route`.

✓ Anchor facts

- ✓ **Virtual router = legacy. Logical router = Advanced Routing Engine.**
- ✓ Enabling ARE is device wide and **requires a reboot**; configuration is not auto converted.
- ✓ ARE brings **route maps, prefix lists, access lists, AS path lists.**
- ✓ Selection: **longest prefix → lowest AD → lowest metric.**
- ✓ Static AD default **10**; OSPF internal **30**, external **110**; eBGP **20**, iBGP **200**.
- ✓ **PBF runs before the route lookup** and only on transit traffic.



Easy to get wrong

- Studying virtual routers when the estate runs the **Advanced Routing Engine**.
- Expecting a floating static to fail over **without path monitoring**.
- Forgetting PBF **does not apply to firewall originated traffic**.
- Deploying PBF with no monitoring, creating a silent blackhole.
- Assuming eBGP outranks OSPF everywhere, compare the actual distances.

	Legacy virtual router	Advanced Routing Engine
Container	Virtual router	Logical router
Protocol config	Inside the VR object	Reusable routing profiles
Filtering	Redistribution profiles	Route maps, prefix lists, access lists, AS path lists
Enablement	Default	Device wide toggle, reboot required
Migration	,	Manual rebuild, not automatic

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Dynamic Routing

The objectives name dynamic routing protocols, redistribution and policies, and route monitoring. You need protocol competence plus how PAN-OS specifically exposes it.

39 OSPF on PAN-OS

Configured per virtual router or logical router. You set a **Router ID**, enable the protocol, define **areas**, and add interfaces to those areas.

Area types: Normal, **Stub** (blocks type 5, optionally with no summary for totally stubby), and **NSSA** (blocks type 5 but permits type 7, with a translation option on the ABR). **Area 0 is the backbone** and all other areas must connect to it, directly or through a **virtual link**.

Interface settings: link type (**broadcast**, **p2p**, **p2mp**), priority for DR/BDR election, cost, hello and dead intervals, and **passive** to advertise a network without forming adjacencies on it.

Authentication profiles support MD5 or simple password.

OSPFv3 is configured separately for IPv6 and uses **IPsec for authentication** rather than built in auth.

Verify with `show routing protocol ospf neighbor` , `... interface` , `... area` .

In practice

Adjacency failures on PAN-OS are the same as anywhere: mismatched **area ID**, **hello/dead timers**, **MTU**, **network type**, or **authentication**. Stuck in ExStart almost always means **MTU**.

40 BGP on PAN-OS

Enable BGP on the router, set the **Router ID** and **local AS**, then create **peer groups** and **peers** within them. Peer group type is **EBGP**, **IBGP**, **EBGP Confed** or **IBGP Confed**.

Per peer: peer AS, local and peer addresses, **multi hop** TTL, keepalive and hold timers, **MD5 authentication**, and address family.

Import and export rules are where PAN-OS does route filtering and manipulation in legacy mode, match on prefix, AS path, community or next hop, then set **local preference**, **MED**, **AS path prepend**, **weight**, **community** or **next hop**. Under the Advanced Routing Engine this moves to **route maps with prefix lists and AS path access lists**.

Other settings: **graceful restart**, **route reflector** client, **confederations**, **aggregation**, and **ECMP** at router level.

Verify with `show routing protocol bgp summary` , `... peer` , `... loc-rib` , `... rib-out` .

In practice

Remember the firewall is a full BGP speaker, so the best path rules apply as normal: **weight**, then **local preference**, then **locally originated**, then **shortest AS path**. The PAN-OS specific part is knowing that import/export rules are where you set those attributes.

Redistribution and route filtering

legacy profiles on the left, the Advanced Routing Engine's toolkit on the right

Legacy · redistribution profiles

create a redistribution profile with a **priority** (1 to 255)

filter by source type, interface, destination, next hop

action **redist** or **no-redist**

then reference it from the receiving protocol

Advanced Routing · the standard toolkit

prefix list: match networks

access list and **AS path access list**

community list

route map ties them together, match, then set

BFD gives subsecond failure detection for OSPF, BGP and static routes. Profiles set transmit and receive intervals and a multiplier. Attach a BFD profile to the neighbour or static route, without it, failure detection waits for protocol hold timers, which can be tens of seconds.

41 Redistribution and routing policies

Legacy mode: you build a **redistribution profile** with a **priority** from 1 to 255 (lower evaluated first), filter by source type, interface, destination or next hop, set the action to **redist** or **no-redist**, then reference that profile from the receiving protocol.

Advanced Routing Engine: standard constructs instead, **prefix lists**, **access lists**, **AS path access lists**, **community lists**, combined in a **route map** that matches and then sets attributes.

Typical redistributions: connected and static into OSPF or BGP, OSPF into BGP, BGP into OSPF, and the default route.

Always consider **loop prevention** when redistributing both directions between two protocols, and **filter** or **tag** routes so they do not come back in.

In practice

Two way redistribution without filtering is how you create a routing loop on any platform. On PAN-OS the tool differs by mode, but the discipline is identical: **tag on the way out, filter on the way back in**.

42 BFD, ECMP and route monitoring

BFD gives **subsecond failure detection** independently of protocol timers. A BFD profile sets the **desired minimum transmit interval**, **required minimum receive interval** and a **detection multiplier**, and is attached to an OSPF or BGP neighbour or to a static route.

ECMP is enabled per router, supporting up to **four equal cost paths**. Load balancing methods are **IP modulo**, **IP hash**, **weighted round robin** and **balanced round robin**. Enabling or disabling ECMP **restarts the routing process**, so it is disruptive.

Route monitoring is the umbrella term for path monitoring on static routes, PBF monitoring, and tunnel monitoring, each withdraws or bypasses something when a probe fails.

In practice

Use **weighted round robin** when the links are not equal capacity, and remember that changing ECMP settings is a **restart**, not a hitless change, schedule it.

✓ Check yourself

dynamic routing

1. An OSPF adjacency is stuck in ExStart. Most likely cause?

MTU mismatch. **Area ID, timers, network type and authentication mismatches prevent adjacency earlier than ExStart.**

2. How does OSPFv3 authenticate?

With IPsec: it has no built in authentication, unlike OSPFv2 which supports MD5 or simple password.

3. Where do you set local preference or AS path prepend in legacy mode, and under the ARE?

Legacy: BGP import and export rules. ARE: route maps with prefix lists and AS path access lists.

4. What range does a redistribution profile priority use, and which is evaluated first?

1 to 255, and lower is evaluated first.

5. How many equal cost paths does ECMP support, and what is the catch when changing it?

Up to four. Enabling or disabling ECMP restarts the routing process, so it is disruptive, schedule it.

6. Requirement is subsecond failure detection on a BGP peering. What do you add?

A BFD profile attached to the neighbour. Protocol hold timers alone can take tens of seconds.

✓ Anchor facts

- ✓ OSPF areas: **Normal, Stub, NSSA**; area 0 is the backbone; virtual links bridge the gap.
- ✓ **OSPFv3 authenticates with IPsec**, not built in auth.
- ✓ BGP peer group types: **EBGP, IBGP, EBGP Confed, IBGP Confed**.
- ✓ Legacy filtering = **import/export rules and redistribution profiles**; ARE = **route maps**.
- ✓ Redistribution profile priority is **1 to 255, lower first**.
- ✓ **ECMP: up to 4 paths**, and toggling it restarts routing.



Easy to get wrong

- Two way redistribution with no filtering → routing loop.
- Forgetting **passive** interfaces advertise the network without forming adjacency.
- Expecting ECMP changes to be hitless. **They restart the routing process.**
- Relying on protocol hold timers when the requirement is subsecond, use **BFD**.
- Looking for route maps on a firewall still in **legacy** routing mode.

Task	Legacy mode	Advanced Routing Engine
Filter redistributed routes	Redistribution profile	Route map + prefix list
Manipulate BGP attributes	Import / export rules	Route map set clauses
Match AS path	Regex in an import rule	AS path access list
Match communities	Import rule	Community list
Container	Virtual router	Logical router

Part 5 • NAT and Traffic Steering ¹³

NAT

You cannot operate a firewall without it, and scenario questions assume it. The golden rule carries most of the page.

43 Source NAT types

Dynamic IP and Port (DIPP): many to one using ports. The translated address is the **interface address** or a **translated address pool**. Standard outbound internet breakout. **Oversubscription** is configurable (1x, 2x, 4x, 8x) and determines how many concurrent sessions one IP supports.

Dynamic IP: one to one from a pool, **no port translation**. When the pool is exhausted new sessions fail, unless **Advanced (Dynamic IP/Port Fallback)** is configured so it falls back to DIPP.

Static IP: fixed one to one, port preserved. Can be **bidirectional**, which **automatically creates the implicit reverse destination NAT**: a rule you did not write.

Configured at `Policies > NAT`. Verify with:

```
show running nat-policy
```

```
show running ippool : DIPP utilisation per rule
```

```
show session all filter source 10.1.1.5
```

In practice

DIPP pool exhaustion presents as **new sessions failing while existing ones keep working**. Check

```
show running ippool
```

 before assuming a policy problem, the symptom looks identical to a broken rule.

44 Destination NAT and port forwarding

Destination NAT translates the destination address of inbound traffic to an internal server, optionally translating the **port** as well, which is port forwarding.

PAN-OS also supports **dynamic destination NAT** to a translated address **pool**, giving basic session distribution across several servers, round robin, not health aware.

The NAT rule's **destination zone is the zone the pre NAT destination routes to**, which is usually the untrust zone. That means the firewall needs a route for the public address so it can resolve the correct egress zone.

Where the public address is **in the same subnet as the interface**, PAN-OS answers ARP for it automatically. An address **outside** that subnet needs the upstream device to route it to the firewall.

In practice

The most common inbound failure is the **NAT rule's destination zone** set to the server's zone. It must be the zone the firewall routes the *public* address to, not where the server lives.

The golden rule, and the U turn case that breaks people

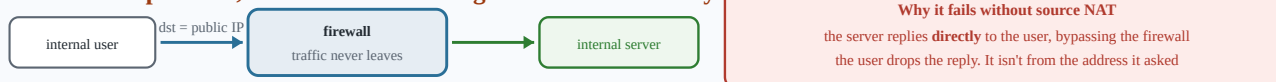
pre NAT address, post NAT zone

Security policy is evaluated with the PRE NAT address but the POST NAT zone

destination = PUBLIC (pre NAT) IP

zone = INTERNAL (post NAT) zone

U turn / hairpin NAT, internal user reaching an internal server by its public IP



The fix: one NAT rule doing **both**: destination NAT to the server's private IP, AND source NAT to the firewall's internal interface. The server then replies to the firewall, which translates back. Source and destination zone on the NAT rule are both the **internal** zone.

45 U turn (hairpin) NAT

The scenario: an **internal user** reaches an **internal server** using the server's **public IP**, usually because internal DNS returns the public record.

Destination NAT alone is not enough. The firewall translates the destination, the server receives the packet, but the **source is still the user's internal address**, so the server replies **directly** to the user, bypassing the firewall. The user discards that reply because it comes from an address it never contacted.

The fix is a single NAT rule doing **both translations**: destination NAT to the server's private address **and** source NAT to the firewall's internal interface. The server replies to the firewall, which translates back correctly.

On that rule, **source zone and destination zone are both the internal zone**, because the traffic never leaves the firewall's inside.

The cleaner alternative is **split DNS**, so internal clients resolve the private address and U turn NAT is never needed.

In practice

"The website works from outside but not from the office" is almost always this. Fix it with split DNS where you control the zone, U turn NAT where you don't.

46 NAT and security policy interaction

The golden rule: **security policy is evaluated using the pre NAT address but the post NAT zone.**

For an inbound DNAT that means the security rule's **destination address is the public IP** (policy is evaluated before translation) while the **destination zone is the internal zone** (the firewall has already done its route lookup to determine egress).

NAT rules are matched **top down, first match**, evaluated on the **original packet**, and **applied at egress**.

Verification, in this order:

`test nat-policy-match from TRUST to UNTRUST source 10.1.1.5 destination 8.8.8.8 protocol 6`

`test security-policy-match from ... to ... source ... destination ... application ...`

then `show session id <id>`: the **c2s and s2c flows show the actual post NAT values** per direction.

In practice

"NAT works but traffic is blocked" is this rule applied half right. Run both test commands: if NAT matches and security doesn't, check the **zone**; if security matches and traffic still fails, check the **route**.

✓ Check yourself

NAT

1. State the golden rule for writing a security rule alongside an inbound DNAT.

Pre NAT address, post NAT zone. **Destination is the public IP; destination zone is the internal zone.**

2. New outbound sessions are failing but established ones work. First check?

`show running ippool`: DIPP pool or port exhaustion. **Looks identical to a broken rule.**

3. What does bidirectional static NAT create that you did not write?

The implicit reverse destination NAT rule. Worth remembering when auditing a rulebase.

4. Why does U turn NAT need source translation as well as destination?

Without it the server replies directly to the internal user, bypassing the firewall, and the user drops the reply.

5. Which zone goes on a NAT rule's destination field for inbound DNAT?

The zone the firewall routes the public address to, usually untrust, not the server's zone.

6. Which two test commands do you run before changing configuration?

`test nat-policy-match` and `test security-policy-match`.

✓ Anchor facts

- ✓ Pre NAT address, post NAT zone.
- ✓ NAT rules: **top down, first match**, on the original packet, **applied at egress**.
- ✓ DIPP many to one with ports · **Dynamic IP** one to one no ports · **Static** fixed.
- ✓ Bidirectional static NAT **auto creates the reverse rule**.
- ✓ U turn needs **destination AND source** translation in one rule.
- ✓ DIPP oversubscription: **1x, 2x, 4x, 8x**.



Easy to get wrong

- Writing the security rule with the **private** destination IP.
- NAT destination zone set to the server's zone rather than the routed pre NAT zone.
- Solving U turn with destination NAT only.
- Missing DIPP exhaustion: **new sessions fail, old ones work.**
- Forgetting the implicit rule bidirectional static NAT created.

NAT type	Mapping	Ports translated?	Exhaustion behaviour
Dynamic IP and Port	Many to one	Yes	New sessions fail once ports run out
Dynamic IP	One to one from a pool	No	Fails unless DIPP fallback configured
Static IP	Fixed one to one	No	N/A
Destination NAT	Public to private	Optional (port forwarding)	N/A
U turn	Destination and source	Source uses DIPP	As DIPP

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Part 5 • NAT and Traffic Steering ¹⁴

Policy Based Forwarding, DHCP, IPv6, Sessions and QoS

Policy Based Forwarding overrides the routing table, and the service features (DHCP, IPv6, sessions and QoS) are the everyday configuration that surrounds it.

47 Policy Based Forwarding

PBF **overrides the routing table** for traffic matching a PBF rule, letting you forward by policy rather than by destination alone. Configured at [Policies › Policy Based Forwarding](#).

Match criteria, richer than routing

Source zone or interface, source address and user, destination address, application, and service. Being able to match on **application and user** is what routing cannot do.

Actions

Forward to a specified egress interface and next hop • **Forward to VSYS** • **Discard** • **No PBF** to exempt traffic and return it to normal routing.

Monitoring makes it safe

With monitoring enabled and **Disable this rule if nexthop is unreachable** set, a failed next hop causes the rule to be bypassed and the packet follows the routing table instead. Without it, a dead next hop is a silent blackhole.

Symmetric return

Uses the **ARP cache** to send return traffic back out the interface it arrived on, preventing asymmetric routing when PBF steers flows down a non default path.

Two scope limits: PBF is evaluated **before the route lookup**, and it applies only to traffic **through** the firewall, not to traffic the firewall originates itself. For firewall sourced traffic use **service routes** instead.

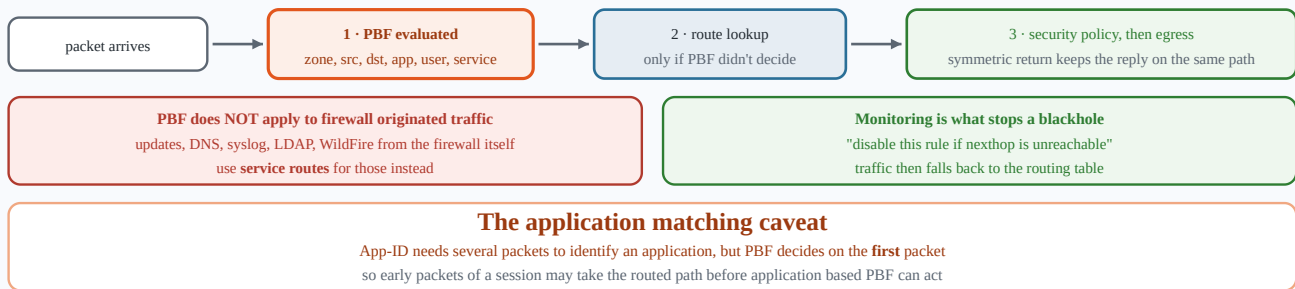
Verify with [show pbf rule all](#) and [test pbf-policy-match](#).

In practice

PBF is the usual answer to "send guest traffic out the broadband link and corporate over MPLS". Always pair it with monitoring. And remember application based PBF has a chicken and egg problem: the firewall needs several packets to identify the application, so the first packets of a session may take the routed path before PBF can act.

Where PBF sits, and what it cannot reach

before the route lookup, and transit traffic only



48 DHCP and IPv6

DHCP server on a Layer 3 interface: an address pool, reserved addresses, lease type (**unlimited** or **timeout**), and options including gateway, subnet mask, DNS, WINS, NTP and domain. Configured at [Network > DHCP](#).

DHCP relay forwards client requests to up to **eight** external servers, with IPv4 and IPv6 relay configured separately. An interface is **either a server or a relay**, never both.

DHCP client on an interface obtains an address and can optionally accept the **default route** from the server, common on branch internet interfaces.

IPv6 is configured alongside IPv4 on the same interface. You enable IPv6 on the interface, add addresses, and choose whether the firewall sends **Router Advertisements** for SLAAC. **Neighbour Discovery** replaces ARP, with a **DAD** (duplicate address detection) setting and an **ND cache**. Security policy and NAT support IPv6, and **NAT66**, **NAT64** and **NPTv6** exist for translation.

Verify with [show dhcp server lease all](#) , [show dhcp client state all](#) , [show ipv6 neighbor](#) .

In practice

An interface cannot be a DHCP server and relay simultaneously, the GUI will let you start configuring both and then complain on commit. And with IPv6 enabled, remember policy must be written for both families or you get silent single stack failures.

49 Session management and QoS

Session settings at `Device > Setup > Session` : global **timeouts** (TCP, UDP, ICMP, default, discard, plus TCP handshake and TIME_WAIT), **TCP settings** including **reject non SYN TCP** and **asymmetric path** handling, and **session rematch**.

Session rematch is important: with it enabled, a **policy change is applied to existing sessions** on the next packet. With it disabled, existing sessions keep their original treatment until they close. That is why tightening a rule sometimes appears to do nothing.

Reject non SYN TCP and **asymmetric path** are the two settings to relax only deliberately. They exist to reject out of state packets, which is exactly what asymmetric routing produces.

QoS on PAN-OS: a **QoS profile** defines eight classes with guaranteed and maximum bandwidth and priority (real time, high, medium, low); a **QoS policy** classifies traffic into a class by zone, address, user, application or service; and the profile is applied to a **physical egress interface** under `Network > QoS` .

QoS is egress only, and **class 4 is the default** for unmatched traffic. Verify with `show qos interface ethernet1/1` .

In practice

Session rematch explains the classic "I tightened the rule and traffic still flows". Either enable rematch or clear the sessions with `clear session id <id>` . And QoS only shapes what leaves the interface. You cannot shape inbound traffic on the link you are receiving it on.

✓ Check yourself

steering and services

1. PBF is configured and the next hop dies. What happens, and what makes that safe?

Without monitoring the traffic blackholes. With monitoring plus "disable this rule if nexthop is unreachable", it falls back to the routing table.

2. Why can PBF not steer the firewall's own update traffic?

PBF applies only to transit traffic. Firewall originated traffic is steered with service routes.

3. What is the catch with application based PBF?

App-ID needs several packets, but PBF decides on the first one, so early packets may take the routed path.

4. Can one interface be both a DHCP server and a DHCP relay?

No: one or the other. Relay forwards to up to eight servers.

5. You tighten a security rule but traffic keeps flowing. Why, and two fixes?

Session rematch is disabled, so existing sessions keep their original treatment. Enable rematch, or `clear session id` .

6. Which direction does PAN-OS QoS work in, and which class catches unmatched traffic?

Egress only, applied to a physical interface. Class 4 is the default.

✓ Anchor facts

- ✓ **PBF runs before the route lookup** and only on transit traffic.
- ✓ PBF matches on **application and user**, which routing cannot.
- ✓ **No PBF** action exempts traffic back to normal routing.
- ✓ DHCP relay: up to **eight** servers; server and relay are mutually exclusive per interface.
- ✓ **Session rematch** decides whether policy changes hit existing sessions.
- ✓ QoS: **8 classes, egress only, class 4 default**.



Easy to get wrong

- Deploying PBF with no monitoring → silent blackhole.
- Expecting PBF to steer firewall sourced traffic.
- Relaxing **reject non SYN TCP** casually to "fix" asymmetric routing.
- Forgetting IPv6 needs its own policy, or you get single stack failures.
- Trying to shape inbound traffic with QoS on the receiving interface.

Steering mechanism	Applies to	Decides on	Failure behaviour
Routing table	All traffic	Destination prefix	Next best route
PBF	Transit only	Zone, src, dst, app, user, service	Blackhole unless monitored
Service routes	Firewall originated only	The service type	Service fails
QoS	Egress per interface	Class from QoS policy	Falls to class 4

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Part 6 · Policy and Objects 15

Rule Evaluation

How a rulebase is read, and the two rules at the bottom that you never wrote. Get this and most policy behaviour becomes predictable rather than surprising.

50 How security rules are evaluated

Rules are read **top down**, and the **first match wins and evaluation stops**. Nothing below a matching rule is consulted. That means **rule order is policy**, not presentation.

The two implicit rules

At the bottom of every rulebase sit two rules you cannot delete:

intrazone-default: traffic within the same zone is **allowed**.

interzone-default: traffic between different zones is **denied**.

Neither logs by default. You can **override** them to enable logging and to attach a log forwarding profile, which is the single most valuable day one change.

What a match is evaluated against

Zone, address, user, application, service and URL category all have to match. A rule with **any** in a field matches everything in that field. Remember from the NAT page that policy uses the **pre NAT address but the post NAT zone**.

Actions

Allow · **Deny** (uses the application's default deny action) · **Drop** (silent) · **Reset client** · **Reset server** · **Reset both**. Drop presents to the user as a **hang**; reset presents as **connection refused**.

Verify with `show running security-policy` and predict a match with

```
test security-policy-match from TRUST to UNTRUST source 10.1.1.5 destination 8.8.8.8 application ssl protocol 6 destination-port 443
```

In practice

If denied traffic never appears in the log, it hit an **implicit rule** that is not logging. Override both on day one. And match the user's described symptom to the action, "it hangs" means drop, "connection refused" means reset.

Reading a rulebase first match stops evaluation, and the bottom two are yours to override

#	Rule	Zones	App / Service	Action
1	block-bad-sources	untrust → any	any	drop
2	allow-web	trust → untrust	web-browsing, ssl	allow
3	allow-web-strict	trust → untrust	web-browsing	allow + profiles
... your other rules ...				
	intrazone-default: same zone → ALLOW · not logged			
	interzone-default: different zones → DENY · not logged			

Rule 2 matches → evaluation STOPS
rule 3 is shadowed: never reached

Override both to enable logging
otherwise your denies are invisible

Actions: allow · deny (application default) · drop (silent → user sees a **hang**) · reset-client / reset-server / reset-both (user sees **connection refused**)

51 Rule types: universal, intrazone, interzone

Every rule has a **Rule Type**, and it changes which traffic the rule can match regardless of the zones you list.

universal (the default): matches traffic **both within and between** the listed zones. So a rule with source TRUST and destination TRUST-DMZ also matches TRUST-to-TRUST traffic.

intrazone: matches only traffic whose source and destination zone are **the same**. The destination zone field is greyed out because it is implied.

interzone: matches only traffic **crossing** from one zone to a different one.

The consequence: a **universal** rule is broader than it looks. If you intend a rule to apply only to traffic leaving a zone, set it to **interzone** rather than relying on the zone fields.

In practice

This silently widens rules. A universal allow with the same zone in both fields permits intrazone traffic you may not have considered, and since intrazone-default already allows it, the effect is easy to miss until you tighten the default.

52 The policy types and their order

PAN-OS has several separate rulebases under **Policies**, each evaluated at a different point:

NAT: evaluated on the original packet, applied at egress.

Policy Based Forwarding: before the route lookup.

Decryption: before App-ID can see inside TLS.

Authentication: triggers a challenge for unidentified users.

Security: the allow or deny decision.

Application Override: bypasses App-ID for matched traffic.

QoS: classifies for egress shaping.

DoS Protection: threshold enforcement.

Tunnel Inspection: inspects inside GRE and non encrypted tunnels.

Each rulebase is independently **top down, first match**. A packet may match one rule in several rulebases. They are not alternatives.

In practice

The ordering that catches people: **decryption happens before security policy** can use App-ID meaningfully on TLS, and **application override happens before App-ID**, which is why an override turns a rule into a port based one and disables threat inspection for that traffic.

✓ Check yourself

rule evaluation

1. Denied traffic is not appearing in the Traffic log. Why, and what is the fix?

It hit an implicit rule, and neither logs by default. Override intrazone-default and interzone-default to enable logging.

2. Same zone in both fields, rule type left at default. What traffic does it match?

Universal matches both intrazone and interzone, so it is broader than it looks. Use interzone to restrict it.

3. A user reports the application hangs. Which action is likely, and which would give "connection refused"?

Drop gives a hang. Reset gives connection refused. Match the symptom to the log action.

4. Why is rule 3 in a rulebase sometimes never reached?

It is shadowed by a broader rule above it. First match wins and evaluation stops.

5. Which two rulebases are evaluated before security policy can use App-ID properly?

Decryption (so App-ID can see inside TLS) and Application Override (which bypasses App-ID entirely).

6. Which command predicts the matching rule without generating traffic?

`test security-policy-match` with from, to, source, destination, application and port.

✓ Anchor facts

- ✓ **Top down, first match, then stop.** Rule order is policy.
- ✓ **intrazone-default allows, interzone-default denies**, neither logs.
- ✓ **universal** matches within *and* between zones, broader than it looks.
- ✓ **Drop = hang. Reset = connection refused.**
- ✓ Each rulebase is independent and separately first match.
- ✓ Policy uses the **pre NAT address, post NAT zone.**



Easy to get wrong

- Leaving the implicit rules unlogged, then having no record of denials.
- Relying on zone fields instead of setting **interzone** rule type.
- Writing a rule below a broader one and wondering why it never hits.
- Treating **deny** and **drop** as identical, the user experience differs.
- Forgetting application override **disables threat inspection** for that traffic.

Rule type	Matches	Destination zone field	Use when
universal (default)	Within and between listed zones	Editable	General purpose, but check the breadth
intrazone	Same source and destination zone only	Greyed out	Controlling traffic inside a zone
interzone	Crossing zones only	Editable	You mean "leaving this zone"

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 6 • Policy and Objects 16

Applications in Policy

The service field, dependencies and shadowing. application-default versus any is the single most consequential setting on a rule, and the one most often left wrong.

53 application-default versus any, and why it matters

The **Service** column on a security rule decides which ports the named applications are permitted on. It has three meanings:

application-default: the application is allowed **only on the ports Palo Alto defines as its standard ports**. So `web-browsing` is permitted on 80, and an attempt to run web-browsing on 8080 is **not** matched by this rule.

any: the application is allowed on **any port**. Convenient, and the reason evasive applications succeed. You have said "this app, anywhere".

A specific service object: your own port list, which is what you use for non standard internal applications.

Why it matters

Using **any** reopens the port hopping gap that App-ID exists to close. The recommendation is **application-default for internet bound rules**, and explicit service objects where an application genuinely runs on a non standard port.

The trade off

application-default can break legitimate traffic that runs an app on an unusual port. The fix is a specific service object for that rule, not switching the whole rule to **any**.

In practice

This is the most common real world misconfiguration. A rule reading "allow web-browsing, service any" looks tight because it names an application, but it permits that application on every port. Check the Service column whenever a rule looks safer than it behaves.

The Service column changes what a rule actually permits

naming an application is not the same as constraining it

application-default

web-browsing on port 80 → matches, allowed

web-browsing on port 8080 → does NOT match this rule

service = any

web-browsing on any port → allowed

reopens the port hopping gap App-ID closes

Application dependencies

you allow: facebook-chat

→ also needs web-browsing + ssl

the rule looks correct and the app still fails

Policy Optimizer

No App Specified · Unused Apps · Rule Usage

shows the applications a port based rule actually carried
so you can convert it to App-ID from real traffic

54 Application dependencies and application override

Dependencies: some App-IDs cannot be identified without their parent applications also being permitted. Allowing `facebook-chat` requires `web-browsing` and `ssl` as well, because the session starts as those before it can be refined.

PAN-OS shows dependencies in the application object, and **Policy Optimizer** will flag rules where a dependency is missing. The symptom is a rule that looks correct and an application that still fails.

Implicit dependency handling exists for some applications where the parent is allowed by the *same* rule, but do not rely on it, list the dependencies.

Application override

A separate rulebase that **bypasses App-ID** for matched traffic, forcing it to a named application, usually a custom one, based on port and zone.

The cost is significant: overridden traffic **skips App-ID and therefore skips threat inspection** for that session, turning the rule into a port based one. Legitimate uses are a latency sensitive internal application, or one App-ID repeatedly misidentifies.

In practice

Application override is a last resort, not a tuning tool. If an application is being misidentified, a **custom App-ID** is almost always the better answer because it preserves inspection.

55 Shadowing, unused rules and Policy Optimizer

Shadowing is a rule that can never match because a broader rule above it catches the same traffic first. PAN-OS warns about some shadowing on commit, but not all of it.

Rule usage tracking at `Policies` shows each rule's **hit count**, **first hit** and **last hit**, with a **Rule Usage** filter for unused rules. This is how you find rules safe to remove, though check the timeframe before concluding a rule is dead.

Policy Optimizer

Purpose built for migrating **port based rules to App-ID**. Its views are:

No App Specified: rules still matching on service only.

Unused Apps: applications listed on a rule that never actually appear.

Rule Usage: hit counts over a period.

The workflow: find a port based rule, view the applications it **actually carried**, add those applications to the rule, set the service to **application-default**, then remove the port based service. Because it is built from observed traffic, the risk of breaking something is far lower than guessing.

Also useful: **tags** and **rule groups** for organising a large rulebase, **audit comments** on rule changes, and **rule hit counters** reset with `clear rule-hit-count`.

In practice

Policy Optimizer is the honest answer to "how do you move a legacy port based rulebase to App-ID without an outage". You migrate from what the rule actually carried, not from what you assume it carried.

✓ Check yourself

applications in policy

1. A rule allows web-browsing with service set to any. What is wrong with it?

It permits that application on every port, reopening the port hopping gap. Use application-default.

2. application-default is breaking a legitimate internal app on a non standard port. Correct fix?

A specific service object for that rule, not switching the rule to any.

3. You allow facebook-chat and it still fails. Why?

Missing dependencies: it also needs web-browsing and ssl, because the session starts as those.

4. What does application override cost you?

Overridden traffic skips App-ID and threat inspection. A custom App-ID is usually better.

5. Name the three Policy Optimizer views.

No App Specified, Unused Apps, Rule Usage.

6. Why is Policy Optimizer safer than manually rewriting port based rules?

It shows the applications the rule actually carried, so you migrate from observed traffic rather than assumption.

✓ Anchor facts

- ✓ **application-default** = standard ports only. **any** = every port.
- ✓ Dependencies must be listed, **facebook-chat** needs **web-browsing** and **ssl**.
- ✓ **Application override** bypasses **App-ID**, so it bypasses threat inspection.
- ✓ Policy Optimizer views: **No App Specified, Unused Apps, Rule Usage**.
- ✓ Rule usage shows **hit count, first hit, last hit**.
- ✓ Custom App-ID beats override, because it keeps inspection.



Easy to get wrong

- Assuming a rule is tight because it names an application, **check the Service column**.
- Switching to **any** to fix a broken app instead of adding a service object.
- Omitting dependencies, then blaming App-ID.
- Using application override as a routine tuning tool.
- Deleting a "0 hits" rule without checking the counter timeframe.

Service setting	Permits the application on	Risk	Use for
application-default	Its standard ports only	Breaks non standard usage	Internet bound rules
any	Every port	Port hopping and evasion	Rarely justified
Service object	The ports you list	Maintenance overhead	Internal apps on odd ports

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 6 • Policy and Objects 17

Objects, EDLs and Dynamic Address Groups

Objects keep a rulebase maintainable, and the dynamic ones, EDLs and Dynamic Address Groups, let policy change without a commit, which is how modern estates stay current.

56 Address, service and application objects

Address objects take an **IP netmask**, an **IP range**, an **IP wildcard mask**, or an **FQDN**. FQDN objects are resolved by the firewall on a refresh interval, which lets policy follow a hostname whose address changes.

Address groups are **static** (a fixed member list) or **dynamic** (membership from a tag match expression).

Service objects define protocol and port, with optional **session timeout overrides** per service, useful for an application needing a longer idle timeout than the global default.

Application groups are static collections; **application filters** are dynamic, selecting by **category**, **subcategory**, **technology**, **risk** and **characteristics**. A filter automatically picks up new applications matching its criteria on each content update, which is powerful and slightly risky.

Regions are built in geographic objects usable as source or destination.

In practice

Prefer an **application filter** over a long static group when the intent is categorical ("all peer to peer file sharing"), because it stays current. But know that a content update can then change what your rule permits without you editing it.

57 External Dynamic Lists

An **EDL** is a list the firewall **fetches from a web server** on a schedule and uses directly in policy, so the contents change without a commit.

Types: **IP address**, **domain**, **URL**, **Predefined IP**, **Predefined URL**, and **Equipment Identity / Subscriber Identity** on supporting platforms.

Palo Alto predefined EDLs ship with the platform and are maintained by Palo Alto, known malicious IPs, bulletproof hosting, Tor exit nodes, high risk IPs. They cost nothing to use and are a sensible first block rule.

Configuration: the source URL, a **check for updates interval** (five minutes to monthly), optional **client certificate** for authenticated sources, and an **exception list** for entries you want to ignore.

Test and verify with `request system external-list show type ip name <name>` and `request system external-list refresh name <name>`. The GUI has a **Test Source URL** button.

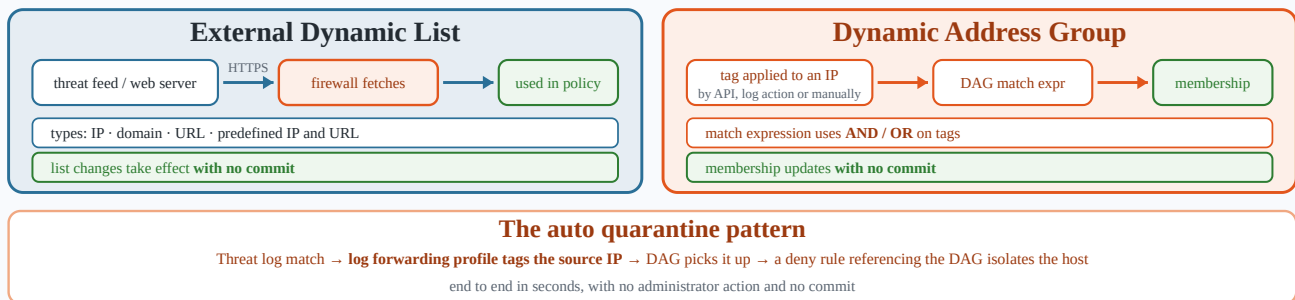
Limits are per platform: there is a maximum number of EDLs and a maximum total entries, so very large lists need checking against the model.

In practice

The reason EDLs matter operationally: a threat intel feed updates the block list **without a configuration change**, so there is no commit, no change window and no approval cycle for each new indicator.

Dynamic policy without a commit

EDLs pull lists from outside; tags drive Dynamic Address Groups from inside



58 Tags and Dynamic Address Groups

Tags are labels with an optional colour, applied to address objects, rules and other objects. Beyond organisation, they are the mechanism behind dynamic membership.

A **Dynamic Address Group (DAG)** defines a **match expression** over tags using **AND** and **OR**, for example 'quarantine' and 'corporate'. Any IP carrying tags that satisfy the expression becomes a member, and **membership changes take effect without a commit**.

Tags reach an IP address three ways: **manually**, via the **XML API** (`register` and `unregister` calls), or from a **log forwarding profile built in action** that tags the source or destination of a matching log.

The auto quarantine pattern

A Threat log entry matches a log forwarding profile, whose built in action **tags the source IP**. The DAG picks it up, and a deny rule referencing that DAG isolates the host, **within seconds, with no administrator action and no commit**.

Verify with `show object registered-ip all` and `show object dynamic-address-group all`. Clear with `debug object registered-ip clear all`.

In practice

Registered IP tags are **runtime state, not configuration**, so they do not survive a reboot and are not part of the saved config. If quarantine must persist, the tagging source has to reapply it.

59 Custom applications, threats and data patterns

Custom applications at `Objects > Applications` define your own App-ID for an in house or unrecognised application, using a **signature** based on context and pattern, or simply port and protocol. Because it is a real App-ID, **threat inspection still applies**, which is why it beats application override.

Custom objects > Spyware and Vulnerability let you write your own threat signatures with a **threat ID in the 41000 to 45000 range**, matched by pattern against a decoder context.

Custom URL categories are your own lists of URLs, usable in URL Filtering profiles and as a match condition in security policy. Types are **URL List** or **Category Match**.

Data patterns underpin Data Filtering: **predefined** patterns (credit card, social security number), **regular expressions**, and **file properties**.

Custom signatures are exported and imported as XML, which is how they are shared between firewalls outside Panorama.

In practice

When App-ID does not recognise an internal application, the ladder is: **custom App-ID first** (keeps inspection), **application override only if that fails** (loses inspection). Reaching straight for override is the common mistake.

✓ Check yourself

objects and dynamic content

1. What is the operational advantage of an EDL over an address group?

The list contents change without a commit, no change window per indicator.

2. Name three ways a tag gets applied to an IP address.

Manually, via the XML API register call, or by a log forwarding profile built in action.

3. Walk the auto quarantine chain.

Threat log → log forwarding profile tags the source IP → DAG membership updates → deny rule referencing the DAG isolates the host. No commit.

4. Do registered IP tags survive a reboot?

No: they are runtime state, not configuration. The tagging source must reapply them.

5. An internal app is unrecognised. Which do you build first, and why?

A custom App-ID, because it keeps threat inspection. Application override loses it.

6. What range do custom threat signature IDs use?

41000 to 45000, for custom spyware and vulnerability objects.

✓ Anchor facts

- ✓ Address objects: **IP netmask, IP range, IP wildcard, FQDN**.
- ✓ **Application filters** are dynamic by category, risk and technology; groups are static.
- ✓ EDL types: **IP, domain, URL, predefined IP, predefined URL**.
- ✓ **EDL and DAG changes need no commit**.
- ✓ DAG match expressions use **AND / OR** over tags.
- ✓ Custom threat IDs: **41000 to 45000**.



Easy to get wrong

- Expecting registered IP tags to **persist across a reboot**.
- Forgetting EDL count and entry **limits are per platform**.
- Not realising an **application filter** can change a rule's scope on a content update.
- Using application override where a **custom App-ID** would keep inspection.
- Assuming an FQDN object updates instantly. It follows a refresh interval.

Object	Static or dynamic	Needs a commit to change?	Driven by
Address group	Static	Yes	Member list
Dynamic Address Group	Dynamic	No	Tag match expression
External Dynamic List	Dynamic	No	Fetches from a URL on a schedule
Application group	Static	Yes	Named applications
Application filter	Dynamic	No (updates with content)	Category, risk, technology
FQDN address object	Semi dynamic	No	DNS, on a refresh interval

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 7 · App-ID and Content-ID (18)

How App-ID Identifies Traffic

App-ID is what makes the firewall next generation rather than a port filter. Learn the identification sequence and the log values it produces, because both turn up constantly in troubleshooting.

60 The App-ID identification sequence

App-ID identifies the application **regardless of port, protocol, encryption or evasive technique**. It works through four mechanisms applied in order, and the sequence matters because each stage can hand over to the next.

1 · Application signatures

Traffic is matched against signatures looking at **unique properties and transaction characteristics**, not just the port. This is where most applications are identified.

2 · TLS/SSH decryption

If the traffic is encrypted and a decryption policy applies, it is decrypted so signatures can be applied to the payload. **Without decryption, App-ID sees only the outer protocol**: typically `ssl` or a partial identification from the certificate and SNI.

3 · Protocol decoders

Decoders understand the protocol and look for applications **tunnelled inside** it. This is how the firewall finds a file transfer inside an HTTP session.

4 · Heuristics

Behavioural analysis for applications that deliberately evade signatures, encrypted peer to peer being the usual example.

Application shift

Identification is **iterative**. A session may start as `web-browsing`, then be refined to `facebook-base`, then to `facebook-chat`. Each shift causes a **fresh security policy lookup**, which is why a session can be allowed initially and dropped moments later.

In practice

Application shift explains the "it connected then died" symptom. The first App-ID matched a permissive rule; the refined App-ID did not. Check the Traffic log for the **final** application, not the one you expected.

App-ID values in the Traffic log four values that each point at a different root cause

incomplete

the handshake never finished

`SYN with no SYN-ACK`

look at routing, the server, or something upstream dropping

insufficient data

handshake completed

`too little payload to identify`

session closed too early
often normal, not a fault

not applicable

the firewall denied it

`before App-ID ever ran`

no rule permitted the port,
so this is a policy problem

unknown-tcp / udp

real traffic, no signature

`usually an in house app`

build a custom App-ID
rather than an override

The distinction that matters most

not applicable is a policy problem you caused · **incomplete** is a connectivity problem somewhere else

Application shift causes a fresh policy lookup

`web-browsing → facebook-base → facebook-chat, allowed at first, dropped on refinement`

61 App-ID values in the Traffic log

Four values appear in the **Application** field instead of a real application name, and each points somewhere different:

incomplete: the **TCP handshake never completed**, so there was no payload to identify. A SYN with no SYN-ACK. This is a **connectivity** problem: routing, the server being down, or something upstream dropping the traffic.

insufficient data: the handshake **did** complete but there was **not enough payload** to identify the application before the session closed. Often entirely normal.

not applicable: the firewall **denied the traffic before App-ID ever ran**, because no rule permitted that port or service. This is a **policy** problem, and it is yours.

unknown-tcp / **unknown-udp**: real traffic that completed, but no signature matched. Usually an in house or bespoke application.

Filter for them in the Traffic log with `( app eq incomplete )` or `( app eq not-applicable )`.

In practice

Learn the **incomplete versus not applicable** pair properly. One sends you to the network, the other to your own rulebase, and mixing them up wastes the first hour of an investigation.

62 Custom App-IDs and content updates

For **unknown-tcp** or **unknown-udp** traffic, build a **custom application** at `Objects > Applications`. You define the protocol and port, and optionally a **signature** matching a pattern within a decoder context.

The advantage over application override is decisive: a custom App-ID is a **real application**, so **threat inspection still applies**. An override bypasses App-ID entirely and takes threat inspection with it.

You also set the application's **characteristics**: risk, category, subcategory, technology, whether it evades, tunnels other applications, or transfers files. These matter because **application filters select on them**.

Content updates and the new App-ID risk

Applications and threats arrive in **content updates**. When Palo Alto releases **new or modified App-IDs**, traffic that previously matched one application may start matching a new, more specific one, and **your rule may not list it**.

PAN-OS mitigates this. The **Applications and Threats** update page shows which App-IDs are new, lets you **disable new App-IDs on install** and enable them later after review, and a **content update schedule** can set a **threshold** delay so updates are not installed the instant they are published.

In practice

This is the one content update risk worth planning for. A working rule can break because the application it permits got split into a more specific one. Review new App-IDs before enabling, and use the threshold delay in production.

✓ Check yourself

App-ID

1. Name the four App-ID mechanisms in order.

Application signatures → TLS/SSH decryption → protocol decoders → heuristics.

2. A session connects then dies seconds later. What is the likely cause?

Application shift. The refined App-ID triggered a fresh policy lookup and no longer matched the permissive rule.

3. Distinguish incomplete from not applicable.

incomplete = handshake never finished → connectivity problem. not applicable = denied before App-ID ran → policy problem.

4. What does insufficient data mean, and is it a fault?

Handshake completed but too little payload to identify before the session closed. Often normal.

5. You see unknown-tcp for an in house app. What do you build, and why not an override?

A custom App-ID: it keeps threat inspection. An override bypasses App-ID and inspection with it.

6. How can a content update break a working rule, and what two controls help?

A new App-ID may take over traffic your rule doesn't list. Use disable new App-IDs on install and a threshold delay on the update schedule.

✓ Anchor facts

- ✓ Four mechanisms: **signatures, decryption, decoders, heuristics.**
- ✓ Identification is **iterative**: each shift triggers a **fresh policy lookup.**
- ✓ **incomplete** = no handshake. **insufficient data** = not enough payload.
- ✓ **not applicable** = denied before App-ID ran. **unknown-tcp** = no signature.
- ✓ Custom App-ID **keeps threat inspection**; override does not.
- ✓ Content updates can introduce **new App-IDs** that change what a rule matches.



Easy to get wrong

- Confusing **incomplete** (network) with **not applicable** (policy).
- Expecting App-ID to see inside TLS **without decryption.**
- Treating **insufficient data** as an error when it is usually normal.
- Installing content updates immediately in production with no threshold.
- Reaching for application override before trying a custom App-ID.

Log value	Means	Look at	Usually
incomplete	Handshake never completed	Routing, server, upstream drops	A network fault
insufficient data	Too little payload to identify	Nothing, usually	Normal behaviour
not applicable	Denied before App-ID ran	Your rulebase	A policy gap
unknown-tcp / udp	No signature matched	The application itself	Build a custom App-ID

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Part 7 · App-ID and Content-ID ¹⁹

Security Profiles · Antivirus to Vulnerability Protection

Where Content-ID becomes configuration. The distinction running through both pages: a rule decides whether traffic is allowed; a profile inspects what is inside it.

63 Security profiles versus security policy rules

They do different jobs at different points. A **security policy rule** decides whether traffic is allowed at all, based on zone, address, user, application and service. A **security profile** decides what happens to the content inside traffic that has already been allowed.

So profiles attach to **allow** rules only. They **never run on a deny rule**: there is nothing left to inspect.

The profile types are **Antivirus, Anti-Spyware, Vulnerability Protection, URL Filtering, File Blocking, WildFire Analysis and Data Filtering**, with **DNS Security** and **Enterprise DLP** as licensed extensions.

Use **Security Profile Groups** rather than attaching profiles individually, so a rule references one group and every rule stays consistent. In Panorama, push a default group so no rule is ever created without inspection attached.

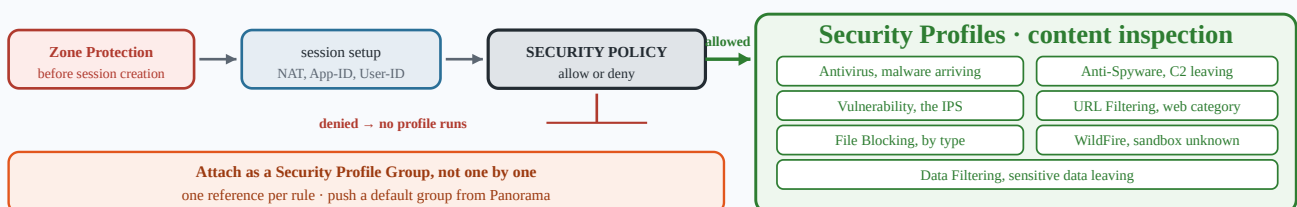
Palo Alto ships **best practice profiles** as a starting baseline, sensible, because building from empty means every signature decision is yours.

In practice

The rule decides who gets through the door; the profile inspects what they are carrying once inside. If a rule denies traffic, no profile ever sees it, which is why threat logs are empty for denied sessions.

Where profiles sit in the flow

zone protection acts before a session exists; profiles act last, on allowed traffic only



Attach as a Security Profile Group, not one by one
one reference per rule · push a default group from Panorama

WildFire's verdict becomes an **antivirus signature** pushed back to every firewall, which is how the sandbox loop closes and why the two profiles are related.

Zone Protection and DoS Protection are NOT security profiles. They are separate objects evaluated before session creation, defending against floods rather than inspecting content.

64 Antivirus and Anti-Spyware

Antivirus scans allowed traffic for known malware using **stream based scanning**: inspecting as the file flows rather than buffering it whole, which is what keeps latency acceptable.

Actions are set **per protocol decoder**: FTP, HTTP, HTTP2, IMAP, POP3, SMB, SMTP. Available actions are **default, allow, alert, drop, reset-client, reset-server, reset-both**, where **default** uses the action the signature itself recommends.

It has **two signature sources** configured independently: standard antivirus signatures, and **WildFire signatures** generated from sandbox verdicts and pushed out within minutes.

Anti-Spyware · the other direction

Antivirus catches malware **arriving**. Anti-Spyware catches **an infection already inside talking out**: command and control, beaconing, spyware phoning home. Rules are built by **severity**, each with an action and packet capture setting.

DNS sinkholing

Malware usually **resolves a domain before connecting out**. Sinkholing spots the malicious lookup and **forges a DNS response pointing at a sinkhole address you control**.

The reason this matters is **visibility**. Without it the DNS query comes from your **internal DNS server**, so the log names the DNS server rather than the infected host. With sinkholing the infected host connects to the sinkhole itself, appearing in the **Traffic log by source address**, so **you can identify patient zero**.

In practice

Sinkholing is the highest value feature on this page. Without it you know you have an infection somewhere; with it you know which machine.

65 Vulnerability Protection

This is the IPS function on a Palo Alto firewall. It protects against exploit attempts against known vulnerabilities: buffer overflows, illegal code execution, protocol anomalies, and scanning or probing behaviour.

Like Anti-Spyware, rules are built by **severity** with an action and packet capture. What differs is the scoping: you can narrow a rule by **CVE, vendor ID, threat name, category and host type**.

The **host type** field is the one worth knowing: **client** protects your users, **server** protects your published services. Server rules face the internet, so they normally carry stricter actions.

This profile generates **most false positives**, because **bespoke and proprietary protocols trip generic vulnerability signatures**. The fix is a **threat exception scoped to that specific threat ID**, restricted to the affected source or zone where possible, and documented. **Never disable the profile**.

In practice

Verify before excepting. Take the threat ID, confirm with a packet capture that it really is a false positive, then scope the exception as narrowly as the situation allows.

✓ Check yourself

profiles, part one

1. Why is the Threat log empty for traffic a rule denied?

Profiles attach to **allow** rules and never run on a deny. There is nothing left to inspect.

2. What does stream based scanning buy you?

Inspection as the file flows **rather than buffering it whole**, which keeps latency acceptable.

3. Antivirus versus Anti-Spyware in one line.

Antivirus catches malware arriving; Anti-Spyware catches an infection already inside calling out.

4. Why does DNS sinkholing identify patient zero when ordinary logging does not?

Without it the query comes from your DNS server. With it, the infected host connects to the sinkhole and appears in the Traffic log by source.

5. What does host type client versus server change on a vulnerability rule?

Client protects your users; server protects published services. **Server rules face the internet and are usually stricter.**

6. A bespoke app keeps tripping a vulnerability signature. Correct fix?

A threat exception scoped to that threat ID, restricted by source or zone, verified first. Never disable the profile.

✓ Anchor facts

- ✓ Profiles run on **allowed traffic only**, never on a deny.
- ✓ Seven profiles, attached as a **Security Profile Group**.
- ✓ Antivirus decoders: **FTP, HTTP, HTTP2, IMAP, POP3, SMB, SMTP**.
- ✓ Antivirus has **two signature sources**: standard and WildFire.
- ✓ **Sinkholing reveals patient zero**.
- ✓ Vulnerability Protection **is the IPS**; scope by CVE and **host type**.



Easy to get wrong

- Expecting threat logs for **denied** traffic.
- Attaching profiles individually instead of as a group.
- Disabling a vulnerability profile to fix one false positive.
- Excepting a threat ID without verifying it first.
- Forgetting sinkholing needs an address you actually control.

Profile	Catches	Keyed by	Threat log type
Antivirus	Known malware arriving	Protocol decoder	virus
Anti-Spyware	C2 and beaconing leaving	Severity; DNS sinkholing	spyware
Vulnerability Protection	Exploit attempts, the IPS	Severity, CVE, host type	vulnerability

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Part 7 · App-ID and Content-ID (20)

URL Filtering, Files, Data and Tuning

The remaining profiles, then the method that matters most in practice: identifying which profile blocked something, and tuning it without loosening protection wholesale.

66 URL Filtering and User Credential Detection

Classifies web traffic into **categories** from the **PAN-DB** cloud database, with a local cache on the firewall and a cloud lookup for anything unknown.

Actions per category: **allow, alert, block, continue, override**. Two are worth explaining. **continue** presents an interstitial the user can click through, for categories you want to discourage rather than forbid. **override** requires a password, letting an administrator unlock a category for a specific user.

Beyond categories the profile handles **URL category exceptions** through allow and block lists, **Safe Search enforcement**, **HTTP header insertion**, and logging of the **HTTP referer and user agent**.

User Credential Detection

Detects when a user submits **corporate credentials into an external website**, which is how phishing actually succeeds. It works by **IP to user mapping**, **group membership**, or **domain credential filter**, and blocks the **submission** rather than the browsing.

The distinction people trip on

URL category is used in two places. As a **match condition in a security policy rule**, where it influences whether traffic is allowed at all. And as an **action inside the URL Filtering profile**, where it controls what happens to allowed web traffic. Mixing them up is a common configuration error.

In practice

User Credential Detection is the feature separating someone who has deployed URL filtering from someone who has read about it. Pair it mentally with Orion's passwordless work, one stops the credential being typed, the other removes it from the flow entirely.

Finding which profile blocked something

the Threat log Type field names it, so you tune the right thing

Threat log · the Type field tells you which profile fired

virus Antivirus	spyware Anti-Spyware	vulnerability Vulnerability Protection	url URL Filtering	file File Blocking	wildfire WildFire Analysis	data Data Filtering
---------------------------	--------------------------------	--	-----------------------------	------------------------------	--------------------------------------	-------------------------------

Then tune it narrowly

take the **threat ID** → verify with a packet capture → **exception scoped to that ID**, restricted by zone or source → document it
never disable the profile

Rolling out new profiles

deploy in **alert only** → run a baseline period → review what it *would* have blocked → move to blocking severity by severity
the only way to avoid taking out a production application with a signature you have never seen fire

67 File Blocking, WildFire Analysis and Data Filtering

File Blocking controls file types by **direction**: upload, download or both. Actions are **alert, block, continue**. A typical policy blocks executables inbound from the internet and blocks encrypted archives that cannot be inspected. Its value is stopping **entire classes of file before any scanning is needed**.

WildFire Analysis decides **which files are forwarded to the sandbox** for detonation, by application, file type and direction. The destination is either the **public WildFire cloud** or a **private WF-500 appliance** where data residency rules apply.

The relationship in one line: **File Blocking decides whether the file is allowed at all; WildFire Analysis decides whether an allowed unknown file gets detonated**. The verdict returns as an antivirus signature, closing the loop.

Data Filtering is the DLP style profile. It looks for sensitive data leaving, using **predefined patterns** (credit card, social security number) or **custom regular expressions**, with a **weight per pattern** and **alert and block thresholds**, so one match alerts and several in a transfer block. **Enterprise DLP** extends this as a licensed cloud service.

In practice

Blocking encrypted archives is the underrated File Blocking rule. If the firewall cannot open it, nothing downstream will inspect it either, so allowing it is a deliberate blind spot.

68 Identifying the culprit and tuning safely

Start at the Threat log. The **Type** field names the profile that fired: `virus`, `spyware`, `vulnerability`, `url`, `file`, `wildfire`, `data`. That tells you which profile to tune without guessing.

From there the method is the same for any drop: take the **threat ID**, **verify with a packet capture** rather than assuming, create an **exception scoped to that ID** restricted by zone or source, and document the justification.

Rollout approach

Deploy new profiles in **alert only mode first**, run a baseline period, review what they *would* have blocked, then move severity by severity to blocking actions.

That is not caution for its own sake. It is the only way to avoid taking out a production application with a signature you have never seen fire before.

Useful commands: `show running security-policy` to confirm which rule applied, and `debug dataplane packet-diag set capture` for the staged capture covered in the troubleshooting part.

In practice

The discipline is **evidence before action, fix narrowly, never loosen wholesale**. It is the same method as any firewall drop, and the Threat log Type field is what makes it fast here.

✓ Check yourself

profiles, part two

1. What do the URL filtering actions continue and override each do?

`continue` = **interstitial the user clicks through**. `override` = **password required, so an admin can unlock a category**.

2. What does User Credential Detection stop, and how does it detect it?

Corporate credentials submitted to external sites. By IP to user mapping, group membership, or domain credential filter: blocking the submission, not the browsing.

3. Where are the two places URL category is used?

As a match condition in a security rule (allowed at all?) and as an action in the URL Filtering profile (what happens to allowed web traffic).

4. File Blocking versus WildFire Analysis in one line.

File Blocking decides whether the file is allowed; WildFire Analysis decides whether an allowed unknown file gets detonated.

5. Why block encrypted archives?

If the firewall cannot open it, nothing downstream will inspect it either, allowing it is a deliberate blind spot.

6. Which log field names the profile that blocked something?

The Threat log Type field, virus, spyware, vulnerability, url, file, wildfire, data.

✓ Anchor facts

- ✓ URL actions: **allow, alert, block, continue, override.**
- ✓ **User Credential Detection** blocks the **submission**, not the browsing.
- ✓ URL category works **in a rule** and **in a profile**: different effects.
- ✓ File Blocking is by **type and direction**; WildFire Analysis picks what gets detonated.
- ✓ Data Filtering uses **pattern weight and thresholds**.
- ✓ Threat log **Type** names the profile. Fix with a **scoped threat ID exception**.



Easy to get wrong

- Confusing URL category **in a rule** with URL category **in a profile**.
- Allowing encrypted archives that nothing can inspect.
- Going straight to blocking on a new profile with no baseline.
- Excepting a threat ID without a packet capture to confirm it.
- Assuming WildFire Analysis blocks files. It **selects what gets detonated**.

Profile	Catches	Keyed by	Threat log type
URL Filtering	Web category, credential submission	PAN-DB category	url
File Blocking	Unwanted file types	File type + direction	file
WildFire Analysis	Unknown files for detonation	App, file type, direction	wildfire
Data Filtering	Sensitive data leaving	Pattern weight and threshold	data

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 8 · Certificates and Decryption

21

Purpose, Value and Network Design

Decryption is the densest topic in this guide, with five distinct subareas. Start with why you decrypt and where it sits in the design, because both shape every later decision.

69 The purpose and value of decryption

The argument is simple: **the overwhelming majority of traffic is encrypted**, and everything the firewall does above Layer 4, App-ID refinement, antivirus, vulnerability signatures, URL categorisation of the full path, file blocking, WildFire, data filtering, **operates on payload**. Without decryption those engines see an opaque tunnel.

What you actually get by decrypting:

Threat visibility: malware and C2 hidden inside TLS become inspectable.

Accurate App-ID, without it the application is typically just `ssl` or a guess from the certificate and SNI.

Data loss control: Data Filtering cannot see what it cannot read.

Certificate verification: expired, untrusted and self signed server certificates get caught rather than silently accepted by the user.

The costs, stated honestly

CPU and latency: decryption is expensive, so it drives platform sizing more than almost anything else.

Privacy and legal obligations: some categories must not be decrypted.

Breakage: certificate pinning and mutual TLS fail when intercepted.

The resolution is not to choose an extreme. You **decrypt selectively at a controlled inspection point**, exclude what must not or cannot be decrypted, and size the hardware for what remains.

In practice

The sentence that settles most design arguments: **you cannot enforce policy on traffic you cannot read**. The debate is then about which traffic, not whether.

The certificate objects, and what each one is for

get these straight and the decryption pages become easy

Certificate

an identity the firewall holds

generate self signed, or import

flags: **Forward Trust** · **Forward Untrust** · **CA**

Certificate Profile

how the firewall **validates others**

which CAs to trust · CRL / OCSP checks

block on unknown status or timeout

SSL/TLS Service Profile

how the firewall **presents itself**

a certificate + min and max TLS version

allowed cipher suites and algorithms

The one line distinction

a **certificate profile** validates certificates presented to the firewall · an **SSL/TLS service profile** governs certificates the firewall presents outward

The master key protects every private key on the box

it must match across an HA pair and across Panorama managed devices, changing it badly is unrecoverable

70 Certificates and PKI integration

Certificates live at `Device > Certificate Management > Certificates`. You can **generate** one on the firewall (self signed, or a CSR to be signed externally) or **import** one with its private key.

Key attributes when generating: **common name**, **signed by** (self signed or a local CA certificate), whether it is a **Certificate Authority**, the **algorithm and key size**, and **Subject Alternative Names**.

A certificate on the firewall carries **usage flags**, and these are the examinable part: **Forward Trust Certificate**, **Forward Untrust Certificate**, **Trusted Root CA**, and use for **SSL Exclude Certificate**.

PKI integration

SCEP lets the firewall obtain and renew certificates automatically from an enterprise CA, used heavily for GlobalProtect client certificates, since manual issuance does not scale.

Revocation checking is by **CRL** or **OCSP**, configured in a certificate profile. The firewall can also act as an **OCSP responder**.

The master key

Encrypts all private keys on the device. It **must match across an HA pair** and across Panorama managed firewalls. Set an expiry reminder, and be careful, a mismatched or lost master key is effectively unrecoverable.

In practice

Get the master key right before deploying certificates at scale. It is the one piece here where a mistake cannot be fixed by reissuing something.

71 Certificate profiles and SSL/TLS service profiles

These two are constantly confused, and the distinction is directional.

Certificate Profile defines how the firewall **validates certificates presented to it**. It lists the **CA certificates to trust**, enables **CRL and/or OCSP** checking, sets whether to **block sessions on an unknown certificate status** or **on a revocation check timeout**, and can require specific certificate attributes for username extraction.

Used for **administrator certificate authentication**, **GlobalProtect client certificate checks**, and **captive portal**: anywhere the firewall must decide whether someone else's certificate is acceptable.

SSL/TLS Service Profile governs how the firewall **presents itself** when it terminates TLS. It binds a **certificate** plus a **minimum and maximum TLS version** and the **allowed cipher suites and algorithms**.

Used for the **management web interface**, the **GlobalProtect portal and gateway**, and captive portal pages.

Setting **minimum TLS 1.2** on management and GlobalProtect service profiles is the standard hardening step, and where Orion's quantum work eventually lands.

In practice

Remember it directionally. **Profile in = certificate profile**. **Profile out = SSL/TLS service profile**. If the question is "should I trust this certificate", it is the former; "which certificate do I show", the latter.

✓ Check yourself

certificates and PKI

1. Why does decryption matter for App-ID specifically?

Without it the application is typically just **ssl** or a guess from the certificate and **SNI**, refinement needs payload.

2. Name the three costs of decryption.

CPU and latency, privacy and legal obligations, and breakage from certificate pinning and mutual TLS.

3. Distinguish a certificate profile from an SSL/TLS service profile.

Certificate profile validates certificates presented to the firewall; SSL/TLS service profile governs certificates the firewall presents outward.

4. What is SCEP used for, and why does it matter?

Automatic certificate issuance and renewal from an enterprise CA, essential for GlobalProtect client certificates, since manual issuance does not scale.

5. What does the master key protect, and what must be true of it?

All private keys on the device. It must match across an HA pair and Panorama managed firewalls. Losing it is unrecoverable.

6. Name the certificate usage flags that matter for decryption.

Forward Trust, Forward Untrust, Trusted Root CA, and SSL Exclude Certificate.

✓ Anchor facts

- ✓ You cannot enforce policy on traffic you **cannot read**.
- ✓ Certificate flags: **Forward Trust, Forward Untrust, Trusted Root CA, SSL Exclude**.
- ✓ **Certificate profile = validates inbound. SSL/TLS service profile = presents outward.**
- ✓ Revocation via **CRL or OCSP**; the firewall can be an OCSP responder.
- ✓ **SCEP** automates issuance and renewal from an enterprise CA.
- ✓ The **master key** must match across HA and Panorama managed devices.



Easy to get wrong

- Swapping certificate profile and SSL/TLS service profile.
- Changing the master key without planning. It is unrecoverable.
- Issuing GlobalProtect client certificates manually instead of via SCEP.
- Leaving revocation check failure set to allow.
- Sizing a platform without accounting for decryption CPU.

Object	Direction	Answers	Used by
Certificate	:	An identity the firewall holds	Everything below
Certificate profile	Inbound	Should I trust this certificate?	Admin auth, GP client certs, portal
SSL/TLS service profile	Outbound	Which certificate do I present?	Web UI, GP portal and gateway
Master key	,	Encrypts all private keys	The whole device

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 8 · Certificates and Decryption

22

Forward Proxy, Forward Trust and Untrust

The outbound case, and the mechanism worth knowing explicitly. Forward trust versus forward untrust is the single most important pair on this page.

72 SSL Forward Proxy, and the two certificates it needs

Used for **outbound** traffic: your users going to the internet. The firewall acts as a **man in the middle you control**, terminating the client's session and opening its own to the server.

The flow

The client connects. The firewall opens its own TLS session to the destination and **inspects the real server certificate**. It then **generates a certificate on the fly** for that site and presents it to the client. Two separate TLS sessions exist, and the firewall sees plaintext between them.

Forward Trust Certificate

Used when the **real server certificate is valid and trusted**. The firewall signs the impersonated certificate with this CA. Because **this CA is installed in your clients' trust stores**, the user sees no warning and browsing is seamless.

Forward Untrust Certificate

Used when the **real server certificate is invalid**: expired, self signed, untrusted issuer, name mismatch. The firewall signs with a CA that is deliberately **not** in client trust stores, so **the user gets a browser warning**.

That design is the clever part: it **preserves the trust signal**. Without two certificates, every site would appear valid to the user because the firewall resigned it, and you would have destroyed the very warning the browser exists to give.

Both must be flagged as **Certificate Authority**. The forward trust CA is distributed to clients by **GPO or MDM**; the forward untrust CA **must never be distributed**.

In practice

If users report no warnings on genuinely broken sites, someone has distributed the **forward untrust** CA to the trust store. That silently removes a protection nobody will notice until it matters.

Forward proxy, and why there are two CAs

the trust signal is preserved, not replaced



Server certificate VALID

firewall signs with the **Forward Trust CA**
that CA is in the client trust store
→ **no warning, seamless**

Server certificate INVALID

firewall signs with the **Forward Untrust CA**
that CA is **deliberately not** in the trust store
→ **the user gets a warning**

With one CA everything would look valid to the user. You would have **destroyed the browser warning**. Two CAs preserve it.

distribute the **trust CA** by GPO or MDM

never distribute the untrust CA

73 Subordinate CA from enterprise PKI

This is the **recommended** way to run forward proxy in an enterprise, and worth calling out explicitly.

The problem with a self signed forward trust CA: it is a brand new root that every client must be told to trust. You have to distribute it to every managed device, every BYOD device, every appliance and every application with its own trust store, and anything you miss breaks.

The subordinate CA approach: the firewall's forward trust certificate is **signed by your existing enterprise root CA** rather than being self signed. Because **clients already trust that root**, they automatically trust the firewall's certificates with **no distribution step at all**.

How you do it: generate a **CSR** on the firewall with the **Certificate Authority** flag set, have your enterprise CA sign it as a **subordinate CA**, import the signed certificate back, and flag it as **Forward Trust Certificate**.

The trade off to state

The firewall now holds a **subordinate CA key** that can issue certificates trusted across your organisation. That is a significant concentration of trust, so it needs **appropriate physical and administrative controls** and careful master key handling.

In practice

This is the difference between a decryption rollout that takes weeks and one that works on day one. If the question mentions an existing enterprise PKI, subordinate CA is almost always the expected answer.

74 SSL Inbound Inspection

The **inbound** case: internet users reaching **your** published servers. Mechanically different from forward proxy, and that difference is the exam point.

There is **no impersonation and no certificate generation**. Instead you **import the server's actual certificate and private key onto the firewall**. The firewall then decrypts the session passively using that key, inspects, and forwards.

Because it holds the real key, the client's TLS session is **with the real server certificate**, so the client sees nothing unusual and no CA distribution is needed anywhere.

Operational consequences

Whenever the **server certificate is renewed**, the new certificate and key **must be reimported** to the firewall. Miss that and inspection silently fails after renewal.

It also imposes a cryptographic constraint: **ephemeral key exchange such as DHE or ECDHE defeats passive key based decryption**, because the session key is not derivable from the server's private key. In those cases the firewall must proxy the session instead.

In practice

The contrast worth memorising: **forward proxy generates certificates; inbound inspection uses the real key**. And add certificate renewal on protected servers to your change checklist, because it is the usual cause of inbound inspection quietly stopping.

✓ Check yourself

forward proxy and inbound

1. Why does forward proxy need two CA certificates?

To preserve the trust signal. Valid sites are signed with the trust CA (no warning); invalid ones with the untrust CA (warning shown).

2. Which CA must never be distributed to clients, and what breaks if it is?

The forward untrust CA. Distributing it means broken sites stop producing warnings.

3. What problem does a subordinate CA solve?

No CA distribution. The forward trust certificate is signed by your existing enterprise root, which clients already trust.

4. What is the trade off of the subordinate CA approach?

The firewall holds a key that can issue org wide trusted certificates: a concentration of trust needing real controls.

5. How does inbound inspection differ mechanically from forward proxy?

No impersonation. You import the server's real certificate and private key; the firewall decrypts passively.

6. Why can ephemeral key exchange defeat inbound inspection?

With DHE or ECDHE the session key is not derivable from the server private key, so the firewall must proxy instead.

✓ Anchor facts

- ✓ Forward proxy = **outbound**, two TLS sessions, certificate generated on the fly.
- ✓ **Forward Trust** for valid server certs, **Forward Untrust** for invalid ones.
- ✓ Both must be flagged **Certificate Authority**.
- ✓ **Subordinate CA** from your enterprise root removes the distribution step entirely.
- ✓ Inbound inspection = **import the real certificate and key**, no impersonation.
- ✓ **Ephemeral key exchange** defeats passive inbound decryption.



Easy to get wrong

- Distributing the **untrust** CA and silently killing browser warnings.
- Self signing the forward trust CA when an enterprise PKI exists.
- Forgetting to reimpor a renewed server certificate for inbound inspection.
- Expecting inbound inspection to work with ECDHE.
- Omitting the **Certificate Authority** flag on a forward certificate.

	SSL Forward Proxy	SSL Inbound Inspection
Direction	Outbound: your users out	Inbound: internet to your servers
Certificate handling	Generated on the fly	Real certificate and key imported
Needs CA distribution?	Yes, unless subordinate CA	No
Client sees	The firewall's certificate	The real server certificate
Breaks on	Pinning, mutual TLS	Renewal, ephemeral key exchange

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 8 · Certificates and Decryption

23

Policy, Profiles, Exclusions and SSH Proxy

Deciding what to decrypt, controlling how, and handling the traffic that must be left alone. Exclusions are not a workaround. They are part of the design.

75 Decryption policy and SSH Proxy

Decryption policy at [Policies › Decryption](#) decides **which traffic is decrypted**. Like every rulebase it is **top down, first match**.

Match criteria: **source and destination zone and address, source user, service, and URL category**. URL category is the important one. It is how you exclude finance and health categories for privacy or legal reasons while decrypting everything else.

Rule types: **SSL Forward Proxy, SSL Inbound Inspection, SSH Proxy, and No Decrypt**.

SSH Proxy

Decrypts SSH to detect **SSH tunnelling**: port forwarding and X11 forwarding used to smuggle other protocols inside an allowed SSH session. It does not inspect the shell session content for threats; the purpose is to identify and control tunnelled traffic.

No certificates are needed for SSH proxy: the firewall generates its own keys. But clients will see a **changed host key**, which needs communicating in advance or it looks exactly like an attack.

Verify with [show session all filter ssl-decrypt yes](#) and [show counter global filter aspect ssl](#).

In practice

Build the decryption rulebase as **explicit No Decrypt rules first**, then a broad decrypt rule. That way the exclusions are visible at the top rather than buried, and a reviewer can see exactly what is not inspected.

76 Decryption profiles

A **decryption profile** attaches to a decryption rule and controls **how** decryption behaves. It has three sections.

SSL Decryption

Forward Proxy checks: block sessions with **expired certificates**, **untrusted issuers**, **unknown certificate status**, **timed out revocation checks**, **unsupported cipher suites or versions**, and **client authentication** (mutual TLS).

Inbound Inspection checks: unsupported modes and versions.

No Decryption checks: even on traffic you do not decrypt, you can still **block expired or untrusted certificates**: a useful control people forget exists.

SSH Proxy

Block on unsupported version or algorithm, and on SSH errors.

Protocol Settings

Minimum and maximum TLS version, allowed **key exchange**, **encryption and authentication algorithms**. This is where you refuse weak cryptography outright.

The **No Decryption** section is worth calling out: it lets you enforce certificate hygiene on excluded traffic without reading it.

In practice

Apply a decryption profile to your **No Decrypt** rules too. Just because you are not inspecting the payload does not mean you should accept an expired or untrusted certificate.

What must be excluded, and the three ways to do it

exclusions are part of the design,
not a failure of it

Traffic that breaks when decrypted

certificate pinning: app rejects any substituted cert

mutual TLS: client certificate cannot be proxied

privacy and legal categories, finance, health

Three ways to exclude

No Decrypt rule, by zone, address, user, URL category

SSL Decryption Exclusion list, by hostname

predefined exclusions: Palo Alto ships a list

Apply a decryption profile to No Decrypt rules too

you can still block **expired and untrusted certificates** on traffic you never read

Build the rulebase exclusions first

explicit **No Decrypt** rules at the top, then a broad decrypt rule, so what is not inspected is visible, not buried

77 Exclusions, pinning and mutual TLS

Certificate pinning is an application that embeds the expected server certificate or its issuer and **rejects anything else**, including your forward proxy certificate, even though clients trust your CA. Banking apps, many mobile apps and some software updaters do this. There is **no fix**; you must exclude them.

Mutual TLS means the client presents its own certificate. A proxy cannot reproduce the client's private key, so the handshake fails. Also excluded.

Three exclusion mechanisms

No Decrypt rule, by zone, address, user or URL category.

SSL Decryption Exclusion list at [Device > Certificate Management > SSL Decryption Exclusion](#), by hostname, for specific applications.

Predefined exclusions: Palo Alto maintains a list of known broken applications, updated with content.

Legal and privacy exclusions are a separate category and are usually done by **URL category** in a No Decrypt rule, which is auditable and easy to justify.

In practice

Check the predefined exclusion list before writing your own. Most of the applications that break under decryption are already on it, and reproducing it by hand is wasted effort that goes stale.

78 Decryption broker and mirroring

Both solve the same problem: **other security tools also want to see the plaintext**, and you do not want each one decrypting independently.

Decryption Broker makes the firewall the **single decryption point** and forwards the decrypted traffic to one or more **inline third party security chains**, then reencrypts on the way back out. The firewall becomes the decryption service for the whole stack.

That removes duplicated decryption cost, gives you one place to manage certificates and exclusions, and means the other tools do not each need their own CA in the trust store.

Decryption Mirroring is simpler and **passive**: a copy of the decrypted traffic is sent out a dedicated **Decrypt Mirror** interface to a DLP system, IDS or packet capture appliance. It is a tap, not a chain, the mirrored copy cannot block anything.

Decrypt Mirror requires a supporting platform and a free licence, and carries obvious **privacy implications** since you are exporting plaintext.

In practice

The distinction: **broker is inline and can block**; **mirroring is passive and can only observe**. Same as the tap versus inline logic from the interfaces part.

✓ Check yourself

policy, profiles and exclusions

1. Name the four decryption rule types.

SSL Forward Proxy, SSL Inbound Inspection, SSH Proxy, No Decrypt.

2. What is SSH Proxy actually for?

Detecting SSH tunnelling: port and X11 forwarding smuggling protocols inside an allowed SSH session.

3. Why does certificate pinning have no workaround?

The app embeds the expected certificate and rejects any substitute, even from a trusted CA. You must exclude it.

4. Name the three exclusion mechanisms.

No Decrypt rule, the SSL Decryption Exclusion list by hostname, and Palo Alto's predefined exclusions.

5. What can a decryption profile still enforce on No Decrypt traffic?

Certificate hygiene: block expired and untrusted certificates without reading the payload.

6. Broker versus mirroring?

Broker is inline and can block; mirroring is passive and can only observe.

✓ Anchor facts

- ✓ Decryption rule types: **Forward Proxy, Inbound Inspection, SSH Proxy, No Decrypt.**
- ✓ Match on **zone, address, user, service and URL category.**
- ✓ Profile sections: **SSL Decryption, SSH Proxy, Protocol Settings.**
- ✓ The **No Decryption** section enforces certificate checks without reading payload.
- ✓ **Pinning and mutual TLS cannot be proxied:** exclude them.
- ✓ **Broker = inline chain. Mirroring = passive copy.**



Easy to get wrong

- Trying to "fix" certificate pinning instead of excluding it.
- Leaving No Decrypt rules with no decryption profile attached.
- Hand writing exclusions already in the predefined list.
- Enabling SSH proxy without warning users about the changed host key.
- Expecting mirrored traffic to block anything.

Exclusion method	Granularity	Maintained by	Best for
No Decrypt rule	Zone, address, user, URL category	You	Privacy and legal categories
SSL Decryption Exclusion	Hostname	You	A specific broken application
Predefined exclusions	Hostname	Palo Alto, via content	Known broken apps, check first

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 8 · Certificates and Decryption

24

Best Practices and Troubleshooting

How to roll decryption out without a support queue, and how to diagnose it when a single site fails. Both are named course sections.

79 Deploying decryption without breaking the business

Decryption breaks more things than any other feature, so the rollout method matters as much as the configuration.

1 · Prepare the trust

Use a **subordinate CA signed by your enterprise root** if you have one. If not, distribute the self signed forward trust CA by **GPO or MDM first**, and account for the trust stores that are not Windows: macOS, Linux, Java, Python, mobile devices, appliances.

2 · Start narrow

Begin with a **pilot group** and a **limited URL category set**, not everything at once. Exclude privacy sensitive categories from the outset so you never have to explain why you read them.

3 · Expand by category

Add categories in tranches, watching for breakage after each. Applications fail loudly and specifically, so incremental expansion localises the cause.

4 · Size for it

Decryption is CPU intensive. Check `show running resource-monitor` as coverage grows, and know the platform's decryption throughput figure rather than its App-ID one.

Standing practices

Check the **predefined exclusion list** before hand writing exclusions · attach a **decryption profile to No Decrypt rules** · set a **minimum TLS version** and refuse weak algorithms · keep exclusions **documented and reviewed**, since a temporary exclusion becomes permanent by default.

In practice

The failure mode of a rushed rollout is a support queue full of unrelated looking complaints. Pilot group, tranches by category, and communicate before enabling, particularly for SSH proxy, where the changed host key looks like an attack.

One site fails under decryption work down this list rather than disabling the rule

1 · Is it pinning or mutual TLS?, if so there is no fix; add it to the exclusion list

2 · Check the **Decryption log**: it names the error and the certificate involved

3 · Is the failure a **profile check**? expired, untrusted issuer, unsupported cipher or version

4 · Does the client trust the **forward trust CA**? check that device's trust store, not just Windows

5 · For inbound: was the server certificate **renewed** and not reimported?

6 · Confirm with counters: **show counter global filter aspect ssl**

Fix it with a **targeted exclusion**, never by disabling the decryption rule for everyone

80 Troubleshooting decryption

The **Decryption log** at `Monitor › Logs › Decryption` is the starting point. It records each decrypted session with the **error reason**, the **TLS version and cipher suite**, and the **certificate details**, which usually names the problem outright.

Useful commands:

`show session all filter ssl-decrypt yes` , which sessions are being decrypted

`show counter global filter aspect ssl` : decryption counters and failure reasons

`show system setting ssl-decrypt certificate` : the forward certificates in use

`debug dataplane show ssl-decrypt ssl-stats` : decryption statistics

The common causes, in the order they occur

Pinning or mutual TLS: excluded, not fixed.

Client does not trust the forward trust CA: check that device's own trust store.

A profile check is blocking: expired, untrusted issuer, unsupported cipher or version.

Inbound certificate renewed and not reimported.

Resource exhaustion: decryption CPU maxed, showing as intermittent failures.

Fix with a **targeted exclusion** for the affected hostname or category, never by disabling the rule for everyone. Same discipline as threat exceptions.

In practice

"It works on my laptop but not on the tablet" is a trust store problem, not a firewall one. Non Windows devices and applications with their own bundled trust stores are where decryption rollouts actually stumble.

81 Decryption and network design

The course treats network design as its own topic, and the question it answers is **where in the path decryption should happen**.

One controlled inspection point is the goal. Decrypting in several places multiplies CPU cost, certificate management and exclusion lists, and creates places where plaintext exists that nobody is tracking.

Design consequences worth knowing:

Place decryption where the traffic is aggregated: typically the internet edge for outbound, and in front of the server farm for inbound.

Avoid asymmetric paths. Decryption requires the firewall to see **both directions** of a session; asymmetric routing breaks it outright.

Account for the doubled session cost: forward proxy maintains two TLS sessions per client session.

Virtual wire and Layer 2 deployments can decrypt, so transparent insertion does not rule it out.

Where other tools need plaintext, use **decryption broker** rather than letting each tool decrypt for itself, the design argument for the single inspection point again.

In practice

Asymmetry is the design trap. A path that works for stateful inspection may still break decryption, because seeing one direction of a TLS session is useless.

✓ Check yourself

best practices and troubleshooting

1. What is the first question when one site fails under decryption?

Is it pinning or mutual TLS? If so there is no fix, exclude it.

2. Which log names the error, cipher and certificate?

The Decryption log at Monitor › Logs › Decryption.

3. Which command shows decryption counters and failure reasons?

`show counter global filter aspect ssl`.

4. "Works on my laptop, not on the tablet." What is it?

A trust store problem on that device, not a firewall fault. Non Windows and app bundled stores are the usual gap.

5. Why does asymmetric routing break decryption specifically?

Decryption needs both directions of the session. Seeing one half is useless, even though stateful inspection might cope.

6. Why prefer a single decryption point?

Decrypting in several places multiplies CPU, certificate management and exclusions, and spreads plaintext nobody tracks.

✓ Anchor facts

- ✓ Roll out by **pilot group**, then **category tranches**.
- ✓ Non Windows and app bundled **trust stores** are the usual failure.
- ✓ The **Decryption log** gives error, TLS version, cipher and certificate.
- ✓ `show counter global filter aspect ssl` for failure reasons.
- ✓ Decryption needs **both directions**: asymmetry breaks it.
- ✓ Forward proxy holds **two TLS sessions** per client session.



Easy to get wrong

- Disabling the decryption rule to fix one site.
- Distributing the CA to Windows only.
- Enabling everything at once instead of by tranche.
- Letting temporary exclusions become permanent undocumented ones.
- Decrypting in several places instead of brokering from one.

Symptom	Likely cause	Action
One app fails, others fine	Certificate pinning	Add to the SSL Decryption Exclusion list
Handshake fails with a client cert	Mutual TLS	No Decrypt rule
Warnings on every site, one device	Trust store missing the CA	Deploy the CA to that platform
Inbound inspection stopped silently	Server certificate renewed	Reimport certificate and key
Intermittent failures under load	Decryption CPU exhaustion	Check resource-monitor; reduce scope or resize

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 8 · Certificates and Decryption

25

Detecting and Decrypting Post Quantum Traffic

Everything on the rest of this page applies unchanged to PQC, the same two decryption modes, the same profiles. What's new is a way to know PQC is in use without decrypting, and a way to give it to clients without upgrading servers.

82 Detecting PQC use in traffic: threat signatures that don't need to break the encryption

This is a genuinely different capability from everything else on this page: these signatures identify **which cryptographic algorithm a session negotiated**, without needing decryption to succeed or even be attempted. Content release **version 8692, March 2023**, added 25 new vulnerability signatures for exactly this.

What the signatures actually detect

Two families, all category **info leak**, default action **alert**, minimum PAN-OS **9.1.0**:

- **Pure post quantum ciphers**: signatures for **Crystals-Kyber, BIKE, FrodoKEM, HQC** detected in use.
- **Hybrid classical plus PQ combinations**: e.g. **ECDH and Crystals-Kyber, ECDH and HQC, ECDH and BIKE, ECDH and FrodoKEM** detected together in the same negotiation.
- **Signature algorithms too**: **Crystals-Dilithium, Falcon, SPHINCS-Plus**, and their hybrid pairings with **ECDSA or RSA**.

Why this matters even before any PQC policy exists

This is the concrete answer to the unsanctioned PQC problem raised back in Part 1: a signature firing on **"Use of Post Quantum Cyphers ... detected"** tells you a session used PQC **whether or not you sanctioned it**, purely from observable handshake behaviour, no decryption required.

What it looks like in the logs

A Threat log entry names the exact cipher, e.g. **Use of Post Quantum Cyphers Crystals-Kyber detected**, severity **informational**. Cross referencing the matching **Decryption log** for the same session shows the mechanics directly: **TLS 1.3**, key exchange algorithm **kyber512**, authentication algorithm **SHA384**: the signature and the decryption log corroborate each other.

In practice

These signatures work on traffic you are **not decrypting** at all. That's the entire point. A no-decrypt policy with logging enabled, paired with these signatures, is how you inventory PQC usage across an estate before deciding on any decryption or migration policy.

83 Decrypting PQC traffic: the same two modes, plus a translation trick neither of them needed before

Decrypting PQC traffic follows **exactly the same principles** as decrypting standard traffic, the same two modes from earlier on this page apply unchanged:

- **SSL Forward Proxy** secures **known clients and host machines** going out to the internet, HQ, branch, mobile, remote users.
- **SSL Inbound Inspection** secures **data center and cloud hosted applications** from unknown clients arriving from the internet.

Hybrid PQC forward proxy, walked through

An internal client establishes a session with the firewall negotiating a **hybrid PQC cipher**, e.g.

x25519_mlkem768: classical X25519 combined with post quantum ML-KEM-768. The firewall decrypts to apply policy, then opens a **separate, new connection** to the actual external server, **also negotiating x25519_mlkem768**. Two distinct hybrid PQC sessions, managed independently, protect the entire path against both classical and quantum threats while the firewall keeps full visibility.

Cipher translation for PQC specifically

This is the genuinely new trick: on **inbound inspection**, the firewall can decrypt external client traffic using the **hybrid post quantum cipher** the client offered, then **reencrypt to the internal server using only the server's own classical cipher**: X25519 alone, no PQC involved on that leg. External clients get quantum resistant protection **immediately**, with **zero upgrade required on internal servers**, enabling a genuinely phased migration rather than a synchronised cutover.

Where this is configured

The **Decryption Profile's** SSL Protocol Settings tab gets a new **Key Exchange Algorithms** section, alongside the existing Classical one:

- **PQC-Standard** and **PQC-Experimental**: separate checkboxes, enabled independently.
- **Preferred Session Types**: Post Quantum SSL preferred for the **client side** session, and separately for the **server side** session.

One operational detail worth knowing: for **unsupported modes or failures**, session information is **cached for 12 hours**, so a repeat connection between the same host and server pair during that window is **not decrypted again**: check those boxes to force blocking those sessions instead if that caching behaviour isn't wanted.

In practice

Cipher translation is the detail worth having ready: a firewall can offer PQC to the outside world today without touching a single internal server, because the PQC leg and the classical leg are two separate, independently managed sessions either side of the firewall, the same forward proxy and inbound inspection pattern already covered, just carrying a different cipher on one side.

✓ Check yourself

PQC detection and decryption

1. Do the PQC threat signatures require decryption to succeed?

No. **They identify the negotiated algorithm from observable handshake behaviour alone.**

2. What content release version added the PQC detection signatures, and when?

Version 8692, March 2023: **25 new vulnerability signatures.**

3. In the hybrid PQC forward proxy example, how many separate hybrid PQC sessions does the firewall manage?

Two: client to firewall and firewall to server, independently negotiated.

4. What does cipher translation let a firewall do for inbound PQC traffic?

Decrypt the external hybrid PQC session, then reencrypt to the internal server using only its classical cipher, no server upgrade needed.

5. Where are PQC-Standard and PQC-Experimental configured?

The Decryption Profile's **SSL Protocol Settings**, under **Key Exchange Algorithms**.

6. What happens to session information for unsupported PQC modes or failures?

It's cached for 12 hours: a repeat connection in that window is not decrypted again unless blocking is explicitly configured.

✓ Anchor facts

- ✓ PQC threat signatures work on **undecrypted** traffic, behavioural detection only.
- ✓ Signatures cover pure PQC (Kyber, BIKE, FrodoKEM, HQC) and hybrid classical+PQ combos.
- ✓ Hybrid forward proxy = **two independent** hybrid PQC sessions either side of the firewall.
- ✓ **Cipher translation**: external PQC in, internal classical out, no server upgrade.
- ✓ PQC-Standard / PQC-Experimental are separate, independent checkboxes on the Decryption Profile.
- ✓ Unsupported mode sessions cache for **12 hours** before reevaluation.



Easy to get wrong

- Assuming PQC detection requires a decrypt policy. It doesn't.
- Thinking cipher translation means the internal server also runs PQC. It never has to.
- Missing that client side and server side PQC preference are configured separately.
- Forgetting the 12 hour cache can mask a fix until it expires or is explicitly cleared.

Capability	Needs decryption?	Configured on
PQC threat signatures	No	Vulnerability Protection Profile
Hybrid PQC forward proxy	Yes	Decryption Policy + Profile
Cipher translation	Yes (inbound)	Decryption Profile, SSL Inbound Inspection
PQC-Standard / Experimental	N/A, toggle	Decryption Profile › SSL Protocol Settings

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | **CONFIGURE** | VERIFY

Part 9 · Authentication and Identity 26

Server Profiles, Authentication Profiles and Sequences

This page covers roles, profiles and sequences together. They stack in a specific order, and knowing which layer does what makes every later identity topic easier.

84 The authentication stack: server profile, authentication profile, sequence

Three layers, each wrapping the one below. Getting them in order is most of the difficulty here.

1 · Server profile: where to ask

At `Device > Server Profiles`. Defines **how to reach an authentication server**: addresses, port, timeouts and any bind credentials. Types are **LDAP**, **RADIUS**, **TACACS+**, **Kerberos**, **SAML Identity Provider** and the local database.

2 · Authentication profile: how to ask

At `Device > Authentication Profile`. References **one server profile** and adds the logic around it: the **allow list** of permitted users and groups, **username modifiers** (whether to strip or add a domain), **account lockout** settings, a **certificate profile** where certificates are involved, and **MFA factors**.

3 · Authentication sequence: who to ask, in order

An **ordered list of authentication profiles**, tried **top down** until one succeeds. This is how you support several identity sources from one login prompt, try the primary directory first, then a secondary, then local.

The behaviour worth memorising

A sequence moves to the next profile when a server is **unreachable**. Whether it moves on after an **explicit reject** depends on the configuration, so the order should be deliberate rather than arbitrary, put the authoritative source first.

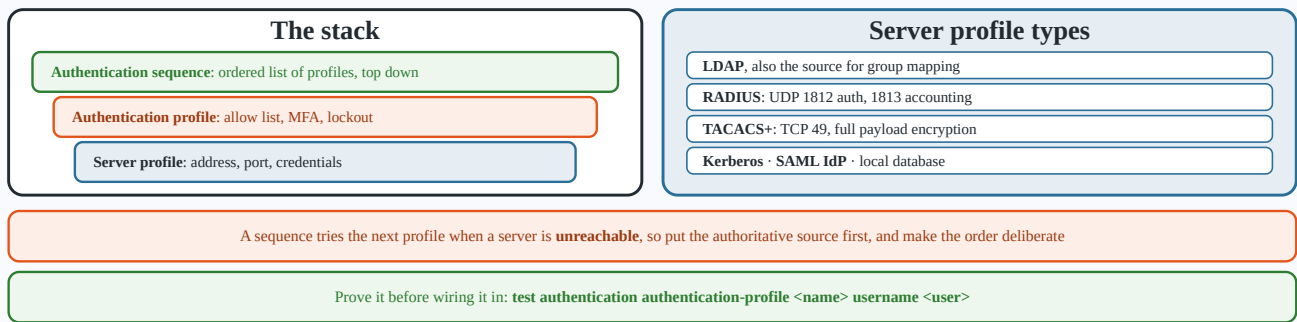
Verify with `test authentication authentication-profile <name> username <user> password`, which is the fastest way to prove a profile works before wiring it into anything.

In practice

Remember it as **where, how, who in what order**. Server profile = where the server is. Authentication profile = how to use it. Sequence = which profiles to try and in what order.

The three layers, and the protocols underneath

each wraps the one below it



85 The authentication protocols compared

LDAP: port **389**, or **636** for LDAPS. Used both for authentication and, importantly, as the source for **group mapping**. A bind account is needed to read the directory.

RADIUS: **UDP 1812** for authentication and **1813** for accounting. Encrypts **only the password**, not the rest of the packet. Commonly fronts MFA systems, and can return **vendor specific attributes** to assign an admin role.

TACACS+: **TCP 49**, and encrypts the **entire payload**. It separates authentication, authorisation and accounting, which is why it is preferred for **device administration** where per command authorisation matters.

Kerberos: ticket based against a KDC, used for single sign on. This is what Orion's passwordless feature builds on, with the firewall acting as a **KCD agent**.

SAML: browser redirect federation to an identity provider. The firewall is the **service provider**; the IdP handles the actual authentication and any MFA. Now the common choice for **GlobalProtect and administrator access**, because MFA and conditional access live at the IdP rather than on the firewall.

In practice

The pair worth stating precisely: **RADIUS encrypts only the password over UDP**; **TACACS+ encrypts the whole payload over TCP 49**. That plus AAA separation is why TACACS+ wins for device administration.

86 Administrator roles and access control

Administrators are configured at `Device > Administrators`, authenticating against the **local database**, an **authentication profile**, or a **certificate profile** for certificate based login.

Role types

Dynamic roles are built in and maintained by PAN-OS: **superuser**, **superuser (read only)**, **device administrator**, **device administrator (read only)**, and on multi-vsyt platforms **vsyt administrator** and its read only form.

Admin Role profiles are custom, letting you enable or disable each individual **web interface tab**, **CLI access level** and **XML API capability**. This is how you give a team visibility into logs without configuration rights.

Remote authentication can carry the role: RADIUS **vendor specific attributes**, TACACS+ attributes, or SAML attributes can map a user directly to an admin role, so role assignment lives in the directory rather than on each firewall.

Hardening around this: set **minimum password complexity** and **failed attempt logout**, restrict management access by **permitted IP**, and enable **Panorama based authentication** so leavers are removed centrally.

In practice

Custom Admin Role profiles are what make least privilege real on a firewall. Everyone being superuser is the default drift, and it is the first thing an audit picks up.

✓ Check yourself

the authentication stack

1. Name the three layers and what each one defines.

Server profile = **where the server is**. Authentication profile = **how to use it**. Sequence = **which profiles to try, in order**.

2. When does a sequence move to the next profile?

When a server is unreachable. Behaviour on explicit reject depends on configuration, order deliberately.

3. RADIUS versus TACACS+ in one line.

RADIUS encrypts only the password over UDP 1812/1813; TACACS+ encrypts the whole payload over TCP 49 and separates AAA.

4. Which protocol is preferred for device administration, and why?

TACACS+: full payload encryption plus AAA separation for per command authorisation.

5. What is the firewall's role in SAML, and what does that buy you?

The firewall is the service provider; the IdP authenticates. MFA and conditional access live at the IdP, not on the firewall.

6. How do you grant log visibility without configuration rights?

A custom Admin Role profile enabling only the required tabs, CLI level and API capabilities.

✓ Anchor facts

- ✓ Stack order: **server profile** → **authentication profile** → **sequence**.
- ✓ LDAP **389 / 636** · RADIUS **UDP 1812 / 1813** · TACACS+ **TCP 49**.
- ✓ **RADIUS = password only**. TACACS+ = **full payload**.
- ✓ Sequences are tried **top down**, next on unreachable.
- ✓ Dynamic roles are built in; **Admin Role profiles** are custom and granular.
- ✓ RADIUS VSAs, TACACS+ and SAML attributes can carry the **admin role**.



Easy to get wrong

- Confusing which layer holds the **allow list**: it is the authentication profile.
- Putting a fallback source first in a sequence.
- Using RADIUS for device administration where per command authorisation is needed.
- Leaving every administrator as superuser.
- Forgetting username modifiers when the directory expects a domain prefix.

Protocol	Transport and port	Encrypts	Best for
LDAP	TCP 389 / 636	TLS if LDAPS	Auth and group mapping
RADIUS	UDP 1812 / 1813	Password only	MFA front end, user auth
TACACS+	TCP 49	Entire payload	Device administration
Kerberos	UDP/TCP 88	Tickets	SSO, KCD passwordless
SAML	HTTPS redirect	TLS	GlobalProtect, admin SSO

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 9 · Authentication and Identity 27

Authentication Policy, MFA and Passwordless

Authentication profiles say how to authenticate. Authentication policy decides when to demand it, and passwordless removes the credential from the exchange entirely.

87 Authentication Policy and the authentication portal

Authentication Policy at [Policies › Authentication](#) triggers an authentication challenge for traffic that matches, which serves two purposes: **identifying users** User-ID could not map, and **demanding step up authentication** before sensitive resources.

It matches on **source and destination zone and address, service, URL category**, and references an **Authentication Enforcement** object, which in turn points at an authentication profile and a method.

Methods

browser challenge: transparent NTLM or Kerberos, no prompt if the browser can respond.

web form: an explicit login page.

no-captive-portal: matched traffic bypasses the challenge.

The **authentication portal** (historically captive portal) is configured at

[Device › User Identification › Authentication Portal Settings](#) with a mode of **redirect** or **transparent**. **Redirect mode is preferred** because it supports authentication cookies and works far better with modern browsers.

A successful challenge creates a **user to IP mapping**, so it feeds User-ID rather than being separate from it. The **timeout** controls how long before the user is challenged again.

In practice

Step up is the more interesting use. You allow a finance application generally but demand fresh MFA at the point of access, rather than trusting a login that happened at 8am.

88 Multi factor authentication

PAN-OS integrates MFA two ways, and the distinction matters.

Through RADIUS or SAML: the firewall simply authenticates against a provider that happens to enforce MFA. The firewall knows nothing about the factors. This is the common path for **GlobalProtect and administrator access**.

Native MFA integration: an **MFA Server Profile** at `Device > Server Profiles > MFA`, with vendor integrations, attached to an authentication profile. Up to **four factors** can be chained, and this path is what **Authentication Policy** uses for step up challenges inside the network.

The distinction: RADIUS and SAML MFA covers **access to the firewall or VPN**; native MFA with Authentication Policy covers **access to resources through the firewall**, which is where step up belongs.

Design notes: exempt service accounts and machine traffic that cannot respond to a challenge, and remember **MFA cannot apply to non browser traffic** that has no way to present a prompt.

In practice

"MFA on the VPN" and "MFA before the finance app" are different mechanisms. The first is RADIUS or SAML at the gateway; the second is Authentication Policy with an MFA profile.

Two different MFA questions

getting into the network, versus reaching a resource inside it

Getting in

GlobalProtect · admin login

MFA enforced by the **RADIUS or SAML provider**

the firewall knows nothing about the factors

authentication profile → server profile

Reaching a resource

step up before a sensitive app

Authentication Policy triggers the challenge

native **MFA server profile**, up to 4 factors

a success also creates a user to IP mapping

Passwordless: the firewall as a Kerberos Constrained Delegation agent

it obtains a ticket on the user's behalf, so Kerberos protected apps are reached without reentering credentials

Why it is called phishing resistant

a credential that is never typed into an application cannot be phished from it or replayed

89 Passwordless authentication

Introduced with **PAN-OS 12.1 Orion** and described as **phishing resistant**, for a straightforward reason: a credential that is **never typed into an application** cannot be phished from it or replayed.

The mechanism

Passwordless Authentication for Kerberos Protected Applications. The firewall acts as a **Kerberos Constrained Delegation (KCD) agent**, obtaining a Kerberos ticket **on the user's behalf** so they reach Kerberos protected enterprise applications **without repeatedly entering credentials**.

Two benefits follow. **Security:** fewer password related vulnerabilities, because individual applications are not handling credentials. **Usability:** access to multiple Kerberos protected applications is streamlined behind a single authentication event.

It complements rather than replaces the rest of this part. You still need authentication profiles, you may still use SAML for GlobalProtect, and MFA still applies at the initial authentication.

The connection worth drawing

Pair it with **User Credential Detection** in URL Filtering. One feature stops users typing corporate credentials into external sites; passwordless removes the credential from the flow altogether. Two angles on the same problem.

In practice

Know the shape rather than the configuration: **the firewall gets the ticket for the user via KCD**. That one sentence covers what the course section is teaching.

90 Certificate based authentication

Certificates authenticate **without a password**, which is why they underpin most large GlobalProtect deployments and hardened administrator access.

The firewall validates a presented certificate using a **certificate profile**, which names the trusted CAs, enables **CRL or OCSP** revocation checking, and defines how to extract the **username** from a certificate field, typically subject, common name or a SAN.

For administrators: assign a certificate profile to the admin account so login requires the certificate rather than a password.

For GlobalProtect: machine or user certificates prove the device is corporate managed. Combined with SAML for the user, this gives you **device identity plus user identity**, which is the usual Zero Trust posture.

At scale the issuing problem dominates, which is why **SCEP** matters, certificates are enrolled and renewed automatically from the enterprise CA rather than by hand.

In practice

Set revocation checking to **block on unknown status** deliberately. Leaving it permissive means a revoked certificate keeps working, which defeats the point of having revocation at all.

✓ Check yourself

authentication policy and MFA

1. What two purposes does Authentication Policy serve?

Identifying users **User-ID could not map**, and step up authentication **before sensitive resources**.

2. Name the three authentication methods and which portal mode is preferred.

browser challenge, web form, no-captive-portal. Redirect mode is preferred. It supports cookies and modern browsers.

3. Distinguish the two MFA integration paths.

RADIUS/SAML covers getting into the firewall or VPN. Native MFA with Authentication Policy covers reaching resources through it.

4. How many factors can a native MFA profile chain?

Up to four.

5. What is the firewall's role in Orion's passwordless feature?

A Kerberos Constrained Delegation (KCD) agent, obtaining a ticket on the user's behalf.

6. Why set revocation checking to block on unknown status?

Otherwise a revoked certificate keeps working, which defeats having revocation at all.

✓ Anchor facts

- ✓ Authentication Policy triggers a challenge; **success creates a user to IP mapping**.
- ✓ Methods: **browser challenge, web form, no-captive-portal**.
- ✓ Portal modes: **redirect (preferred)** or transparent.
- ✓ Native MFA chains up to **four factors**.
- ✓ Passwordless = firewall as **KCD agent** for Kerberos protected apps.
- ✓ Certificate profiles extract the **username** from a certificate field.



Easy to get wrong

- Conflating **VPN MFA** with **step up MFA**: different mechanisms.
- Applying Authentication Policy to traffic that cannot present a prompt.
- Using transparent portal mode with modern browsers.
- Leaving revocation checking permissive.
- Issuing GlobalProtect certificates by hand rather than via SCEP.

Requirement	Mechanism	Where configured
MFA to connect the VPN	RADIUS or SAML provider	Authentication profile on the GP gateway
MFA before a sensitive app	Authentication Policy + MFA profile	Policies › Authentication
Identify an unmapped user	Authentication portal challenge	Authentication Enforcement object
No password at all	KCD passwordless or certificates	Authentication profile / certificate profile

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 9 · Authentication and Identity 28

User-ID Mapping Methods

User-ID turns an IP address into a username so policy can be written about people. Group mapping, user to IP mapping and user context all start here.

91 User-ID concepts and the two halves

User-ID does **two separate jobs**, and conflating them is the usual source of confusion.

1 · User to IP mapping

Learning that **10.1.1.50 is currently domain\alice**. This is dynamic, changes constantly, and is what lets a security rule match a user.

2 · Group mapping

Learning that **alice is in the Finance group**. This comes from **LDAP**, is relatively static, and is what lets a rule reference a group rather than individuals.

Both must work for a group based rule to function. A rule for "Finance" fails if the user is mapped but the group membership is not, and equally if the groups are known but the IP is unmapped.

Where mappings are used

Security policy as a match condition · **logs and reports**, so the Traffic log shows a name instead of an address · **Authentication Policy** · **QoS and PBF**, which can both match on user.

Enable User-ID **per zone**, and use the zone's **include and exclude lists** to constrain which subnets are mapped. Enabling it on an untrusted zone is a well known bad practice. You do not want to accept mapping data from outside.

Verify with `show user ip-user-mapping all`, `show user group list`, and `show user group name "cn=finance,..."`.

In practice

When a user based rule misbehaves, establish which half is broken first: `show user ip-user-mapping ip <addr>` for the mapping, `show user group name` for membership. They fail independently and have different fixes.

The mapping methods, and what each one can actually see

most estates need several

Security log monitoring

reads DC security event logs
the classic Windows method
needs an agent or agentless
misses non domain devices

GlobalProtect

the user authenticated to us
most reliable source there is
no guessing involved
remote users solved

Syslog listener

parses logins from any system
802.1X, VPN, wireless, proxies
needs a parse profile
covers non Windows

The rest

XML API · port mapping
client probing (WMI/NetBIOS)
authentication portal
probing: avoid, noisy

Terminal servers need Terminal Services agent

many users share one IP, so mapping is by **port range** instead

Enable User-ID per zone

use include and exclude lists · **never on an untrusted zone**

Two halves, two failure modes

user to IP mapping (who is at this address, dynamic) · **group mapping** (which groups is this user in, from LDAP, static)

92 The mapping methods

Security log monitoring: reads **domain controller security event logs** for logon events. The classic Windows method. Covers domain joined devices and misses everything else.

GlobalProtect: the user **authenticated to the firewall**, so the mapping is authoritative. The most reliable source available, and it solves remote users completely.

Syslog listener: parses login events from **any system that logs them**: 802.1X and NAC, wireless controllers, VPN concentrators, proxies, Linux hosts. Needs a **syslog parse profile** defining the regex and fields. This is how you cover non Windows.

XML API: a script or system posts mappings directly. Useful for bespoke sources and automation.

Authentication portal: challenge the user and map on success, as covered on the previous page.

Client probing (WMI and NetBIOS), the firewall or agent probes endpoints. **Deprecated in practice:** noisy, slow, and a security concern because it sends credentials to endpoints. Avoid it.

Terminal Services agent: where **many users share one IP**, mapping is done by **source port range** instead of address.

In practice

Real estates use several methods at once: security log monitoring for the Windows fleet, GlobalProtect for remote users, and a syslog listener for everything else. Client probing should not be one of them.

93 Agent versus agentless User-ID

Two deployment models for reading domain controller logs, and the choice is about scale.

Windows User-ID agent: software installed on a Windows server that reads DC logs and sends mappings to the firewall. Suits **large environments and many domain controllers**, because one agent serves many firewalls and the load sits off the firewall.

Agentless (PAN-OS integrated): the firewall reads the domain controller logs **itself**, with no software to deploy. Simpler, and the usual choice for **smaller environments** or a handful of DCs. The cost is **management plane load** on the firewall.

Either way the account used needs the rights to read security event logs, and configuration lives at

Device > User Identification .

Two related agents worth naming: the **Terminal Services agent** for shared IP environments, and the **credential enforcement** component supporting User Credential Detection in URL Filtering.

Check with `show user user-id-agent statistics` and `show user server-monitor statistics` .

In practice

The scaling rule: **agentless for a few domain controllers, agent when there are many**. Agentless is simpler but every DC it monitors is work the firewall's management plane is doing instead of something else.

✓ Check yourself

User-ID mapping

1. Name the two halves of User-ID and how each fails.

User to IP mapping (**dynamic, who is at this address**) and group mapping (**static, from LDAP**). A group rule needs both.

2. Which mapping method is the most authoritative, and why?

GlobalProtect: **the user authenticated to the firewall directly, so there is no inference.**

3. How do you map users on non Windows systems?

A syslog listener **with a parse profile, reading login events from 802.1X, wireless, VPN or proxies.**

4. Why is client probing discouraged?

Noisy, slow, and it sends credentials to endpoints: a security concern in its own right.

5. How are users mapped on a terminal server?

By source port range **via the** Terminal Services agent, **because many users share one IP.**

6. When do you choose the agent over agentless?

Many domain controllers or large scale. **Agentless puts the load on the firewall's management plane.**

✓ Anchor facts

- ✓ Two halves: **user to IP mapping** and **group mapping**.
- ✓ Group mapping comes from **LDAP** and is relatively static.
- ✓ Methods: DC log monitoring, **GlobalProtect**, syslog, XML API, portal, probing, TS agent.
- ✓ **Terminal servers map by port range.**
- ✓ Enable User-ID **per zone**, never on an untrusted zone.
- ✓ **Agentless for few DCs, agent for many.**



Easy to get wrong

- Assuming a user based rule failure is always the IP mapping.
- Enabling User-ID on an **untrusted** zone.
- Relying on DC log monitoring alone and missing non domain devices.
- Leaving **client probing** enabled.
- Expecting per user policy on a terminal server without the TS agent.

Method	Covers	Reliability	Notes
Security log monitoring	Domain joined Windows	Good	Agent or agentless
GlobalProtect	Remote and mobile users	Highest	User authenticated to us
Syslog listener	Non Windows, NAC, wireless	Good	Needs a parse profile
XML API	Bespoke sources	As good as the source	Scripted or automated
Client probing	Windows endpoints	Poor	Avoid: noisy and risky
Terminal Services agent	Shared IP servers	Good	Maps by port range

Part 9 · Authentication and Identity 29

Group Mapping, Redistribution and User Context

How identity gets from the source to every firewall that needs it. Redistribution and segments is the part most often skipped.

94 Group mapping and directory sync

Group mapping at `Device > User Identification > Group Mapping Settings` tells the firewall **which groups a user belongs to**, so policy can reference a group rather than a list of names.

It needs an **LDAP server profile**, a **bind account** with read rights, and a **search base** (a distinguished name such as `dc=corp,dc=example,dc=com`). You then choose an **update interval**: the default is one hour, since group membership changes slowly.

The include list matters at scale

By default the firewall retrieves **every group** in the search base. In a large directory that is an enormous amount of data the firewall does not need. Use the **group include list** to retrieve only the groups actually referenced in policy.

Custom groups can be defined from an LDAP filter where the directory's own groups do not match how you want to write policy.

Two gotchas. **Nested groups** need the directory to be queried appropriately or members of child groups are missed. And **primary group membership** in Active Directory is stored differently from other memberships, so "Domain Users" frequently appears empty.

Verify with `show user group list` and `show user group name "<dn>"`.

In practice

The classic symptom is a rule for "Domain Users" matching nobody. That is **primary group membership**, not a broken mapping, reference a different group or adjust the mapping configuration.

95 User context and Device-ID

User context is the broader picture the firewall holds about an identity, beyond the bare IP to name mapping: which **groups** the user is in, **where the mapping came from** and when, the **device** they are on, and their **HIP posture** if GlobalProtect is in use.

Richer context enables finer policy. Rather than "Finance can reach the finance app", you can write "Finance, on a compliant corporate device, during business hours", which is the practical shape of Zero Trust on a firewall.

PAN-OS 12.1 Orion raised the ceiling to **10 million user contexts**, a **20x increase**, specifically so user based policy stays usable in very large and cloud scale environments.

Device-ID

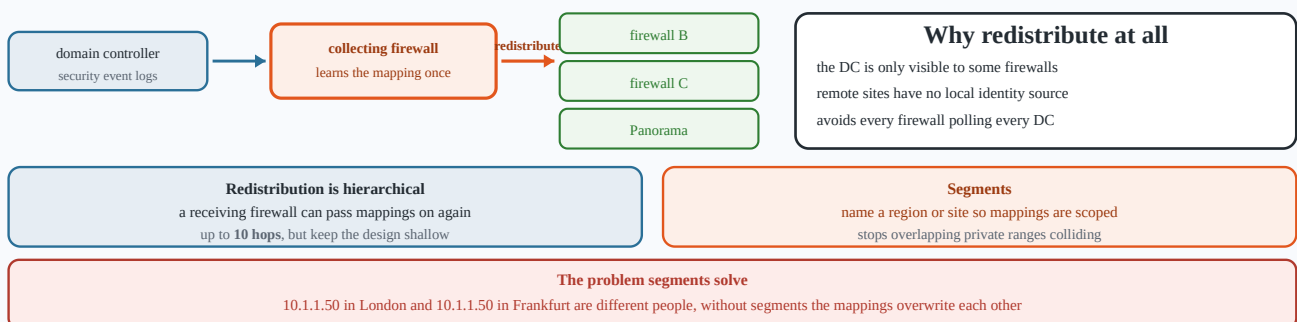
The parallel idea for devices rather than users. It identifies endpoints and lets policy match on device attributes (category, profile, vendor, model, OS) which matters for IoT and unmanaged equipment that will never have a user logged into it. Orion **expanded Device-ID capabilities**.

Together they answer the two questions policy actually wants: **who is this**, and **what are they on**.

In practice

Device-ID is the answer for anything that cannot authenticate, cameras, sensors, printers, building systems. You cannot write user policy for a device with no user, but you can write device policy.

Redistribution and segments learn the mapping once, use it everywhere



96 Identity data redistribution and segments

A firewall that learns a user to IP mapping can **redistribute it to other firewalls**, so identity is learned once and used everywhere.

Why it exists

The domain controllers may only be reachable from some firewalls. Remote sites may have no local identity source at all. And you do not want **every firewall polling every domain controller**: that scales badly and loads the directory.

How it is configured

On the **sending** device, enable redistribution under `Device > Data Redistribution` and choose what to share: **IP-User mappings**, **IP-Tag** and **User-Tag** mappings, **HIP** information, and **quarantine list** data.

On the **receiving** device, add the sender as an **agent** of type Redistribution. **Panorama can act as a redistribution point**, which is the usual design in a large estate.

Redistribution is **hierarchical**: a receiver can pass mappings on again, up to **10 hops**. Keep the design shallow anyway; every hop adds delay before a mapping is usable.

Segments, the part people miss

A **segment** names a region or site, so mappings are scoped to where they came from. Without them, **overlapping private address ranges collide**: 10.1.1.50 in London and 10.1.1.50 in Frankfurt are different people, and the mappings would overwrite each other.

With segments the firewall keeps them distinct and applies the right identity in the right place. Any estate with repeated RFC 1918 ranges across sites needs this.

Verify with `show user user-id-agent statistics` and inspect a specific address with `show user ip-user-mapping ip <addr>`, which shows the **source** of the mapping.

In practice

Segments are the detail that separates someone who has read about redistribution from someone who has deployed it across sites. If two sites reuse the same private range, mappings without segments will actively give you the wrong username.

✓ Check yourself

group mapping and redistribution

1. A rule for "Domain Users" matches nobody. Why?

Primary group membership **is stored differently in Active Directory**, so that group often appears empty.

2. Why use a group include list?

By default the firewall **retrieves every group in the search base**, in a large directory that is far more data than policy needs.

3. Give three reasons redistribution exists.

DCs are **only reachable from some firewalls**; remote sites have no local source; and you **avoid every firewall polling every DC**.

4. What can be redistributed besides IP-User mappings?

IP-Tag and User-Tag **mappings**, HIP **data**, and the **quarantine list**.

5. How many redistribution hops are supported?

Up to 10: but keep it shallow, since each hop delays when a mapping becomes usable.

6. What problem do segments solve?

Overlapping private ranges. **Without them, the same IP at two sites collides and you get the wrong username.**

✓ Anchor facts

- ✓ Group mapping needs **LDAP profile, bind account, search base**; default update **1 hour**.
- ✓ Use a **group include list** at scale.
- ✓ Redistribution shares **IP-User, IP-Tag, User-Tag, HIP and quarantine** data.
- ✓ **Panorama can be a redistribution point**.
- ✓ Hierarchical up to **10 hops**.
- ✓ **Segments** scope mappings so overlapping ranges do not collide.



Easy to get wrong

- Blaming group mapping for the **Domain Users** primary group behaviour.
- Pulling every group in a large directory.
- Missing **nested group** members.
- Deploying redistribution across sites with **no segments**.
- Building deep redistribution chains and wondering why mappings lag.

Redistributed data	Carries	Used for
IP-User mapping	Who is at an address	User based policy and logs
IP-Tag	Tags applied to addresses	Dynamic Address Groups, quarantine
User-Tag	Tags applied to users	Dynamic user groups
HIP	Endpoint posture	Compliance based rules
Quarantine list	Isolated devices	Consistent isolation estate wide

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 9 · Authentication and Identity 30

Cloud Identity Engine, SCIM and Health Alerts

The Identity course is built around the Cloud Identity Engine. It answers the problem on premises User-ID cannot: identity for users who never touch your network.

97 What the Cloud Identity Engine is and why it exists

The **Cloud Identity Engine (CIE)** is a cloud delivered identity service that collects identity data centrally and makes it available to firewalls, Panorama and Prisma Access, **without each device connecting to the directory itself**.

The problem it solves

Traditional User-ID assumes users are **on your network**, authenticating against **domain controllers you can reach**. That assumption has broken down: users are remote, directories are in the cloud, and applications are SaaS. A firewall in a branch office may have no path to a domain controller at all.

CIE connects to the identity sources instead, **Azure AD / Entra ID, Okta, PingOne, Google Workspace**, and on premises **Active Directory** through a lightweight agent, and distributes the result.

What it provides

Directory sync for users, groups and attributes · **authentication** via SAML or OIDC through the cloud · a **single consistent identity source** across on premises firewalls, Prisma Access and Panorama.

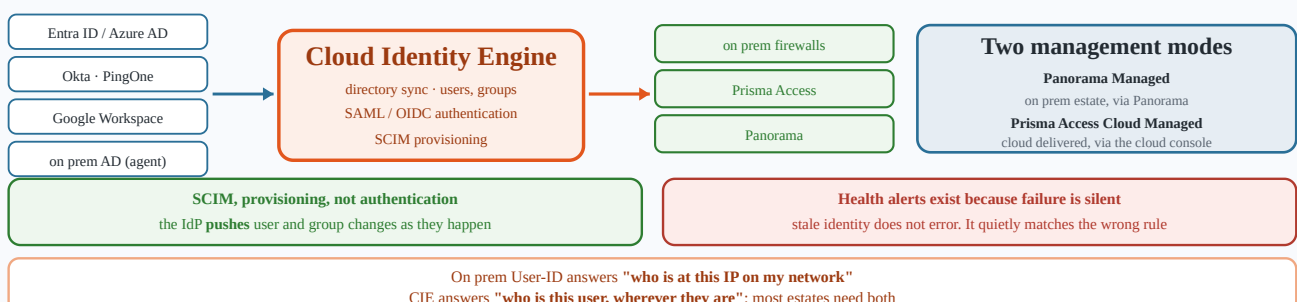
The benefits follow from being cloud delivered: **no directory connection per firewall**, works for users who never touch the corporate network, and one place to configure identity rather than repeating it on every device.

It does not replace on premises User-ID. They coexist, and most real estates run both, CIE for cloud and remote identity, traditional User-ID for on network mapping from domain controller logs.

In practice

The one line framing: **on premises User-ID answers "who is at this IP on my network"; CIE answers "who is this user, wherever they are"**. Different questions, both usually needed.

Cloud Identity Engine identity collected once in the cloud, consumed everywhere



98 SCIM and the two management modes

SCIM, System for Cross domain Identity Management, is a standard for **automated user and group provisioning** between an identity provider and a service.

The distinction that matters: **SCIM is provisioning, not authentication**. SAML authenticates a user at login. SCIM keeps the **user and group data itself** in sync, so when someone joins Finance or leaves the company, the change **arrives without waiting for a scheduled sync**.

It is **push rather than pull**, which is why it is faster and lighter than polling a directory on an interval, a leaver is removed promptly rather than at the next cycle.

The two management modes

The course covers CIE in two deployments.

Panorama Managed: CIE configured and consumed through Panorama, distributing identity to an on premises firewall estate.

Prisma Access Cloud Managed: CIE configured through the cloud management console, serving cloud delivered infrastructure.

The identity data and the sources are the same; what differs is **where you configure it and which infrastructure consumes it**. Estates running both on premises firewalls and Prisma Access encounter both.

In practice

Remember the pairing: **SAML authenticates, SCIM provisions**. They are complementary, SAML at login time, SCIM continuously in the background.

99 Health alerts and preventing stolen credentials

CIE health alerts notify you when identity collection degrades, a directory sync failing, an agent losing connectivity, or authentication errors rising.

They exist because **identity failure is silent**. When mappings go stale, traffic does not error: it simply **stops matching user based rules** and falls through to something more general, or gets denied for reasons nobody connects to identity. Without alerting you discover it from user complaints.

Preventing use of stolen credentials

A named course section, and the controls are ones already covered here, assembled into a strategy:

User Credential Detection in URL Filtering, blocking submission of corporate credentials to external sites.

MFA, so a stolen password alone is insufficient.

Passwordless and certificates, so there is no reusable credential to steal.

Authentication Policy for step up before sensitive resources.

Credential based Dynamic Address Group quarantine when a compromise is detected.

The through line: **reduce what a stolen password gets you, and detect when one is being used**.

In practice

Set the health alerts up when you deploy CIE, not after an incident. Stale identity is the failure mode you will not notice until policy has been quietly wrong for days.

✓ Check yourself

Cloud Identity Engine

1. What assumption of traditional User-ID has broken down?

That users are on your network **authenticating against** reachable domain controllers. Remote users and cloud directories break it.

2. Does CIE replace on premises User-ID?

No: they coexist. CIE for cloud and remote identity, traditional User-ID for on network DC log mapping.

3. SAML versus SCIM.

SAML authenticates at login; SCIM provisions user and group data continuously **by push rather than scheduled pull**.

4. Name the two CIE management modes.

Panorama Managed **for the on premises estate**, and Prisma Access Cloud Managed **via the cloud console**.

5. Why do health alerts matter specifically for identity?

Identity failure is silent: traffic stops matching user rules rather than erroring, so you find out from complaints.

6. List three controls that reduce the value of a stolen password.

MFA, passwordless or certificates, **and** step up Authentication Policy, **plus** User Credential Detection to stop submission.

✓ Anchor facts

- ✓ CIE sources: **Entra ID, Okta, PingOne, Google Workspace, on prem AD via agent.**
- ✓ Provides **directory sync** and **SAML/OIDC authentication** from the cloud.
- ✓ **SCIM provisions; SAML authenticates.**
- ✓ Modes: **Panorama Managed** and **Prisma Access Cloud Managed.**
- ✓ **CIE does not replace on premises User-ID: they coexist.**
- ✓ Orion supports **10 million user contexts.**



Easy to get wrong

- Treating CIE as a replacement for User-ID.
- Confusing **SCIM provisioning** with **SAML authentication**.
- Deploying CIE without **health alerts**.
- Assuming stale identity produces errors. It produces **wrong rule matches**.
- Relying on MFA alone while credentials are still submitted to external sites.

	On premises User-ID	Cloud Identity Engine
Answers	Who is at this IP, on my network	Who is this user, wherever they are
Source	DC logs, GP, syslog, API	Cloud IdPs and on prem AD via agent
Connection	Each firewall to the directory	Cloud service, no per firewall link
Best for	On network mapping	Remote users, cloud directories, Prisma Access
Most estates run both.		

IKE, IPSec and the Site to Site Build

IPSec, quantum resistant cryptography and GRE are covered here as equals. Start with the two phases every tunnel depends on, then the objects PAN-OS builds them from.

100 IKE Phase 1 and Phase 2, and how PAN-OS configures each

IKE negotiates the keys that IPSec uses. It runs in two phases, and PAN-OS separates the configuration to match.

Phase 1, the IKE Gateway

Establishes a secure, authenticated channel between the two peers. Configured at

`Network > Network Profiles > IKE Gateway`: **peer IP or FQDN**, **local and peer identification**, **authentication** (pre shared key or certificate), **IKE version** (IKEv1, IKEv2, or IKEv2 preferred with v1 fallback), and an **IKE Crypto Profile**.

The IKE Crypto Profile sets **DH group**, **encryption algorithm**, **authentication (hash) algorithm**, **key lifetime**, and an **IKEv2 authentication multiple** for rekey timing.

Phase 2, the IPSec Tunnel

Negotiates the actual **data plane security association** that encrypts traffic, running inside the Phase 1 channel.

Configured at `Network > IPSec Tunnels`: the **tunnel interface**, the **IKE Gateway** to use, and an **IPSec Crypto Profile** setting **ESP or AH**, encryption, authentication and lifetime, plus **Perfect Forward Secrecy** via its own DH group.

IKEv1 versus IKEv2

IKEv1 splits Phase 1 into **Main Mode** (6 messages, identity protected) or **Aggressive Mode** (3 messages, faster but exposes identity). **IKEv2** collapses this into fewer exchanges, has **built in NAT-T** and **dead peer detection**, and is the current recommendation. It is also the version the quantum resistant options attach to.

Verify with `show vpn ike-sa` for Phase 1 and `show vpn ipsec-sa` for Phase 2, and troubleshoot with `show vpn flow` and the **System log filtered to the ike subtype**.

In practice

When a tunnel won't come up, check **which phase failed** first. No IKE SA means Phase 1 parameters mismatch, DH group, encryption, PSK, or peer ID. An IKE SA that exists but no IPSec SA points at Phase 2, proxy IDs or the IPSec crypto profile.

Two phases, two PAN-OS objects

Phase 1 builds the channel; Phase 2 runs inside it

Phase 1, IKE Gateway

peer address, local/peer ID, auth method

IKE Crypto Profile: DH group, encryption, hash

builds the **authenticated channel**

verify: **show vpn ike-sa**

Phase 2, IPSec Tunnel

tunnel interface + which IKE Gateway

IPSec Crypto Profile: ESP/AH, encryption, PFS

runs **inside** the Phase 1 channel

verify: **show vpn ipsec-sa**

Diagnose by which phase failed

no IKE SA → Phase 1 mismatch (DH, encryption, PSK, peer ID) · **IKE SA but no IPSec SA** → Phase 2 (proxy IDs, crypto profile)

101 Proxy IDs and tunnel interfaces

A **tunnel interface** is logical, with no physical port, assigned a **zone** and a **router** like any other interface. It is what routing and security policy actually reference, the IPSec tunnel object below it is the encryption machinery.

Route based versus policy based VPN

PAN-OS is **route based**: traffic reaches the tunnel because a route points at the tunnel interface, and what gets encrypted is **whatever the routing table sends there**. Many other vendors, including older Check Point and some ASA configurations, are **policy based**: an access list defines exactly which traffic is interesting, and that definition **is** the encryption domain.

Proxy IDs bridge the two worlds

When the far end is **policy based**, it expects a specific source and destination network pair as the encryption domain. **Proxy IDs** on the PAN-OS tunnel let you specify that pair explicitly, so the two mismatched models agree on what "interesting traffic" means.

The trap: a **route based** peer with **no proxy ID configured** sends a default `0.0.0.0/0 to 0.0.0.0/0` proxy ID, which many policy based devices reject outright, or accept and then encrypt far more than intended.

Multiple proxy ID pairs create **multiple Phase 2 SAs over one Phase 1 SA**, which is normal and expected when several subnets need to cross the same tunnel to a policy based peer.

In practice

If Phase 1 comes up but Phase 2 never does against a specific vendor, check proxy IDs first. It is the most common route based to policy based mismatch, and it is invisible until you look for it.

102 Tunnel monitoring and dead peer detection

Two separate mechanisms detect a failed VPN, operating at different layers.

Dead Peer Detection (DPD)

An **IKE layer** keepalive. If the peer stops responding to DPD messages, the IKE and IPsec SAs are torn down. Configured on the IKE Gateway, with an interval and retry count. This detects a **peer that has genuinely gone away**.

Tunnel Monitoring

A PAN-OS feature that sends **ICMP probes across the tunnel itself** to a destination IP on the far side, configured on the tunnel interface. It detects something DPD cannot: **the IKE and IPsec SAs are still up, but traffic is not actually passing**: a routing problem or an ACL on the far end blocking the probe's return path.

On failure, the action is **wait-recover** (keep retrying, tunnel interface stays up) or **failover** (bring the tunnel interface down so routing fails over to a backup path or route).

The design point: **tunnel monitoring is what makes a backup VPN path actually useful**. Without it, a tunnel with dead SAs but no traffic flowing would sit there looking healthy while a static route through it kept sending traffic into a black hole.

Verify with `show vpn ike-sa`, `show vpn ipsec-sa`, and `show vpn flow tunnel-name <name>` for tunnel monitor state specifically.

In practice

DPD answers "is the peer alive". Tunnel monitoring answers "is traffic actually getting through". A tunnel can pass DPD and still be useless, that gap is exactly what tunnel monitoring closes.

✓ Check yourself

IKE, IPsec and proxy IDs

1. Which PAN-OS object corresponds to IKE Phase 1, and which to Phase 2?

IKE Gateway = Phase 1. IPsec Tunnel (with its IPsec Crypto Profile) = Phase 2, running inside the Phase 1 channel.

2. A tunnel has no IKE SA. Which phase, and what would you check?

Phase 1. DH group, encryption, PSK or certificate, and peer ID mismatch.

3. Is PAN-OS route based or policy based, and what does that mean?

Route based. Whatever the routing table sends to the tunnel interface gets encrypted; there is no access list defined encryption domain.

4. What do proxy IDs solve?

They let a route based PAN-OS tunnel agree with a policy based peer on the exact source/destination pair to encrypt.

5. Distinguish DPD from tunnel monitoring.

DPD (IKE layer) detects a dead peer. Tunnel monitoring (ICMP across the tunnel) detects SAs that are up but traffic not actually passing.

6. What are the two tunnel monitor actions on failure?

wait-recover (keep retrying) or failover (bring the interface down so routing fails over).

✓ Anchor facts

- ✓ **IKE Gateway = Phase 1. IPSec Tunnel = Phase 2**, inside Phase 1.
- ✓ PAN-OS is **route based**; the routing table decides what gets encrypted.
- ✓ **Proxy IDs** bridge to policy based peers by naming the encryption domain.
- ✓ No proxy ID configured defaults to **0.0.0.0/0 to 0.0.0.0/0**.
- ✓ **DPD** = is the peer alive. **Tunnel monitoring** = is traffic passing.
- ✓ IKEv2 has built in **NAT-T and DPD**, and is the current recommendation.



Easy to get wrong

- Assuming a passed DPD means the tunnel is actually useful.
- Forgetting proxy IDs when peering with a policy based device.
- Confusing which show command checks which phase.
- Leaving tunnel monitoring off a backup path that depends on failover.
- Using Aggressive Mode where identity exposure matters.

	DPD	Tunnel Monitoring
Layer	IKE	Data plane, ICMP
Detects	Peer unreachable	SAs up but traffic not passing
Configured on	IKE Gateway	Tunnel interface
On failure	Tears down SAs	wait-recover or failover

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

GRE and Quantum Resistant Cryptography

GRE is the plain tunnelling protocol IPsec is layered onto for confidentiality; quantum resistant crypto is where Orion's headline theme actually attaches to a configuration.

103 GRE tunnels on PAN-OS

GRE encapsulates one packet inside another so payloads that would not otherwise cross a network can be carried, multicast, non IP protocols, or simply a stable logical link across a path that changes underneath.

Configured at `Network > GRE Tunnels`: local and peer address, a **tunnel interface**, **keepalive** settings, and an optional **copy ToS header** option so QoS marking survives encapsulation.

The point worth stating plainly

GRE provides encapsulation. It provides no encryption and no authentication. Traffic inside a plain GRE tunnel is as visible on the wire as unencrypted traffic anywhere else.

The standard pattern is therefore **GRE over IPsec**: GRE does the encapsulation, carrying whatever needs carrying, and IPsec wraps around it for confidentiality and integrity. This is also how you get **multicast routing protocols across a VPN**, since plain IPsec alone will not carry multicast, but GRE will, and IPsec can then protect the GRE tunnel.

Protocol number **47**, and it adds roughly **24 bytes** of overhead, worth accounting for in path MTU, same discipline as the MSS clamping covered on the interfaces page.

In practice

If a design needs multicast, dynamic routing protocols, or non IP traffic across a VPN, the answer is GRE over IPsec, not IPsec alone. State that GRE alone is unencrypted before anyone has to ask.

104 Quantum resistant cryptography on IPSec: the mechanics, the limits, and which PAN-OS ships which RFC

This is where Orion's **harvest now, decrypt later** concern (covered in Part 1) becomes an actual tunnel setting rather than a slide.

RFC 8784, Post Quantum Pre Shared Keys, the actual mechanism

RFC 8784 adds an extra round to key generation, not a replacement for the existing one:

- Performed **after** the normal IKEv2 Diffie-Hellman key exchange, as an additional round.
- Mixes in a **static PPK string** alongside the DH derived SKEYSEED.
- The resulting key is **not based on primes**, so it is **not vulnerable to Shor's algorithm** even if the DH portion eventually is.

What the PPK itself actually is

The security of the whole scheme rests on this one secret:

- **32 to 128 characters** (16 to 64 bytes), a genuinely complex secret, not a passphrase.
- A **strong random string**, e.g. a hex value, not something a person composes.
- **Never sent in the IKEv2 exchange itself**, so it cannot be intercepted on the wire.
- A separate **PPK KEYID is sent as a reference**, telling the peer which PPK to use without revealing the PPK itself.

RFC 8784 isn't perfect

Three real limitations, and they're operational, not cryptographic:

- **Not scalable**: manually configuring PPKs gets genuinely tough past roughly **25 peers**.
- **The PPK must stay secret**, and a leaked or lost one significantly raises risk. You can control your own staff; you cannot control a **partner's employees** who also hold the same PPK.
- **Humans are bad at this**: a PPK needs high entropy (minimum 32 characters) and must resist a dictionary attack, which makes it hard to remember, and a genuinely tough PPK is exactly the kind that gets **written down somewhere** and eventually lost or leaked.

RFC 9242 and RFC 9370, advancing to hybrid KEM solutions

These exist specifically as the **precursor to PQ IKEv2 hybrid keys**, and RFC 8784 can be **layered with them during the PQC transition** for added insurance rather than one replacing the other.

- Together they're IKEv2 enhancements supporting **key exchange with large amounts of data**: something the original IKEv2 exchange wasn't sized for.
- They add up to **7 additional key exchange rounds** on top of the classic one.

What actually runs in those rounds

The classic round is **Diffie-Hellman, Group 21**. The additional PQ KEM rounds, in order, are named directly:

- **Round 1**: ML-KEM, "Kyber".
- **Round 2**: BIKE.
- **Round 3**: HQC.
- ... up to **Round 7**: KEM X.

Best practice

is **one classic round (DH) plus one or more PQC rounds**, not seven by default. Each additional round buys **crypto agility** (if one KEM is later broken, the others still hold) at the cost of larger exchanges, so the number actually used is a deliberate design choice, not a maximum to chase.

Which PAN-OS ships which RFC

The two mechanisms landed in different releases, and it's worth knowing which is which:

- **PAN-OS 11.1 (Cosmos): Post Quantum Pre Shared Keys, RFC 8784.** The fastest way to a quantum resistant VPN, and it needs **no crypto upgrade** on either side.
- **PAN-OS 11.2 (Quasar): PQC Hybrid Keys, RFC 9242 and RFC 9370.** Up to 7 additional KEMs, and it's what actually provides crypto agility.

Both are described as **defence in depth**: they can be used **independently or together**, and combining them is what meets current government recommendations rather than either alone.

Where these sit relative to conventional parameters

All of this is **additive on the IKE Crypto Profile and IKE Gateway**, alongside the DH group, encryption and hash settings already covered. It does not replace conventional IKEv2 negotiation; it strengthens the key material that negotiation produces. Practical requirement: **both peers must support the same options**, so this is generally deployed between PAN-OS devices you control, or where a third party peer has confirmed support. It is not yet a universal default.

This connects directly to **Part 1's Quantum Readiness Dashboard**: it is what tells you which tunnels are carrying long lived confidential data and should be prioritised for these settings first.

In practice

Know the version split cold: **RFC 8784 (PPK) is 11.1 Cosmos, no crypto upgrade needed. RFC 9242/9370 (hybrid KEM) is 11.2 Quasar, up to 7 extra rounds.** Both defend against harvest now decrypt later, and layering both is the recommended posture, not a choice between them.

Classic plus PQ KEM rounds, and which PAN-OS ships which RFC

best practice is one classic round plus one or more PQC rounds, not all seven

RFC 8784 · PAN-OS 11.1 (Cosmos) Post Quantum Pre Shared Keys	RFC 9242 + 9370 · PAN-OS 11.2 (Quasar) PQC Hybrid Keys, up to 7 extra rounds
Extra keygen round, after IKEv2 DH exchange	Classic: Diffie-Hellman, Group 21
PPK: 32 to 128 chars, never sent, KEYID as reference	Round 1 · ML-KEM "Kyber"
Fastest path, no crypto upgrade needed	Round 2 · BIKE
	Round 3 · HQC ... Round 7 · KEM X
Isn't perfect: not scalable past ~25 peers · PPK secrecy depends on partners too · humans write down tough PPKs	Best practice: 1 classic round (DH) + 1 or more PQC rounds not all seven by default

✓ Check yourself

GRE and quantum resistant crypto

1. Does GRE provide encryption on its own?

No. GRE is encapsulation only. It is paired with IPSec for confidentiality and integrity.

2. Why is GRE over IPSec used for multicast across a VPN?

Plain IPSec cannot carry multicast; GRE can, and IPSec then wraps around the GRE tunnel to protect it.

3. GRE's protocol number and approximate overhead?

Protocol 47, roughly 24 bytes of overhead, account for it in path MTU.

4. When exactly does RFC 8784's extra keygen round happen, and what does it mix in?

After the normal IKEv2 DH exchange. It mixes in a static PPK string, producing a key not based on primes.

5. How long must a PPK be, and why does it never appear in the IKEv2 exchange?

32 to 128 characters (16 to 64 bytes). **It's never sent, so it can't be intercepted, only a PPK KEYID is sent, as a reference.**

6. Name two real limitations of RFC 8784 in practice.

Any two: doesn't scale past ~25 peers, partner employees **also hold the secret**, or humans write down **PPKs tough enough to resist a dictionary attack**.

7. What's the classic round in a hybrid key exchange, and what's the recommended number of PQC rounds?

Classic: Diffie-Hellman, Group 21. **Best practice is 1 or more PQC rounds, not all 7 by default.**

8. Which PAN-OS release ships RFC 8784, and which ships RFC 9242/9370?

11.1 Cosmos ships RFC 8784 (PPK). 11.2 Quasar ships RFC 9242/9370 (hybrid KEM).

9. Should you choose between PPK and hybrid KEM, or use both?

Both: they're defence in depth, independently or together, and combining them meets current government recommendations.

✓ Anchor facts

- ✓ **GRE = encapsulation only, no encryption.** Protocol 47, ~24 bytes overhead.
- ✓ **GRE over IPSec** is the standard pattern for multicast or dynamic routing across a VPN.
- ✓ **RFC 8784:** extra round after DH, mixes a 32 to 128 char PPK, PPK never sent (only its KEYID).
- ✓ RFC 8784 breaks down past **~25 peers**, and depends on partner side PPK secrecy too.
- ✓ **RFC 9242/9370:** up to **7** extra KEM rounds; best practice is DH + 1 or more, not all 7.
- ✓ **PAN-OS 11.1 Cosmos** = RFC 8784. **PAN-OS 11.2 Quasar** = RFC 9242/9370.



Easy to get wrong

- Assuming a GRE tunnel is private on its own.
- Trying to carry multicast over plain IPSec.
- Forgetting the ~24 byte GRE overhead when sizing MTU.
- Thinking RFC 8784/9242/9370 replace conventional DH rather than adding to it.
- Treating RFC 8784 as scalable at enterprise size. It isn't, past roughly 25 peers.
- Mixing up which PAN-OS ships which RFC, 11.1 is the PPK, 11.2 is the hybrid KEM.

Mechanism	Adds	PAN-OS	Configured on
GRE	Encapsulation for otherwise uncarriable traffic	N/A	GRE Tunnel object
RFC 8784	Symmetric PQ preshared key, extra round after DH	11.1 Cosmos	IKE Gateway
RFC 9242	IKEv2 support for large key exchange data	11.2 Quasar	IKE Gateway / Crypto Profile
RFC 9370	Up to 7 extra KEM rounds, e.g. ML-KEM, BIKE, HQC		

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | **CONFIGURE** | VERIFY

Configuring Quantum Resistant IPsec, and a Different Answer Entirely

The previous page covered what RFC 8784 and the hybrid KEM do. This page covers actually building them in the GUI, the specific algorithms behind the hybrid KEM, and a completely different mechanism, QKD, that answers the same weaknesses by physical means instead of maths.

105 Configuring RFC 8784 and the hybrid KEM, step by step, and the NIST round 4 candidates behind it

RFC 8784, the IKE Gateway build

Under [Network > Network Profiles > IKE Gateways > Add](#), the version has to be set correctly first: **IKEv2 only mode** aborts the connection if the peer doesn't support IKEv2; **IKEv2 preferred mode** falls back to IKEv1 if the peer doesn't, but post quantum VPN features **require IKEv2** and simply won't be available if the session falls back.

On the **Advanced Options** tab, a new **PQ PPK** section holds **Enable Post Quantum Pre Shared Key (PPK)**, disabled by default and supported **only on IKEv2**. Clicking Add opens the **Add Post Quantum Pre Shared Key** popup: a **PPK KeyID**, the **PPK Secret** and its confirmation, an **Activate** checkbox, and a **PPK length** field. PAN-OS will **generate a strong PPK for you** at the length you set (32 to 128 characters) via a **Generate Strong PPK** button, then **auto copies it** so it can be pasted straight into the peer's own configuration, genuinely useful given the entropy requirements covered on the previous page.

Two details worth being precise about

PPK is independent of the authentication method: the gateway can still use Pre Shared Key or Certificate authentication as normal; PPK is an addition, not a replacement for that choice. And **up to ten PPK KEYID pairs** can be configured at once, each individually activated or deactivated; with **multiple active keys**, the initiator selects one at random for each negotiation.

Negotiation mode

Preferred permits fallback to classic IKEv2 if the peer can't do PQ PPK; **Mandatory** forces the PQ PPK path and refuses the connection otherwise, the same preferred versus mandatory pattern as the IKEv2 versus IKEv1 choice above it.

The hybrid KEM build, on top of RFC 8784

Under the same Advanced Options, a **PQ KEM** tab holds **Enable Post Quantum Key Exchange**, also disabled by default, with a further **Block IKEv2 if vulnerable cipher is used** option to refuse a negotiation that falls back to something no longer considered safe. The **IKE Crypto Profile** is where the actual KEMs to use are selected. Because PQ KEM material is large, **enable IKEv2 fragmentation** when using PQC KEMs, and set **both VPN termination devices to the same fragmentation values**: a mismatch here is a real, practical cause of failed negotiation.

The actual NIST round 4 candidates supported

These are the algorithms crypto agility is built around, if one is later found vulnerable, swap to another without waiting for a whole new RFC:

- **ML-KEM:** Kyber512, Kyber768, Kyber1024.
- **BIKE:** L1, L3, L5.
- **Classic McEliece:** 348864, 348864f.
- **HQC:** 128, 192, 256.
- **FrodoKEM:** 640-AES, 640-SHAKE, 976-AES, 976-SHAKE, 1344-AES, 1344-SHAKE.
- **NTRU-Prime:** sntrup761.

The limitation worth stating plainly: **if a previously harvested session used a KEM that later proves vulnerable, swapping the KEM going forward does not protect that already captured traffic**: crypto agility protects future sessions, not the harvest now decrypt later exposure of past ones.

Growing industry support for RFC 8784

worth knowing this isn't a PAN-OS only feature: **Cisco Catalyst 8000V, 8300 Series and ASR 1000 Series** from IOS XE 17.11, **JunOS SRX** from 22.4R1, **Aruba VIA Client v4.3**, **Samsung Galaxy** on Android 11, and the open source **StrongSwan v5.9** and **Libreswan v3.25** projects all support it, genuine interoperability rather than a proprietary extension.

In practice

The fragmentation mismatch detail is worth having ready specifically: PQ KEM payloads are large enough that IKEv2 fragmentation has to be enabled and matched on both ends, or negotiation fails in a way that looks unrelated to fragmentation at first glance.

106 Quantum Key Distribution: a completely different answer to the same IKEv2 weaknesses

Everything on this page so far (PPK, hybrid KEM) is **post quantum cryptography**: new maths, run on an ordinary classical computer, believed hard for a quantum computer to break. **QKD is not that**. It's a physically different mechanism, and it exists because IKEv2 itself has structural weaknesses PQC alone doesn't fully close.

The three IKEv2 weaknesses QKD is built to answer

- **Key derivation**: IKEv2 uses **Diffie-Hellman** to derive the symmetric encryption key, and DH is a **classical algorithm vulnerable to a quantum computer**, exactly the harvest now decrypt later exposure covered throughout this guide.
- **Key exchange and payload share one path**: IKEv2 sends the key exchange and the encrypted data payload over the **same communication path**. An attacker who eavesdrops on the key exchange has everything needed to later decrypt the payload too.
- **No built in tamper or MITM detection**: IKEv2 has no mechanism to detect an attacker intercepting or altering traffic, so interception can happen **without being noticed** at all.

What QKD actually does about it

A pair of dedicated **QKD devices** generate keys from genuine **quantum random numbers**, and instead of sending those keys over IKEv2 with everything else, the devices **synchronise them out of band**, over a physically **separate quantum channel** that cannot be intercepted the way a normal network path can. Only a **key ID**, never the key itself, travels over IKEv2 to the responder, which is what closes the same path weakness above.

ETSI-014 is the interoperability layer

a standardised **API** that lets an ordinary key management system **request and receive** keys from QKD hardware without needing to understand the quantum mechanics underneath, the firewall talks ETSI-014, not physics.

The six step exchange

- **1**: Initiator sends a **key request** to its own QKD device.
- **2**: The QKD device returns a **key and a key ID**.
- **3**: The two QKD devices **sync the key and key ID** between themselves, over the isolated quantum channel.
- **4**: The **key ID** (not the key) is sent to the responder over ordinary **IKEv2 + IPSec**.
- **5**: The responder requests the matching key from **its own** QKD device, using that key ID.
- **6**: The **same key** is returned to the responder, both ends now hold an identical key that never crossed the network they're protecting.

The practical constraint worth knowing: this needs **dedicated QKD hardware at both ends** and a genuine physical quantum channel between them. It is not a software feature you simply enable, the way PPK or hybrid KEM are.

In practice

Keep the distinction sharp if asked: **PPK and hybrid KEM are post quantum cryptography, new maths on classical hardware**. **QKD is a different mechanism entirely, distributing keys over a physical quantum channel so the key itself never travels the path an attacker could eavesdrop on**. They solve overlapping problems by genuinely different means.

✓ Check yourself

configuring PQ VPN and QKD

1. What happens if IKEv2 preferred mode falls back to IKEv1, with PQ PPK configured?

Post quantum VPN features are unavailable: they require IKEv2, and a fallback session won't have them.

2. What does PAN-OS do to help generate a compliant PPK?

A Generate Strong PPK button creates one at the configured length and auto copies it for pasting into the peer's config.

3. Why must both VPN termination devices share the same IKEv2 fragmentation settings for PQ KEM?

PQ KEM key material is large; a fragmentation mismatch is a real, practical cause of failed negotiation.

4. Name three of the NIST round 4 KEM candidates supported for the hybrid exchange.

Any three: ML-KEM, BIKE, Classic McEliece, HQC, FrodoKEM, NTRU-Prime.

5. Does swapping to a different KEM protect traffic already harvested under a now vulnerable one?

No. Crypto agility protects future sessions, not past harvest now decrypt later exposure.

6. What are the three IKEv2 weaknesses QKD specifically addresses?

DH key derivation vulnerable to quantum attack, key exchange and payload sharing one path, and no MITM/tamper detection.

7. In the QKD exchange, what actually crosses the IKEv2 path: the key, or something else?

Only the key ID: the key itself is synced separately, out of band, over the quantum channel.

8. Is QKD a software feature you simply enable, like PPK or hybrid KEM?

No: it needs dedicated QKD hardware at both ends and a physical quantum channel between them.

✓ Anchor facts

- ✓ PQ VPN features need **IKEv2**; a fallback to IKEv1 loses them entirely.
- ✓ PAN-OS **generates and auto copies** a compliant PPK for you.
- ✓ Up to **10 PPK KEYID pairs**; multiple active keys are chosen at random by the initiator.
- ✓ PQ KEM needs **matched IKEv2 fragmentation** on both ends.
- ✓ RFC 8784 has real industry support: **Cisco, JunOS, Aruba, StrongSwan, Libreswan** and more.
- ✓ **QKD ≠ PQC**. QKD is a physical mechanism; only a key ID ever crosses IKEv2.
- ✓ ETSI-014 is the **API** that lets ordinary key management talk to QKD hardware.



Easy to get wrong

- Leaving the IKE Gateway on preferred mode and assuming PQ features still apply after an IKEv1 fallback.
- Forgetting to match fragmentation settings on both VPN termination devices.
- Treating KEM crypto agility as retroactive protection for already harvested traffic.
- Confusing QKD with post quantum cryptography. They solve overlapping problems by entirely different means.
- Assuming QKD is a config checkbox rather than dedicated hardware and a physical channel.

Mechanism	What crosses IKEv2	Hardware needed	Protects against
RFC 8784 (PPK)	Nothing, PPK never sent	None beyond the firewall	Shor's algorithm on the DH portion
RFC 9242/9370 (Hybrid KEM)	KEM ciphertext (large, needs fragmentation)	None beyond the firewall	DH compromise, via crypto agility
QKD	Only the key ID	Dedicated QKD devices + quantum channel	DH derivation, shared path eavesdropping, undetected MITM

Architecture and the Sequential Build

Portals, gateways, authentication and split tunnelling are the four pieces covered here. Learn the architecture first, then the order you actually build it in, the order matters, because later steps depend on objects the earlier ones create.

107 Portal, gateway and agent: what each one does

Three components, and the distinction between the first two is the one most often blurred.

Portal

The **single point of first contact**. The agent connects here once to receive its **configuration**: which gateways exist, how to choose between them, client certificate settings, and the agent software itself for initial install and updates. The portal **authenticates the user** but does not carry their tunnel traffic.

Gateway

Where the **actual VPN tunnel terminates** and where security policy is enforced on the user's traffic. A deployment can have **one portal and many gateways**: regional gateways for latency, or internal versus external gateways for different access models.

Agent (GlobalProtect app)

Runs on the endpoint. Contacts the portal first, downloads its configuration, then connects to the **selected or best available gateway** based on that configuration. Available for Windows, macOS, Linux, iOS, Android and Chrome OS.

Both portal and gateway are typically configured as a **loopback interface** with an SSL/TLS service profile, though a gateway can also sit on a physical or subinterface directly facing where clients connect from.

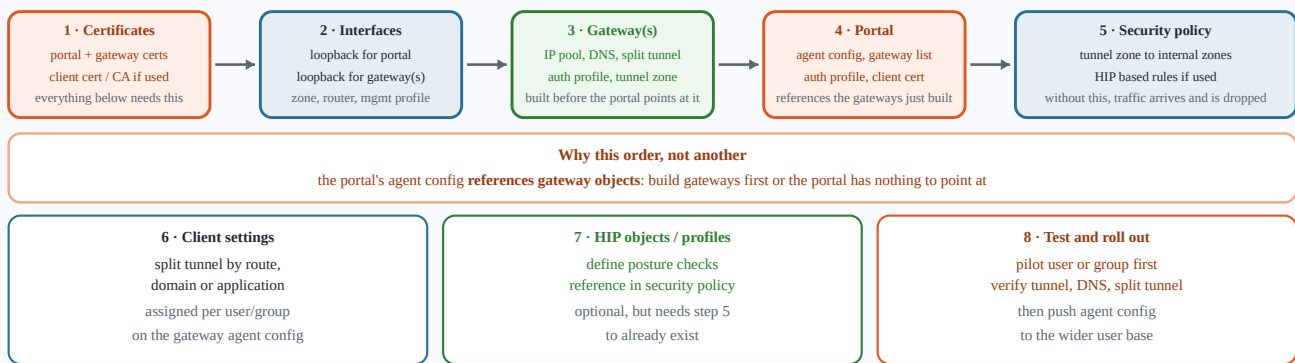
Internal versus external gateways

An **external gateway** serves remote users over the internet. An **internal gateway** serves users already inside the corporate network, and is what makes **HIP based internal enforcement** possible, the same posture check applies whether someone is remote or sitting at their desk.

In practice

Say it as one line if asked to summarise: **the portal hands out the map, the gateway is the destination on it.** Confusing the two is the most common conceptual slip on this topic.

The sequential build each step depends on an object the previous step created



108 The sequential implementation, step by step

Portals, gateways, authentication and split tunnelling are separate concerns, but they are not built in that order. Here is the order that actually works, and why each step has to come before the next.

1 · Certificates first

Generate or import the **portal certificate** and **gateway certificate(s)**, and if using certificate based client authentication, a **client certificate or CA** to validate against. Everything downstream references these through an SSL/TLS service profile, so they have to exist before anything else is configured.

2 · Interfaces

Create the **loopback interfaces** the portal and each gateway will bind to, with the correct zone, router and an Interface Management Profile allowing HTTPS. Most deployments use loopbacks specifically because they are always up and decoupled from any single physical interface.

3 · Gateway(s) before the portal

This is the step people get backwards. Build the **gateway** object first: its interface, IP pool for client addresses, DNS servers to push, tunnel zone, authentication profile, and split tunnel settings. The gateway must exist as a configured, addressable object **before** the portal can reference it.

4 · Portal

Now build the **portal**: its interface and certificate, the **agent configuration** naming which gateways to offer and how to choose between them (manual, or automatic by lowest latency or a priority list), and the authentication profile users authenticate against to fetch that configuration.

5 · Security policy

Traffic arriving through the tunnel lands in the **tunnel zone**. Without a security rule permitting tunnel zone to the internal zones it needs, the VPN connects successfully and then **every application fails**: a very common early support call.

6, 7, 8 · Client settings, HIP, then roll out

Refine the **client configuration** (split tunnel, connect method) on the gateway's agent config. Add **HIP objects and profiles** if posture based policy is required, referencing the security rules from step 5. Finally **test with a pilot group** before pushing the configuration estate wide.

In practice

State the order as a sentence if asked directly: **certificates, interfaces, gateway, portal, policy, then client settings and HIP, then a piloted rollout**. The one detail worth emphasising is that the gateway is built before the portal, which is the reverse of how people usually think about it since the portal is what users see first.

109 Gateway selection and connect methods

Once the agent has the portal's configuration, it has to decide which gateway to use, and PAN-OS gives three models.

User-Logon (Always On): the tunnel connects automatically at login and stays up, before the user does anything. Highest security posture, standard for managed corporate devices.

Pre-Logon: establishes a machine only tunnel using a machine certificate **before** the user logs in, so domain login and startup scripts work over the VPN. Solves the chicken and egg problem of a remote laptop needing network access to authenticate the user who would normally establish that access.

On Demand: the user manually triggers the connection. Lowest friction, lowest assurance, typically reserved for contractor or BYOD scenarios.

Choosing between gateways

The portal's agent config can list gateways as **manual** (user picks from a list), or with **automatic selection by priority and calculated latency**: the agent probes each candidate gateway and connects to the best performing one that the user is authorised for.

External versus internal gateway detection: the agent can determine whether the endpoint is already inside the corporate network and behave differently, skip the tunnel entirely, or connect to an internal gateway for consistent policy enforcement rather than an external one.

In practice

Pre-Logon is the answer whenever a scenario mentions domain login scripts, group policy, or any process that needs to run before a human authenticates on a remote machine.

✓ Check yourself

architecture and the build order

1. In one line, distinguish the portal from the gateway.

The portal hands out configuration (**which gateways, how to connect**); the gateway terminates the tunnel **and enforces policy on the traffic**.

2. Which is built first, the gateway or the portal, and why?

The gateway. The portal's agent configuration references gateway objects, so they must already exist.

3. A user connects successfully but every internal application fails. What was missed?

Security policy **permitting the tunnel zone to the internal zones it needs**. The connection working does not mean **traffic is permitted**.

4. Which connect method solves domain login scripts on a remote laptop?

Pre-Logon: **a machine certificate tunnel established before the user authenticates**.

5. What makes an internal gateway useful beyond serving on network users?

It enables HIP based enforcement on network, **applying the same posture checks whether someone is remote or in the office**.

6. State the full build order in one sentence.

Certificates, interfaces, gateway, portal, security policy, then client settings and HIP, then a piloted rollout.

✓ Anchor facts

- ✓ **Portal = configuration. Gateway = tunnel termination and policy.**
- ✓ Build order: **certs → interfaces → gateway → portal → policy → client settings/HIP → pilot.**
- ✓ Both typically bind to a **loopback interface**.
- ✓ Connect methods: **User-Logon, Pre-Logon, On Demand.**
- ✓ **Pre-Logon** uses a machine certificate, before user authentication.
- ✓ Internal gateways enable **HIP enforcement on network**.



Easy to get wrong

- Building the portal before the gateways it needs to reference.
- Forgetting security policy for the **tunnel zone** entirely.
- Using On Demand where domain login scripts are required.
- Confusing the portal's job (configuration) with the gateway's (traffic).
- Rolling out to the whole estate before piloting.

Step	Object	Depends on
1	Certificates	,
2	Loopback interfaces	Zone, router already exist
3	Gateway	Interface, certificate
4	Portal	Gateway objects from step 3
5	Security policy	Tunnel zone from the gateway
6 to 7	Client settings, HIP	Gateway config, security policy
8	Pilot rollout	Everything above

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Split Tunnelling, Authentication and HIP

The two named subobjectives left: authentication, and split tunnelling. Then HIP and clientless access, which round out what a GlobalProtect deployment actually enforces.

110 Split tunnelling by route, domain and application

Split tunnelling decides **which traffic goes through the tunnel** versus straight to the internet from the endpoint. Configured in the gateway's **Client Settings**, under three separate mechanisms that can combine.

By route (network): **Include** and **Exclude** lists of networks. Include narrows the tunnel to specific destination subnets; Exclude carves exceptions out of an otherwise full tunnel. **No entries in Include means everything routes through the tunnel:** full tunnel is the default, not split.

By domain: DNS based: **Include** and **Exclude Domain** lists send or exempt entire domains, resolving through the tunnel or not accordingly. This is how you keep, say, all corporate SaaS domains in the tunnel without needing to know their address ranges.

By application: identifies specific applications on the endpoint (by process name or path) and includes or excludes their traffic from the tunnel regardless of destination. The most granular option, useful for exempting a specific piece of software that should never traverse the corporate network.

The trade off to state

Full tunnel maximises visibility and control, since all traffic can be inspected, but costs bandwidth at the gateway and adds latency to internet bound traffic. **Split tunnel** is efficient and low latency for excluded traffic, but that traffic **receives none of the firewall's inspection**.

The three mechanisms are commonly combined: exclude well known SaaS domains by domain based split tunnel to save bandwidth, while keeping everything else full tunnelled for inspection.

In practice

The default with nothing configured is full tunnel, which surprises people expecting split to be the default. And route based split with no Include entries doesn't split anything. It is a no-op, not an error.

111 Authentication and SAML on GlobalProtect

Authentication happens at both the portal and, if configured separately, the gateway, and it draws directly on the authentication stack from Part 9.

The methods available

Local database or LDAP/RADIUS via a standard authentication profile. **Certificate based**, using a client certificate issued via SCEP or imported, validated against a certificate profile. **SAML**, where the firewall is the **service provider** and an external IdP performs the actual authentication, including any MFA the IdP enforces.

Why SAML is now the common choice

MFA, conditional access and session policy all live at the **identity provider** rather than needing separate configuration on the firewall. It also integrates naturally with the **Cloud Identity Engine** covered in Part 9, the same identity source serving both on premises User-ID and GlobalProtect authentication.

Combining methods

Client certificate **plus** SAML is a common pattern: the certificate proves the **device** is corporate managed, SAML proves the **user**, and together they give device and user assurance without either check alone being sufficient. This is the practical shape of Zero Trust remote access.

An **authentication cookie** can be issued on successful login so the user is not reprompted on every reconnect within the cookie's lifetime, balancing security against a VPN that constantly demands reauthentication.

In practice

If a scenario names an external identity provider, MFA, or conditional access, the answer is **SAML**. If it names device compliance or corporate managed devices specifically, add **certificate based auth alongside it**.

Split tunnel: three mechanisms, one decision

and what nothing configured actually means

By route

Include / Exclude networks
tunnels specific subnets
or carves exceptions out
empty Include = full tunnel

By domain

Include / Exclude Domain
DNS based, not address based
good for SaaS you don't
know the address range for

By application

match on process/path
most granular option
independent of destination
exempt one specific app

Full tunnel

max visibility, costs gateway bandwidth

Split tunnel

low latency, **excluded traffic gets no inspection**

Common pattern: **exclude known SaaS by domain**, keep everything else full tunnelled for inspection

112 HIP objects and HIP profiles

Host Information Profile (HIP) is endpoint posture assessment: the agent collects data about the device and reports it, so policy can differ for a compliant device versus a risky one.

Two layer object model. A **HIP Object** checks one category of thing, **disk encryption, patch management, antivirus status, firewall enabled, a specific process running or not running, mobile device attributes, custom registry or plist checks**. A **HIP Profile** combines HIP objects with **boolean logic (AND/OR/NOT)** into a named condition, such as "Corporate-Compliant" or "High Risk".

The HIP Profile is then referenced as a **match condition in security policy**, the same way a user or application is. A rule can permit a corporate application only for traffic matching the "Corporate-Compliant" HIP profile, and deny or restrict it otherwise.

The data itself: the agent collects it and sends it to the gateway, which evaluates HIP profiles and can log a **HIP Match** event, visible in the Traffic and HIP Match logs, useful for auditing posture even where policy does not yet act on it.

Ties directly back to Part 9: HIP data can be **redistributed** between firewalls the same way user to IP mappings are, so posture based policy remains consistent estate wide rather than being known only to the gateway a user happened to connect to.

In practice

The two layer model is the detail worth stating precisely: **objects check individual facts, profiles combine them with logic into something policy can reference**. That is exactly the pattern security profile groups use for content inspection.

113 Clientless VPN

Clientless VPN provides **browser based access to specific internal web applications** without installing the GlobalProtect agent, reverse proxying the application through the gateway.

The use case is narrow and deliberate: **unmanaged devices** (personal laptops, kiosk machines), **contractors** who should not have full network access or agent software installed, or quick access to a single web application rather than the whole network.

Configured via **Clientless VPN application objects** on the gateway, defining exactly which internal web applications are published, referencing standard HTTP/HTTPS.

The limitations worth stating unprompted

It only supports **web based applications**, no native client server traffic, no arbitrary ports, nothing that is not HTTP or HTTPS under the hood. Complex JavaScript heavy applications can break under the URL rewriting the reverse proxy performs. **No HIP data is available**, since there is no agent to collect it, so posture based policy cannot apply to a clientless session.

The trade off in one line: **it trades network level access and posture visibility for zero client side footprint**, which is exactly the right trade for a small set of published web apps reached from devices you do not manage.

In practice

If a scenario mentions contractors, BYOD, or "access to one specific application without a full VPN client", clientless is the answer. If it mentions any non web protocol or needs posture checking, it is not.

✓ Check yourself

split tunnel, auth, HIP, clientless

1. What does an empty route based Include list actually do?

Nothing splits: **the default is full tunnel. It is a no-op, not an error.**

2. Name the three split tunnel mechanisms.

By route, by domain, by application. **They can be combined.**

3. Why is client certificate combined with SAML a common pattern?

The certificate proves the device is corporate managed; SAML proves the user. Together, device and user assurance.

4. Distinguish a HIP object from a HIP profile.

An object checks one fact (disk encryption, AV status). A profile combines objects with AND/OR/NOT into a named condition used in policy.

5. Why does clientless VPN carry no HIP data?

There is no agent to collect it, clientless is browser based reverse proxy only.

6. A scenario mentions contractors needing access to one internal web app with no client install. What's the answer?

Clientless VPN: the narrow, browser based, no footprint case.

✓ Anchor facts

- ✓ Split tunnel: **route, domain, application**: combinable.
- ✓ Default with nothing configured is **full tunnel**.
- ✓ SAML: firewall is the **service provider**; MFA lives at the IdP.
- ✓ **Certificate + SAML** = device assurance + user assurance.
- ✓ HIP: **objects check facts, profiles combine them with logic**.
- ✓ Clientless VPN: **web apps only, no HIP, reverse proxied**.



Easy to get wrong

- Assuming split tunnel is the default. It is not, full tunnel is.
- Excluded split tunnel traffic still getting **zero inspection** by design.
- Expecting HIP data from a clientless session.
- Trying to publish a non web protocol through clientless VPN.
- Forgetting HIP data can be **redistributed** like User-ID mappings.

Mechanism	Basis	Grain	Good for
Route based split	Destination network	Subnet	Known address ranges
Domain based split	DNS	Domain	SaaS with unknown IP ranges
App based split	Process / path on endpoint	Most granular	Exempt one specific app
Clientless VPN	Reverse proxy	Per published web app	Contractors, BYOD, unmanaged

Active/Passive, the HA Links and Election

Active/active, active/passive, and link and path monitoring are the three pieces covered here. Start with the simpler mode and the links both modes share, then split off active/active.

114 Active/Passive HA and the HA links

Two firewalls, one **active** and processing all traffic, one **passive** and holding a synchronised configuration and session state, ready to take over. **Active/Passive is the default recommendation** for most designs: failover behaviour is predictable, and troubleshooting a single active path is far simpler than reasoning about two.

HA1, control link

Carries **heartbeats, hello messages, and configuration synchronisation**. This is how each peer knows the other is alive and how a change made on one is replicated to the other. Can be a dedicated port or, on smaller platforms, shared with a data interface as **HA1 backup**.

HA2, data link

Carries **session state synchronisation**: the session table, forwarding table, IPSec security associations, ARP table, and User-ID mappings. This is what makes failover **stateful**: existing sessions survive rather than needing to reconnect. Can run in **HA2 keepalive** mode to detect a data link failure without waiting for something worse.

HA3, packet forwarding (Active/Active only)

Used in active/active for forwarding packets between the two units when a session's return traffic arrives on the peer that is not its session owner. Not present in active/passive, since there is only ever one active path.

Backup links

Both HA1 and HA2 can have a **dedicated backup link**, so a single link failure does not force an unnecessary failover. Best practice is to run HA1 and HA2 on separate physical interfaces, with backups for both where the platform allows it.

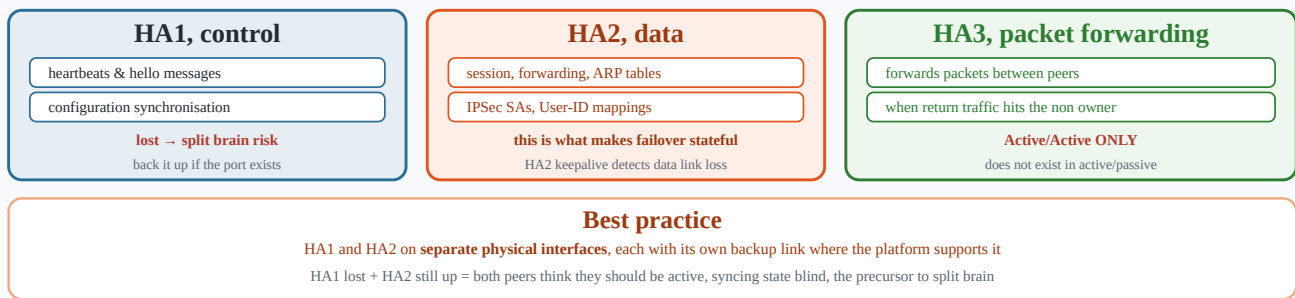
Verify with `show high-availability state` , `show high-availability link-monitoring` , and `show high-availability control-link state` .

In practice

If HA1 fails but HA2 stays up, the peers cannot exchange heartbeats but sessions keep synchronising blind, a classic **split brain precursor**. This is why HA1 backup is worth the extra port far more than HA2 backup in most designs.

The three HA links, and what each one carries

HA3 only exists in active/active



115 HA states and election

An HA firewall moves through a defined set of states, and knowing the sequence is what lets you read `show high-availability state` output correctly.

Initial → the firewall is starting HA and has not yet determined its role.

Active / Passive → the steady states in active/passive mode.

Active-Primary / Active-Secondary → the equivalent steady states in active/active mode.

Tentative → a transitional state, typically entered by a unit rejoining after a failure, while it resynchronises before becoming fully active.

Suspended → the administrator has manually taken the unit out of the HA pair, or a failure condition has forced it out.

Non functional → a hardware or software fault has made the unit unable to participate at all.

Election

Decided by **device priority** (lower value wins) and whether **preemption** is enabled. With preemption **on**, a unit that returns after being down and has a better (lower) priority will **reclaim active status**. With preemption **off**, whichever unit is active stays active even if the original primary returns, avoiding an unnecessary second failover.

Most designs run with **preemption disabled** for exactly that reason: a flapping link that keeps triggering failover and preemption back is worse than staying on the current active unit until a deliberate, planned failback.

Heartbeat polling and **hello interval** settings control how quickly a peer is declared dead, trading detection speed against false positives from transient network noise.

In practice

If a scenario mentions a unit "flapping" between active and passive after a maintenance window, the first thing to check is whether **preemption** is enabled and whether the priority difference is intentional.

116 Link monitoring and path monitoring

Two failure detection mechanisms beyond the HA1 heartbeat itself detect different classes of failure.

Link Monitoring

Watches a group of **physical interfaces** on the active unit. If a monitored interface goes down, the unit's **priority is degraded** or it triggers failover outright, depending on configuration. Interfaces are grouped, and you choose whether failover requires **any** link in the group to fail or **all** of them, "any" is stricter, "all" tolerates a single redundant link going down.

Path Monitoring

Sends **ICMP pings to a specified destination** beyond the firewall's own interfaces, the next hop router, or further upstream. This catches a failure link monitoring cannot see: the local interface is up, but the path **beyond** it is broken, so an interface only check would report everything fine while the unit is actually unreachable from the network's point of view.

Path monitoring can be configured at the **virtual router level** (monitoring the general health of routes via that router) or at the **destination IP level** (monitoring reachability to one specific address), giving finer control over exactly what "the path is broken" means for this deployment.

Both mechanisms feed the same outcome: a degraded or failed condition causes the unit to **lower its priority**, which, combined with the election rules from the previous topic, causes the healthier peer to become or remain active.

In practice

The pairing to remember: **link monitoring answers "is my interface up"**, **path monitoring answers "can I actually reach something through it"**. A unit can pass the first and fail the second, and only path monitoring catches that.

✓ Check yourself

HA links, states and monitoring

1. Which HA link carries session state, and why does that matter for failover?

HA2. It is what makes failover stateful: existing sessions survive rather than resetting.

2. What happens if HA1 is lost but HA2 stays up?

Peers cannot exchange heartbeats but keep syncing state blind, the precursor to split brain.

3. Which HA link exists only in active/active mode?

HA3, for forwarding packets when return traffic lands on the non owner peer.

4. With preemption disabled, what happens when the original primary rejoins?

The currently active unit stays active. No automatic fallback.

5. Distinguish link monitoring from path monitoring.

Link monitoring: is the interface up. Path monitoring: can I actually reach something beyond it, via ICMP.

6. A unit's interface is up but the network beyond it is broken. Which mechanism catches this?

Path monitoring: link monitoring alone would report the interface as healthy.

✓ Anchor facts

- ✓ **HA1** = control, heartbeats, config sync. **HA2** = session state. **HA3** = A/A packet forwarding only.
- ✓ Best practice: HA1 and HA2 on **separate interfaces**, each backed up.
- ✓ States: Initial, Active/Passive (or Active-Primary/Secondary), **Tentative**, Suspended, Non functional.
- ✓ Election: **lower priority wins**; preemption controls failback.
- ✓ **Most designs disable preemption** to avoid flapping.
- ✓ Link monitoring = interface state. Path monitoring = **ICMP beyond the interface**.



Easy to get wrong

- Running HA1 and HA2 on the same physical link with no backup.
- Enabling preemption without understanding the flap risk.
- Assuming link monitoring alone catches an upstream path failure.
- Confusing **Tentative** (transitional) with **Suspended** (deliberately out).
- Forgetting HA3 only applies to active/active.

Link	Carries	Present in	Loss causes
HA1	Heartbeats, config sync	Both modes	Split brain risk
HA2	Session, forwarding, ARP, IPSec SAs	Both modes	Failover loses state
HA3	Inter peer packet forwarding	Active/Active only	N/A in active/passive

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Active/Active and HA Upgrades

Active/active is worth its own page. It trades the simplicity of active/passive for both boxes doing useful work, at the cost of real design complexity.

117 Active/Active: session owner, session setup and floating IPs

In active/active, **both firewalls process traffic simultaneously**. That immediately raises a question active/passive never has to answer: for any given session, which of the two units is actually responsible for it? PAN-OS splits the answer into two separate roles.

Session owner

The unit that **owns the session for the lifetime of that session**: performs the security processing, holds the session table entry, and logs it. Assigned when the session is created and does not move.

Session setup

The unit that **handles the initial setup** of a new session, the first packet processing. This can be a different unit from the eventual session owner, decided by the configured **session setup method**: **IP Modulo** (a hash of source and destination IP decides which unit) or **Primary Device** (one designated unit always handles setup, then hands ownership across if needed).

If setup and ownership end up on different units, or if **return traffic for an existing session arrives on the non owner** peer, that traffic is forwarded to the owner across **HA3**, which is exactly why HA3 exists only in this mode.

Floating IPs

An IP address that can move between the two units, so a client facing gateway address is **always answered by whichever unit currently owns it**, without the client needing to know which physical firewall that is. Typically one floating IP is assigned to each unit as its "home", and it moves to the peer if that unit fails. **ARP load sharing** is the alternative mechanism, where the two units answer ARP for a shared virtual address using a hash based or round robin split.

Verify session distribution and ownership with `show session all` filtered per unit, and

`show high-availability interface ha3` for the forwarding link's own state.

In practice

The complexity active/active adds is exactly this session owner versus setup split, and it is why active/passive remains the default recommendation for critical paths, predictable failover is worth more than using both boxes' capacity, and it is far easier to troubleshoot when something is wrong.

Active/Active session handling two roles, and why HA3 exists to bridge them

Session setup

handles the first packet
decided by IP Modulo
or Primary Device
may differ from the eventual owner

Session owner

processes and logs
for the session's lifetime
assigned at creation
does not move

Mismatch? → HA3

return traffic on the wrong peer
gets forwarded to the owner
this is the ONLY reason HA3 exists

Floating IP

moves to the peer on failure
one per unit, client never needs to know which box

ARP load sharing

shared virtual address, split by hash or round robin
the alternative to per unit floating IPs

This session owner/setup split is exactly why active/passive stays the default recommendation for critical paths

118 Config sync and split brain

HA1 carries **configuration synchronisation**: a commit on one peer is **automatically replicated** to the other, so both units run identical policy. This happens continuously, not only at failover.

Status is visible per object in the GUI and via `show high-availability state`, which reports whether the peer's configuration is in sync. A firewall that has drifted out of sync, typically from a change made while HA1 was down, needs a manual **full sync push** to correct it once the link is restored.

Split brain

The failure state where **both units believe they are active simultaneously**, because they have lost the ability to communicate (HA1 down) but each still considers itself healthy. Both then process traffic and, worse, may have diverging configuration if changes were made independently during the outage.

The defences: a **backup HA1 link** so a single link failure does not cause a communication loss; keeping HA1 and HA2 on **separate physical paths** so one fault does not take out both simultaneously; and, on some platforms, an out of band **heartbeat backup** mechanism using a different transport entirely for an extra layer of certainty.

After any split brain event, the recovery step is to identify which unit has the **correct, most recent configuration** and push a full sync from it, never assume both sides converged safely on their own.

In practice

Split brain is rare precisely because HA1 backup exists to prevent it. When it is asked about in an interview, the expected answer names the cause (communication loss with both sides believing they're healthy) and the specific defence (a backup control link), not just the definition.

119 HA upgrade sequence and config sync during maintenance

Upgrading PAN-OS on an HA pair follows a specific order, and skipping steps is the most common way to cause an avoidable outage.

1 · Suspend the passive unit

Take the **passive** unit out of the HA pair manually `request high-availability state suspend` so it stops being a failover target mid upgrade.

2 · Upgrade the passive unit

Install the new PAN-OS image and reboot. It comes back up still suspended, running the new version, while the **active unit keeps serving all traffic unaffected** on the old version.

3 · Verify, then unsuspend

Confirm the upgraded unit is healthy before rejoining the pair `request high-availability state functional`. It comes up as **passive**, now on a different PAN-OS version than its active peer, a deliberately mismatched, temporary state.

4 · Fail over deliberately

Manually trigger failover so the **now upgraded unit becomes active**. Traffic moves to it, and this is the moment to watch closely, let it run and stabilise before touching the other unit.

5 · Upgrade the former active

With the newly active unit (upgraded) now carrying traffic, upgrade the remaining unit the same way: suspend, upgrade, verify, rejoin. Both units end up on the same version.

Panorama can **orchestrate this entire sequence** from the UI rather than requiring each CLI step by hand, and current PAN-OS additionally supports **hot, warm and cold patches** for targeted fixes that do not require the full sequence at all, both covered in Part 1's Orion material.

In practice

The step people skip is **letting the newly active unit run and stabilise** before upgrading the second one. Upgrading both back to back with no pause removes your only safety net if the new version has a problem.

✓ Check yourself

active/active and upgrades

1. Distinguish session owner from session setup.

Setup = handles the first packet. Owner = processes and logs for the session's lifetime. They can be different units.

2. Why does HA3 only exist in active/active?

To forward return traffic to the session owner when it arrives on the non owner peer, a scenario only possible with two active units.

3. What causes split brain, and what is the primary defence?

Loss of HA1 communication while both units believe they are healthy. Defence: a backup HA1 link and separating HA1/HA2 paths.

4. Why does a config sync mismatch need a manual full push?

Drift typically happens when HA1 was down during a change, automatic sync could not run, so a manual push corrects it after the link returns.

5. In the HA upgrade sequence, which unit is upgraded first?

The passive unit, suspended first so it is not a failover target during the upgrade.

6. What step do people skip, and why does it matter?

Letting the newly active (upgraded) unit run and stabilise before upgrading the second, skipping it removes your rollback safety net.

✓ Anchor facts

- ✓ **Session owner** processes for the session's life. **Session setup** handles the first packet.
- ✓ Setup method: **IP Modulo** or **Primary Device**.
- ✓ Mismatch between owner and traffic path → forwarded via **HA3**.
- ✓ Floating IP per unit, or **ARP load sharing** on a shared address.
- ✓ **Split brain**: both active, HA1 lost. Defence: backup HA1 link.
- ✓ Upgrade order: **suspend passive → upgrade → verify → rejoin → fail over → stabilise → upgrade the other**.



Easy to get wrong

- Treating active/active as strictly better because both boxes do work.
- Forgetting HA3 has one job: bridging owner/setup mismatches.
- Assuming config always stays in sync with no verification.
- Upgrading both HA units back to back with no stabilisation pause.
- Not knowing Panorama can orchestrate the upgrade sequence.

Step	Action	State after
1	Suspend passive unit	Out of the pair, not a failover target
2	Upgrade passive unit	New version, still suspended
3	Verify, rejoin	Passive, mismatched version (temporary)
4	Fail over deliberately	Upgraded unit now active
5	Let it stabilise, then upgrade the other	Both on the same version

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

VSYS Concepts and What Lives Inside One

Interfaces and zones, virtual routers, logical routers, and inter VSYS routing and security are the four pieces covered here. Start with what a VSYS actually is before the routing between them.

20 Virtual systems and multi tenancy

A **virtual system (VSYS)** is a logically separate firewall running on shared physical hardware, with its own **interfaces, zones, security policy, NAT, routing, and set of administrators**. One appliance can host multiple VSYS, each behaving as an independent firewall to whoever manages it.

This is the Palo Alto answer to the same problem Cisco solves with **ASA security contexts**: put several logically distinct firewalls on one box rather than buying separate hardware for each.

Why organisations use it

Multi tenancy: a managed service provider running separate customers on shared appliances, each customer's administrators only able to see and change their own VSYS.

Internal separation: isolating a regulated business unit, a lab environment, or an acquired company's network from the rest of the estate on the same physical pair.

Delegated administration: giving a team control of their own segment's policy without touching the shared platform configuration.

VSYS is licensed per platform, with a **maximum count that varies by model**: smaller appliances may support none or a handful, larger and chassis based platforms support many more. Enabling multi-vsys mode is a device wide setting under `Device › Setup › Management`.

A **vsys1** exists by default even on a single-vsys firewall; enabling multi-vsys mode is what exposes the ability to create additional ones alongside it.

In practice

The mental model that makes the rest of this part make sense: a VSYS is a firewall within a firewall. Everything that follows (which objects it owns, how traffic crosses between VSYS, who administers it) falls out of that one idea.

One appliance, several independent firewalls

what belongs to a VSYS, and what is shared across all of them



121 Interfaces, zones and routers inside a VSYS

Every physical or logical interface is assigned to **exactly one VSYS** when multi-vsyz mode is enabled, the assignment happens on the interface itself, before zone configuration. A VSYS's zones can only contain interfaces that already belong to that VSYS.

Zones are VSYS scoped: a zone named TRUST in VSYS1 is a completely different object from a zone named TRUST in VSYS2, even sharing the same name. They cannot reference each other's interfaces.

Virtual and logical routers

Each VSYS gets its own routing instance, a **virtual router** under legacy routing, or a **logical router** under the Advanced Routing Engine, exactly as covered in Part 4. A VSYS's routing table is **independent**: VSYS1 and VSYS2 can both run entirely different routing protocols, both use overlapping private address space, and neither is aware of the other's routes by default.

One router can technically be shared across VSYS in some configurations, but the normal and recommended design gives **each VSYS its own router**, preserving full routing isolation between tenants.

Security policy is per VSYS

Each VSYS has its own complete security rulebase, its own NAT rulebase, and its own implicit intrazone-allow / interzone-deny rules, independently overridable per VSYS.

In practice

The isolation is the whole point: a misconfiguration or compromise inside one VSYS's zones, routing table, or policy has **no direct visibility** into another VSYS's traffic, which is exactly the guarantee multi tenancy requires.

122 Shared versus VSYS specific objects

Not everything on a multi-vsyz firewall has to be duplicated. Objects can be created at two scopes.

VSYS specific: visible and usable only within the VSYS it was created in. This is the default and the right choice for tenant specific address objects, application groups, or anything that should stay isolated with the rest of that VSYS's configuration.

Shared: created at the **Shared** location and visible to **every VSYS** on the device. Useful for genuinely common objects: a security profile group every VSYS should use as a baseline, an EDL of known malicious IPs relevant regardless of tenant, or standard logging settings.

The trade off is exactly the isolation guarantee from the previous topic: a **shared object is a shared dependency**. Change it once, and it changes for every VSYS using it, which is convenient for a genuinely universal threat feed, and dangerous for anything a tenant should control independently.

Objects can be **cloned from shared into a VSYS** if a tenant needs their own independently editable copy going forward, breaking the shared link deliberately.

Certificates, in particular, are commonly kept at the **Shared** location so a single enterprise CA issued certificate for decryption or GlobalProtect can serve every VSYS without reimporting it per tenant.

In practice

Default to VSYS specific unless there is a deliberate reason for shared. The failure mode people hit is a change to a shared object landing unexpectedly across every tenant, when only one tenant's behaviour should have changed.

✓ Check yourself

VSYS concepts

1. What is a VSYS, in one line?

A logically separate firewall (own interfaces, zones, policy, NAT, router, admins) running on shared hardware.

2. What is the Cisco equivalent of a VSYS?

ASA security contexts. Same purpose: several logical firewalls, one physical box.

3. Can two VSYS have zones with the same name?

Yes: zones are VSYS scoped, so identically named zones in different VSYS are entirely separate objects.

4. Do VSYS share a routing table by default?

No. Each VSYS has its own independent virtual or logical router; overlapping address space between VSYS is fine.

5. What is the risk of a shared object?

It is a shared dependency: a change made once affects every VSYS using it, which can surprise a tenant expecting isolation.

6. What is the default scope for a new object on a multi-vsys firewall?

VSYS specific, unless deliberately created at the Shared location.

✓ Anchor facts

- ✓ VSYS = **logical firewall** on shared hardware; equivalent to **ASA contexts**.
- ✓ Interfaces belong to **one VSYS**; zones are VSYS scoped and cannot cross.
- ✓ Each VSYS gets its own **virtual/logical router**: independent, can overlap addressing.
- ✓ Security policy, NAT and implicit rules are **per VSYS**.
- ✓ **Shared** objects are visible to all VSYS; a change affects everyone using it.
- ✓ Certificates are commonly kept **Shared** to avoid reimporting per tenant.



Easy to get wrong

- Assuming VSYS share a routing table or can see each other's routes.
- Editing a Shared object without realising every VSYS is affected.
- Forgetting interface to VSYS assignment happens before zone config.
- Expecting a same named zone in another VSYS to be the same object.
- Not licensing enough VSYS for the platform before designing around them.

Scope	Visible to	Change impact	Default for
VSYS specific	One VSYS only	That VSYS alone	New objects, by default
Shared	Every VSYS	All VSYS using it	Certificates, common EDLs, baseline profiles

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Inter VSYS Routing, External Zones and Administration

If VSYS are fully isolated by default, how does traffic ever get from one to another? This is where most of the practical configuration actually sits.

123 Inter VSYS routing: external zones and the shared gateway

By design, VSYS cannot see each other's traffic. Two mechanisms deliberately open a controlled path between them, and they solve different problems.

External zone, VSYS talking to VSYS

An **external zone** is created in one VSYS and configured to reference **another VSYS** as its "external" system. Traffic can then be routed between the two VSYS **through this zone**, and, critically, **security policy still applies** on both sides, exactly as it would for any other zone to zone traffic. This is how you deliberately allow VSYS1 to reach a specific service in VSYS2 without collapsing the isolation between them entirely.

Each VSYS needs a route pointing toward the other via this external zone relationship, and the traffic is genuinely routed. It is not a shortcut that bypasses inspection.

Shared gateway, VSYS sharing one internet exit

A **shared gateway** is a special interface, owned by no single VSYS, that **multiple VSYS route through to reach a common external network**: typically the internet. Rather than every VSYS needing its own dedicated internet facing interface and public IP allocation, they all route out through one shared exit point.

The shared gateway has its own zone and its own security policy layer, so traffic from any VSYS crossing it is still inspected at that boundary, in addition to whatever policy the originating VSYS itself applied.

The distinction to hold

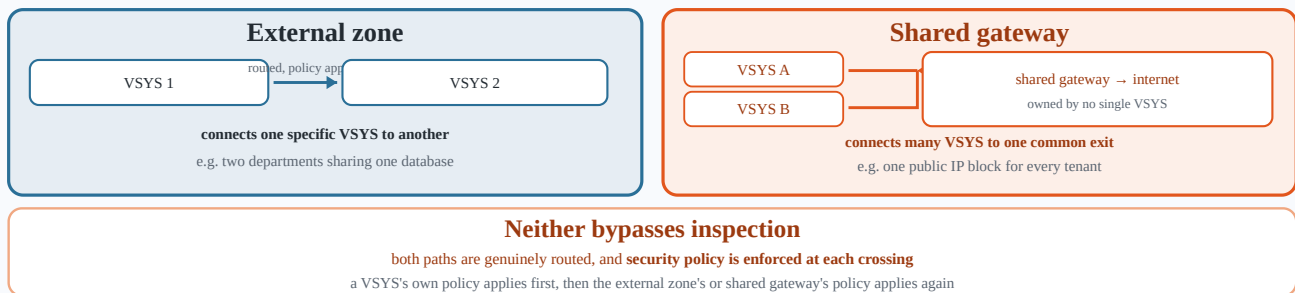
External zone connects two specific VSYS to each other. Shared gateway connects many VSYS to one common external network. A design can use both: external zones for VSYS that genuinely need to talk to one specific peer, a shared gateway for the internet exit every VSYS needs but none should own individually.

In practice

If a scenario says "two departments need to reach a common database", that is an **external zone** between their two VSYS. If it says "every tenant needs internet access but we only have one public IP block", that is a **shared gateway**.

Two controlled paths through the isolation

VSYS to VSYS versus VSYS to the outside world



124 VSYS administrators and resource limits

A **VSYS administrator** role, configured at `Device > Administrators`, is scoped to **one or more specific VSYS** rather than the whole device. An admin assigned only to VSYS2 cannot see or change VSYS1's configuration, logs, or objects at all, from their perspective, the shared hardware and every other VSYS simply do not exist.

This is the delegated administration use case from the first topic made concrete: a managed service provider gives each customer's team a VSYS admin account, and no customer can see another customer's policy, traffic, or logs, even though all of them run on the same physical appliance pair.

Above VSYS admins sits the **device wide superuser**, who can see and configure every VSYS, the shared objects, and the platform level settings (licensing, PAN-OS version, HA) that no VSYS admin can touch regardless of role.

Resource limits

Because VSYS share physical resources, PAN-OS lets you **cap what each VSYS can consume**: maximum sessions, maximum security rules, and maximum number of zones, among other limits, configured per VSYS. This prevents one noisy or misconfigured tenant from starving the session table or rule capacity that every other VSYS on the box also depends on.

Overall platform maximums (total sessions, total rules the hardware can hold) are fixed by the model; VSYS resource limits divide that fixed pool **deliberately** rather than leaving it to whichever VSYS happens to consume it first.

In practice

The two together are the actual multi tenancy guarantee: **administrative isolation** (a VSYS admin cannot see other tenants) plus **resource isolation** (one tenant cannot exhaust shared capacity). Neither alone is sufficient for a genuine multi tenant design.

125 When would you choose VSYS over separate physical firewalls?

This is a genuine design judgement question, and it is worth having the trade off ready rather than just the mechanics.

VSYS makes sense when

Tenants or business units need **genuine policy and administrative separation** but do not individually need dedicated hardware, the workloads are modest enough that sharing a platform's capacity is efficient, and the cost or rack space of separate physical appliances per tenant is not justified.

It also makes sense where **centralised lifecycle management** matters: one PAN-OS image, one set of hardware to patch and maintain, versus that multiplied by however many separate firewalls a fully physical design would need.

Separate firewalls make more sense when

A tenant has **compliance requirements demanding physical separation**, not just logical, some regulatory regimes specifically require this, and VSYS logical isolation does not satisfy it. Or when a tenant's traffic volume is large enough that **sharing platform resource limits** with other VSYS becomes a genuine capacity risk regardless of how carefully those limits are tuned. Or when a **single point of hardware failure** affecting every VSYS simultaneously is not an acceptable risk profile for that workload.

A shared HA pair running multi-vsyz still shares the fate of that pair: if the platform itself has a catastrophic failure, every VSYS on it goes down together, which physically separate firewalls per tenant would not.

In practice

I would frame it as: VSYS buys efficient logical separation on shared hardware; separate firewalls buy true physical and fault isolation at the cost of managing more boxes. The choice comes down to what the compliance and blast radius requirements actually demand, not just how many tenants there are.

✓ Check yourself

inter VSYS routing and administration

1. Distinguish external zone from shared gateway.

External zone **connects one VSYS to another specific VSYS**. Shared gateway **connects many VSYS to one common external network**.

2. Does traffic crossing an external zone or shared gateway bypass security policy?

No. **Both are genuinely routed, and policy is enforced at each crossing**.

3. A scenario says "two departments need to reach a shared database". Which mechanism?

External zone **between their two VSYS**.

4. What can a VSYS administrator see outside their assigned VSYS?

Nothing: **other VSYS and shared platform settings are invisible to them entirely**.

5. Why does PAN-OS let you set per VSYS resource limits?

To stop one VSYS exhausting shared capacity (sessions, rules, zones) that every other VSYS on the platform also depends on.

6. Name one reason to choose separate physical firewalls over VSYS.

A compliance requirement for physical, not just logical, separation, or an unacceptable shared blast radius on hardware failure.

✓ Anchor facts

- ✓ **External zone** = VSYS to a specific other VSYS, policy still applies.
- ✓ **Shared gateway** = many VSYS to one common external network.
- ✓ Neither bypasses inspection, **both are genuinely routed**.
- ✓ **VSYS admins** see only their assigned VSYS; the **superuser** sees all.
- ✓ Resource limits cap **sessions, rules, zones** per VSYS.
- ✓ VSYS shares the fate of the underlying hardware/HA pair.



Easy to get wrong

- Assuming external zone traffic skips policy because it's "internal".
- Confusing external zone (VSYS to VSYS) with shared gateway (VSYS to internet).
- Believing VSYS gives physical isolation. It does not.
- Not setting resource limits and letting one VSYS starve the rest.
- Forgetting a VSYS admin cannot even see other VSYS exist.

	External zone	Shared gateway
Connects	One VSYS to another	Many VSYS to one external network
Owned by	A VSYS, referencing another	No single VSYS
Policy enforced?	Yes, both sides	Yes, at the gateway boundary
Typical use	Two tenants sharing one service	Shared internet exit for all tenants

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Part 14 · Web Proxy and Logging⁴⁰

Standard Inspection, Explicit and Transparent Web Proxy

Configuring web proxy on PAN-OS is a broad topic in practice, covering two genuinely different deployment models with different client requirements and different visibility.

126 Standard network inspection versus explicit versus transparent proxy

The distinction underneath all three modes is whether the firewall **terminates the TCP session** at all. Get this one fact right and the rest of the comparison follows from it.

Standard network inspection (Routed / Gateway)

This is the **default mode** almost every deployment runs in, whether the interfaces are Virtual Wire, Layer 2 or Layer 3. The firewall sits as a **gateway the traffic routes through**, inspecting the **same client to server TCP session** as it passes. It never ends that session and starts a new one. App-ID, Content-ID and decryption all still run in full, but structurally the firewall is a **Layer 3 hop**, not an endpoint the client is talking to.

Web proxy mode is a genuinely different architecture: the firewall **becomes an actual TCP endpoint**. It terminates the client's connection, then **opens a separate, new connection** to the destination server on the client's behalf. That's what makes proxy mode a true man in the middle at the application layer, rather than an in path inspector, and it's why proxy configuration lives apart from ordinary security policy. The two proxy modes below only differ in whether the **client knows** it's talking to a proxy at all.

Explicit proxy

The client is **deliberately configured** to send its web traffic to the firewall's proxy address and port, either by **manual browser settings** or, far more commonly at scale, a **PAC (Proxy Auto Configuration) file** that the browser fetches and evaluates to decide which requests go to the proxy.

Because the client is aware, explicit proxy naturally supports **proxy authentication**: the browser can present credentials directly to the proxy, independent of any other identification method. It is configured with a dedicated **proxy port** and a **PAC file location** the firewall serves out.

Transparent proxy

The client sends traffic **normally, with no awareness** of a proxy. The firewall intercepts it in path (the traffic was already routed through it as a Layer 3 gateway) and proxies it without any client side configuration at all.

Transparent mode's advantage is **zero endpoint configuration**: nothing to push to every device, nothing that can be bypassed by a user changing their own proxy settings. Its cost is that **proxy level authentication is harder**, since the client never explicitly addressed a proxy and so has no natural place to present credentials to it; identification typically falls back to User-ID mapping instead.

The distinction worth stating plainly

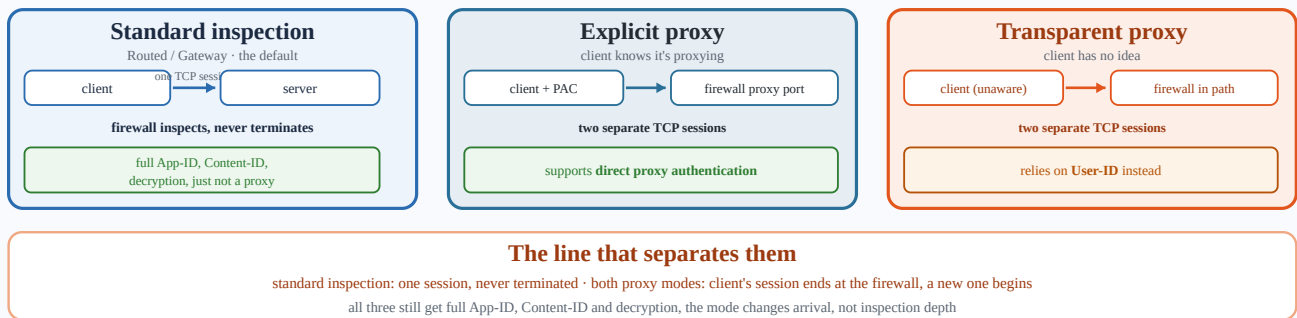
Standard inspection passes one continuous session through and never terminates it. Explicit and transparent proxy both terminate the client's connection and originate a new one: they differ only in whether the client is aware that's happening. All three still get full App-ID, Content-ID and decryption; the mode decides how traffic *arrives and is structured*, not what happens to it once there.

In practice

If a scenario describes ordinary firewall deployment with no proxy configuration mentioned, that's standard inspection, the default. If it mentions PAC files or proxy credentials, that's explicit proxy. If it mentions BYOD, unmanaged devices, or "no client configuration" specifically for web traffic, that's transparent proxy.

Three modes, one real dividing line

does the firewall terminate the TCP session, or just inspect it in flight?



127 PAC files and proxy authentication

A **PAC (Proxy Auto Configuration) file** is a JavaScript file a browser fetches and evaluates for every request, returning which proxy (if any) to use. The firewall **hosts and serves** the PAC file, and clients are pointed at its URL, typically via **DHCP option 252** or **Group Policy** pushing the setting centrally rather than configuring each browser by hand.

The PAC file's logic can differentiate by **destination**, sending internal traffic direct and only proxying internet bound requests, or by other conditions the script defines, giving central control over proxy behaviour without touching every endpoint individually when the logic changes.

Proxy authentication

Under explicit proxy, the firewall can require the client to **authenticate directly to the proxy** before requests are served, using **Basic, NTLM, or Kerberos** authentication methods, tied back to the same authentication profiles and server profiles covered in Part 9.

This gives an **additional identification path** alongside User-ID mapping, useful specifically where a device is not on a domain and cannot be mapped by the usual DC-log or agent methods, but the user can still present credentials directly to the proxy.

Successful proxy authentication can also feed a **user to IP mapping**, the same way an Authentication Policy challenge does, so downstream security policy written against users benefits even when the identification method was the proxy layer rather than User-ID's usual sources.

In practice

PAC based deployment via DHCP or GPO is what makes explicit proxy viable at scale, manually configuring proxy settings on every browser does not survive contact with a real estate. Kerberos proxy auth in particular gives single sign on with no separate login prompt for domain joined machines.

128 Log types and what writes to each

PAN-OS separates logs by **subject matter**, and knowing which log a given event lands in is what makes an investigation fast rather than a search across everything.

Traffic log: every session matching a security rule with logging enabled: source, destination, application, rule matched, action, bytes, and the session end reason covered in the troubleshooting material. The broadest log, and the starting point for most investigations.

Threat log: anything a security profile caught, with the **Type** field naming the profile (virus, spyware, vulnerability, url, file, wildfire, data), exactly as covered in Part 7.

URL Filtering log: a more detailed view specifically of URL category decisions, sometimes consolidated into the Threat log depending on configuration.

Decryption log, from Part 8: cipher, TLS version, certificate details, and the error reason for any decryption failure.

System log: device level events: HA state changes, commit results, licence expiry, interface up/down.

Configuration log: every configuration change, who made it, and what changed, which is the audit trail for compliance.

GlobalProtect log: connection and authentication events from Part 11's VPN material.

HIP Match log: posture evaluation results, also from Part 11.

User-ID log: mapping events: source, method, and timing of every user to IP mapping learned.

Each log type can be viewed at [Monitor > Logs](#) with its own filter syntax, and each can be **independently forwarded** to different destinations via log forwarding profiles, covered next.

In practice

Match the investigation to the log by what you are actually asking. "Was this allowed" is Traffic. "What did a profile catch" is Threat. "Who changed this rule" is Configuration. Picking the wrong log wastes time searching in the wrong place.

✓ Check yourself

standard inspection, proxy modes and log types

1. What's the fundamental structural difference between standard network inspection and either proxy mode?

Standard inspection passes one continuous TCP session **through and never terminates it. Both proxy modes** terminate the client's session **and originate a new one to the server.**

2. What is the core difference between explicit and transparent proxy?

Whether the client is aware it is talking to a proxy. Explicit is configured deliberately; transparent intercepts unannounced.

3. Which mode naturally supports direct proxy authentication, and why?

Explicit: the client knowingly addresses the proxy, giving it a place to present credentials. Transparent relies on User-ID instead.

4. How is a PAC file typically distributed at scale?

DHCP option 252 or Group Policy, rather than manual per browser configuration.

5. Which log names the security profile that caught a threat?

The Threat log, via its Type field.

6. Which log is the audit trail for who changed what configuration?

The Configuration log.

7. A scenario says "no client configuration allowed, BYOD environment". Which proxy mode?

Transparent: zero endpoint configuration is its defining advantage.

✓ Anchor facts

- ✓ **Standard inspection** = default mode, one session, never terminated. **Both proxy modes** terminate and reoriginate.
- ✓ **Explicit** = client aware, PAC file, supports direct proxy auth.
- ✓ **Transparent** = client unaware, in path, relies on User-ID.
- ✓ PAC distributed via **DHCP option 252** or GPO.
- ✓ Proxy auth methods: **Basic, NTLM, Kerberos**.
- ✓ Ten log types, each independently forwardable.
- ✓ Match the investigation to the log by **what question you're asking**.



Easy to get wrong

- Assuming transparent proxy supports proxy level auth as easily as explicit.
- Manually configuring browsers instead of using a PAC file at scale.
- Searching the Traffic log for something that only appears in Threat.
- Forgetting App-ID and decryption apply the same regardless of proxy mode.
- Confusing System log (device events) with Configuration log (change audit).

Log	Records	Key field
Traffic	Every logged session	App, rule, action, session end reason
Threat	Security profile hits	Type names the profile
Decryption	TLS sessions and failures	Cipher, version, error reason
System	Device level events	HA, commits, licensing
Configuration	Config changes	Admin, what changed
User-ID	Mapping events	Source, method, timing

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 14 · Web Proxy and Logging⁴¹

Log Forwarding Profiles and Built in Actions

Logs are only useful once they leave the firewall. Forwarding profiles decide where each log goes, and built in actions turn a log event into an automatic response.

129 Log forwarding profiles and match lists

A **Log Forwarding Profile** at `Objects > Log Forwarding` decides where logs go once generated. It is attached to security rules (for Traffic and Threat logs) or configured at a system level (for System, Configuration and other device logs), and without one attached, logs stay local to the firewall.

Inside a profile, you define **match list profiles** (one per log type (Traffic, Threat, URL Filtering, WildFire, Decryption, and so on)) each specifying a **filter** for which log entries qualify, and a list of **forwarding destinations**: syslog, email, SNMP trap, HTTP, and the Panorama or Strata Logging Service pipeline covered on the next page.

Filtering, not just forwarding everything

A match list's filter narrows what actually gets sent. A common pattern is forwarding **only high and critical severity Threat log entries** to an email destination for immediate attention, while sending the **full unfiltered log** to the long term Strata Logging Service or Panorama collector for retention and later analysis.

This is the same principle as tuning a security profile to reduce noise, applied to what gets sent onward rather than what gets blocked.

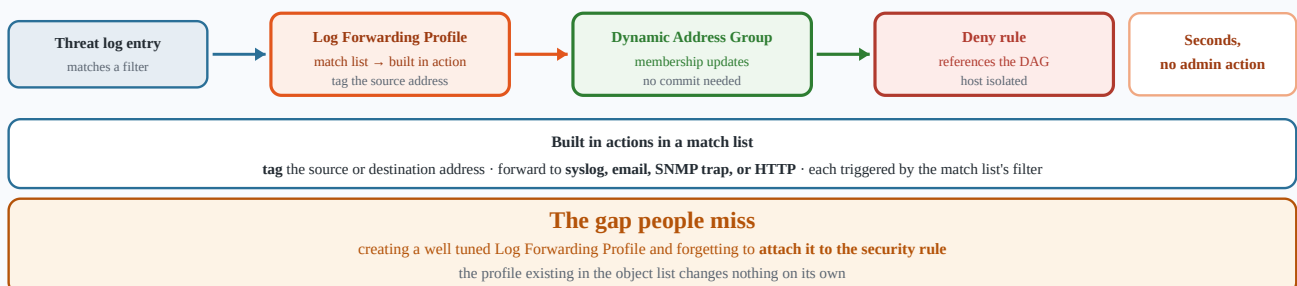
Verify a profile is actually attached and working with `show log-settings` and by checking the destination directly, a syslog server with nothing arriving usually means the profile was never attached to the rule generating the traffic, not a network problem between the firewall and the collector.

In practice

The most common early deployment gap is creating a well configured Log Forwarding Profile and forgetting to **attach it** to the security rules that need it. The profile existing changes nothing on its own.

The auto quarantine pattern, revisited with the actual objects

this is the same chain from Part 6, now with the forwarding profile named



130 Built in actions and tagging

A match list's **built in action** is what turns a log event from passive record keeping into an active response, and it is configured directly inside the log forwarding profile rather than needing an external script or SIEM to react.

The tag action, the one to know cold

On a match, the firewall can **automatically tag the source or destination IP address** involved in that log entry. This is exactly the mechanism covered as the auto quarantine pattern in Part 6: a Threat log entry matching "critical severity" triggers a tag, a Dynamic Address Group already built around that tag picks up the newly tagged address with **no commit required**, and a deny rule already referencing that DAG isolates the host within seconds.

The whole chain from detection to containment requires **no administrator to be watching** at the moment it happens, the objects were built once, and they react automatically every time the pattern matches again.

Forwarding actions

Beyond tagging, a match list forwards matching entries to one or more destinations: **syslog** servers (feeding a SIEM), **email** for direct alerting, **SNMP trap** for integration with existing monitoring, or **HTTP** for a webhook style integration with an external system.

Multiple actions can attach to the same match list, tag the address **and** forward to syslog **and** send an email, all from one matching log entry.

In practice

The tag action is the detail that turns "we log everything" into "we respond automatically". When a scenario describes automatic quarantine or isolation with no human intervention, this built in tag action attached to a match list is the mechanism, not a separate orchestration platform.

131 Designing a logging and forwarding strategy

Pulling the log types and forwarding mechanisms together into an actual design, the questions worth answering deliberately rather than defaulting on.

What needs immediate attention versus long term retention

Critical and high severity Threat entries, HA state changes, and licence expiry warnings typically warrant **email or SNMP trap** for immediate visibility. The full, unfiltered log volume goes to **Panorama or the Strata Logging Service** for retention, correlation and later investigation, different urgency, different destination.

What feeds automated response

Anything with a Dynamic Address Group waiting on it needs the **tag** built in action on a tightly scoped match list, too broad a filter tags too much and quarantines legitimate traffic; too narrow misses genuine incidents.

What feeds the SIEM

Syslog forwarding to a central platform, typically with **most or all log types** included, since correlation across Traffic, Threat, User-ID and Configuration logs is exactly what a SIEM is for and narrowing the feed defeats that purpose.

Retention and compliance

How long logs are kept is often a **compliance driven decision** (from Part 8's material on this) rather than a technical one, PCI-DSS and similar frameworks specify minimum retention periods that the forwarding destination needs to actually support.

I would design it as: **tag and quarantine narrow and precise, SIEM feed broad and inclusive, retention driven by whatever compliance framework applies**, and confirm every profile is actually attached before assuming it is working.

In practice

This is where logging strategy stops being a checklist item and becomes a design decision with real trade offs, over broad automated tagging causes outages, under broad SIEM feeds miss the correlation that justified buying a SIEM in the first place.

✓ Check yourself

forwarding profiles and built in actions

1. What does a Log Forwarding Profile actually contain?

Match list profiles: **one per log type, each with a filter and a set of forwarding destinations or actions.**

2. Why does a syslog server sometimes receive nothing despite a correctly built profile?

The profile was never attached **to the security rule (or system log setting) generating the traffic.**

3. Which built in action drives the auto quarantine pattern from Part 6?

The tag action: tagging the source or destination address, which a waiting Dynamic Address Group picks up automatically.

4. Name the four forwarding destination types a match list can send to.

Syslog, email, SNMP trap, HTTP.

5. Why would you filter the email destination to critical/high severity only, but not the SIEM feed?

Email is for immediate human attention: **too much noise defeats it. The SIEM needs** broad, correlatable data **to do its job.**

6. What risk does an overly broad tag and quarantine filter create?

It tags and isolates legitimate traffic, turning an automated defence into a self inflicted outage.

✓ Anchor facts

- ✓ A **Log Forwarding Profile** holds match list profiles, one per log type.
- ✓ A profile does **nothing until attached** to a rule or log setting.
- ✓ Built in actions: **tag** (address), plus forward to syslog/email/SNMP/HTTP.
- ✓ The **tag** action is the mechanism behind auto quarantine.
- ✓ Filter narrowly for automated response; keep the **SIEM feed broad**.
- ✓ Retention is often a **compliance driven** decision.



Easy to get wrong

- Building a profile and forgetting to attach it anywhere.
- Filtering the SIEM feed too narrowly and losing correlation value.
- An overly broad tag filter quarantining legitimate hosts.
- Assuming a match list can only do one thing, several actions can stack.
- Ignoring compliance mandated retention when designing the destination.

Need	Filter breadth	Destination	Action
Immediate human alert	Narrow: critical/high only	Email, SNMP trap	Forward
SIEM correlation	Broad: most/all entries	Syslog	Forward
Automated quarantine	Precise: tightly scoped	Local DAG	Tag
Long term retention	Full, unfiltered	Panorama / Strata Logging Service	Forward

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 14 · Web Proxy and Logging⁴²

Strata Logging Service, Collectors and Collector Groups

Strata Logging Service, log forwarding, and log collectors and collector groups are the three pieces covered here. The cloud path and the on premises path, and how they relate.

132 Strata Logging Service

Strata Logging Service (SLS) is Palo Alto's **cloud delivered** log storage and management platform, firewalls send logs directly to the cloud rather than to on premises log collectors, and it is licensed by **retention volume and period** rather than by hardware.

Why it exists

On premises Panorama log collectors require sizing, patching, storage expansion and hardware lifecycle management. SLS removes that entirely, Palo Alto operates the storage and scaling, and the firewall estate just **sends logs to a cloud endpoint** rather than managing collector capacity locally.

Firewalls are onboarded to SLS via a **Customer Support Portal** tenant association, and log forwarding to it is configured the same way as any other destination in a Log Forwarding Profile, SLS is simply one more forwarding target alongside syslog, Panorama, email and the rest.

The relationship with Panorama

Panorama can **query SLS directly** for log viewing, reporting and ACC dashboards, rather than needing its own local collector to hold the data. This means an architecture can be **Panorama for management and policy, SLS for log storage**, decoupling the two roles that a traditional on premises Panorama with local collectors would combine.

SLS also underpins several of the AI driven and cloud native features referenced elsewhere, Cortex integration, AIOps, and cross session correlation from Orion in Part 1 all depend on logs actually being centrally available at cloud scale, which is precisely what SLS is built to provide.

In practice

The one line contrast: **Panorama log collectors are infrastructure you own and size; SLS is a service you subscribe to and never size at all.** Which one a design uses is largely a build versus buy decision on log infrastructure.

Two log storage models, and where each firewall's logs actually land

build it yourself, or
subscribe to it

On premises collectors

Panorama Log Collectors, **you size and patch**

grouped into **Collector Groups** for redundancy

data plane capacity is a fixed, owned resource

Strata Logging Service

cloud delivered, licensed by volume and period

firewalls send logs **directly to the cloud**

no hardware sizing, **never runs out of room**

Panorama can query either

management and policy through Panorama, log storage through **local collectors OR SLS**
the one line frame: *collectors are infrastructure you own; SLS is a service you subscribe to*

133 Log collectors and log collector groups

A **Log Collector** is a Panorama role, either a dedicated **M-Series/CN-Series appliance** running in Log Collector mode, or Panorama itself acting as its own local collector for smaller deployments, that receives and stores logs forwarded from managed firewalls.

Multiple collectors are organised into a **Log Collector Group**, which provides two things a single collector cannot: **redundancy**, so a collector failure does not lose incoming logs, and **capacity**, since log volume is distributed across the group rather than limited to one appliance's storage.

How firewalls choose a collector

A managed firewall is assigned a **Collector Group** to forward to (not an individual collector), and within that group PAN-OS uses **forwarding preference** settings and current load to decide which specific collector actually receives each log stream, giving the group both load distribution and automatic failover if the preferred collector becomes unreachable.

Collectors within a group **replicate configuration and can be configured for log redundancy**, so a single collector's hardware failure does not mean permanent loss of the logs it held, depending on the redundancy settings chosen for that group.

Sizing a collector group is a genuine design exercise: **expected log rate, retention period, and desired redundancy level** together determine how many collectors and how much storage the group needs, exactly the sizing burden SLS on the previous page removes entirely.

In practice

The layering to keep straight: **firewall → Collector Group (not an individual collector) → specific collector chosen by preference and load** within that group. Misconfiguring the group assignment, rather than an individual collector, is the usual root cause when logs stop arriving from a specific site.

134 Forwarding preference and redundancy

Forwarding preference is the setting that decides, within a Collector Group holding several collectors, **which one a given firewall prefers to send its logs to first**, before falling back to others in the group.

This lets you deliberately steer a site's logs toward a **nearby collector** for latency reasons, a branch office preferring the regional collector over one across a WAN link, while still having the rest of the group available as automatic failover if that preferred collector becomes unreachable.

Redundancy within the group

PAN-OS supports configuring log redundancy across collectors in a group, so a given log entry is **stored on more than one collector** rather than existing as a single copy. The trade off is direct: redundancy protects against losing logs to a hardware failure, at the cost of the **storage capacity** and **write throughput** that duplication consumes.

Whether redundancy is worth that cost depends on what the logs are for, compliance mandated retention (Part 8's material) usually justifies it, while a purely operational troubleshooting log with a short retention window may not.

Health of a Collector Group is monitored via `show log-collector-es-cluster health` and Panorama's own collector status dashboard, which reports per collector storage utilisation, connection state, and whether the group's configured redundancy level is actually being maintained.

In practice

Forwarding preference is a **latency and locality** decision; redundancy is a **durability versus cost** decision. They are configured together in the same Collector Group but answer two genuinely different design questions.

✓ Check yourself

SLS, collectors and collector groups

1. What is the fundamental difference between SLS and a Panorama log collector?

SLS is cloud delivered and unsized, **subscribed to by volume/period**. A collector is infrastructure you own **and must size and maintain**.

2. Can Panorama query logs stored in SLS?

Yes: Panorama can query SLS directly for viewing, reporting and ACC, decoupling management from log storage.

3. Does a firewall forward to an individual collector or to a group?

A Collector Group. The specific collector within it is chosen by forwarding preference and current load.

4. What two things does grouping collectors provide that one collector cannot?

Redundancy (**survives a collector failure**) and capacity (**log volume spread across the group**).

5. What does forwarding preference actually control?

Which collector in the group a firewall prefers first, typically for locality/latency, before falling back to others.

6. What is the cost of enabling log redundancy across a collector group?

Storage capacity and write throughput: **each log entry is stored on more than one collector**.

✓ Anchor facts

- ✓ **SLS** = cloud delivered, licensed by volume/period, no sizing.
- ✓ Panorama can query **SLS or local collectors** for the same views.
- ✓ Firewalls forward to a **Collector Group**, not an individual collector.
- ✓ Groups provide **redundancy** and **capacity** a single collector cannot.
- ✓ **Forwarding preference** = locality/latency. **Redundancy** = durability vs cost.
- ✓ `show log-collector-es-cluster health` for group status.



Easy to get wrong

- Assuming logs missing from a site means a single collector fault, not a group assignment issue.
- Treating SLS and Panorama as mutually exclusive rather than combinable.
- Enabling redundancy without accounting for the storage cost.
- Not setting forwarding preference, leaving a branch defaulting across a WAN link.
- Sizing on premises collectors for peak rather than sustained log rate.

	Panorama Log Collector	Strata Logging Service
Where it runs	On premises, your hardware	Cloud delivered
Sizing	You size and maintain it	None, scales automatically
Licensing	Hardware + Panorama	Volume and retention period
Grouped for	Redundancy, capacity	N/A, managed by Palo Alto
Queried by	Panorama	Panorama, directly

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 15 • Panorama and ACC ⁴³

Modes, Onboarding and Device Groups

Panorama, templates and device groups, and pre/post rulesets are the three pieces covered here. Start with what Panorama actually is, then the container that holds shared policy.

¹³⁵ Panorama modes and adding firewalls

Panorama is **centralised management** for an estate of firewalls: policy, objects, software, licensing, and logging can all be controlled from one place instead of touching every device individually. It runs in one of three modes.

Management only: Panorama's role is purely configuration and policy management; log storage is handled elsewhere, either by dedicated Log Collectors or by Strata Logging Service.

Panorama mode (management + local log collector): the default for a standalone M-Series or VM appliance, combining management with its own local log collection.

Log Collector mode: a dedicated appliance running **only** as a Log Collector, with no management function, joined to a Panorama management instance and a Collector Group as covered in Part 14.

Adding a firewall to Panorama

Three things need to line up: the firewall's **management connectivity to Panorama** (reachable, correct Panorama IP configured on the firewall), the firewall added to Panorama's **managed devices** list by serial number, and the firewall assigned into a **device group** and a **template stack** so it actually receives shared configuration rather than sitting unmanaged.

Once connected, the firewall's **local configuration** and **Panorama pushed configuration** coexist: templates and device groups push their content, but a firewall retains some settings that only exist locally and are never overridden.

Verify with `show panorama-status` on the firewall (connection state, last push and commit results) and the **Managed Devices** health view in Panorama itself.

In practice

A firewall showing as "connected" in Panorama but not receiving policy updates is almost always a **device group or template assignment gap**, not a connectivity problem, connectivity and configuration assignment are two separate things that both have to be right.

Two hierarchies, two jobs

device groups carry policy and objects; templates carry device and network settings

Device Groups

policy, NAT, objects

hierarchical: child groups inherit from parent

a firewall belongs to **exactly one** device group

"what security rules and objects apply"

Templates & Template Stacks

interfaces, zones, routing, device settings

stacked: several templates layer, top wins conflicts

a firewall is assigned a **stack**, not one template alone

"what the device itself looks like"

Both must be assigned, or the firewall receives nothing from Panorama

device group without a template stack: policy pushed, but no shared interface/zone/routing config, and vice versa

136 Device groups and hierarchy

A **Device Group** is a logical grouping of managed firewalls that share **security policy, NAT policy, and objects**. It is the vehicle for pushing rules and address/service/application objects to many firewalls at once rather than configuring each individually.

Hierarchy

Device groups can be nested: a child device group **inherits** the shared policy and objects of its parent, and can add its own on top. A common design uses a top level group for organisation wide baseline rules (block known bad EDLs, compliance requirements) with child groups per region, business unit, or function adding their own specific rules beneath that baseline.

A firewall is assigned to **exactly one device group**, typically the most specific one relevant to it, and receives the **full inherited chain** of policy from every ancestor group above it in the hierarchy, not just its own group's content.

Shared is the topmost, implicit level above all device groups, for objects and policy meant to apply organisation wide regardless of hierarchy position.

Objects can be defined at **Shared, at any device group level, or locally on the firewall itself**, and PAN-OS resolves naming conflicts by a defined precedence, with more specific (closer to the firewall) generally winning over more general.

In practice

Design the hierarchy around what should genuinely be common versus what is truly local, over centralising into Shared makes every change affect the whole estate; under using hierarchy means duplicating the same rule across many device groups by hand.

37 Device group objects and policies

Within a device group, the policy and object model mirrors what a standalone firewall has, but scoped to **however many firewalls belong to that group** rather than one device.

Objects: address, service, application, and security profile objects created at the device group level are pushed to every member firewall, keeping definitions consistent estate wide rather than drifting per device.

Policies: security and NAT rules created here push to every firewall in the group (and its descendants, given inheritance). A rule referencing an object defined at the **same device group level or above** resolves correctly on push; a rule accidentally referencing a purely local object on one firewall will not push consistently to others that lack it.

Overrides

A specific firewall can be given a **local override** of a pushed object or, in some cases, policy behaviour, for a genuine device specific exception. This should be the deliberate exception, not the normal pattern, overusing overrides defeats the purpose of centralising through a device group in the first place.

Managing which firewalls belong to which device group, and reviewing what each group actually pushes, is done from [Panorama > Device Groups](#), with a **Preview Changes** function letting you see the exact configuration diff a commit and push will produce **before** it happens.

In practice

Always use **Preview Changes** before a device-group push affecting many firewalls at once. It is the equivalent of Terraform's plan step, showing the delta before it is applied, and it is the single best way to avoid an unintended estate wide policy change.

✓ Check yourself

Panorama modes and device groups

1. Name Panorama's three modes.

Management only, Panorama mode (management + local collector), Log Collector mode (**dedicated, no management**).

2. A firewall shows connected in Panorama but never receives policy. Most likely cause?

Missing device group or template stack assignment: **connectivity and configuration assignment are separate**.

3. How many device groups can one firewall belong to?

Exactly one, **typically the most specific, but it inherits from every ancestor above it**.

4. What is the topmost, implicit level above all device groups?

Shared: **for objects and policy meant to apply organisation wide**.

5. What does Preview Changes let you do before a push?

See the exact configuration diff **a commit and push will produce, before it actually happens**.

6. Should local overrides on a firewall be common or exceptional?

Exceptional. **Overusing them defeats the purpose of centralising through device groups**.

✓ Anchor facts

- ✓ Modes: **management only**, **Panorama (mgmt + collector)**, **Log Collector only**.
- ✓ A firewall needs **both** a device group and a template stack assigned.
- ✓ Device groups carry **policy, NAT, objects**; hierarchical, one group per firewall.
- ✓ **Shared** sits above all device groups.
- ✓ Firewalls inherit the **full ancestor chain**, not just their own group.
- ✓ **Preview Changes** shows the diff before a push, use it every time.



Easy to get wrong

- Assuming "connected" means "configured". They are separate checks.
- Over centralising everything into Shared.
- Pushing policy referencing an object that does not exist on every target firewall.
- Skipping Preview Changes on a broad push.
- Treating local overrides as the normal pattern rather than the exception.

Mode	Manages policy?	Stores logs?
Management only	Yes	No, elsewhere
Panorama (default)	Yes	Yes, locally
Log Collector mode	No	Yes, only this

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 15 · Panorama and ACC ⁴⁴

Pre/Post Rulesets and Templates

Two named subitems in one page: how device group rules interleave with a firewall's own local rules, and how templates push device level configuration.

138 Pre rules, post rules and evaluation order

This is the most examinable Panorama concept, because it determines **whose rules win** when both Panorama and a local firewall administrator write policy.

Every device group's rulebase splits into **Pre Rules** and **Post Rules**, and a managed firewall's own local rulebase sits **between the two** when everything is combined at evaluation time.

The full evaluation order on a managed firewall is:

1› **Device group Pre Rules** (from every ancestor in the hierarchy, most general first, most specific last) → 2› **the firewall's own local rules** → 3› **device group Post Rules** (most specific first, most general last) → 4› **the two implicit rules** (intrazone-allow, interzone-deny) from Part 6.

Why this ordering exists

Pre Rules are for **mandatory, organisation wide policy that must be evaluated first** and cannot be bypassed by a local rule, block known malicious EDLs, deny to sanctioned bad categories, enforce a compliance baseline. Because they run before local rules, a local administrator cannot accidentally or deliberately create a rule that overrides them.

Post Rules are for **catch all policy that should apply only if nothing more specific matched first**: a default deny and log everything else rule, or logging requirements that need to see whatever local rules did not already handle. Because they run after local rules, they act as a safety net rather than a blocker.

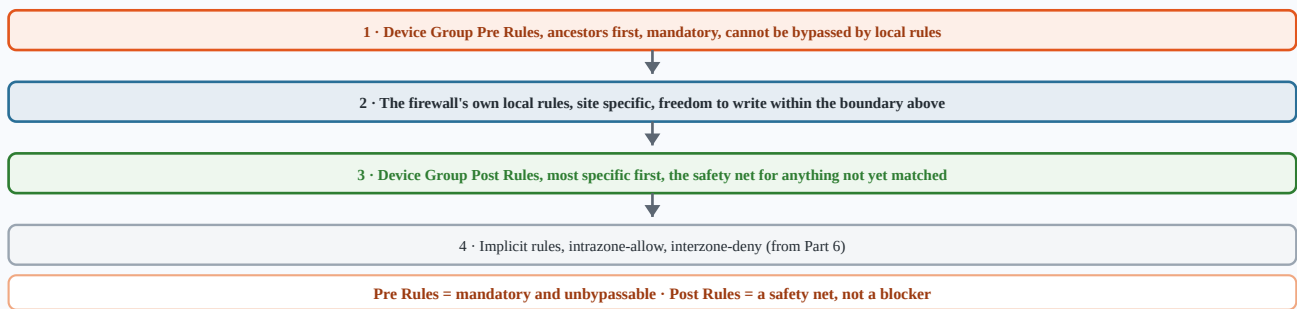
A local administrator's own rules sit in the middle, giving them freedom to write site specific policy **without being able to override the organisation wide Pre Rules** above them, and knowing their rules will still be caught by the Post Rules safety net if something falls through.

In practice

State the order precisely if asked: **Pre Rules, then local rules, then Post Rules, then the implicit rules**. This is the mechanism that makes centralised governance actually enforceable rather than advisory, a local admin physically cannot write a rule that bypasses a Pre Rule sitting above it in evaluation order.

The full evaluation order on a managed firewall

Pre Rules cannot be bypassed; Post Rules catch what falls through



139 Templates and template stacks

A **Template** pushes **device and network configuration** to managed firewalls: interfaces, zones, virtual/logical routers, HA settings, log forwarding profiles applied at the device level, and other settings from **Device** and **Network** tabs, the configuration a device group's policy and objects model does not cover.

Template stacks

A firewall is assigned a **Template Stack**, an ordered list of one or more individual templates, rather than a single template directly. Where multiple templates in a stack configure the **same setting**, the **template higher in the stack order wins** the conflict.

This lets you layer configuration the same way device groups layer policy: a **base template** with organisation wide standards (NTP servers, syslog destination, a baseline interface naming convention) with a **site specific template** stacked on top adding or overriding only what that site genuinely needs to differ.

Variables can be defined in a template and given a **different value per firewall** (or per template stack), a single template referencing a variable for "management IP" or "site specific VLAN ID" can be reused across many firewalls, each substituting its own value, rather than needing a nearly identical separate template per site.

As with device groups, a firewall can hold **local overrides** of template pushed settings for genuine device specific exceptions, visible and manageable from the firewall's own configuration view.

In practice

The variables feature is what makes a small number of templates scale across a large, heterogeneous estate, without it you would need a near duplicate template per site just to change one IP address, defeating the purpose of centralising the configuration at all.

140 Designing a device group and template hierarchy

Pulling the last two topics together into an actual design decision, since the real test is whether you can architect this, not just define the terms.

Device group design

Start with a **top level group for genuinely universal policy** (Pre Rules blocking known bad categories and EDLs, compliance mandated baseline rules) then branch into child groups by whatever dimension actually varies: region, business unit, or security tier. Keep the hierarchy as shallow as the estate genuinely needs; excessive depth makes it harder to reason about what a given firewall actually inherits.

Template design

Mirror the same instinct: a **base template** for settings true everywhere (NTP, DNS, syslog, a naming standard), with **site or role specific templates stacked above it** for genuine local variation, using **variables** rather than near duplicate templates wherever only a single value differs.

Where Pre/Post Rules fit into the design

Put anything that is a **compliance or security mandate**, the kind of control from Part 8's compliance material that should never be bypassable, into **Pre Rules** at the top level device group. Put catch all logging and default deny policy into **Post Rules** so local rules still get their chance to match first.

The design failure to avoid: putting **too much** in Pre Rules removes local flexibility entirely and turns every site specific need into an exception request to the central team; putting too little there means the "mandatory" baseline is not actually enforceable, since a local rule evaluated first could override it.

In practice

This is the Panorama equivalent of the automation design question from Part 6's Policy Optimizer material, centralise what should genuinely be centralised, leave room for legitimate local variation, and use the tooling (Preview Changes, variables, hierarchy) to keep the whole thing auditable rather than accumulating undocumented exceptions.

✓ Check yourself

pre/post rules and templates

1. State the full four stage evaluation order on a managed firewall.

Device Group Pre Rules → local rules → Device Group Post Rules → implicit rules.

2. Why can a local administrator never bypass a Pre Rule?

Pre Rules are evaluated before local rules in the order, so nothing local is ever reached first.

3. What is the purpose of Post Rules specifically?

A safety net for logging or default deny that should apply only if nothing more specific (including local rules) already matched.

4. What does a Template push that a Device Group does not?

Device and network configuration: interfaces, zones, routers, HA, not policy or objects.

5. In a template stack with a conflict, which template wins?

The one higher in the stack order.

6. What do template variables solve?

Letting one template be reused across many firewalls with per firewall values, avoiding near duplicate templates per site.

✓ Anchor facts

- ✓ Order: **Pre Rules** → **local** → **Post Rules** → **implicit**.
- ✓ **Pre Rules** = mandatory, unbypassable. **Post Rules** = safety net.
- ✓ **Templates** = device/network config. Assigned via a **Template Stack**.
- ✓ Stack conflict: **higher template wins**.
- ✓ **Variables** let one template serve many firewalls with different values.
- ✓ Local overrides exist on both device groups and templates, keep them rare.



Easy to get wrong

- Forgetting local rules sit **between** Pre- and Post Rules, not after both.
- Assuming Post Rules can override something. They only catch what's left.
- Duplicating near identical templates instead of using variables.
- Putting too much in Pre Rules and eliminating local flexibility.
- Putting compliance mandated rules in Post Rules where they can be bypassed.

Stage	Source	Purpose	Bypassable by local rules?
1	Device Group Pre Rules	Mandatory baseline	No
2	Local rules	Site specific policy	N/A, this is local
3	Device Group Post Rules	Catch all, logging, default deny	Yes, by design
4	Implicit rules	intrazone-allow, interzone-deny	Yes, always last

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 15 · Panorama and ACC ⁴⁵

The Commit Model, Panorama HA and Log Collection

Not a single feature by itself, but the single most operationally important Panorama concept, commit and push are two separate actions, and confusing them is the most common real world mistake.

④ The commit model: candidate, running, commit versus push

Panorama has its own **candidate configuration**, separate from any managed firewall's candidate configuration, and the relationship between changing something and it actually taking effect on a firewall involves **two distinct actions** that are easy to conflate.

Commit

Applies Panorama's **candidate configuration** to **Panorama's own running configuration**: this makes the change real **on Panorama itself**: visible in the GUI, saved, versioned. It does **not**, by itself, send anything to a single managed firewall.

Push

A separate action that takes Panorama's **already committed** device group and template configuration and sends it out to the **managed firewalls** assigned to those device groups and templates, where it becomes part of **their** candidate configuration, which they then must **commit locally** (automatically, as part of the push, in normal operation) to make active.

The critical implication

A Panorama **commit alone changes nothing on any firewall**. Only a **push** does. It is entirely possible to commit a change on Panorama and never push it, leaving every managed firewall running exactly what it was before, which is sometimes deliberate (staging several changes before pushing them together) and sometimes an operator simply forgetting the second step.

Commit Type options at push time let you choose **which device groups, templates, or even individual firewalls** receive this particular push, rather than always pushing to the entire managed estate at once, useful for staged rollouts of a risky change.

Locks

Panorama supports **configuration locks** the same way a standalone firewall does, preventing two administrators from committing conflicting changes simultaneously, scoped per device group or globally.

In practice

The two step model is the answer whenever a scenario says "I made a change in Panorama and nothing happened on the firewall", the change was **committed but never pushed**. This is the single most common Panorama operational mistake, and it is worth stating that plainly if asked.

Commit versus push

two separate actions, and the mistake that happens when only one is remembered



The mistake this model produces

committing on Panorama and stopping there, **changes nothing on any firewall**
the single most common real world Panorama operational error

Why it is sometimes deliberate

stage several committed changes on Panorama, then push them together in one **Commit Type** scoped to the intended firewalls
the deliberate version and the forgotten version look identical until you check

142 Panorama HA

Panorama supports **Active/Passive HA** between two Panorama instances, following the same conceptual shape as firewall HA from Part 12, applied to the management layer itself, because Panorama managing an entire estate is itself a single point of failure worth protecting.

Configuration synchronisation keeps the candidate and running configuration identical between the active and passive Panorama, and both instances can independently receive logs from managed firewalls (log data is **not** something that only flows through the active peer, unlike firewall session state over HA2).

Failure detection and election use the same underlying concepts as firewall HA (heartbeats, priority, and a defined active/passive relationship) though the specific link architecture differs since Panorama is a management platform rather than an inline traffic device.

Where this connects back

If Panorama itself is the single point of policy control for an entire estate, its own HA design deserves the same rigour as any critical piece of infrastructure covered elsewhere in this guide, and it is often the piece people forget to make redundant while carefully planning firewall HA everywhere else.

Verify with `show high-availability state` on Panorama itself (the same command family used for firewall HA), and the GUI's dedicated Panorama HA status view.

In practice

The point worth making explicitly: **losing Panorama does not stop firewalls from passing traffic**, they continue operating on their last pushed configuration, but it does stop **any centralised policy change or new firewall onboarding** until Panorama is restored. Panorama HA protects management continuity, not traffic continuity.

143 Log collection and forwarding through Panorama

This topic connects directly to Part 14: Panorama is the **management point** for the Log Collector Groups, forwarding preferences, and redundancy settings already covered there, but seen now from the Panorama configuration side rather than the collector side.

From **Panorama > Collector Groups**, you define which collectors belong to which group, configure the **forwarding preference and redundancy** settings per group, and assign managed firewalls to the appropriate Collector Group so their logs land in the right place.

Log forwarding profiles, revisited

The Log Forwarding Profiles from Part 14 are themselves objects that can be **pushed via device groups**, ensuring every firewall in an estate applies the same forwarding logic (which log types go where, which built-in actions fire) without configuring it individually per device, centralising the logging strategy the same way security policy is centralised.

Panorama's own log view at **Monitor > Logs** queries across **every managed firewall's forwarded logs** in one place, whether stored on local Collector Groups or in Strata Logging Service, giving a single estate wide search rather than needing to check each firewall's local logs individually.

This centralised view is also what feeds **ACC**, covered on the next page, the dashboards and reports there are only as complete as the logs actually being forwarded to Panorama or SLS in the first place.

In practice

If ACC or a Panorama report looks incomplete for a given firewall, check whether that firewall's **log forwarding profile** and **Collector Group assignment** are actually correct before assuming the reporting feature itself is broken.

✓ Check yourself

commit model, HA and log collection

1. Does a Panorama commit alone change anything on a managed firewall?

No. **Commit makes the change real on Panorama only; push is what sends it to firewalls.**

2. What must happen on the firewall after a push for the change to become active?

A local commit **on the firewall, which happens automatically as part of normal push behaviour.**

3. What does losing Panorama do to firewalls already passing traffic?

Nothing immediately: **they keep running their last pushed configuration. Panorama HA protects management, not traffic.**

4. Does log data only flow through the active Panorama HA peer?

No: **unlike firewall HA2 session state, both Panorama peers can independently receive logs.**

5. Can Log Forwarding Profiles be pushed via device groups?

Yes: **centralising logging strategy the same way security policy is centralised.**

6. ACC looks incomplete for one firewall. What would you check first?

That firewall's log forwarding profile and Collector Group assignment, not assume ACC itself is broken.

✓ Anchor facts

- ✓ **Commit** = real on Panorama. **Push** = sent to firewalls.
- ✓ Firewall still needs a **local commit**, automatic during a normal push.
- ✓ Commit without push is the most common real world Panorama mistake.
- ✓ Panorama HA protects **management continuity**, not traffic flow.
- ✓ Both Panorama HA peers can **independently receive logs**.
- ✓ Log Forwarding Profiles push via device groups, same as policy.



Easy to get wrong

- Assuming a committed change is already live on firewalls.
- Forgetting to scope **Commit Type** and pushing to the whole estate unintentionally.
- Thinking Panorama going down stops traffic. It does not.
- Blaming ACC for gaps caused by a missing forwarding profile.
- Not planning Panorama's own HA while carefully planning firewall HA.

Action	Affects	Requires a follow up step?
Commit (Panorama)	Panorama's own running config	Yes, push, or nothing reaches firewalls
Push	Managed firewalls' candidate config	Local commit, usually automatic
Commit (firewall, local)	That firewall's running config	No, final step

● NETWORKING ● DEVICE SETTINGS ● INTEGRATION ● CONTEXT

KNOW | CONFIGURE | VERIFY

Part 15 • Panorama and ACC ⁴⁶

Administration, ACC Dashboards and Troubleshooting

This page covers building ACC dashboards and custom reports. Pair it with RBAC and access domains, then close the part with how to troubleshoot Panorama itself.

144 Administrative accounts, RBAC and access domains

Panorama administration mirrors the firewall administration model from Part 9's Admin Role profiles, but adds one concept specific to managing an estate: **Access Domains**.

Admin roles on Panorama

Built in **dynamic roles** (Superuser, Panorama Administrator, and read only variants) plus **custom Admin Role profiles** defining exactly which tabs, features and commit/push capabilities an administrator has, the same granular model covered for firewalls, applied at the Panorama layer.

Access Domains, the Panorama specific piece

An **Access Domain** restricts an administrator's visibility and control to **specific device groups and templates**, rather than the entire managed estate. An administrator assigned to an Access Domain covering only the EMEA device groups cannot see, commit to, or push configuration affecting APAC firewalls at all, even though both are managed by the same Panorama instance.

This is the Panorama equivalent of **VSYS scoped administrators** from Part 13, the same delegated administration pattern, applied across an estate of separate physical firewalls rather than within one shared appliance.

Access Domains and Admin Role profiles combine: an administrator's **role** defines what actions they can take, and their **Access Domain** defines what they can take those actions on, a regional network team might get a role permitting full policy edits, scoped by Access Domain to only their own region's device groups.

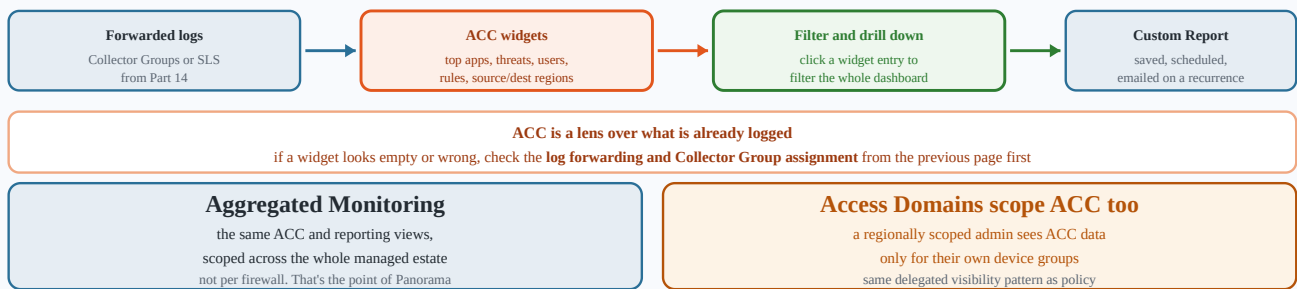
Authentication for Panorama administrators uses the **same authentication profile and sequence model** from Part 9, so RADIUS, SAML or certificate based login all apply here exactly as they do on a standalone firewall.

In practice

Access Domains are the answer whenever a scenario describes regional teams, managed service provider customers, or any requirement that one Panorama instance serve multiple administrative boundaries without those boundaries seeing each other's configuration.

ACC: built from logs, not policy

and the workflow from a dashboard widget to a saved report



145 ACC dashboards and custom reports

The **Application Command Center (ACC)** is a dashboard built from **logged data** (Traffic, Threat, URL Filtering and the other log types from Part 14) presenting it as widgets: top applications by session count, top threats, top source and destination users, rule usage, geographic breakdowns of traffic.

On Panorama, ACC operates in **Aggregated Monitoring** mode: the same widgets and drill down interactions, but scoped across the **entire managed estate** rather than one firewall's logs, which is the whole reason to view it from Panorama rather than logging into each device individually.

Interactivity

Widgets are **interactive**: clicking an entry (a specific application, a specific threat) filters the rest of the dashboard to that context, letting you narrow from "what does overall traffic look like" down to "show me everything related to this one application" without leaving the view.

Custom Reports

Beyond the built in ACC widgets, **Monitor › Manage Custom Reports** (or the Panorama equivalent) lets you define your own report against specific log types and filters, then **schedule it** to run on a recurrence and **email the results** automatically, turning an ad hoc investigation into a recurring, hands off piece of operational reporting.

Report Groups combine several individual reports into one scheduled bundle, useful for a weekly security summary combining threat trends, top denied applications, and admin activity into a single email rather than several separate ones.

Because ACC and reports are built entirely from forwarded logs, their completeness depends directly on the **log forwarding and Collector Group configuration** from the previous page, a firewall not forwarding logs correctly will simply be under represented or absent from these views, with no error to flag it.

In practice

Building ACC dashboards and custom reports is really about turning raw logs into something a non technical stakeholder can read on a schedule. Scheduled, emailed Custom Reports are the concrete deliverable that answers that.

146 Panorama troubleshooting

A structured approach for the most common Panorama problems, following the same evidence before action discipline used throughout this guide's troubleshooting material.

Firewall not receiving policy

Check, in order: is it **connected** at all (`show panorama-status` on the firewall), does it have both a **device group and template stack assigned**, and was the change actually **pushed** after being committed, the three part chain from earlier in this part, any of which failing produces the same symptom.

Commit or push failures

Check the **Task Manager** in Panorama for the specific commit or push job's error detail, a failed push often names exactly which firewall and which validation error caused it, rather than requiring guesswork.

Logs missing from ACC or Monitor

Work backward through Part 14's chain: is the **Log Forwarding Profile** attached to the relevant rules, is the firewall's **Collector Group assignment** correct, and is that Collector Group itself healthy (`show log-collector-es-cluster health`).

Panorama itself unreachable or degraded

Check Panorama's own **system resources** the same way Part 1 covers for a firewall's management plane, a Panorama instance managing a very large estate can itself become resource constrained, and its symptoms (slow GUI, delayed pushes) look exactly like the management plane symptoms covered there.

Configuration sync issues between an HA pair of Panoramas, or between Panorama and a managed firewall, generally resolve with a **manual full sync push** once the underlying connectivity problem causing the drift has been fixed, the same recovery pattern as firewall HA config sync from Part 12.

In practice

Nearly every Panorama problem in this guide reduces to one of three things: a **missing assignment** (device group, template, Collector Group), a **forgotten push**, or a **resource constraint**. Check those three before assuming something more exotic is wrong.

✓ Check yourself

administration, ACC and troubleshooting

1. What does an Access Domain restrict, specifically?

An admin's visibility and control to specific device groups and templates, **not the whole managed estate**.

2. What is the VSYS equivalent of a Panorama Access Domain?

VSYS scoped administrators **from Part 13, same delegated visibility pattern, different scale**.

3. What mode does ACC run in on Panorama, and what does it show?

Aggregated Monitoring: **the same widgets, scoped across the entire managed estate**.

4. What turns an ad hoc ACC investigation into recurring reporting?

A Custom Report, **scheduled with a recurrence and emailed automatically**. Report Groups **bundle several together**.

5. ACC is missing data for one firewall with no error shown. What do you check?

That firewall's log forwarding profile and Collector Group assignment: incomplete forwarding shows as absence, not an error.

6. Name the three most common root causes behind nearly every Panorama problem.

A missing assignment (device group/template/collector group), a forgotten push, or a resource constraint.

✓ Anchor facts

- ✓ **Access Domains** scope an admin to specific device groups/templates.
- ✓ ACC on Panorama = **Aggregated Monitoring**, estate wide.
- ✓ Widgets are **interactive**: click to filter and drill down.
- ✓ **Custom Reports** can be scheduled and emailed; **Report Groups** bundle them.
- ✓ ACC completeness depends entirely on **log forwarding being correct**.
- ✓ Most Panorama issues: **missing assignment, forgotten push, or resource constraint**.



Easy to get wrong

- Confusing Access Domain scoping (visibility) with Admin Role (permissions). They combine.
- Assuming an empty ACC widget means an error rather than missing forwarding.
- Forgetting Report Groups exist and building several separate scheduled reports instead.
- Not checking Task Manager for the actual push failure detail.
- Treating Panorama's own resource exhaustion as a mysterious fault rather than the same pattern as firewall management plane load.

Symptom	Check first
Firewall not receiving policy	Connection → device group/template assignment → was it pushed
Push or commit fails	Task Manager for the specific error
Logs / ACC data missing	Log forwarding profile and Collector Group assignment
Panorama slow or unresponsive	Panorama's own system resources
Config out of sync	Manual full sync push after fixing connectivity

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 15 • Panorama and ACC ⁴⁷

Quantum Readiness Across the Managed Estate

The same inventory problem the Quantum Readiness Dashboard solved for one firewall, back in Part 1, needs an answer at fleet scale too. This is that answer, inside Strata Cloud Manager.

⁴⁷ The Quantum Readiness view in Strata Cloud Manager

This is an **alternative view of the Command Center** in Strata Cloud Manager, reached via the settings panel, and it's built to solve four specific problems at once rather than being a single report.

What it actually solves

- **Eliminates the inventory blind spot:** the same "you don't know what crypto you're using" problem the Quantum Readiness Dashboard answers at the individual firewall level, here rolled up across the whole managed estate.
- **Provides a single pane of glass:** one view rather than checking each firewall's own posture separately.
- **Delivers actionable remediation steps,** not just a finding, a next action.
- **Tracks migration progress:** the same multi year sequencing problem covered in Part 1, now with something to actually measure against.

Evaluating posture: three states

the view classifies cryptographic posture into three tiers, based on the algorithms and protocols actually detected across the network:

- **Secure:** quantum safe algorithms.
- **Weak:** algorithms at quantum risk, still functional today but on the list to migrate.
- **Vulnerable:** deprecated algorithms, the highest priority remediation targets.

Minimum requirements, worth knowing precisely

To use the Quantum Readiness view at all, NGFWs onboarded into SCM must be:

- At least **generation four hardware**.
- Running **PAN-OS 11.1 or higher**.
- Configured with a **no-decrypt decryption policy rule** that enables **both successful and unsuccessful session logging**: the same no decrypt plus logging pattern that makes the threat signatures covered on the Decryption page usable without actually decrypting anything.
- Carrying a **vulnerability protection profile**.

The view itself is **interactive**: widgets update based on cursor position or click, and links drill through to more detailed data rather than everything being visible on one static screen.

In practice

The requirements list is the part worth having exact, gen4+ hardware, PAN-OS 11.1+, a no-decrypt rule logging both outcomes, and a vulnerability protection profile. Missing any one of the four means the view either won't populate or won't show the full picture, and it's a genuinely common setup gap.

✓ Check yourself

Quantum Readiness view

1. Name the three posture states the Quantum Readiness view classifies algorithms into.

Secure, Weak, Vulnerable.

2. What's the minimum PAN-OS version and hardware generation to use this view?

PAN-OS 11.1 or higher, **on** generation four **hardware or later**.

3. What kind of decryption policy rule does the view require, and what must it log?

A no-decrypt **rule**, **logging** both successful and unsuccessful **sessions**.

4. What fourth requirement, beyond hardware/OS/logging, does the view need?

A vulnerability protection profile **attached**.

✓ Anchor facts

- ✓ **Secure / Weak / Vulnerable**: the three posture tiers.
- ✓ Requires **gen4+ hardware**, **PAN-OS 11.1+**, a **no-decrypt rule logging both outcomes**, and a **vulnerability protection profile**.
- ✓ It's an alternative **Command Center** view in SCM, not a separate product.
- ✓ Solves the same inventory blind spot as the single firewall dashboard, at fleet scale.



Easy to get wrong

- Enabling the view without a no-decrypt rule that logs unsuccessful sessions too, only logging successes leaves a gap.
- Assuming any onboarded firewall qualifies, gen4+ hardware and 11.1+ are hard floors.
- Forgetting the vulnerability protection profile requirement entirely.

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 16 · Software Updates 48

Content Updates, PAN-OS Upgrades and Backup

A recurring operational duty that touches almost everything else in the guide, rather than a single self contained topic. HA upgrade sequencing itself was covered in Part 12 · T119. This page covers what runs on top of that: content, version paths, and getting back to a known good state.

148 Dynamic content updates and the new App-ID risk

PAN-OS separates **content** from the underlying software image. **Content updates** are smaller, more frequent packages, App-ID signatures, Threat and Anti-Spyware signatures, Antivirus signatures, WildFire signature updates, and URL category data, that refresh what the firewall can recognise without a full PAN-OS upgrade.

Managed from Device > Dynamic Updates: each content type shows its **current version**, can be checked for new releases, downloaded, and installed, with a schedule for automatic download and installation at a defined recurrence and time.

The risk that matters most, new App-IDs

A content update can introduce **new application signatures** that PAN-OS did not previously recognise. Traffic that used to match a broad rule via application-default or a general category can suddenly be identified more specifically and **match a different, more restrictive rule**, or no rule at all, and get denied. This is a genuine, if narrow, availability risk from what looks like routine maintenance.

The mitigation PAN-OS provides

Content release notes explicitly flag which App-IDs are new in that release. The disciplined pattern is: **review the release notes before installing**, use **threshold based deployment** (install content only after it has been available for a defined number of days, giving Palo Alto's own telemetry time to surface any widespread issue), and stage installation to a **pilot device or group** before an estate wide push via Panorama.

Antivirus and WildFire content are commonly set to **install more aggressively** (same day) since the risk of a missed threat signature usually outweighs the App-ID reclassification risk, while App-ID and Threat content are more often given a **short delay** for exactly this reason.

In practice

If a scenario describes traffic that was working suddenly breaking with **no policy or network change made**, a recent content update introducing a new App-ID match is one of the first things to check, alongside show system info for the content version and the Traffic log's application field for what the session is now being classified as.

Content updates versus PAN-OS upgrades

different cadence, different risk, different rollback story

Content updates

App-ID, Threat, AV, WildFire, URL, **frequent**

no reboot, applied live

risk: **new App-IDs reclassify existing traffic**

mitigate with threshold delay + release notes

PAN-OS upgrades

the base image, **infrequent**, planned

requires a **reboot**, real outage window

risk: **cannot always skip versions**

HA sequencing already covered in Part 12 · T119

The distinction worth stating precisely

content changes **what the firewall recognises**: PAN-OS upgrades change **what the firewall can run at all**

149 PAN-OS upgrade paths and version compatibility

PAN-OS releases are versioned major.minor (for example 11.1, 12.1), and **you generally cannot skip a major version** when upgrading, the firewall must pass through each intermediate major release's base image before installing the next, even if you never intend to run that intermediate version day to day.

Palo Alto publishes an **official upgrade path** per release on the support portal, and following it matters: jumping straight from an old major version to a much newer one is **not supported** and can leave the device in an inconsistent state, because each major release can carry schema or migration changes that the installer applies incrementally.

Preferred versus latest

Palo Alto marks certain releases as a **Preferred Release** (a maturity signal, distinct from simply the newest available build) and the general guidance for production is to run a **Preferred Release** rather than chasing the latest release the day it ships, mirroring the same "let the ecosystem catch problems first" thinking behind content threshold deployment.

Before any upgrade

Check the **release notes** for known issues and deprecated features, confirm **hardware and feature compatibility** (a feature relied on in the current configuration is occasionally deprecated in a newer release), verify sufficient **disk space** for the new image, and back up the configuration, the topic covered next.

Verify the current and available versions with `show system info` and `request system software info`, and check the upgrade path itself against the support portal's published matrix rather than assuming.

In practice

The specific trap worth naming: an admin upgrading a firewall that is several major versions behind and attempting to jump straight to current will find the installer refuses or the device behaves unpredictably. The fix is following the published path step by step, not forcing a shortcut.

150 Configuration backup and restore

PAN-OS automatically saves a configuration snapshot at every commit, visible and downloadable from

Device › Setup › Operations

, but relying only on automatic snapshots is not the same as a deliberate backup discipline.

Export options

Export named configuration snapshot downloads the current running configuration as an XML file. **Export configuration version** lets you pull any specific historical commit's saved version, useful for comparing what changed between two points in time. Both should be taken and stored **off device** before any risky change, a config backup that only exists on the device being changed does not help if that device is the thing that fails.

Restore

Load named configuration snapshot loads a previously exported file back into the **candidate** configuration, which then still needs a **commit** to become active, the same candidate/running distinction that governs every other configuration change on the platform. This means you can load an old configuration, review the diff against what is currently running, and decide deliberately before committing, rather than an immediate irreversible rollback.

Panorama managed firewalls

A firewall's **local** configuration and its **Panorama pushed** configuration (device groups, templates) are distinct, and a full recovery needs both: restoring the firewall's local backup, and confirming its device group and template assignment in Panorama is intact so the next push reapplies the shared configuration correctly.

Before any major change (a PAN-OS upgrade, a significant policy restructure, a decryption rollout) the standard practice is: **export a named snapshot immediately beforehand**, store it somewhere the firewall's own failure cannot also take out, and confirm you know the exact restore procedure before you need it under pressure.

In practice

The habit worth having cold for an interview answer: **before any risky change, export a named configuration snapshot off device first**. The automatic per commit history is a safety net, not a substitute for a deliberate, exportable, offsite backup taken at the moment that actually matters.

✓ Check yourself

content updates, upgrade paths and backup

1. Why can a content update break traffic with no policy change made?

A new App-ID can reclassify traffic that used to match a broader rule, causing it to hit a different, more restrictive rule, or none.

2. Name two mitigations for the new App-ID risk.

Threshold based deployment (delay installation by a set number of days) and reviewing content release notes before installing.

3. Can you generally skip a PAN-OS major version when upgrading?

No: you must follow the published upgrade path through each intermediate major release.

4. What does Palo Alto's "Preferred Release" designation mean?

A maturity signal for production use, distinct from simply the newest available build.

5. After loading a named configuration snapshot, is it immediately active?

No: it loads into the candidate configuration and still needs a commit.

6. Why is an on device automatic commit history not sufficient as a backup strategy?

It does not help if the device itself fails: a deliberate, exportable, off device backup is needed before risky changes.

✓ Anchor facts

- ✓ Content updates: App-ID, Threat, AV, WildFire, URL, **frequent, no reboot**.
- ✓ Biggest content risk: **new App-IDs reclassifying existing traffic**.
- ✓ PAN-OS upgrades: **cannot skip major versions**, follow the published path.
- ✓ **Preferred Release** ≠ latest release.
- ✓ Export a **named snapshot off device** before any risky change.
- ✓ A restored snapshot loads to **candidate**; still needs a commit.



Easy to get wrong

- Assuming AV/WildFire and App-ID/Threat content should follow the same deployment delay.
- Attempting to jump PAN-OS versions without following the published path.
- Relying only on automatic per commit history as the backup strategy.
- Forgetting Panorama pushed configuration is separate from a firewall's local backup.
- Not checking hardware/feature compatibility before an upgrade.

Change type	Cadence	Needs a reboot?	Primary risk
Content update	Frequent, often daily	No	New App-ID reclassifies traffic
PAN-OS upgrade	Planned, infrequent	Yes	Cannot skip major versions
Config restore	As needed	No, until committed	Loads to candidate, not live immediately

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 17 • Cloud and Orchestration 49

Deployment Across AWS, Azure and GCP, and Bootstrapping

Two deployment models exist for every major cloud, and confusing them is the most common design mistake. Then the mechanics of getting a VM-Series instance from zero to configured with no human logging in at all.

151 Self managed VM-Series versus Cloud NGFW

Every public cloud offers the same fundamental choice, and it is the first design decision on any cloud security project: **who manages the firewall instance itself?**

Self managed VM-Series

You deploy the VM-Series image yourself (from the AWS, Azure or GCP marketplace, or via Terraform) into your own VPC or VNet. You own the instance: sizing, patching, PAN-OS upgrades, and scaling are all your responsibility, typically managed through Panorama the same way an on premises estate is. This gives **full configuration control**, including features a managed service might not expose.

Cloud NGFW, the managed service

Cloud NGFW for AWS and **Cloud NGFW for Azure** are Palo Alto managed services, consumed natively through each cloud provider's own marketplace and billing, with the underlying firewall infrastructure, scaling, and PAN-OS version entirely managed by Palo Alto rather than the customer. Policy is still defined by the customer (via the cloud provider's own console, Panorama, or Strata Cloud Manager) but there is **no instance to patch or size**.

The trade off in one line

Self managed gives full control at the cost of operational overhead. Cloud NGFW gives zero infrastructure management at the cost of some configuration depth. A team already running Panorama at scale, with established automation, often prefers self managed for consistency. A team wanting cloud native simplicity, or without deep firewall operations experience, often prefers Cloud NGFW.

Both models still rely on the same underlying App-ID, Content-ID and threat prevention engine covered throughout this guide, the difference is entirely in **who operates the infrastructure**, not in inspection capability.

In practice

If a scenario asks about patching cadence, instance sizing, or PAN-OS version control in the cloud, that is **self managed VM-Series**. If it asks about a firewall with no instance to manage at all, consumed like any other cloud native service, that is **Cloud NGFW**.

Who operates the firewall

the same inspection engine, two very different operational models

Self managed VM-Series

deployed into your own VPC/VNet

you size, patch, upgrade, scale

full control, more operational overhead

managed via Panorama, same as on prem

Cloud NGFW (managed)

native marketplace service, AWS or Azure

Palo Alto sizes, patches, upgrades, scales

zero infrastructure management

policy still yours, via console/Panorama/SCM

The engine is identical either way

App-ID, Content-ID and threat prevention do not change, only **who operates the infrastructure** does

152 Bootstrapping: init-cfg.txt and bootstrap.xml

Bootstrapping is how a VM-Series instance goes from a blank image to a configured, licensed, managed firewall with **no administrator logging in at all**: essential for cloud auto scaling and any automated deployment pipeline.

The bootstrap package is a defined folder structure, `/config`, `/content`, `/license`, `/software`, `/plugins` : supplied at launch, in AWS typically as an S3 bucket referenced at instance creation.

The two configuration files, and how they combine

`init-cfg.txt` is **mandatory**: basic network settings for the management interface (static or DHCP), DNS, and optionally the Panorama IP, VM auth key, device group and template stack.

`bootstrap.xml` is **optional**: a full configuration, either written by hand or exported from an existing firewall's running configuration.

If both are present and their settings overlap, **init-cfg.txt wins**: the firewall merges the two but treats init-cfg.txt as authoritative on any conflict.

Auto registration with Panorama

A **VM auth key** generated on Panorama, included in init-cfg.txt, lets the newly booted firewall authenticate itself to Panorama on first contact. Supplying a device group and template stack in the same file means the firewall is **fully onboarded and receiving policy within minutes of boot**, with no manual step in Panorama at all.

The `/license` folder needs an **auth code bundle**, not individual auth codes, supplying individual codes means the firewall only retrieves the license tied to the first one it finds.

In practice

This is what makes VM-Series genuinely cloud native rather than "a firewall someone happened to also put in the cloud": launch, bootstrap, and the instance is licensed, configured, and managed, the entire lifecycle scriptable and repeatable, which is the actual point of an autoscaling group.

153 Autoscaling and lifecycle automation

Bootstrapping is what makes **autoscaling** possible: a cloud autoscaling group can launch a new VM-Series instance in response to load, and that instance bootstraps itself into a working, policy enforcing firewall with no human involved in the loop at all.

The pattern

A load based trigger (CPU, session count, throughput) causes the cloud provider's autoscaling group to launch a new instance from a template referencing the bootstrap package. The instance boots, reads its bootstrap configuration, registers with Panorama using the VM auth key, receives its device group and template stack assignment, and begins passing traffic, the whole sequence typically completing in minutes.

Scaling **in** is the less obvious half of the design: an instance being terminated needs its sessions **drained gracefully** rather than dropped mid flow, and needs to be **cleanly removed from Panorama's managed devices list** so it does not linger as a stale, unreachable entry.

Where Terraform fits

Terraform (covered later in this part) typically provisions the surrounding cloud infrastructure (the autoscaling group itself, load balancers, the bootstrap S3 bucket) while the bootstrap package handles what happens **inside** each instance once it launches. The two are complementary layers of the same automation, not competing tools.

This connects directly to the **Multicloud Security Fabric** covered later in this part: MSF's automated deployment and lifecycle management is, at its core, this same bootstrap and register pattern, extended and orchestrated across an entire multicloud estate rather than one autoscaling group.

In practice

The design question worth asking in an interview: **scaling out is the easy half; scaling in gracefully, with sessions drained and Panorama's inventory kept clean, is where automation projects actually fail.**

✓ Check yourself

cloud deployment models and bootstrapping

1. What is the core difference between self managed VM-Series and Cloud NGFW?

Who operates the infrastructure: **self managed means you size/patch/scale it; Cloud NGFW means Palo Alto does, as a native marketplace service.**

2. Which bootstrap file is mandatory, and which is optional?

`init-cfg.txt` is mandatory. `bootstrap.xml` is optional, for a full configuration.

3. If `init-cfg.txt` and `bootstrap.xml` both set the same value differently, which wins?

`init-cfg.txt` wins on any conflict.

4. What does the VM auth key in `init-cfg.txt` enable?

The newly booted firewall authenticates itself to Panorama on first contact, for automatic registration.

5. Why must the license folder use an auth code bundle rather than individual codes?

With individual codes, the firewall only retrieves the license tied to the first one found: a bundle lets it fetch all associated licenses.

6. What is the harder half of autoscaling design: scaling out or scaling in?

Scaling in: draining sessions gracefully and cleanly removing the instance from Panorama's inventory.

✓ Anchor facts

- ✓ **Self managed** = you operate it. **Cloud NGFW** = Palo Alto operates it, native to AWS/Azure.
- ✓ Bootstrap folders: `/config, /content, /license, /software, /plugins` .
- ✓ `init-cfg.txt` mandatory and authoritative on conflict; `bootstrap.xml` optional.
- ✓ **VM auth key** in `init-cfg.txt` enables Panorama auto registration.
- ✓ License folder needs an **auth code bundle**, not individual codes.
- ✓ Scaling in requires **graceful session drain** and Panorama cleanup.



Easy to get wrong

- Assuming Cloud NGFW means giving up policy control, only infrastructure ops changes.
- Forgetting `init-cfg.txt` overrides `bootstrap.xml` on conflicting settings.
- Supplying individual auth codes and getting only one license activated.
- Designing autoscaling around scale out only, with no scale in plan.
- Treating Terraform and bootstrapping as the same layer rather than complementary.

	Self managed VM-Series	Cloud NGFW
Instance ops	You size, patch, scale	Palo Alto manages it
Consumption	Marketplace image / Terraform	Native cloud provider service
Policy control	Full, via Panorama	Full, via console/Panorama/SCM
Best fit	Established Panorama estates	Cloud native simplicity

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 17 • Cloud and Orchestration⁵⁰

NSX-T Service Insertion and CN-Series on Kubernetes

Two private cloud and container integrations that follow the same underlying pattern seen already in this guide: a distributed, per host option and a centralised, cluster option, trading operational simplicity against consolidated capacity.

154 VMware NSX-T service insertion

NSX-T's own built in firewall is capable but lacks App-ID and the deeper inspection this guide has covered throughout. **Service insertion** is how a VM-Series firewall is attached into the NSX-T fabric to inspect traffic that NSX-T itself would otherwise just switch or route.

North south, attached to a Tier-0 gateway

The **Tier-0 (T0) gateway** is NSX-T's edge, handling traffic leaving or entering the software defined data centre. Attaching a VM-Series service insertion here inspects **north south** traffic, internet bound or inter datacentre. Only one service insertion per gateway is supported, though the firewall itself can run in HA. The **T0 gateway must be Active/Standby** for this attachment; Active/Active is not a supported attachment point.

East west, attached to a Tier-1 gateway

A **Tier-1 (T1) gateway** sits at an application or tenant boundary, handling traffic **between segments within** the data centre. Attaching service insertion here inspects **east west** traffic, lateral movement between workloads, which is exactly the traffic a traditional perimeter only design misses entirely.

Two east west deployment modes

Distributed (per host): a firewall instance runs on every ESXi host, inspecting that host's local traffic with no hop off the host at all. **Centralised (service cluster):** firewall instances run in a dedicated cluster, and traffic is redirected to it. Distributed minimises latency and blast radius per host; centralised is simpler to size and scale as one pool rather than per host capacity planning, the same distributed versus cluster trade off that reappears with CN-Series on Kubernetes in the next topic.

Keeping policy in sync with a dynamic environment

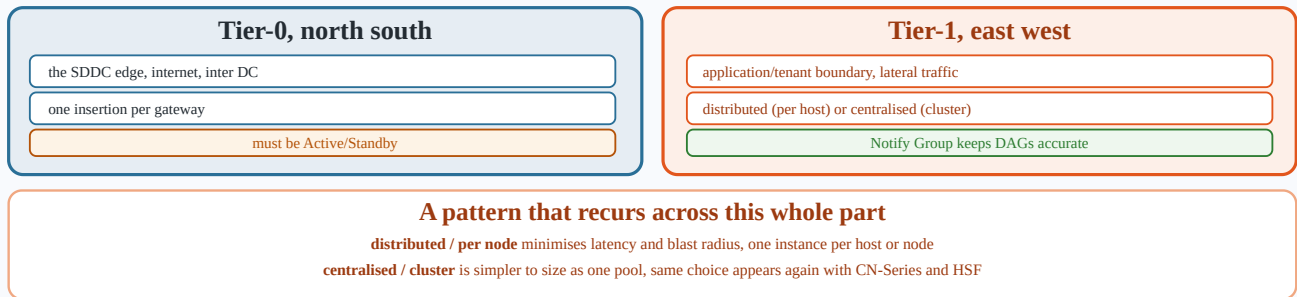
A **Notify Group**, configured in the service definition, tells NSX-T integrated firewalls about VM creation, deletion and movement in real time, which is what keeps **Dynamic Address Groups** from Part 6 accurate as workloads are created and destroyed, policy written against a DAG stays correct without anyone updating an address object by hand.

In practice

T0 for north south, T1 for east west is the fact to have instantly available. And the Notify Group to DAG connection is worth stating unprompted. It is what makes NSX-T integration genuinely dynamic rather than a one time insertion that immediately goes stale.

T0 versus T1, and the distributed/centralised pattern that keeps recurring

the same trade off as HSF and CN-Series



155 CN-Series and Kubernetes

CN-Series is the **container native** firewall, running as Kubernetes native pods rather than a virtual machine, purpose built to inspect traffic **between pods** that a VM based firewall sitting outside the cluster would never see.

The two pods

CN-MGMT is the management plane, connecting to **Panorama** the same way any managed firewall does, licensing, device group and template stack assignment.

CN-NGFW is the dataplane, doing the actual App-ID, Content-ID and threat prevention work on pod to pod traffic.

PAN-CNI: how traffic actually reaches CN-NGFW

A **Container Network Interface (CNI)** plugin configures networking for pods as they are created. **PAN-CNI** is inserted into the cluster's CNI plugin chain as a DaemonSet, and it reads an **annotation on each application pod or namespace**: `paltoaltonetworks.com/firewall: pan-fw`, to decide whether that pod's traffic should be redirected to CN-NGFW for inspection as it enters and leaves the pod.

This is a genuinely different redirection mechanism from the VM-Series world: instead of a route or a security group steering traffic to a firewall, a **Kubernetes annotation** is what turns inspection on for a given namespace or workload.

Distributed versus cluster deployment, the same pattern again

Distributed mode: CN-NGFW runs as a **DemonSet**, one dataplane pod per node. **Cluster (service) mode:** CN-NGFW runs as a **Kubernetes Service** in dedicated security nodes, and is the **recommended** deployment mode. Same trade off as NSX-T's distributed versus centralised choice on the previous page, per node simplicity versus pooled, centrally sized capacity.

Deployment is via **Helm charts**, the standard Kubernetes package manager, and **namespaces**, Kubernetes' own tenant separation construct, are commonly used as the security boundary, letting policy be written per team or per application the same way zones separate traffic elsewhere in this guide.

In practice

The detail worth having exact: **an annotation, not a route, is what turns on inspection** for a pod or namespace. That single fact answers most "how does traffic actually get to the CN-Series firewall" questions.

✓ Check yourself

NSX-T and CN-Series

1. Which gateway tier handles north south traffic, and which handles east west?

Tier-0 = north south. Tier-1 = east west, **at the application/tenant boundary**.

2. What HA mode must the T0 gateway be in for a service insertion attachment?

Active/Standby: **Active/Active is not a supported attachment point**.

3. What does a Notify Group do, and what does it keep accurate?

Tells firewalls about VM creation/deletion/movement in real time, keeping Dynamic Address Groups accurate.

4. Name the two CN-Series pods and their roles.

CN-MGMT (**management, connects to Panorama**) and CN-NGFW (**dataplane, App-ID/Content-ID/threat prevention**).

5. What actually decides whether a pod's traffic gets inspected by CN-NGFW?

A Kubernetes annotation on the pod or namespace, read by PAN-CNI, not a route or security group.

6. What is the recommended CN-Series deployment mode, and what pattern does it echo?

Cluster (service) mode: **the same distributed versus centralised trade off as NSX-T service insertion.**

✓ Anchor facts

- ✓ **T0 = north south** (SDDC edge). **T1 = east west** (app/tenant boundary).
- ✓ T0 attachment requires **Active/Standby**; one insertion per gateway.
- ✓ **Notify Group** keeps DAG membership accurate as VMs change.
- ✓ CN-Series: **CN-MGMT** (control) + **CN-NGFW** (data), via **PAN-CNI**.
- ✓ Traffic redirect is by **pod/namespace annotation**, not routing.
- ✓ CN-Series **cluster (service) mode** is the recommended deployment.



Easy to get wrong

- Swapping T0 and T1, T0 is the edge, T1 is internal boundaries.
- Assuming Active/Active works for a T0 service insertion attachment.
- Forgetting the Notify Group is what keeps DAGs live, not a one time sync.
- Expecting CN-Series traffic redirection to work like VM-Series routing.
- Treating distributed and cluster/centralised modes as interchangeable with no trade off.

Integration	Distributed / per node	Centralised / cluster	Recommended
NSX-T east west	Per ESXi host	Dedicated service cluster	Depends on scale
CN-Series	DaemonSet, per node	Kubernetes Service, dedicated nodes	Cluster mode

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | **CONFIGURE** | VERIFY

Part 17 • Cloud and Orchestration (51)

The Two Orion Fabrics

Part 1 named these as Orion's two fabrics and deferred the detail here. Both are 12.1 Orion capabilities, both are genuinely new, and this guide states plainly where the material is recent enough that a live check is worth doing before an exam or an interview.

156 Multicloud Security Fabric (MSF): the four pillars

The **Multicloud Security Fabric** is Palo Alto's answer to a specific, named problem: security teams running **fragmented visibility, inconsistent policies, and unsustainable manual effort** across AWS, Azure, GCP and private data centres, each managed differently. MSF is described as four capabilities working together, not four separate products.

1 • Deep Visibility

Automatic discovery of the **entire multi cloud estate**: VMs, Kubernetes clusters, AI applications and LLMs, plus the underlying network topology of VPCs/VNets, subnets and cloud native gateways. This builds a single topology map spanning clouds that would otherwise each need their own console.

2 • Intelligent Automation

Automated **deployment and lifecycle management** for the software firewalls doing the inspection, plus automatic traffic steering, programming routes to redirect VM, container and AI workload traffic through the firewalls without manual route configuration. Palo Alto states this reduces what was previously **20 plus manual steps** for firewall insertion down to an automated workflow.

3 • A Unified Data Plane

An automated **network mesh** interconnecting application environments across clouds, providing consistent inspection for east west and cross cloud traffic. This introduces **macro segments**: a consistent Layer 3 segmentation strategy applied uniformly across the entire multicloud footprint, regardless of which cloud a given segment happens to sit in.

4 • Centralized Management

The whole fabric is governed from **Strata Cloud Manager (SCM)**, defining and enforcing one uniform security policy across every cloud, with tools for cross cloud session insight and a **drift detector** to flag risky configuration changes.

This is the automated, multicloud scale version of the bootstrap and register pattern from earlier in this part, the same idea of a firewall configuring and registering itself, extended to an entire estate and orchestrated centrally rather than one instance at a time.

In practice

Know the four pillars by function, not by name recall alone: **discover everything, deploy and steer traffic automatically, inspect consistently across clouds, manage from one console**. This is genuinely new material from the current PAN-OS 12.1 Orion release, worth a quick check against Palo Alto's own site for anything that has moved since this guide was written.

Multicloud Security Fabric: four pillars, one governing console

discover, deploy,
inspect, manage

1 • Deep Visibility

discovers VMs, K8s, AI/LLM
apps, VPCs/VNets, subnets
one topology map,
all clouds at once

2 • Intelligent Automation

automated FW deploy
and lifecycle mgmt
auto traffic steering,
20+ manual steps → automated

3 • Unified Data Plane

automated network mesh
across all clouds
macro segments,
consistent L3 segmentation

4 • Centralized Mgmt

via Strata Cloud
Manager (SCM)
one uniform policy,
drift detector

The problem this solves, stated plainly

fragmented visibility, inconsistent policy, unsustainable manual effort across AWS, Azure, GCP and private data centres
MSF is the four pillars working together, not four separate products bought individually

157 Hyperscale Security Fabric (HSF): P-Nodes and S-Nodes

Where MSF solves multicloud consistency, **Hyperscale Security Fabric** solves a narrower, sharper problem: capacity planning for a data centre where traffic can spike far above what any single firewall can absorb, without buying hardware sized for a theoretical peak that mostly sits idle.

HSF is a **dynamic cluster of software firewalls with a built in load balancer**, engineered for active active elasticity, scale, and resilience, without needing a third party external load balancer at all.

P-Nodes and S-Nodes

P-Nodes are fixed capacity NGFWs that do double duty: they **inspect traffic and load balance** to the layer behind them, exposed as a single IP using **ECMP**. **S-Nodes** sit behind the P-Nodes and **auto scale** based on session utilisation, absorbing the actual traffic volume as it fluctuates.

This is the same distributed versus centralised design language as NSX-T and CN-Series from the previous page, but resolved differently: rather than choosing one mode, HSF **splits the two jobs across two node types**: a fixed, predictable inspection and routing layer, and an elastic capacity layer behind it.

Scale and platform support

HSF scales from roughly **1 Gbps up to 200 Gbps**. It is available on **ESXi and KVM**, with public cloud platform support described as coming; check current documentation before relying on a specific cloud target. A **CN-Series equivalent** exists for containerised, Kubernetes based deployments, aimed specifically at 5G and mobile service provider use cases.

Session resiliency

HSF provides **session failover to healthy instances** within the cluster and auto scaling driven by real time session load, so both a node failure and a genuine traffic spike are absorbed without manual intervention.

In practice

The two node type architecture is the fact worth having exact: **P-Nodes inspect and load balance; S-Nodes just scale**. This, like MSF, is current release material, the platform support list in particular is worth a live check rather than treating this guide as the final word.

✓ Check yourself

MSF and HSF

1. Name the four MSF pillars.

Deep Visibility, Intelligent Automation, a Unified Data Plane, Centralized Management (via Strata Cloud Manager).

2. What does MSF's "macro segment" concept provide?

A consistent Layer 3 segmentation strategy applied uniformly across the entire multicloud footprint.

3. Roughly how many manual steps does MSF claim to reduce firewall insertion from?

20 plus manual steps, down to an automated workflow.

4. What does a P-Node do in HSF?

Fixed capacity NGFW that inspects traffic and load balances to the S-Nodes behind it, exposed via ECMP.

5. What does an S-Node do, and how does it differ from a P-Node?

It auto scales based on session utilisation. It absorbs capacity, it does not load balance.

6. What throughput range does HSF scale across?

Roughly 1 Gbps to 200 Gbps.

✓ Anchor facts

- ✓ MSF pillars: **Deep Visibility, Intelligent Automation, Unified Data Plane, Centralized Management.**
- ✓ MSF is managed from **Strata Cloud Manager**, with a drift detector.
- ✓ HSF: **P-Nodes inspect + load balance (fixed); S-Nodes auto scale.**
- ✓ HSF exposes as one IP via **ECMP**, no third party load balancer needed.
- ✓ HSF scale: roughly **1 to 200 Gbps**, on ESXi and KVM today.
- ✓ Both are **current PAN-OS 12.1 Orion** material, verify against live docs.



Easy to get wrong

- Confusing MSF (multicloud consistency) with HSF (single data centre elasticity).
- Assuming P-Nodes auto scale. They are the fixed layer; S-Nodes scale.
- Treating HSF as needing an external load balancer. It is built in.
- Stating platform support (especially public cloud) as fixed rather than evolving.
- Forgetting MSF is four pillars working together, not four separate purchases.

	Multicloud Security Fabric	Hyperscale Security Fabric
Solves	Multicloud consistency and visibility	Single site elastic capacity
Scope	AWS, Azure, GCP, private DC	One data centre / cluster
Managed via	Strata Cloud Manager	Panorama / SCM, per platform
Core mechanism	Discovery + automated steering	P-Nodes + S-Nodes

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Part 17 • Cloud and Orchestration (52)

APIs, Terraform, and CLARA

The last piece: the two APIs everything in this part is actually built on, the Terraform provider that wraps them, and CLARA, the assessment that tells an organisation where to start.

158 XML API, REST API and the Terraform provider

Every automation covered in this part (bootstrapping, autoscaling, MSF's traffic steering, Terraform) ultimately calls one of two APIs on the firewall or Panorama. Knowing which is which, and that Terraform sits on top of them rather than being a third option, ties this whole part together.

The XML API, full control, older interface

A **low level API covering every feature** the firewall has, usable directly against a firewall or Panorama. Its `type` parameter selects the operation: `config` for reading or writing configuration, `op` for operational commands, `commit`, and `log` for retrieving logs, the same log-query syntax covered in Part 18 works here too. Depth and completeness are its strength; verbosity is its cost.

The REST API, newer, resource shaped

Exposes policy objects as **top level URIs** (address objects, security rules, NAT rules) each independently addressable with standard HTTP verbs. Simpler to reason about for straightforward CRUD style automation, but it does not yet cover every corner the XML API does.

The Terraform provider, built on top of both

The official **panos** Terraform provider is now on its **2.x generation**, auto generated from the same normalised PAN-OS schemas that generate the Go SDK, rather than hand maintained resource by resource. Under the hood it drives the **XML API**, and a performance feature called **multi-config batching** combines up to **500 XML API calls into a single HTTP request** by default, a direct answer to the "many small changes are slow" problem that plain scripted API calls run into at scale.

A separate official **Terraform module** (distinct from the provider) generates and uploads a complete **bootstrap package to an AWS S3 bucket**: the exact bootstrap mechanics from earlier in this part, expressed declaratively rather than assembled by hand.

Where Ansible and Python fit, cross referencing the interview material

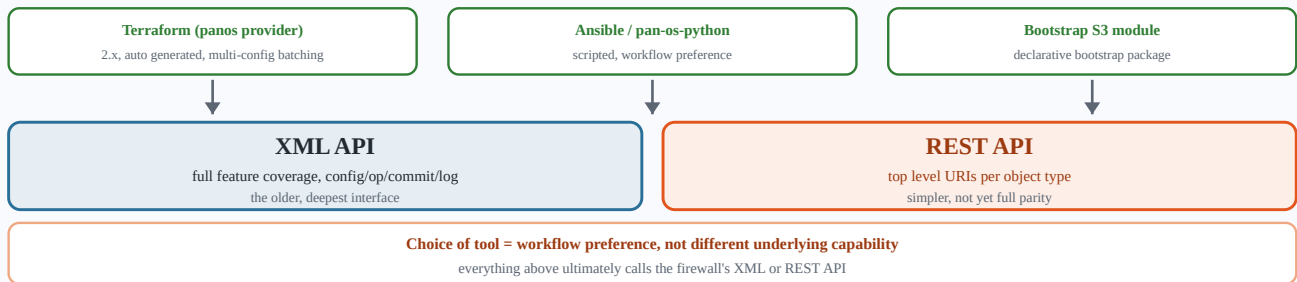
Ansible's PAN-OS collection and the **pan-os-python** SDK (covered in the interview supplement's automation page) both also call these same two APIs underneath, the choice between Terraform, Ansible and raw Python is a question of **workflow and team preference**, not of which one has access to different underlying capability.

In practice

The layering to state precisely: **XML API is the full featured original**; **REST API is the newer, simpler, resource shaped alternative**; **Terraform, Ansible and pan-os-python all sit on top of one or both, adding declarative or scripted convenience rather than new capability.**

Everything in this part calls one of two APIs

Terraform, Ansible and Python are convenience layers on top, not separate capability



159 CLARA: Cloud Network & AI Risk Assessment

CLARA (Cloud Network & AI Risk Assessment) is a Palo Alto delivered assessment that scans an organisation's cloud and AI estate to produce a **risk brief**, explicitly positioned as having **zero impact on production environments**: it is a read only assessment, not a deployment.

The problem it targets

Palo Alto's own framing: as organisations shift from multicloud expansion into AI native workloads, **shadow AI** emerges, assets and integrations that exist without visibility or governance, hidden behind the operational noise of a large, fragmented estate. CLARA is pitched as the first, low friction step to find out where that exposure actually is before committing to a broader remediation project.

Where it fits relative to everything else in this part

CLARA is the **assessment**; MSF's Deep Visibility pillar is the **ongoing discovery capability** that keeps working after the one time assessment. In practice, CLARA is a reasonable entry point for an organisation that has not yet deployed MSF at all, establishing the baseline that justifies the investment, rather than assuming the investment first and discovering the gaps later.

This mirrors a pattern already covered in the interview material: it is the same "**assess before you commit**" instinct behind Policy Optimizer's evidence based rule rewriting, or a staged rollout in the automation topic, look before acting, on a larger, organisation wide scale.

Sign up and delivery is through Palo Alto directly, producing a report intended for a **leadership or board level audience** as much as a technical one. It is explicitly framed as decision support, turning "we don't know our exposure" into a quantified starting point for budget and prioritisation conversations.

In practice

Know the acronym expansion cold, **Cloud Network and AI Risk Assessment**, and the one line that captures its purpose: a **zero impact scan that turns unknown multicloud and AI exposure into a quantified, prioritised starting point**.

✓ Check yourself

APIs, Terraform and CLARA

1. What does the XML API's **type** parameter select between?

config, op, commit, log: the operation being performed.

2. How does the REST API differ structurally from the XML API?

It exposes policy objects as top level URIs with standard HTTP verbs, rather than one general purpose XML endpoint.

3. Which API does the Terraform provider actually drive underneath?

The XML API: Terraform is a declarative layer on top, not a separate capability.

4. What does multi-config batching do, and what's the default limit?

Combines multiple XML API calls into one HTTP request; default up to 500 calls per batch.

5. What does CLARA stand for?

Cloud Network & AI Risk Assessment.

6. What is CLARA's defining operational characteristic?

Zero impact on production: a read only assessment, not a deployment.

✓ Anchor facts

- ✓ XML API = full feature coverage, older, config/op/commit/log.
- ✓ REST API = newer, resource shaped, not yet full parity.
- ✓ Terraform **panos** provider is 2.x, auto generated, drives the XML API.
- ✓ **Multi config batching**: up to 500 XML calls per HTTP request, default.
- ✓ **CLARA** = Cloud Network & AI Risk Assessment, zero production impact.
- ✓ CLARA targets **shadow AI** and fragmented multicloud exposure specifically.



Easy to get wrong

- Treating Terraform as a third API rather than a layer on the XML API.
- Assuming the REST API already has full XML API feature parity.
- Forgetting CLARA is an assessment, not an ongoing capability like MSF's discovery.
- Missing that tool choice (Terraform/Ansible/Python) is workflow, not capability.
- Not verifying REST API coverage before committing automation to it for an edge case feature.

Layer	Talks to	Best for
XML API	Firewall or Panorama, direct	Full feature coverage, log queries
REST API	Firewall or Panorama, direct	Simple, object shaped CRUD
Terraform (panos)	XML API, underneath	Declarative, staged infrastructure
Ansible / pan-os-python	XML or REST API	Scripted operational tasks
CLARA	N/A, a delivered assessment	Establishing the baseline first

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Packet Flow, First Packet and the Session Table

Every troubleshooting question in this guide has assumed you know what happens to a packet between arrival and departure. This page makes that path explicit, because reading a capture or a counter means nothing without it.

160 Packet flow: first packet versus fast path

Single Pass Parallel Processing from Part 1 describes **what** the data plane does to a packet. This topic is **when** it does it, not every packet in a session gets the same treatment, and knowing which stage a packet is in explains most confusing behaviour.

First packet, the slow path

The **first packet of a new session** triggers the expensive work: a route lookup, a zone lookup, a security policy match walking the rulebase top to bottom, NAT translation lookup, and App-ID's initial classification attempt. This is genuinely more CPU intensive than everything after it, because none of the answers exist yet. They all have to be computed.

Once that work completes, the result is written into the **session table** as a new entry, keyed on the session's 6 tuple (source/destination IP, source/destination port, protocol, and the security zone).

Every subsequent packet, the fast path

A packet belonging to an **existing session** is matched against the session table directly. If it matches, the firewall reuses the policy decision, zone, and NAT translation already computed on the first packet. It does **not** rewalk the security rulebase or rerun the route lookup. This is dramatically cheaper, which is why session setup rate and throughput are reported as two separate performance numbers on a datasheet.

Where App-ID complicates this

App-ID is not always fully resolved on the first packet, some applications only reveal their identity after several packets of payload. The session is created on packet one with a **provisional identification** (often the app matching the port, or "incomplete"), and policy can **reevaluate** once App-ID reaches a final verdict, potentially changing which security profile or even which rule now applies to that already open session. This is the session end reason **policy deny** appearing mid session rather than at session start, and it is a frequent source of "why did this get blocked after it was already working" questions.

Verify session state and which path a flow is taking with `show session all filter` (narrow by source, destination, or application) and `show session id <id>` for the full detail on one session, including its current App-ID verdict.

In practice

The one line version worth having ready: **first packet is expensive and decides everything; every packet after it is cheap and just checks the session table**: except when App-ID's identification changes mid flow and forces a reevaluation.

First packet versus every packet after it

the session table is what makes the fast path fast

First packet, slow path

route lookup · zone lookup
walk the full security rulebase
NAT lookup · initial App-ID attempt

result written to the session table

Every packet after, fast path

match against the session table directly
reuse the policy, zone, NAT already decided
no rulebase walk, no route lookup again

dramatically cheaper. This is throughput

The complication: App-ID can change its mind mid session

a provisional identification on packet one can be revised once enough payload arrives, forcing **policy reevaluation**
this is why a session can be denied after it was already flowing, not a bug, App-ID simply reached a final verdict

161 The session table and session states

The **session table** is the data plane's live record of every active flow, the object the fast path checks against, and the single most useful place to look when a flow is behaving unexpectedly right now.

Each entry carries the **6 tuple key** (source/destination IP, source/destination port, protocol, zone), plus the resolved **application**, **NAT translation**, **security rule matched**, and **current state**.

Session states worth recognising

INIT: the session is being set up, first packet processing in progress.

ACTIVE: established and passing traffic normally.

CLOSING / CLOSED: a graceful TCP teardown (FIN) in progress or complete.

DISCARD: the firewall has decided to drop this session's packets but is **still tracking the session itself**, typically to avoid reevaluating a denied flow on every subsequent packet, a discard entry existing is not evidence the firewall is passing that traffic.

Reading a specific session

`show session id <id>` gives the full picture: which rule matched, the current application verdict, byte and packet counters in each direction, the NAT translation applied, and **which HA peer owns it** if the platform is in active/active from Part 12.

Session **timeouts** are protocol- and application specific (a TCP session has a different idle timeout than UDP or ICMP, and some applications carry their own timeout override), configured at `Objects > Applications` or the session settings under `Device > Setup > Session`. A session sitting idle past its timeout is aged out and removed from the table, which is itself a common cause of an application that "randomly disconnects" after a period of inactivity.

In practice

If traffic that should be denied is somehow still flowing, or traffic that should work is not, checking

`show session all filter` for that specific flow and reading its **state** and **matched rule** directly is almost always faster than rereading the policy configuration from scratch.

162 Global counters

Global counters are the data plane's internal instrumentation, every packet processed increments some set of counters as it moves through the pipeline, and reading them tells you **where in the pipeline packets are actually being dropped**, rather than just that they are being dropped.

Viewed with `show counter global filter delta yes`: the **delta yes** filter is the detail worth remembering, since it shows only counters that have changed since the last time you ran the command, turning an overwhelming full list into just the counters actively incrementing right now while you reproduce a problem.

Counter categories worth recognising

Counters are grouped by the stage of processing they represent: **flow** (session setup and packet forwarding decisions), **tcp** (TCP specific state handling), **pktproc** (packet processing pipeline stages), **ssl** (decryption related), and others matching the pipeline stages covered throughout this guide. A counter name like `flow_policy_deny` or `flow_fwd_l3_noroute` names the exact reason a packet was dropped, directly.

The workflow

Run the filtered command, reproduce the problem (send the traffic that is failing), run the command again, and compare, whichever counters incremented during the reproduction window are the ones that actually fired for this specific problem, cutting through the hundreds of counters that exist for scenarios unrelated to what you are chasing.

This pairs directly with the session table from the previous topic: the session table tells you **what** happened to a specific flow; global counters tell you **why**, at the pipeline stage level, when the session table's own fields do not make the cause obvious.

In practice

The counter workflow is the answer whenever a scenario describes drops with no clear cause in the Traffic log, a packet dropped before a session is even created (a malformed packet, a route lookup failure) may never generate a Traffic log entry at all, but it will always increment a global counter.

✓ Check yourself

packet flow and the session table

1. What work happens on the first packet of a session that does not happen again?

Route lookup, zone lookup, the full security rulebase walk, NAT lookup, and initial App-ID classification.

2. Why can a session be denied mid flow with no configuration change made?

App-ID's identification changed **once enough payload arrived**, forcing policy reevaluation against the new verdict.

3. Does a session in DISCARD state mean the firewall is passing that traffic?

No: DISCARD means the firewall is still tracking the session while actively dropping its packets.

4. What does the "delta yes" filter do on a global counter command?

Shows only counters that have changed since the last run, isolating what fired during a specific reproduction.

5. Why might a dropped packet never appear in the Traffic log?

A packet dropped before a session is created (malformed packet, route failure) may generate no log entry, but always increments a global counter.

6. What is the relationship between the session table and global counters when troubleshooting?

The session table shows what happened to one flow; global counters show why, at the pipeline stage level.

✓ Anchor facts

- ✓ **First packet** = full lookup and rulebase walk. **Every packet after** = session table match only.
- ✓ App-ID can be **provisional** on packet one and revise later, forcing reevaluation.
- ✓ Session states: **INIT, ACTIVE, CLOSING/CLOSED, DISCARD**.
- ✓ `show session id <id>` gives rule, app, NAT, counters, HA owner.
- ✓ `show counter global filter delta yes` isolates counters firing right now.
- ✓ A dropped packet with no session yet may have **no Traffic log entry at all**.



Easy to get wrong

- Assuming all packets in a session get the same expensive processing as the first.
- Treating a mid session deny as a bug rather than an App-ID reevaluation.
- Reading DISCARD as "not being tracked" rather than "being tracked and dropped".
- Running `show counter global` without the delta filter and drowning in noise.
- Searching only the Traffic log for a drop that never created a session.

State	Meaning
INIT	First packet setup in progress
ACTIVE	Established, passing traffic
CLOSING / CLOSED	Graceful TCP teardown in progress or done
DISCARD	Still tracked, actively dropping packets

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Packet Capture and a Structured Approach

Packet capture was mentioned in passing across earlier parts, certificates, decryption, tunnels. This page is the dedicated treatment: the four stages, and how to sequence a troubleshooting session so capture is the last resort, not the first reflex.

163 Packet capture: the four stages

PAN-OS packet capture is **staged**: a packet can be captured at up to four distinct points as it moves through the pipeline, and **which stage it appears in tells you where it died** before you have read a single byte of the capture itself. This is the detail that makes Palo Alto capture more diagnostically useful than a plain interface level tcpdump.

Receive (RX): the packet as it arrives on the ingress interface, before any firewall processing at all. If a packet does not appear here, it never reached the firewall, the problem is upstream, in cabling, switching, or routing before this box.

Firewall: the packet as it passes through the security processing engine itself: policy lookup, App-ID, Content-ID, decryption. A packet appearing at Receive but not Firewall means the packet was **consumed by an earlier stage** before reaching security processing, commonly a zone or interface configuration issue.

Drop: captures only packets the firewall has **decided to discard**, at whatever stage that decision was made. A packet appearing here, and here only, confirms the firewall itself made an active decision to drop it, check the matched rule and the session's discard reason from the previous page next.

Transmit (TX): the packet as it leaves the firewall on the egress interface, after all processing completed. A packet reaching Firewall but never appearing at Transmit, with no corresponding Drop entry either, points at something failing silently inside processing rather than a clean policy decision, worth escalating with a support case if global counters do not explain it either.

Reading the four stages together

Enable all four simultaneously with a tight filter (specific source, destination, port) at [Monitor › Packet Capture](#) or the CLI equivalent, then reproduce the problem. **Which stages the packet appears in, and which it does not, localises the failure** without needing to read the packet contents at all, the stage pattern alone is often the entire diagnosis.

In practice

State the four stages in order if asked: **Receive, Firewall, Drop, Transmit**. A packet at Receive but nowhere else is a pre firewall problem. A packet at Receive and Drop is a firewall decision. A packet at all four except Transmit, correlated against global counters, is where deeper investigation actually needs to start.

The four capture stages, and what each pattern of presence/absence means

diagnosis before
you've read a single
byte



Reading the pattern of presence and absence

Absent everywhere → never reached the firewall, problem is upstream, before this box

RX only → consumed before security processing, check zone/interface config

RX + Drop → the firewall made an active decision, check the matched rule next

RX + Firewall, no TX, no Drop → failing silently inside processing, check global counters, then escalate

All four stages → the packet passed through cleanly, look elsewhere for the problem

164 A structured troubleshooting approach

Every tool covered in this part has a cost, and the discipline that runs through this whole guide applies here most directly: **work from cheap and broad to expensive and narrow**, gathering evidence before changing anything.

1 · Define the symptom precisely

Exact source, destination, port, application, and time. "It's slow" or "it doesn't work" is not yet a troubleshooting problem, a specific flow that fails in a specific way is.

2 · Check the cheap sources first

The **Traffic and Threat logs** from Part 14, was the session even seen, what rule matched, what was the session end reason. This alone resolves the majority of "why was this blocked" questions without touching anything else.

3 · Check the session table

From the previous page, is there an active session for this flow right now, what state is it in, which rule and application does the firewall currently believe apply.

4 · Check global counters

If the logs show nothing and no session exists, the packet may be failing before a session is ever created, global counters with the delta filter will show this even when nothing else does.

5 · Packet capture last

Only once the above has narrowed the problem to a specific flow and a rough idea of where it fails. Capture confirms a hypothesis; it rarely generates one efficiently on its own, and an unfiltered capture on a busy interface costs real CPU.

The habits that cut across every part of this guide

Filter before capturing. Check show system resources before assuming a data plane problem when the symptom might be management plane (Part 1). **Check the obvious layer first** (DNS, then reachability, then application) rather than reaching for the most sophisticated tool immediately.

In practice

This five step order is the answer to almost any "how would you troubleshoot X" question in an interview: **define the symptom, logs, session table, counters, capture**: cheapest and broadest evidence first, expensive and narrow tools only once the problem is already mostly localised.

165 Common symptom to cause patterns

A quick reference set of patterns this guide has built toward, pulled together as the kind of pattern matching an experienced engineer does automatically.

Worked yesterday, fails today, no changes made: check the **content update version** from Part 16; a new App-ID reclassifying traffic is the leading cause with literally no configuration change involved.

One application fails, everything else on the same path is fine: almost always **certificate pinning or mutual TLS** breaking under decryption (Part 8), or an **App-ID dependency** issue (Part 7) where the parent application was denied.

Works from one site, fails from another, same rule and objects: check **device group and template assignment** in Panorama (Part 15) before assuming a policy problem; the site may not actually be receiving what you think it is.

VPN connects, but no traffic passes: check **security policy for the tunnel zone** (Part 10/11) before rechecking Phase 1 and Phase 2; a successful tunnel with no policy permitting it is the single most common GlobalProtect and site to site support call.

Slow GUI or commit, traffic flowing fine: management plane, not data plane (Part 1); check

[show system resources](#) before considering a reboot.

HA failover happened with no obvious cause: check **link and path monitoring** thresholds (Part 12) and whether **preemption** is enabled and flapping against a marginal priority difference.

The thread running through all six: the actual root cause is almost never where the symptom first appears to point, which is exactly why the structured, cheap to expensive approach from the previous topic outperforms guessing based on the symptom alone.

In practice

These six patterns cover a large share of real support calls on this platform. Recognising the shape of a problem, not the specific config, is what separates fast diagnosis from starting the five step process from zero every time.

✓ Check yourself

capture stages and structured troubleshooting

1. Name the four packet capture stages, in order.

Receive, Firewall, Drop, Transmit.

2. A packet appears at Receive only, nowhere else. What does that tell you?

It was consumed before security processing: **check zone or interface configuration, not policy.**

3. A packet appears at Receive and Drop. What's confirmed?

The firewall made an active decision to discard it: **check the matched rule and session discard reason next.**

4. State the five step troubleshooting order.

Define the symptom, logs, session table, global counters, packet capture: **cheap and broad before expensive and narrow.**

5. Why should packet capture be the last resort, not the first tool reached for?

It confirms a hypothesis **rather than efficiently generating one, and an unfiltered capture costs real CPU.**

6. Traffic that worked yesterday fails today with no config change. First thing to check?

✓ Anchor facts

- ✓ Capture stages, in order: **Receive** → **Firewall** → **Drop** → **Transmit**.
- ✓ The **pattern** of which stages a packet appears in is the diagnosis.
- ✓ Troubleshooting order: **symptom** → **logs** → **session table** → **counters** → **capture**.
- ✓ Always **filter before capturing**.
- ✓ Rule out **management plane** before assuming a data plane fault.
- ✓ "No changes made, but it broke" often means a **content update** did.



Easy to get wrong

- Reaching for packet capture before checking logs and the session table.
- Running an unfiltered capture on a production interface.
- Misreading a Drop stage capture as evidence of a network problem rather than a policy decision.
- Assuming "no config change" rules out content updates as a cause.
- Diagnosing a VPN traffic failure by rechecking IKE before checking the tunnel zone's security policy.

Pattern	Present in	Absent from	Meaning
Never arrived	,	All four	Problem is upstream of the firewall
Pre processing drop	Receive	Firewall, Drop, Transmit	Zone/interface issue, not policy
Policy decision	Receive, Drop	Firewall (partial), Transmit	Firewall actively discarded it
Silent failure	Receive, Firewall	Drop, Transmit	Failing inside processing, check counters
Clean pass through	All four	,	Not a firewall problem, look elsewhere

Actually Querying the Logs

Part 14 covered what each log type records, and the previous page assumed you'd "check the logs" as step two of any investigation. This page is the part that was missing: the actual filter syntax, in the GUI and the CLI, to pull the entry you need out of millions of others.

166 GUI log queries: the filter syntax

Every log view under `Monitor > Logs > Traffic` (or Threat, URL Filtering, Decryption, System, Configuration, and the rest from Part 14) shares the **same filter language**, entered in the search bar at the top of the log view. Learning it once covers every log type.

The basic shape

A filter is `( field operator value )`, and filters chain together with `and` / `or`. The GUI's **Add Filter** builder generates this syntax for you by clicking on any value in an existing log row, clicking a source IP in a log entry automatically adds `( addr.src in x.x.x.x )` to the search bar, which is the fastest way to build a filter without memorising field names.

Common fields worth knowing directly

`addr.src` / `addr.dst` : source or destination address, accepts a single IP, a subnet, or a range.

`zone.src` / `zone.dst` : source or destination zone.

`port.src` / `port.dst` : source or destination port.

`app` : the identified application.

`action` : allow, deny, drop, reset-client, reset-server, reset-both.

`rule` : the security rule name that matched.

`receive_time` : useful with `geq` / `leq` to bound a time window precisely, tighter than the GUI's preset time range dropdown.

Worked examples

`( addr.src in 10.1.1.5 ) and ( app eq ssl )` : everything from one host using SSL/TLS.

`( action eq deny ) and ( zone.src eq trust )` : every denied session originating from the trust zone, the fastest way to spot a rule that's blocking more than intended.

`( port.dst eq 443 ) and ( action neq allow )` : anything to port 443 that was **not** allowed, useful when a specific service is intermittently failing.

The **Threat log** adds fields specific to it: `severity` (critical, high, medium, low, informational) and `category` (the threat type) are the two used most, e.g. `( severity geq high )` to cut straight to what actually matters during an active incident.

In practice

The habit worth having: **click before you type**. Clicking a field value in any existing log row builds the correct filter syntax automatically, which is faster and less error prone than typing field names from memory, save the memorised syntax for when you're working from the CLI instead.

Building a filter without memorising syntax

the GUI generates it; the CLI needs you to know it

GUI, click to build

click a value in any log row

filter syntax written to the search bar for you

chain more clicks with **and** / **or**

fastest path, no memorisation required

CLI, you type the filter

same field names, same operators

no click to build. You supply the syntax

the only option over SSH with no GUI access

this is why the syntax is worth knowing regardless

Same language, two entry points

(addr.src in 10.1.1.5) and (app eq ssl) works identically in both places learn it once from the GUI's click to build, and it transfers directly to the CLI when that's all you have

167 CLI log commands and live tailing

The CLI equivalent matters whenever you only have **SSH access**: a management interface that's up but with the GUI degraded, or simply working faster from a terminal already open for other commands.

Pulling historical entries

show log traffic direction equal backward query "(addr.src in 10.1.1.5)" : the **query** argument takes **exactly the same filter syntax** as the GUI search bar, in quotes. Swap **traffic** for **threat**, **url**, **system**, or any other log type from Part 14 to query that log instead. **direction equal backward** returns the most recent entries first, which is what you want during an active investigation.

Watching logs live, the most useful command for active troubleshooting

tail follow yes filter "(addr.src in 10.1.1.5)" , run against the log type in the current CLI context (**less mp-log** and **tail mp-log** also apply to specific management plane log files by name). This streams matching entries **as they happen**, which is the single most effective technique for an in progress problem: apply a tight filter, reproduce the issue on the affected host, and watch the exact log entry appear in real time rather than searching after the fact.

Live tailing pairs directly with the global counters workflow from the previous page, run the counter command and the tail in two terminal panes, reproduce the problem once, and you see **both** what the logs recorded and which counters incremented, from the same single reproduction.

On Panorama

The same **show log** syntax works from the Panorama CLI, querying across managed firewalls' forwarded logs rather than one device's local log, which is the CLI equivalent of the estate wide Monitor view from Part 15.

In practice

For an active, reproducible problem, **tail follow yes** with a tight filter is usually faster than anything else in this guide. You see the exact log entry the moment the problem happens, with no need to guess a time window or page back through history afterward.

168 Choosing GUI versus CLI, and combining both

Neither is strictly better. They suit different moments in an investigation, and knowing when to switch is itself part of working efficiently.

GUI strengths

Click to build filters mean **no syntax to remember**, columns can be added or removed to see exactly the fields relevant to this investigation, and a result set can be exported directly to CSV for sharing with someone who doesn't have firewall access. This is the right choice for **exploratory** investigation. You don't yet know exactly what you're looking for.

CLI strengths

Works when the **GUI itself is the problem** (Part 1's management plane symptoms), scripts and repeats identically across many devices, and `tail follow yes` has no GUI equivalent at all, the GUI can auto refresh a search, but it does not stream matching entries the instant they occur the way the CLI tail does. This is the right choice once you know **precisely** what you're filtering for and need to watch it happen live.

The combined workflow that actually works

Start in the **GUI**, click through a few log entries to understand the shape of the problem and build the filter syntax by clicking rather than guessing. Once the filter is right, take that exact syntax to the **CLI** and run it as a live tail while reproducing the problem, getting the best of both: the GUI's ease of exploration, and the CLI's real time visibility that nothing else in this guide provides.

This mirrors the structured approach from the previous page exactly: **broad and cheap first (GUI exploration), narrow and precise once you know what you're looking for (CLI live tail)**: the same cost discipline applied to log tooling rather than to the choice between logs and packet capture.

In practice

If asked how to troubleshoot a live issue, this two step answer is stronger than naming either tool alone: **build the filter visually in the GUI, then run it as a live CLI tail while reproducing the problem**: it demonstrates both the syntax knowledge and the operational judgement of when each interface earns its place.

✓ Check yourself

querying and tailing logs

1. What's the fastest way to build a correct GUI log filter without memorising field names?

Click a value in an existing log row: **the GUI writes the filter syntax for you.**

2. Which field would you use to find every session denied from the trust zone?

`( action eq deny ) and ( zone.src eq trust )`

3. What does the CLI's `query` argument accept?

The exact same filter syntax as the GUI search bar, passed in quotes.

4. What can `tail follow yes` do that the GUI cannot?

Stream matching log entries the instant they occur, **no GUI equivalent exists for true real time tailing.**

5. What's the two step workflow that combines GUI and CLI strengths?

Build the filter visually in the GUI, then run it as a live CLI tail **while reproducing the problem.**

6. Why is the CLI sometimes the only option, regardless of preference?

When the GUI itself is degraded, a management plane symptom from Part 1, SSH and the CLI may be all that's available.

✓ Anchor facts

- ✓ Filter shape: (field operator value) , chained with and / or .
- ✓ Key fields: addr.src/dst, zone.src/dst, port.src/dst, app, action, rule .
- ✓ Click a log value to auto build the filter, fastest GUI method.
- ✓ CLI: show log <type> query "(...)" for history.
- ✓ tail follow yes filter "(...)" streams entries live, no GUI equivalent.
- ✓ Workflow: build in GUI, tail in CLI.



Easy to get wrong

- Typing filter syntax from memory when clicking a log value is faster and error free.
- Forgetting the CLI query syntax is identical to the GUI's, not a separate language.
- Reaching for GUI auto refresh when a true live tail is what's actually needed.
- Not knowing any CLI log syntax when the GUI is the thing that's down.
- Skipping straight to CLI tailing before knowing what filter is actually worth watching for.

Field	Matches	Example
addr.src / addr.dst	Source / destination address	addr.src in 10.1.1.5
zone.src / zone.dst	Source / destination zone	zone.src eq trust
app	Identified application	app eq ssl
action	allow, deny, drop, reset-*	action eq deny
rule	Matched security rule name	rule eq 'Allow-Web'
severity (Threat log)	critical → informational	severity geq high

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Parts 1 to 9 · Everything in the Room, No Explanation

No questions, no reveal, no quiz, just the facts this guide already earned across 51 pages, compressed to what you'd want in front of you five minutes before you walk in. If a line doesn't make sense on sight, that topic needs another pass before today.

✓ 1 · Platform

- ✓ Single pass = inspected once. Parallel = networking/security on dedicated hardware at once.
- ✓ Mgmt plane vs data plane diagnosed separately,
`show system resources` vs `show running resource-monitor`.
- ✓ PA = hardware. VM = any hypervisor/laaS. CN = Kubernetes pods.
- ✓ Orion 12.1: Quantum Readiness Dashboard, cipher translation, 10M user contexts, passwordless/KCD.
- ✓ HSF and MSF are Orion's two fabrics, single site elastic vs multicloud consistency.

✓ 2 · Prisma AIRS

- ✓ 5 capabilities: Runtime Firewall, Runtime API, Model Security, Red Teaming, Posture Mgmt.
- ✓ Network Intercept = inline traffic. API Intercept = direct model/app API calls.
- ✓ AI threats: prompt injection, agentic identity impersonation, memory manipulation, tool misuse.
- ✓ Arc: Discover → Protect → Defend.
- ✓ AI Runtime Security is a 5th deployment option alongside PA/VM/CN/Cloud NGFW.

✓ 3 · Interfaces & Zones

- ✓ Modes: tap (copy only), vwire (no IP, transparent), L2, L3, tunnel.
- ✓ A zone needs a type match to its interface(s); traffic can't cross zones without policy.
- ✓ Aggregate = bandwidth/redundancy bundle. Subinterface = 802.1Q tag, own zone possible.
- ✓ Loopback = always up, used for GP gateway, router ID, management services.
- ✓ Zone Protection = per zone floods/recon. DoS Protection = specific src/dst pairs. Packet Buffer Protection = global, per session.

✓ 4 · Routing

- ✓ Legacy virtual router = one object, protocols configured inside it.
- ✓ Advanced Routing Engine = logical router + reusable routing profiles (route maps, prefix lists).
- ✓ AD: static 10 · OSPF int 110/ext 110 · iBGP 200 · eBGP 20 · RIP 120.
- ✓ Path monitoring on a static route/PBF withdraws it if the next hop dies, same idea as HA path monitoring.
- ✓ Redistribution profiles (legacy) vs route maps + prefix lists (ARE) filter what crosses protocols.

✓ 5 · NAT

- ✓ Golden rule: **pre NAT address, post NAT zone** in security policy.
- ✓ Static 1:1 · dynamic IP · dynamic IP and port (PAT) · U turn for internal to public IP internal.
- ✓ NAT rule evaluation is top down, first match, separate from security rule order.
- ✓ PBF overrides the routing table for policy based path selection; still needs its own health checks.

✓ 6 · Policy & Objects

- ✓ Rule order top down, first match wins. App-ID often needs several packets to finalise.
- ✓ application-default lets App-ID's real ports through; "any" service is a common over permission.
- ✓ Policy Optimizer: rewrite port based rules from **observed** traffic, not guesswork.
- ✓ EDL = external list, auto refreshed. DAG = tag based, membership changes need no commit.
- ✓ Tag action in a Log Forwarding Profile + DAG + deny rule = auto quarantine in seconds.

✓ 7 • App-ID & Content-ID

- ✓ App-ID sequence: signatures → decryption if needed → decoders → heuristics.
- ✓ Log app values: incomplete, insufficient data, not applicable, unknown-tcp/udp.
- ✓ 7 security profiles: AV, Anti-Spyware, Vulnerability Protection, URL Filtering, File Blocking, WildFire, Data Filtering.
- ✓ Security Profile Group bundles them; applied per rule, not globally.
- ✓ Content-ID = single pass scanning stream, not stored then scanned.

✓ 8 • Certificates & Decryption

- ✓ Forward trust CA on client trust stores; forward untrust CA is never distributed.
- ✓ Subordinate CA from enterprise PKI is the recommended forward proxy pattern, avoids a brand new root.
- ✓ SSH proxy inspects SSH without decrypting content, detects tunnelling/port forward abuse.
- ✓ Decryption Exclusion list for pinning/mutual TLS; Decryption log has the error reason.
- ✓ Decryption broker/mirroring sends cleartext to external tools, a real security control on its own.

✓ 9 • Auth & Identity

- ✓ Stack: server profile → auth profile → auth sequence. Layer order matters.
- ✓ LDAP 389/636 • RADIUS 1812/1813 • TACACS+ 49 (full payload encrypted, device admin).
- ✓ KCD = passwordless Kerberos constrained delegation.
- ✓ User-ID methods: security log monitoring, GlobalProtect, syslog listener, agent/agentless.
- ✓ Redistribution carries IP-User, IP-Tag, User-Tag mappings between firewalls, segments define the propagation path.
- ✓ Cloud Identity Engine centralises identity for both User-ID and GlobalProtect SAML.

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY

Parts 10 to 18 · Everything in the Room, No Explanation

Second half. Same rule: if a line doesn't land instantly, that's the topic to reopen before you close this guide for the last time today.

✓ 10 · VPN & Tunnels

- ✓ IKE Gateway = Phase 1. IPSec Tunnel = Phase 2, runs inside Phase 1's channel.
- ✓ PAN-OS is route based; proxy IDs bridge to a policy based peer's fixed encryption domain.
- ✓ DPD = is the peer alive (IKE). Tunnel monitoring = is traffic actually passing (ICMP, data plane).
- ✓ GRE = encapsulation only, no encryption, pair with IPSec for confidentiality, needed for multicast over a VPN.
- ✓ RFC 8784 = symmetric PQ preshared key. RFC 9370 = extra key exchange (e.g. ML-KEM). Both additive to IKEv2.

✓ 11 · GlobalProtect

- ✓ Portal = hands out config. Gateway = terminates the tunnel and enforces policy.
- ✓ Build order: certs → interfaces → **gateway before portal** → security policy → client settings/HIP → pilot.
- ✓ Connect: User-Logon (always on) · Pre-Logon (machine cert, before login) · On Demand.
- ✓ Split tunnel by route/domain/app; empty Include list = full tunnel by default, not split.
- ✓ HIP object = one fact checked. HIP profile = objects combined with AND/OR/NOT, used in policy.
- ✓ Clientless VPN: browser only, published web apps, no HIP data at all.

✓ 12 · High Availability

- ✓ HA1 = heartbeats + config sync. HA2 = session state. HA3 = A/A packet forwarding only.
- ✓ HA1 down + HA2 up = split brain precursor. Back up HA1.
- ✓ Election: lower priority wins. Preemption off = no automatic failback (most designs).
- ✓ Link monitoring = is the interface up. Path monitoring = can I reach beyond it (ICMP).
- ✓ A/A: session owner (processes, fixed) vs session setup (first packet, may differ), mismatch forwards via HA3.
- ✓ Upgrade order: suspend passive → upgrade → verify, rejoin → fail over → stabilise → upgrade the other.

✓ 13 · Virtual Systems

- ✓ VSYS = logical firewall on shared hardware, the ASA contexts equivalent.
- ✓ Interfaces belong to one VSYS; zones cannot cross VSYS boundaries.
- ✓ Each VSYS gets its own virtual/logical router, independent, addressing can overlap.
- ✓ External zone = one VSYS to a specific other VSYS. Shared gateway = many VSYS to one common exit.
- ✓ VSYS admins see only their assigned VSYS; resource limits cap sessions/rules/zones per VSYS.

✓ 14 · Web Proxy & Logging

- ✓ Explicit proxy = client aware, PAC file, supports direct proxy auth. Transparent = unaware, relies on User-ID.
- ✓ 10 log types, each independently forwardable via its own match list.
- ✓ Log Forwarding Profile does nothing until attached to a rule/log setting.
- ✓ Tag built in action + waiting DAG + deny rule = the auto quarantine chain, no commit needed.
- ✓ Strata Logging Service = cloud, unsized, subscription. Panorama collectors = you own and size it.

✓ 15 · Panorama & ACC

- ✓ Device group = policy/objects, hierarchical. Template stack = device/network config, higher wins conflicts.
- ✓ Eval order: Pre Rules (unbypassable) → local rules → Post Rules (safety net) → implicit rules.
- ✓ **Commit** = real on Panorama only. **Push** = sent to firewalls. Commit without push is the #1 mistake.
- ✓ Access Domains scope an admin to specific device groups/templates, the Panorama scale VSYS admin.
- ✓ ACC = Aggregated Monitoring on Panorama; blank widgets usually mean broken log forwarding, not a bug.
- ✓ 3 root causes behind most Panorama issues: missing assignment, forgotten push, resource constraint.

✓ 16 • Software Updates

- ✓ Content updates (App-ID/Threat/AV/WildFire/URL) are frequent, no reboot, biggest risk is a new App-ID reclassifying live traffic.
- ✓ Mitigate with threshold based deployment + reading release notes first.
- ✓ PAN-OS upgrades cannot skip major versions, follow the published path. Preferred Release ≠ latest.
- ✓ Export a named config snapshot off device before any risky change, per commit history is not a backup strategy.

✓ 17 • Cloud & Orchestration

- ✓ Self managed VM-Series = you operate it. Cloud NGFW = Palo Alto operates it, native marketplace service.
- ✓ Bootstrap: init-cfg.txt mandatory (wins conflicts), bootstrap.xml optional. VM auth key = Panorama auto register.
- ✓ NSX-T: T0 = north south (Active/Standby only). T1 = east west. Notify Group keeps DAGs live.
- ✓ CN-Series: CN-MGMT + CN-NGFW pods, PAN-CNI. A pod **annotation** turns on inspection, not a route.
- ✓ MSF = 4 pillars (Visibility, Automation, Unified Data Plane, Central Mgmt via SCM). HSF = P-Nodes (fixed, inspect+LB) + S-Nodes (auto scale).
- ✓ XML API = full feature depth. REST API = simpler, resource shaped. Terraform/Ansible sit on top of both.

✓ 18 • Troubleshooting

- ✓ First packet = full rulebase walk + lookups. Every packet after = session table match only.
- ✓ App-ID can revise its verdict mid session, forcing a policy reevaluation, not a bug.
- ✓ Session states: INIT, ACTIVE, CLOSING/CLOSED, DISCARD (still tracked, actively dropping).
- ✓ Capture stages in order: **Receive → Firewall → Drop → Transmit**. The pattern is the diagnosis.
- ✓ Order: symptom → logs → session table → global counters (delta yes) → capture, always filtered.
- ✓ GUI: click a log value to build the filter.
CLI: `tail follow yes filter "( ... )"` for live.



The six things worth saying without being asked

- **Pre NAT address, post NAT zone:** the single most tested NAT/policy detail in the platform.
- **Gateway before portal** in every GlobalProtect build, the portal references objects the gateway must already have.
- **Commit ≠ push** on Panorama, a committed, unpushed change has changed nothing on any firewall.
- **Tag action + DAG + deny rule** is the whole auto quarantine story, and it needs no commit to take effect.
- **Capture stage pattern is the diagnosis:** Receive only is upstream, Receive+Drop is a policy decision, all four is clean.
- **A new content update, not a config change,** is the first thing to check when something "just stopped working."

• NETWORKING • DEVICE SETTINGS • INTEGRATION • CONTEXT

KNOW | CONFIGURE | VERIFY